

중첩 학습: 심층 학습 아키텍처의 착각

Nested Learning: The Illusion of Deep Learning Architecture

Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, Vahab Mirrokni

문서 성격 원문 순서 · 수치 · 한정어를 보존한 한국어 구조 보존형 의역본이다. 본문 뒤의 원문 보존 부록은 원 논문의 equation, table, figure, caption 을 검증 가능하게 남긴다. 인용과 재현에는 반드시 고정 원문을 사용한다.

고정 버전 arXiv:2512.24695v1

배포 상태 internal-only — arXiv/proceedings rights; translated derivative and source plate are internal-only

Contents

Nested Learning: The Illusion of Deep Learning Architecture —한국어 전문 번역 본 (무축약)	2
중첩 학습 (Nested Learning): 심층 학습 아키텍처의착각	2
초록 (Abstract)	2
1. 서론 (Introduction)	3
2. 예비개념 (Preliminaries)	7
3. 중첩 학습 (Nested Learning)	9
4. 학습 모듈로서의 옵티마이저 (Optimizers as Learning Modules)	19
5 신경 학습 모듈로서의 기존 아키텍처들 (Existing Architectures as Neural Learn- ing Modules)	29
6 핵심 요점 및 혼한 용어의재조명	33
7. 연속체 다중 시간척도 메모리 시스템 (Continuum Multi-Timescale Memory System)	36
7.1 연속체 메모리 시스템 (Continuum Memory System, CMS)	36
7.2 옵티마이저에서의 연속체 메모리시스템	38
7.3 애드혹 레벨 스택킹 (Ad-hoc Level Stacking): 사전 학습된모델로 CMS 초기화 하기	39
8. Hope: 연속체 메모리를 갖춘 자기 참조적 학습 모듈 (A Self-Referential Learning Module with Continuum Memory)	40
9. 실험 (Experiments)	44
9.3	52
9.7 표현력 있는 옵티마이저 (Expressive Optimizers)	56
10 결론 (Conclusion)	57
참고문헌 (References)	57
A. 중첩 학습 (Nested Learning) 과 중첩 시스템 (Nested Systems) 을 위한 일반화 된정식화	70
B. 연상 기억 모듈로서의 Adam, AdaGrad 및 기타 유사옵티마이저	71
C. 정규화를 포함한 델타 경사하강법 (Delta Gradient Descent with Normalization)	73
번역 · 검증 고지	75
원문 구조 · 그림 보존 부록	76

Nested Learning: The Illusion of Deep Learning Architecture — 한국어 전문 번역본 (무축약)

arXiv:2512.24695 「Nested Learning: The Illusion of Deep Learning Architecture」(Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, Vahab Mirrokni (Google))의 전문 무축약 한국어번역입니다. 스터디용이며 공식 번역이 아닙니다. 원문의 모든문장·수식·표·각주를 빠짐없이 옮기는 것을 목표로 했고, 수식은 PDF 추출 텍스트 기반이라 일부 기호가 손상될 수 있습니다. 그림은 원저자의 것입니다.

중첩 학습 (Nested Learning): 심층 학습 아키텍처의 착각

Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, Vahab Mirrokni

arXiv:2512.24695v1 [cs.LG] 2025 년 12 월 31 일

초록 (Abstract)

지난 수십 년 동안, 더 강력한 신경망 아키텍처를 개발하고 동시에 그것을 효과적으로 훈련하기 위한 최적화 알고리즘을 설계하는 일이 머신러닝 모델의 능력을 향상시키기 위한 연구 노력의 핵심이었다. 특히 언어 모델 (Language Models, LMs) 개발에서의 최근 진전에도 불구하고, 그러한 모델이 어떻게 지속적으로 학습/기억하고, 스스로를 개선하며, 효과적인 해를 찾을 수 있는지에 대해서는 근본적인 도전 과제와 미해결된 질문들이 존재한다. 본 논문에서 우리는 **중첩 학습 (Nested Learning, NL)** 이라 불리는 새로운 학습 패러다임을 제시하는데, 이는 머신러닝 모델을 각자 자신만의 “맥락 흐름 (context flow)” 을 갖는, 중첩적이고 (nested), 다중수준 (multi-level) 이며, 그리고/또는 병렬적인 (parallel) 일련의 최적화 문제들로 일관성 있게 표현한다. NL의 관점을 통해 보면, 기존의 심층 학습 (deep learning) 방법들은 자신의 맥락 흐름을 압축함으로써 데이터로부터 학습하며, 인-컨텍스트 학습 (in-context learning) 은 큰 모델에서 자연스럽게 창발한다 (emerge). NL은 더 많은 “수준 (level)” 을 가진 더 표현력 있는 학습 알고리즘을 설계하는 철학을 제안하며, 이는 더 높은 차수 (higher-order) 의 인-컨텍스트 학습을 낳고 잠재적으로 효과적인 지속학습 (continual learning) 능력을 열어준다. 신경과학적 동기에 더하여, 우리는 세 가지 핵심 기여를 제시함으로써 NL을 옹호한다: (1) **표현력 있는 최적화기 (Expressive Optimizers)**: 우리는 Adam, 모멘텀을 가진 SGD 등과 같은 알려진 경사 기반 (gradient-based) 최적화기들이 사실은 (경사 하강을 통해) 경사 (gradient) 정보를 압축하려는 연상 기억 (associative memory) 모듈임을 보인다. 이 통찰에 기반하여, 우리는 깊은 기억 (deep memory) 그리고/또는 더 강력한 학습규칙을 갖춘 다른 “더 표현력 있는” 최적화기들을 제시한다; (2) **자기 수정 학습 모듈 (Self-Modifying Learning Module)**: 학습 알고리즘에 대한 NL의 통찰을 활용하여, 우리는 자신만의 갱신 알고리즘을 학습함으로써 스스로를 어떻게 수정할지 학습하는 시퀀스 모델을 제시한다; 그리고 (3) **연속체 기억 시스템 (Continuum Memory System)**: 우리는 “장기/단기 기억 (long-term/short-term memory)” 이라는 전통적 관점을 일반화하는 새로운 기억 시스템 정식화 (formulation) 를 제시한다. 우리의 자기 수정 시퀀스 모델을 연속체 기억 시스템과 결합하여, 우리는 **Hope** 라 불리는 지속 학습 모듈을 제시하며, 이는 언어 모델링, 지식 통합 (knowledge incorporation), 소수 샷 일반화 (few-shot generalization) 과제, 지속 학습, 그리고 장문 맥락 추론 (long-context reasoning) 과제에서 유망한 결과를 보인다.

“우리는 우리의 문제를 그것을 만들어냈을 때 사용했던 것과 동일한 사고방식으로는 해결할 수 없다!” — 알베르트 아인슈타인의 말로 전해짐

1. 서론 (Introduction)

수십 년 동안, AI 연구는 데이터로부터 학습하거나 (Pitts 1943; McCulloch et al. 1948; McCulloch 1949; Samuel 1959) 경험으로부터 학습하는 (Sutton et al. 1998; Connell et al. 1999; Silver et al. 2025) 머신러닝 알고리즘을 설계하는 데 집중해 왔다; 이는 흔히 경사 기반 방법으로 파라미터 $\theta \in \Theta$ 에 대한 목적함수 $L(\theta)$ 를 최적화함으로써 이루어진다. 전통적인 머신러닝 기법은 특징 추출기 (feature extractor)를 설계하기 위해 세심한 엔지니어링과 도메인 전문성을 요구하여, 자연 데이터를 직접 처리하고 학습하는 능력을 제한했지만 (LeCun et al. 2015), 심층 표현 학습 (deep representation learning)은 과제에 필요한 표현을 발견하는 완전히 자동화된 대안을 제공했다. 그 이후로, 심층 학습은 대규모 계산 모델의 때려야 뗄 수 없는 부분이 되었으며, 화학과 생물학 (Jumper et al. 2021), 게임 (Silver et al. 2016, 2018), 컴퓨터 비전 (Krizhevsky et al. 2012; Dosovitskiy et al. 2021), 그리고 멀티모달 및 자연어 이해 (Achiam et al. 2023; Liu et al. 2024a; Comanici et al. 2025)에서 획기적인 성공을 거두었다.

심층 학습 모델에서 이루어지듯이 여러 층 (layer)을 쌓는 것은, 복잡한 특징을 표현하는 데 있어 모델에 더 나은 표현력 (expressive power)을 제공하고, 더 많은 내부 계산 (예: #FLOPS)을 제공한다 (Montufar et al. 2014; Poole et al. 2016; Hestness et al. 2017). 이 모든 것은 사전에 고정된 집합에 대한 분포 내 (in-distribution) 예측을 요구하는 정적 (static) 과제에 대해 결정적으로 중요하고 바람직한 특성이다. 그러나 이러한 깊은 설계는 모든 도전 과제에 대한 보편적 해법은 아니며, 여러 측면에서 모델의 표현력을 도울 수 없다. 예를 들어: (i) 심층 모델의 계산적 깊이 (computational depth)는 층을 더 쌓아도 변하지 않을 수 있어 (Merrill et al. 2022; Sanford et al. 2024), 복잡한 알고리즘을 구현하는 능력은 전통적인 얕은 (shallow) 접근법에 비해 손대지 않은 채 남는다 (Merrill et al. 2024); (ii) 어떤 종류의 파라미터의 용량 (capacity)은 모델의 깊이/너비를 늘려도 미미한 개선만 보일 수 있다 (Kaplan et al. 2020); (iii) 훈련 과정이 최적이지 아닌 해로 수렴할 수 있는데, 이는 주로 최적화기 (optimizer)

§ 교신 저자: {alibehrouz, razaviyayn, mirrokni}@google.com

및 peilin.zhong@columbia.edu.

§ 본 연구의 한 판본은 Neural Information Processing Systems (NeurIPS) 2025에 게재되었다.

그림 1(Figure 1): 뇌의 균일하고 (uniform) 재사용가능한 (reusable) 구조와 다중 시간 척도 (multi time scale) 갱신은 인간의 지속 학습을 열어주는 핵심 구성요소이다. 중첩 학습 (NL)은 뇌의 각 구성 요소에 대해 다중 시간 척도 갱신을 허용하며, 동시에 Transformer 같은 잘 알려진 아키텍처들이 사실은 서로 다른 빈도 (frequency)로 갱신되는 선형층 (linear layer) 들임을 보여준다.

또는 그 하이퍼파라미터의 최적이지 아닌 선택 때문이다; 그리고 (iv) 모델이 새로운 과제에 빠르게 적응하고, 지속적으로 학습하며, 그리고/또는 분포 외 (out-of-distribution) 데이터로 일반화하는 능력은 층을 더 쌓아도 변하지 않을 수 있으며 더 세심한 설계를 요구한다.

위의 도전 과제들을 극복하고 심층 학습 모델의 능력을 향상시키기 위한 노력의 핵심 부분은 다음에 집중되어 있다: (1) 더 표현력 있는 부류의 파라미터 (즉, 신경망 아키텍처) 개발 (Fukushima 1980; Schmidhuber et al. 1997; Krizhevsky et al. 2012; Vaswani et al. 2017; Behrouz et al. 2025c); (2) 과제를 더 잘 모델링할 수 있는 목적함수의 도입 (Rumelhart et al. 1986; Kingma et al. 2014b; Hjelm et al. 2019; Goodfellow et al. 2020; Alshammari et al. 2025); (3) 더 나은 해를 찾거나 망각에 대한 회복력이 더 큰, 더 효율적/효과적인 최적화 알고리즘의 설계 (Kingma et al. 2014a; Gupta et al. 2018; Farajtabar et al. 2020; Jordan et al. 2024); 그리고 (4) 아키텍처, 목적함수, 최적화 알고리즘의 “올바른” 선택이 이루어졌을 때, 표현력을 향상시키기 위해 모델 크기를 확장 (scaling) 하는 것 (Brown et al. 2020; Kaplan et al. 2020; Hoffmann et al. 2022). 종합적으로, 심층 모델의 확장 패턴에 관한 이러한 진전과 새로운 발견들은 대규모 언어 모델 (Large Language Models, LLMs)이 구축된 기초를 확립했다.

LLM의 발전은 심층 학습 연구의 중대한 이정표를 표시한다: 과제특화 (task-specific) 모델에서, “올바른” 아키텍처를 확장한 결과로 다양한 창발적 (emergent) 능력을 갖춘 더 범용적인 (general-

purpose) 시스템으로의 패러다임 전환이다 (Brown et al. 2020; Schaeffer et al. 2023). 다양한 과제집합에서의 모든 성공과 놀라운 능력에도 불구하고 (Nijkamp et al. 2023; Wang et al. 2023; Comanici et al. 2025), LLM은 초기 배포 단계 이후로 대체로 정적 (static)이며, 이는 사전 훈련 (pre-training) 이나 사후 훈련 (post-training) 중에 학습한 과제는 성공적으로 수행하지만, 자신의 즉각적인 맥락 (immediate context) 을 넘어서는 새로운 능력을 지속적으로 획득하지는 못한다는 것을 의미한다. LLM의 유일하게 적응 가능한 구성요소는 그들의 인-컨텍스트 학습 능력이다—이는 (창발적인 것으로 알려진) LLM의 특성으로, 맥락에 빠르게 적응하여 제로 샷 또는 소수 샷 (zero- or few-shot) 과제를 수행할 수 있게 한다 (Brown et al. 2020). 인-컨텍스트 학습을 넘어서 LLM의 정적 본성을 극복하려는 최근의 노력들은 계산적으로 비싸거나, 외부 구성요소를 요구하거나, 일반화가 부족하거나, 그리고/또는 파국적 망각 (catastrophic forgetting) 으로 어려움을 겪을 수 있는데 (Akyurek et al. 2024a; Eyuboglu et al. 2025; Yu et al. 2025), 이는 연구자들로 하여금 머신러닝 모델을 어떻게 설계할지 재검토할 필요가 있는지, 그리고 지속적 (continual) 설정에서 LLM의 능력을 발휘하기 위해 층을 쌓는 것을 넘어서는 새로운 학습 패러다임이 필요한지 질문하게 만들었다.

현재의 모델은 오직 즉각적인 현재만을 경험한다 (Current Models only Experience the Immediate Present). 유비로서, 그리고 LLM의 정적 본성을 더 잘 예시하기 위해, 우리는 전향성 기억상실증 (anterograde amnesia)의 예를 사용한다—이는 장애 발병 이후에 새로운 장기 기억을 형성하지 못하지만, 기존 기억은 온전히 남아 있는 신경학적 상태이다 (Scoville et al. 1957). 이 상태는 그 사람의 지식과 경험을 짧은 현재의 창 (window) 과 먼 과거—장애 발병 이전—로 제한하며, 그 결과 즉각적인 현재를 마치 언제나 새로운 것처럼 지속적으로 경험하게 된다. 현재 LLM의 기억 처리 시스템은 유사한 패턴으로 어려움을 겪는다. 그들의 지식은, 자신의 맥락 창에 들어맞는 즉각적인 맥락, 또는 “사전 훈련의 끝”이라는 발병 이전의 먼 과거를 저장하는 MLP 안의 지식 중 하나로 제한된다. 이 유비는 우리로 하여금 신경생리학 (neurophysiology) 문헌과 뇌가 자신의 단기 기억을 어떻게 공고화 (consolidate) 하는지로부터 영감을 얻도록 동기를 부여했다.

1.1 인간 뇌의 관점과 신경생리학적 동기 (Human Brain Perspective and Neurophysiological Motivation)

인간의 뇌는 지속 학습에 있어 매우 효율적이고 효과적인데, 이는 흔히 신경가소성 (neuroplasticity)—새로운 경험, 기억, 학습, 그리고 심지어 손상에 반응하여 스스로를 변화시키는 뇌의 놀라운 능력 (Pascual-Leone et al. 2005; Johnston 2009)—에 기인한다. 최근 연구들은 장기 기억의 형성이 적어도 두 개의 구별되지만 상호 보완적인 공고화 과정을 포함한다는 것을 뒷받침한다 (Frey et al. 1997; Goto et al. 2021; Yang et al. 2024): (1) 빠른 “온라인 (online)” 공고화 (시냅스 공고화 (synaptic consolidation) 로도 알려짐) 단계는 학습 직후 또는 곧이어서, 심지어 깨어 있는 동안에도 일어난다. 이는 새롭고 초기에는 취약한 기억 흔적 (memory trace) 이 안정화되고 단기 저장에서 장기 저장으로 이전되기 시작하는 시점이다; (2) “오프라인 (offline)” 공고화 (시스템 공고화 (systems consolidation) 로도 알려짐) 과정은 최근에 부호화된 패턴의 재생 (replay) 을 반복하는데—해마 (hippocampus)에서의 예리파 (sharp-wave ripples, SWRs) 동안, 피질 (cortical) 수면 방추 (sleep spindles) 및 느린 진동 (slow oscillations) 과 협응하여—기억을 강화하고 재조직하며 피질 부위로의 이전을 지원한다 (Foster et al. 2006; Ji et al. 2007; Peyrache et al. 2009).

전향성 기억상실증의 유비로 돌아오면, 증거는 이 상태가 두 단계 모두에 영향을 미칠 수 있지만, 특히 온라인 공고화 단계에 영향을 미친다는 것을 시사하는데, 이는 주로 해마가 새로운 서술 기억 (declarative memory) 을 부호화하는 관문 (gateway) 이며, 따라서 그 손상은 새로운 정보가 결코 장기 기억에 저장되지 않음을 의미하기 때문이다. 위에서 언급했듯이, LLM의 설계, 더 구체적으로는 Transformer 기반 백본 (backbone) 은 사전 훈련 단계 이후 유사한 상태로 어려움을 겪는다. 즉, 맥락에서 제공된 정보는 결코 장기 기억 파라미터 (예: 피드포워드 (feedforward) 층) 에 영향을 미치지 않으며, 따라서 정보가 여전히 단기 기억 (예: 인-컨텍스트 또는 어텐션) 에 저장되어 있지 않는 한, 모델은 새로운 지식이나 기술을 획득할 수 없다. 이를 위해, 비록 두 번째 단계가 기억의 공고화에 있어 동등하게, 또는 심지어 더 결정적으로 중요하며, 그것의 부재가 그 과정을 손상시키고 기억 상실을 야기할 수 있지만 (Drummond et al. 2000; Yoo et al. 2007), 본 연구에서 우리는 첫 번째 단계에 집중한다: 온라인 과정으로서의 기억 공고화. 앞서 논의했듯이, 인간의 기억 처리, 그 온라인 공고화, 그리고 지속 학

습 능력은 신경가소성뿐만 아니라 신경 진동 (neural oscillations) 에도 크게 의존하는 것으로 알려져 있다 (Bliss et al. 1993; Buzsaki et al. 2004; Klinzing et al. 2019).

다중 시간 척도 처리 시스템 (Multi Time scale Processing System). 뇌 진동 (brain oscillations, 뇌파 (brainwaves) 로도 알려짐)—뇌 활동의 율동적인 변동—은 단순히 뇌 기능의 부산물이 아니라, 주의, 기억, 의사결정과 같은 다양한 인지 기능에서 결정적인 역할을 하며, 신경 계산을 조직하고, 뇌 영역 간의 통신을 협응하며, 학습과 기억의 기저를 이루는 시냅스 가소성 (synaptic plasticity) 을 개폐 (gating) 하는 핵심 메커니즘인 것으로 점점 더 이해되고 있다 (Fell et al. 2011; Cavanagh et al. 2014; Fries 2015). 이러한 뇌파는 뇌가 서로 다른 시간 척도와 갱신 빈도에서 자신의 계산을 협응한 결과이며, 각 빈도는 뇌 뉴런 집단이 얼마나 자주 활성화되고 갱신된 정보를 공유하는지를 결정한다. 더 구체적으로, 그러한 신경 진동은 전형적으로 구별되는 빈도들로 분류되며, 각각은 서로 다른 인지 기능과, 결정적으로, 서로 다른 정보 처리의 시간 척도와 연관되어 있다: (1) 주로 감각 정보와 연관된 빠른 감마파 (Gamma waves, 30-150 Hz 빈도) 로부터, (2) 주로 능동적 사고와 연관된 베타파 (Beta waves, 13-30 Hz 빈도) (Buzsaki et al. 2004; Buschman et al. 2007; Lundqvist et al. 2016), 그리고 (3) 주로 기억 공고화와 학습을 담당하는 느린 델타파와 세타파 (Delta and Theta waves, 0.5-8 Hz 빈도) (Marshall et al. 2006; Diekelmann et al. 2010; Ngo et al. 2013; Staresina et al. 2015; Heusser et al. 2016; Daume et al. 2024) 에 이르기까지 걸쳐 있다.

그러나 심층 학습 모델에서는, 아키텍처의 가중치가 테스트 시점에 고정되어 있으며, 또한 사전 훈련에서 모델의 모든 블록/층에 대해 동일한 갱신 속도 (update rate) 를 사용하는 것이 일반적이다. 그러나 나중에 6 절에서, 우리는 인-컨텍스트 학습이 이 설계의 극단적인 경우를 제공하며, 사실 Transformer 아키텍처가 갱신의 두 극단적 빈도에 기반하고 있음을 보인다: 즉, 어텐션 (attention) 블록과 MLP 블록에 대해 각각 ∞ 와 0 의 빈도이다.

뇌의 균일하고 재사용 가능한 구조 (Brain's Uniform and Reusable Structure). 앞서 논의했듯이, 신경가소성은 새로운 기억, 지식, 그리고 심지어 손상에 반응하여 스스로를 변화시키는 뇌의 놀라운 능력이다 (Pascual-Leone et al. 2005; Johnston 2009). 이 특성은 신경요소들이 하나의 기능에 경직되게 전담되지 않고, 대신 재사용 가능하며, 서로 다른 인지적 필요를 지원하기 위해 유연하게 재배치될 수 있는 균일한 아키텍처를 시사한다. 신경 재사용성 (neural reusability) 의 실제 사례 중 하나는 대뇌반구 절제술 (hemispherectomy) 이다—이는 보통 심각한 뇌전증 (epilepsy) 을 완화하기 위해 하나의 대뇌 반구를 외과적으로 제거하거나 무력화하는 것이다. 놀랍게도, 이 수술이 유년기에 이루어지면, 환자들은 성인기까지 높은 기능의 인지 능력과, 전형적인 양반구 뇌에 존재하는 모든 동일한 핵심 뇌 신경망 (언어, 시각 등을 위한 신경망) 을 포함하는 온전한 신경망 조직을 갖추고 대체로 정상적인 삶을 영위할 수 있다. 이 놀라운 결과는 뇌의 균일한 아키텍처에 대한 실제적 증거를 제공한다. 즉, 뇌의 절반만으로도 자원을 재활당하고 재조직하여 그 사람이 극도로 잘 기능할 수 있게 한다. 이러한 사례들은, 피질의 일부가 없는 상태로 비교적 정상적으로 사는 개인들의 문서화된 사례들과 함께, 뇌의 균일하고 재사용 가능한 구조를 강조한다.

그림 2 (Figure 2): 머신러닝 모델과 그 훈련 절차를 일련의 중첩된 최적화 문제로 표현하는 중첩 학습 패러다임. (왼쪽) 하이브리드 (Hybrid) 아키텍처의 예시. NL 의 평면화된 이미지 (flattened image) 로서의 심층 학습 관점은 블록 내부의 계산 깊이에 대한 통찰을 제공하지 않지만, NL 은 모든 내부 경사 흐름 (gradient flow) 을 투명하게 표현한다. (오른쪽) 신경 학습 모듈 (Neural Learning Module): 자신의 맥락 흐름을 어떻게 압축할지 학습하는 계산 모델. 예를 들어, 첫 번째 수준은 모델의 가장 바깥쪽 루프 (outer-loop) 훈련에 해당하며, 흔히 “사전 훈련 (pre-training)” 단계라 불린다.

더 나아가, 뇌의 균일하고 재사용 가능한 구조에 대한 이러한 해석은 인간 뇌에서 기억이 어떤 특정 영역에 있는 고립된 시스템이 아니며, 주로 뇌 전체에 분산되어 (distributed) 있음을 보여준다. 즉, 서로 다른 종류의 기억이 별개의 뇌 구조에 자리한다고 흔히 암시했던 전통적인 기억 모델 (예: 전두 피질 (frontal cortex) 의 단기 기억 대 해마와 피질의 장기 기억) 과는 반대로, 현대 연구는 여러 영역에 걸친 분산된 신경 회로 (distributed neural circuits) 기억 처리를 옹호한다 (Christophel et al. 2017; Kitamura et al. 2017; Roy et al. 2022).

그러나 최근 몇 년간의 현대 심층 학습 아키텍처는, 적어도 표면적으로는, 이질적 (heterogeneous) 으로 보이며, 자기 어텐션 (self-attention) 변형들의 부분 집합 (Vaswani et al. 2017), 현대 순환 신경망 (recurrent neural networks) (Katharopoulos et al. 2020; Schlag et al. 2021; Behrouz et

al. 2025c; Peng et al. 2025b), 캐논 층 (canonical layers)(Allen-Zhu 2025), 전역 컨볼루션 (global convolutions)(Hasani et al. 2023; Poli et al. 2023), 그리고 MLP 블록 (Shazeer 2020) 의 조합에 기반하고 있다. 이는우리가 새로운 균일 아키텍처를 필요로 하는지, 아니면 현재 모델의 이질성에 대한 우리의 믿음을 재검토할 필요가 있는지에 대한 질문을 제기한다.

1.2 기여와 로드맵 (Contributions and Roadmap)

본 논문에서 우리는 기존 알고리즘, 방법, 아키텍처에 대한 새로운 통찰을제공할 뿐만 아니라, 계산적 깊이와 모델의 지속 학습 능력을 향상시키면서심층 학습에서 층을 쌓는 것에 대한 새로운 차원을 드러내는 통합적 학습패러다임을 제시하고자 한다. § 2 에서 예비 개념과 배경을 논의한 후, 우리는다음을 제시한다:

중첩 학습 패러다임 (Nested Learning Paradigm, § 3). 위에서 제기된질문들에 답하고 지속 학습, 아키텍처 설계, 현대 심층 학습 모델의 계산적깊이에서의 설계 도전 과제를 극복하는 것에 대한 새로운 통찰을 제공하기위해, 우리는 중첩 학습 (NL) 을 제시한다—이는 머신러닝 모델의 각 구성요소여러 수준에서 자신만의 맥락에 대한 자신만의 내부 경사 흐름을 갖도록허용하는 학습 패러다임으로, 모델과 그 학습 과정 (즉, 최적화) 을중첩적이고, 다중 수준이며, 그리고/또는 병렬적인 최적화 문제들의 상호 연결된 시스템으로 표현한다. 우리는 최적화 과정과 학습알고리즘/아키텍처가 근본적으로 동일한 개념이지만 서로 다른맥락 (즉, 경사 대 토큰) 을 갖는 시스템의 서로 다른 수준에 있는 것이라고주장한다. 더 나아가, 그것들은 학습 알고리즘/아키텍처가최적화기를 위한 맥락 (즉, 경사) 을 생성하는 두 개의 상호 연결된구성요소이며, 이는 아키텍처 특화 (architecture-specific) 최적화기를설계할 것을 옹호한다. 우리는 수준 간의 지식 전달 (knowledge transfer) 의서로 다른 방식들을 논의하며, 이는 메타 학습 (meta-learning), 인-컨텍스트학습, 순환 신경망, 하이퍼네트워크 (hypernetwork) 등과 같은 개념들을통합하고 일반화하는 결과를 낳는다.

학습 모듈로서의 최적화기와 아키텍처 (Optimizers and Architectures as Learning Module, § 4, § 5). NL 의 관점에 기반하여, 우리는역전파 (backpropagation) 과정과 경사 하강으로 심층 신경망을 훈련하는것이, 층의 입력을 예측에서의 해당 국소 오차 (local error) 로 매핑하는 연상기억을 훈련하는 것을 목표로 하는 압축 및 최적화 문제라고 주장한다. 따라서 우리는 사전 훈련이 인-컨텍스트 학습의 한 형태라고 주장하는데, 여기서 맥락은 전체 사전 훈련 데이터이며 층들은 그 맥락을 자신의파라미터로 압축하고 있다. 우리는 그러한 주장이 다른 인기 있는 경사 기반최적화기들에도 유효하며, 그것들이 경사를 자신의 파라미터로 압축하려는연상 기억임을 입증한다. NL 의 용어로, 모멘텀을 가진 경사 하강, Adam(Kingma et al. 2014a), AdaGrad(Duchi et al. 2011) 와 같은 경사 기반최적화기들은 두 수준의 중첩된 최적화 문제로 분해될 수 있으며, 각각은단순한 경사 하강으로 최적화된다. 특히, 이 관점은 경사를 압축하는 데있어, 이론적으로 Adam 이 원소별 (element-wise) L2 회귀 (regression) 목적함수에 관해 최적의 연상 기억임을 명백하게 만든다.

우리는 아키텍처를 연상 기억으로 표현하는 것에 관한 이전의 발견들 (Behrouz et al. 2025b) 을 재검토하고, 그들의 최적화 과정을 일련의 중첩된 최적화문제들로 분해하며, 이 모두는 경사 하강으로 최적화된다. 위의 발견들—즉, 인기 있는 경사 기반 최적화기와 현대 아키텍처가 모두 일련의 중첩적 그리고/또는 병렬적 최적화 문제라는 것—에 기반하여, 우리는이 둘의 조합—즉, 특정 최적화기로 아키텍처를 훈련하는 것—또한 일련의중첩적 그리고/또는 병렬적 최적화 문제로 표현될 수 있다고주장한다. 따라서 신경 학습 모듈 (neural learning module, 아키텍처와 그훈련/최적화 과정의 결합 시스템) 은 균일한 모델로서, 여기서모든 요소는 선형 (linear) 또는 깊은 MLP이며, 동시에 그들은 서로 다른빈도로 서로 다른 수준에서 자신만의 내부 목적함수를 최적화하고 있다.

최적화기의 연상 기억 관점에 기반하여, 우리는 경사를 압축하는 데 있어 더표현력 있는 기억 구조 또는 기억 관리 (memory management) 를 갖춘 일련의새로운 학습 갱신 (최적화 단계) 을 설계한다. 특히, 우리는 최적화기의 선택이최적화의 맥락에 의존한다고 주장한다. 경사를 압축하는 데 강력한최적화기가 토큰을 압축하는 데는 최선의 선택이 아닐 수 있다. 이를 위해, 우리는 델타 경사 하강 (Delta Gradient Descent, DGD) 이라 불리는 새로운변형의 경사 하강을 제시하는데, 그 갱신은 현재 입력에만 의존하는 것이아니라 신경망 가중치의 상태에도 의존하여, i.i.d. 가정 없이 데이터표본들의 의존성을 포착하는 결과를 낳는다.

주요 시사점과 흔한 용어의 재검토: 지속 및 인-컨텍스트 학습, 사전훈련, 그리고 학습 (Main Takeaways and Revisiting Common Terms: Continual and In-context Learning, Pre-Training, and Learning, § 6). 우리는 원리적개념들에 대한 NL의 주요 시사점을 논의하고 몇몇 흔한 용어들을 재검토한다: (1) 우리는 지속 학습이 들어오는 맥락 (context) 또는 에피소드 (episode)의 시퀀스에 대한 학습 문제로 볼 수 있으며, 여기서 서로 다른 수준들이 자신만의 인-컨텍스트 지식을 압축하고 그것을 더 높은 수준으로 전달하는 것을 담당한다고 주장한다. 이에 기반하여, 우리는 테스트/훈련단계에 의존하지 않고, 오히려 자신의 지식과 기억을 지속적으로 관리하는 모델과 파이프라인을 설계할 것을 옹호한다; (2) 인-컨텍스트 학습은 “여러개의 중첩된 수준을 가지는 것”의 특성이다. 따라서, Transformer의 인-컨텍스트 학습은 토큰에 대한 특정 회귀 목적함수의 비모수적 (non-parametric) 해법이라는 것에서 비롯되는 반면, 현대 순환모델은 그들의 하위 수준에서 모수적 (parametric) 학습 과정을 사용한다; (3) 우리는 더 나아가 학습/기억화 (learning/memorization), 하이브리드 아키텍처, 루프형 아키텍처 (looped architectures), 학습된 최적화기 (learned optimizers)와 같은 다른 용어들도 재검토한다.

연속체 기억 시스템, 자기 참조적 Titans, 그리고 Hope (Continuum Memory System, Self-Referential Titans, and Hope, § 7, § 8). 우리는 연속체 기억 시스템 (Continuum Memory Systems, CMSs)을 제시하고 기억을 빈도 갱신의 스펙트럼을 갖는 분산되고 상호 연결된 시스템으로 봄으로써 “장기/단기 기억 (long-term/short-term memory, LSM)”이라는 전통적 관점을 일반화한다. 이 설계에서, 더 높은 빈도의 뉴런은 빠른 적응을 담당하지만 기억/지식을 짧은 기간 동안 저장하며, 더 낮은 빈도의 뉴런은 더 지속적인 지식을 담당한다. LSM과 비교하여, 우리는 이 다중 빈도 (multi-frequency) 설계가 모델의 기억에 대한 루프과정 (loop process)을 낳는다는 것을 보이는데, 이는 지식이 망각되었을 때 부분적으로 복구될 수 있음을 의미한다. 우리는 주로 이 기억 시스템을 Transformer의 MLP 블록의 대체물로 설계하지만, 이 직관을 활용하여 다중척도 모멘텀 Muon (Multi-scale Momentum Muon, M3) 최적화기—여러 개의 모멘텀 항 (momentum terms)을 갖춘 최적화 알고리즘—을 설계하며, 이는 서로 다른 맥락에서 CMS 설계의 중요성을 더욱 뒷받침한다.

평가 (Evaluations, § 9). 우리의 개념 증명 (proofs-of-concept)의 효과성뿐만 아니라 중첩 학습 설계의 중요성을 뒷받침하기 위해, 우리는 다음에 대한 실험적 평가를 수행한다: (1) 지속 학습 및 인-컨텍스트 학습 과제로 (i) 새로운 언어 학습, (ii) 클래스 증분 학습 (class incremental learning), (iii) 새로운 말뭉치 (corpus)에 대한 질의응답을 포함; (2) 건초더미 속 바늘 찾기 (needle-in-a-haystack) (Hsieh et al. 2024)와 BABILong (Kuratov et al. 2024) 벤치마크를 포함하는 장문 맥락 이해 (long context understanding) 과제; (3) 언어 모델링 및 상식 추론 (common-sense reasoning) 과제; (4) 인-컨텍스트 회상 (in-context recall) 및 기억화 과제; (5) 언어 인식 (language recognition) 과제; 그리고 (6) 우리의 M3 최적화기를 포함한 서로 다른 최적화기들의 비교. 우리의 결과는 지속 학습 능력, 다중 수준의 계산, 자기 참조적 (self-referential) 과정을 갖춘 모델을 설계하는 데 있어 NL 관점의 효과성을 나타낸다.

2. 예비개념 (Preliminaries)

이 절에서 우리는 표기법을 논의하고 배경 개념들을 검토한다.

표기법 (Notations). 우리는 $x \in \mathbb{R}^{N \times d_{in}}$ 을 입력이라 하고, M_t 는 시간 t 에서 기억/모델 M 의 상태를 나타내며, K 는 키 (keys), V 는 값 (values), Q 는 질의 (query) 행렬이라 한다. 우리는 첨자 t 를 가진 굵은 소문자를 입력 t 에 해당하는 벡터를 지칭하는 데 사용한다 (즉, k_t, v_t, q_t). 우리는 더 나아가 임의의 확률 변수 T 의 분포를 $p(T)$ 로 지칭한다. 논문 전반에 걸쳐, 우리는 기억 모듈 $M(\cdot)$ 의 아키텍처로 잔차연결 (residual connection)을 가진 $L_M \geq 1$ 개의 층을 갖는 단순한 MLP를 사용한다. 필요할 때, 우리는 기억 모듈을 $\theta_M \supseteq \{W_1, W_2, \dots, W_{L_M}\}$ 로 파라미터화하며, 이는 적어도 다음의 파라미터들을 포함한다

MLP 내부 선형 계층들의. 우리는 중첩 학습 (nested learning)의 서로 다른 수준 (서로 다른 갱신 빈도)에 있는 파라미터를 가리키기 위해 괄호를 동반한 위첨자를 사용한다. 즉, 수준 인덱스 $W^{(\ell)}$ 를 쓰거나 그에 대응하는 빈도 $W^{(f_\ell)}$ 를 쓴다.

경사 하강법 (Gradient Descent). 경사 하강법은 고차원 문제에 대해 가장 널리 사용되는 최적화 알고리즘 중 하나이며, 그 변형들은 대규모 모델을 학습시키는 표준 도구다. 목적 함수 $\mathcal{L}(\cdot; \cdot)$ 가 주어졌을 때, 스텝 크기 $\eta_t > 0$ (일명 학습률) 을 갖는 확률적 경사하강법 (stochastic gradient descent, SGD) 은 다음과 같이 파라미터를 갱신한다:

$$W_{t+1} = W_t - \eta_t \nabla_{W_t} \mathcal{L}(W_t; \mathbf{x}_t),$$

이는 학습 집합으로부터 얻은 데이터 표본 $\mathbf{x}_t$ 에 대한 것이다. 경사 하강법 정식화는 분석에 유용한 여러 등가적 특성화 (characterization) 를 허용한다. 그 등가적 정식화 중 하나는 유클리드계량 (Euclidean metric) 에서의 최급강하 (steepest-descent) 로서, 경사하강법의 한 스텝은 다음과 등가다:

$$W_{t+1} = \arg \min_W \left\{ \langle \nabla_W \mathcal{L}(W_t; \mathbf{x}_t), W \rangle + \frac{1}{2\eta_t} \|W - W_t\|_2^2 \right\},$$

이는 이차 근접항 (quadratic proximal term) 으로 정규화된 1 차 테일러근사 (first-order Taylor approximation) 를 최소화하는 것이다. 위의 GD 스텝은 정확히 W_t 에서 $\mathcal{L}(\cdot; \cdot)$ 를 선형화한 것에 대한 근접 갱신 (proximal update) 이며, L_2 -거리에서 작은 이동을 향하는 암묵적 편향 (implicit bias) 을 드러낸다. 스텝들을 (상수 학습률 η 로) 누적하면 다음의 follow-the-regularized-leader (FTRL) 형태를 얻는다:

$$W_{t+1} = \arg \min_W \left\{ \left\langle \sum_{s=1}^t \nabla \mathcal{L}(W_s; \mathbf{x}_s), W \right\rangle + \frac{1}{2\eta} \|W - W_1\|_2^2 \right\},$$

그 해는 $W_{t+1} = W_1 - \eta \sum_{s=1}^t \nabla \mathcal{L}(W_s; \mathbf{x}_s)$ 이다. 이 두 정식화는 본 논문에서 상호 교환적으로 사용된다. 다만, 우리의 논의와 정식화는 일반적으로 다른 많은 최적화 알고리즘에 대해서도 유효하다.

메타 학습 (Meta Learning). 효과적인 기계 학습 모델을 설계하는 것은 흔히 $\theta \in \Theta$ 로 파라미터화된 그 아키텍처, 목적 함수 $\mathcal{L}(\theta)$, 그리고 목적 함수를 반복적으로 최적화하는 것을 목표로 하는 최적화기 (optimizer) 에 대한 결정을 내리는 것을 요구한다. 메타 학습 패러다임 (또는 학습하는 법을 학습하기, learning to learn) (Schmidhuber 외 1996; Finn 외 2017; Akyurek 외 2022; Irie 외 2025a) 은 이러한 결정의 일부를 2-수준 최적화 절차로 모델링함으로써 자동화하는 것을 목표로 하는데, 여기서 외부 (outer) 모델은 일련의 과제 (task) 들에 걸쳐 성능을 최대화하도록 내부 (inner) 절차의 파라미터를 설정하는 법을 학습하는 것을 목표로 한다. 즉, 파라미터 Φ 로 파라미터화된 목적 함수, 곧 $\ell(\theta; \mathcal{D}; \Phi)$ 가 주어졌을 때, 외부 루프 과정을 일련의 과제들에 걸쳐 파라미터 Φ 를 최적화하는 것으로 정식화할 수 있다:

$$\Phi^* = \arg \min_{\Phi} \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})} [\ell(\theta; \mathcal{T}_i; \Phi)],$$

여기서 $p(\mathcal{T})$ 는 과제들의 분포다. 메타 학습에 관한 초기 연구들은 외부 루프에 지도 학습 (supervised) 설정을 사용했지만 (Schmidhuber 외 1996), 최근에는 외부 루프에 비지도 (unsupervised) 과정을 사용하는 더 유연한 방법론 계열이 인기를 얻고 있다 (Finn 외 2017; Brown 외 2020; Akyurek 외 2022; Chen 외 2022; Qu 외 2025). 다양한 하류 (downstream) 과제 집합에 메타 학습 방법을 사용하려는 관심이 커지는 것에 더하여 (Finn 외 2017; Munkhdalai 외 2019; Chen 외 2022; Irie 외 2025a; Qu 외 2025), 최근 몇 년간 그것은 강력한 시퀀스 모델을 설계하는 패러다임으로서도 인기를 보여 왔다 (Sun 외 2024; Behrouz 외 2025b,c).

빠른 가중치 프로그래머 (Fast Weight Programmers, FWPs). 빠른 가중치 프로그래머 (더 최근에는 선형 트랜스포머라고 불림) (Hinton 외 1987; Schmidhuber 1992; Ba 외 2016; Schlag 외 2021) 는—신경과학적 관점에서도 동기가 부여되었으며 (Gershman 외 2025)—그 메모리 (또는 은닉 상태) 가 행렬값을 갖는 순환 신경망 (recurrent neural network) 으로서, 단기 기억 (short-term memory) 의 역할을 하는 시변 (time-varying) 빠른-가중치 행렬 $M_t \in \mathbb{R}^{d_{out} \times d_{key}}$ 를 갖는다. 별개의

“프로그래머” (느린 망, slow net) 는 각 입력 $x_t \in \mathbb{R}^{d_{in}}$ 을 질의 (query), 키 (key), 값 (value) 벡터로 사상하고 빠른 가중치를 온라인으로 갱신한다. 기본적인 (헤비안/외적, Hebbian/outer-product) FWP—흔히바닐라 (vanilla) FWP 라고 불림—는 다음과 같이 그 파라미터를 갱신한다:

$$M_t = \alpha_t M_{t-1} + v_t \phi(k_t)^\top,$$

그리고 $y_t = M_t \phi(q_t)$ 로 메모리로부터 검색 (retrieve) 하는데, 여기서 $\phi(\cdot)$ 는 (흔히 키와 질의 모두에 적용되는) 원소별 (element-wise) 특징 사상 (feature map) 이다. 벡터 상태를 갖는 전통적인 RNN 이나 현대 RNN 의 초기 변형들 (Schmidhuber 외 1997; Sun 외 2023; Botev 외 2024) 과 달리, 행렬 M_t 가 순환 상태 (recurrent state) 다. 그것은 랭크-1(rank-one) 갱신으로 기록되고 행렬-벡터 곱으로 읽히며, 시간에 걸쳐 일정한 상태 크기를 갖는 간결하고 학습 가능한 키-값메모리를 제공한다.

맥락 내 학습 (In-context Learning). “맥락 내 학습 (in-context learning)” 개념은 초기에 Brown 외 (2020) 에 의해, 언어 모델이 오직 그 맥락 (예: 소수의 예시, 또는 자연어 지시) 만을 근거로 새로운 과제를 추론하고 수행하기 위해 사전 학습 (pre-training) 동안 획득한 지식을 활용하는 능력으로 정의되었다. 임의의 아키텍처 백본 (backbone) 및/또는 목적 함수를 갖는 어떤 언어 모델에도 단순히 적용가능한 이 광범위하고 일반적인 정의는, 이후에 다음 토큰 예측 (next token prediction) 목적 함수로 학습된 트랜스포머 아키텍처에 대해서만 맥락 내 학습을 기술하는 방식으로 정식화되었다. 따라서 트랜스포머 기반 모델이 맥락 내에서 학습할 수 있는 알고리즘/문제에 관한 광범위한 연구 (Akyurek 외 2022, 2024b; Zhang 외 2024a; Dherin 외 2025) 에도 불구하고, 그 일반적 형태로서의 맥락 내 학습은 상대적으로 덜 탐구되어 있다. 본 논문 전반에 걸쳐 우리는 “맥락 내 학습” 의 가장 일반적인 정의를 사용하며, 그것을 모델이 주어진 맥락에 스스로 적응하고 그로부터 학습하는 능력으로 지칭한다. 우리의 NL 정식화는 ICL 을 연상 기억 (associative memory) 개념과 연결하여, 아키텍처 백본 및/또는 목적 함수에 관계없이 모델의 ICL 능력에 대한 통합적 설명을 제공한다.

3. 중첩 학습 (Nested Learning)

이 절은 중첩 학습 (Nested Learning, NL) 의 동기, 형식적 정의, 그리고 일반적인 고수준 함의를 논의한다. 우리는 연상 기억의 정식화로 시작한다음, 단계별 예시들을 사용하여 아키텍처 분해 (architecture decomposition) 의 배후 직관과, 신경망을 최적화 문제들의 통합된 시스템으로 모델링하는 것과의 연결을 구축한다. 우리는 먼저 딥러닝의 기존 방법과 개념들이 어떻게 NL 패러다임에 속하는지를 보인 다음, 전통적 방법을 넘어서거나 기존 알고리즘 및 설계를 개선하는 방법에 대한 통찰을 제공하는 새로운 정식화들을 제시하는 것을 목표로 한다.

3.1 연상 기억 (Associative Memory)

연상 기억—사건들 사이의 연결을 형성하고 검색하는 능력—은 근본적인 정신과정이며 인간 학습의 분리 불가능한 구성 요소다 (Terry 2017). 문헌에서 흔히 기억 (memorization) 과 학습 (learning) 개념은 상호 교환적으로 사용되지만, 신경심리학 문헌에서는 이 둘이 명확히 구별된다. 더 구체적으로, 신경심리학 문헌 (Okano 외 2000) 을 따라, 우리는 다음의 기억과 학습의 정의에 기반하여 용어를 구축한다:

학습 대 기억 (Learning vs. Memorization): 기억 (Memory) 은 입력에 의해 야기된 신경적 갱신 (neural update) 이며, 학습 (learning) 은 효과적이고 유용한 기억을 획득하는 과정이다.

이 연구에서 우리의 목표는 먼저, 최적화기와 신경망을 포함한 계산적 시퀀스모델의 모든 요소들이 자기 자신의 맥락 흐름 (context flow) 을 압축하는 연상기억 시스템을 보이는 것이다. 대략적으로 말하면, 연상 기억은 키 (key) 들의 집합을 값 (value) 들의 집합으로 사상하는 연산자다. 우리는 Behrouz 외 (2025b) 의 연상 기억에 대한 일반적 정의를 따른다:

정의 1(연상 기억). 키패들의 집합 $\mathcal{K} \subseteq \mathbb{R}^{d_k}$ 와 값들의 집합 $\mathcal{V} \subseteq \mathbb{R}^{d_v}$ 가 주어졌을 때, 연상 기억은 키패들의 집합 $\mathcal{K}$ 를 값들 $\mathcal{V}$ 로 사상하는 연산자 $\mathcal{M}(\cdot)$ 이다. 데이터로부터 그러한 사상을 학습하기 위해, 목적 함수 $\tilde{\mathcal{L}}(\cdot; \cdot)$ 가 사상의 품질을 측정하며 $\mathcal{M}$ 은 다음에 의해 계산될 수 있다:

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \tilde{\mathcal{L}}(\mathcal{M}(\mathcal{K}); \mathcal{V}).$$

연산자 자체는 기억이고 사상은 기억화 과정 (memorization process)(즉, 맥락속 사건들의 연결을 기억하기)의 역할을 하는 반면, 데이터에 기반하여 그러한 효과적인 연산자를 획득하는 것은 학습 과정이다. 여기서 키패와 값은 기억이 사상하고자 하는 임의의 사건일 수 있으며 토큰에 국한되지 않음에 유의하라. 나중에 우리는 맥락 흐름이 주어졌을 때 키패와 값이 토큰, 그래디언트, 하위 시퀀스 등이 될 수 있음을 논의할 것이다. 더 나아가, 연상기억이라는 용어는 신경과학과 신경심리학 문헌에서 더 흔하지만, 위의를정식화는 데이터 압축 (data compression) 및 저차원 표현 (low-dimensional representation) 과도 밀접하게 관련되어 있다. 즉, 식 6의 최적화 과정을, 사상들을 자신의 파라미터로 압축하여 더 낮은 차원의 공간에 그것들을 표현하는 것을 목표로 하는 망 $\mathcal{M}(\cdot)$ 의 학습 과정으로 해석할 수 있다.

시퀀스 모델링에서, 키패와 값이 입력 토큰 (예: 토큰화된 텍스트) 일 때, 식 6을 푸는 목적 함수와 최적화 과정의 선택은 전역/국소 소프트맥스어텐션 (global/local softmax attention)(Vaswani 외 2017) 이나 다른 현대 순환 모델들 (Katharopoulos 외 2020; Sun 외 2023; Behrouz 외 2025c) 과 같은 서로 구별되는 시퀀스 모델링 아키텍처를 낳을 수 있다 (Liu 외 2024b 및 Behrouz 외 2025b 참조). 이러한 시퀀스 모델의 간단한 정식화는 그들의 내부 과정에 대한 더 나은 이해와, 그들의 목적 함수 및 최적화 과정에 기반하여 그들의 모델링 능력 (modeling power) 을 단순히 비교하는 도구를 우리에게 제공한다. 다음에서, 단계별 예시들을 사용하여, 우리는 이 정식화가 신경 아키텍처의 모든 구성 요소 (사전 학습에서의 그 최적화 과정을 포함하여) 에 어떻게 적용될 수 있는지, 그리고 실제로 어떻게 모델이 각기 자신의 맥락 흐름을 갖는 다수준 (multi-level), 중첩된 (nested), 잠재적으로 병렬적인 (parallel) 기억들의 통합된 시스템인지를 논의한다.

MLP 학습의 간단한 예시. 우리는 간단한 예시로 시작하는데, 여기서 우리는 과제 $\mathcal{T}$ 에 대해 그리고 데이터셋 $\mathcal{D}_{train} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{D}_{train}|}\}$ 위에서 목적 함수 $\mathcal{L}(\cdot; \cdot)$ 를 경사 하강법으로 최적화함으로써 (파라미터 W 로 파라미터화된) 1-계층 MLP 를 학습시키는 것을 목표로 한다. 이 경우, 학습 과정의 목적은 다음 최적화 문제를 푸는 것이다:

$$W^* = \arg \min_W \mathcal{L}(W; \mathcal{D}_{train}),$$

이를 (확률적/온라인) 경사 하강법으로 최적화하면 다음의 가중치 갱신 규칙을 얻는다:

$$W_{t+1} = W_t - \eta_{t+1} \nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1}) = W_t - \eta_{t+1} \underbrace{\underbrace{\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})}_{\text{출력에서의 서프라이즈 (Surprise in the Output)}}}_{\text{서프라이즈 (Surprise)}} \otimes \mathbf{x}_{t+1}, \quad \text{여기서 } \mathbf{x}_{t+1} \sim \mathcal{D}_{train},$$

여기서 $y_{t+1} = W \mathbf{x}_{t+1}$ 은 입력 $\mathbf{x}_{t+1}$ 에 대한 모델의 출력이며, 우리는 단순화 표기 $\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) := \frac{\partial \mathcal{L}}{\partial y} \big|_{y=W \mathbf{x}_{t+1}}$ 를 사용했다. 이 경우, $\nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1})$ 은 현재 입력이 이전에 관측된 데이터와 얼마나 다른지를 보여주는 서프라이즈 (surprise) 척도다. 마찬가지로, $\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$ 은 출력에 대한 서프라이즈 척도다 (또는 더 정확히는, 현재 출력과 목적 함수 $\mathcal{L}(\cdot; \cdot)$ 가 강제하는 구조 사이의 불일치를 정량화하는 표현 공간 (representation space)에서의 국소 서프라이즈 신호 (local surprise signal)) — 이는 이 입력에 대해 모델의 예측이 얼마나 놀라운지를 측정한다. 이 정식화가 주어졌을 때, 출력의 서프라이즈 값을 $u_{t+1} = \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$ 로 두고, 역전파 (backpropagation) 과정을, 입력 데이터 점들 $\mathcal{D}_{train} = \{\mathbf{x}_t\}_{t=1}^{|\mathcal{D}_{train}|}$ 을 그에 대응하는 $u_{t+1} = \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$ 로 사상하는 연상 기억을 찾는 최적화 문제의 해로서 재정식화할 수 있다. 즉, 우리는 $\mathcal{M}(\cdot) = W_t \cdot$ 이 기억을 파라미터화하도록 하고, W_t 가 $\mathbf{x}_{t+1}$ 과 $\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$ 사이에서 하는 사상의 품질을 측정하기 위해 내적 (dot-product) 유사도를 사용한다:

$$W_{t+1} = \arg \min_W \langle W \mathbf{x}_{t+1}, u_{t+1} \rangle + \frac{1}{2\eta_{t+1}} \|W - W_t\|_2^2 = \arg \min_W \langle W \mathbf{x}_t, \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \rangle + \frac{1}{2\eta_{t+1}} \|W - W_t\|_2^2.$$

따라서 이 정식화는 모델의 학습 단계를, 데이터 표본들을 표현 공간에서 그들의 국소 서프라이즈 신호 (Local Surprise Signal, LSS) 로 사상하는 효과적인 기억을 획득하는 과정으로 번역한다—이는 그에 대응하는 출력이 얼마나 놀라운지를 측정한다. 이 그래디언트는 예측에서의 오차 (error) 로 볼 수 있다 (손실이 최소화될 때 그래디언트는 0 이 됨). 나중에 4 절에서 우리는 역전파 과정을 연상 기억으로서 더 자세히 논의하지만, 이 간단한 예시로부터의 예비적 시사점으로:

서프라이즈 기반 기억으로서 역전파로 학습된 선형 계층: 역전파로 학습된 선형 계층은 예측된 출력이 얼마나 놀라운지를 기억함으로써 데이터로부터 학습한다. 즉, 역전파는 각 데이터 표본을 그에 대응하는 예측의 오차로 사상하는 연상 기억으로 볼 수 있다.

따라서, 이 예시에서 우리의 모델은 데이터 표본들에 걸친 단일 그래디언트 흐름 (gradient flow) 을 가지며, 이는 오직 데이터셋 $\mathcal{D}_{train} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{D}_{train}|}\}$ 에 대해서만 활성화되고 그 이후의 다른 어떤 데이터 표본 (즉, 추론 또는 테스트 시점) 에 대해서는 동결 (frozen) 될 것이다.

위의 예시에서, 우리는 경사 하강법 알고리즘을 그 모멘텀 기반 (momentum-based) 변형으로 대체할 수 있으며, 이는 다음의 갱신 규칙을 낳는다:

$$W_{t+1} = W_t - \mathbf{m}_{t+1},$$

$$\mathbf{m}_{t+1} = \mathbf{m}_t + \eta_{t+1} \nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1}) = \mathbf{m}_t + \eta_{t+1} \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \otimes \mathbf{x}_{t+1}.$$

식 11 에서, (시점 t 에서의) 식 10 의 이전 상태가 주어졌을 때, $\nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1})$ 의 값 또는 마찬가지로 $\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$ 의 값은 식 11 에서의 순환 (recurrence) 의 출력에 의존하지 않으며 따라서 미리 사전계산될 수 있다. $u_{t+1} = \nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1})$ 로 두면, 식 11 은 다음과 같이 재정식화될 수 있다:

$$\mathbf{m}_{t+1} = \arg \min_{\mathbf{m}} -\langle \mathbf{m}, \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \rangle + \frac{1}{2\eta_{t+1}} \|\mathbf{m} - \mathbf{m}_t\|_2^2 = \arg \min_{\mathbf{m}} -\langle \mathbf{m} \mathbf{x}_{t+1}, \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \rangle + \frac{1}{2\eta_{t+1}} \|\mathbf{m} - \mathbf{m}_t\|_2^2.$$

$$W_{t+1} = W_t - \mathbf{m}_{t+1},$$

이 정식화가 주어졌을 때, 모멘텀 항을 다음 둘 중 하나로 해석할 수 있다: (1) 그래디언트들을 자신의 파라미터로 압축하는 값 없는 (value-less) 연상 기억, 또는 (2) 데이터 점들을 그에 대응하는 LSS-값으로 사상하는 법을 학습하는 연상 기억. 흥미롭게도, 이 정식화는 모멘텀을 갖는 경사 하강법이 2-수준 최적화 절차로 볼 수 있음을 드러내는데, 여기서 기억은 단순한 경사 하강법 알고리즘에 의해 최적화된다.¹

위의 예시들을 결론지으며, 우리는 1-계층 MLP 의 학습 과정이 다음과 같음을 관찰했다: (1) 경사 하강법은 데이터 점들을 그에 대응하는 LSS-값으로 사상하는 법을 학습하는 1-수준 연상 기억이다; 그리고 (2) 모멘텀을 갖는 경사 하강법은 2-수준 연상 기억 (또는 최적화 과정) 으로서, 내부수준 (inner-level) 은 그래디언트 값들을 자신의 파라미터에 저장하는 법을 학습하고, 그 다음 외부 수준 (outer-level) 은 내부 수준 기억의 값으로 느린가중치 (slow weight) (즉, W_t) 를 갱신한다. 이들은 아키텍처와 최적화 알고리즘 양쪽 모두에 관해 가장 단순한 예시이지만, 더 복잡한 설정에서도 유사한 결론을 내릴 수 있는지 물을 수 있다.

아키텍처 분해의 예시. 다음 예시에서, 우리는 이전 예시의 MLP 모듈을 선형 어텐션 (linear attention) (Katharopoulos 외 2020) 으로 대체한다. 즉, 우리는 과제 $\mathcal{T}$ 에 대해 그리고 시퀀스 $\mathcal{D}_{train} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{D}_{train}|}\}$ 위에서 목적 함수 $\mathcal{L}$ 을 경사 하강법으로 최적화함으로써 1-계층 선형 어텐션을 학습시키는 것을 목표로 한다. 정규화되지 않은 (unnormalized)

선형 어텐션 (linear attention) 정식화:

$$\mathbf{k}_t = W_{\mathbf{k}} \mathbf{x}_t,$$

$$\mathbf{v}_t = W_{\mathbf{v}} \mathbf{x}_t,$$

$$\mathbf{q}_t = W_{\mathbf{q}} \mathbf{x}_t,$$

$$M_t = M_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top,$$

$$y_t = M_t \mathbf{q}_t.$$

앞선 연구들 (Liu et al. 2024b; Behrouz et al. 2025b) 에서 논의된 바와같이, 식 15 의 순환 (recurrence) 은 키와 값의 매핑을 자신의 파라미터로 압축하는 행렬값 연상 메모리 (matrix-valued associative memory) $M_t(\cdot)$ 의 최적화 과정으로 재정식화될 수 있다. 구체적으로, 정의 1 에서 $\tilde{\mathcal{L}}(M_{t-1}; \mathbf{k}_t, \mathbf{v}_t) := -\langle M_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle$ 로 두고 이 메모리를 경사하강법 (gradient descent) 으로 최적화하고자 하면, 메모리 업데이트 규칙은 다음과 같다 (단, $\nabla \tilde{\mathcal{L}}(M_{t-1}; \mathbf{k}_t, \mathbf{v}_t) = -\mathbf{v}_t \mathbf{k}_t^\top$ 이고 학습률 $\eta_t = 1$ 로 둔다):

$$M_{t+1} = \arg \min_M -\langle M \mathbf{k}_{t+1}, \mathbf{v}_{t+1} \rangle + \frac{1}{2} \|M - M_t\|_2^2$$

$$\Rightarrow M_{t+1} = M_t - \nabla \tilde{\mathcal{L}}(M_t; \mathbf{k}_{t+1}, \mathbf{v}_{t+1}) = M_t + \mathbf{v}_{t+1} \mathbf{k}_{t+1}^\top,$$

이는 식 15 의 정규화되지 않은 (unnormalized) 선형 어텐션의 업데이트 규칙과 동등하다. 또한, 첫 번째 예제에서 관찰했듯이, 선형 계층 (linear layer) 을 경사하강법으로 학습하는 것은 연상 메모리의 1-레벨 최적화 (식 8) 로 볼 수 있으며, 따라서 사영 계층 (projection layer) 들 (즉, $W_{\mathbf{k}}, W_{\mathbf{v}}, W_{\mathbf{q}}$) 의 일반적인 학습/업데이트 과정 그 자체가 연상 메모리의 최적화과정이다. 그러므로 선형 어텐션을 경사하강법으로 학습하는 것은 2-레벨 최적화 과정으로 볼 수 있는데, 여기서 바깥 루프 (outer-loop, 학습과정이라고도 함) 는 사영 계층들을 경사하강법으로 최적화하고, 안쪽 루프 (inner-loop) 는 M_t 의 내부 메모리를 경사하강법으로 최적화한다.

지금까지 논의한 예제들에서 우리는 두 개의 연상 메모리를 가지며, 각각은 자기 자신의 최적화 과정과 경사 흐름 (gradient flow) 을 갖는다. 즉, 바깥레벨 파라미터인 $W_{\mathbf{k}}, W_{\mathbf{v}}, W_{\mathbf{q}}$ 의 최적화에서는 파라미터 $M(\cdot)$ 에 대한 경사

¹ 우
리는
최적
화 절
차를
기술
하기
위해
2-레
벨 또
는 다
중-레
벨
(multi-
level)
이라
는 용
어를
사용
한다.
이는
최적
화 문
제들
이 계
층적
으로
배열
되는
고전
적인
다중-
레벨
최적
화
(multi-
level
opti-
miza-
tion)
와는
다
르다.

그림 3: 경사하강법으로 최적화하는, Transformer 기반 백본에서의 FFN(예: MLP) 과 선형 어텐션을 비교하는 예시. 빨간색 구성요소는 첫 번째 레벨 (빈도 1) 에 속하는 블록이고, 파란색 구성요소는 두 번째 레벨 (빈도 L) 에 속하는 블록이다. 학습 가능한 초기 메모리 상태를 갖는 선형어텐션 (Linear Attention++라고 부름) 은 MLP 계층과 동일하지만, in-context learning 능력과 입력 시퀀스에 대한 적응 (adaptation) 능력을 추가로 갖는다.

가 없으며 따라서 그것을 통한 역전파 (backpropagation) 도 없다. 마찬가지로, 안쪽 레벨에서는 사

영 계층들을 통한 역전파가 없으며 그것들은 동결된 (frozen) 것으로 간주된다. 나아가 주목할 점은, 이 예제에서 위의 정식화가 선형 어텐션에 대한 FWP(Fast Weight Programmers) 관점 (Schlag et al. 2021) 과도 밀접하게 연결된다는 것이다. 이 관점에서 사영들은 느린가중치 (slow weights) 로 간주되고, 식 15의 메모리 업데이트는 빠른가중치 (fast weight) 업데이트 규칙이 된다.

더 많은 레벨을 갖는 아키텍처 분해 (Architectural Decomposition with More Levels). 위의 두 예제 모두에서, 우리는 그것들이 (그들의 FWP 해석과 일치하는) 2-레벨 최적화 과정으로 볼 수 있음을 논의했다. 그러나 실제로는 더 강력한 최적화 과정, 그리고/또는 더 강력한 메모리 순환 업데이트 규칙을 사용해야 할 수도 있다. 간단한 예로, 선형 어텐션 모델을 학습하기 위해 모멘텀을 갖는 경사하강법 (gradient descent with momentum) 을 사용한다고 가정하자. 위에서 보았듯이, 선형 어텐션 구성요소는 두 개의 중첩된 (nested) 최적화 과정으로 분해될 수 있다. 마찬가지로, 여기서의 모델은 2-레벨 최적화 문제로 표현될 수 있는데, 여기서 (1) 안쪽 레벨은 경사하강법 (식 17) 을 사용하여 컨텍스트를 압축하도록 메모리를 최적화하고, (2) 바깥 레벨은 모멘텀을 갖는 경사하강법으로 사영 계층들을 최적화한다. 흥미롭게도, 우리는 “모멘텀을 갖는 경사하강법” 알고리즘 그 자체가 2-레벨 최적화 과정으로 볼 수 있음을 확인했는데, 여기서 모멘텀 항 자체가 과거의 경사들을 자신의 파라미터로 압축하는 연상 메모리이다.

3.2 중첩 최적화 과정 (Nested Optimization Processes)

이전 절에서 우리는 기계학습 모델을 어떻게 중첩된 (nested) 또는 다중-레벨 최적화 절차의 집합으로 분해할 수 있는지 보이는 예제들을 제시했다. 다음으로, 우리는 먼저 중첩 학습 (nested learning) 문제의 정식화를 제시하고, 그런 다음 데이터로부터 학습하는 통합된 계산 시스템인 신경 학습 모듈 (Neural Learning Module) 을 정의한다.

이전 절들에서 우리는 모델을 최적화 과정의 집합으로 분해했다. 그러나 이러한 과정들에 대한 계층 (hierarchy) 혹은 순서 (order) 를 정의하고, 모델들이 형식으로 유일하게 표현할 수 있는지는 여전히 불분명하다. 뇌파 (brain waves) 의 계층—각 부분의 정보 처리 빈도율 (frequency rate) 을 나타냄 (1 절에서 논의)—에서 영감을 받아, 우리는 각 최적화 과정의 업데이트 속도 (update rate) 를 사용하여 구성요소들을 여러 레벨로 정렬한다. 이를 위해, 하나의 데이터 포인트에 대한 하나의 업데이트 단계 (step) 를 시간의 단위로 두고, 각 구성요소의 업데이트 빈도율을 다음과 같이 정의한다:

정의 2 (업데이트 빈도, Update Frequency). A 의 임의의 구성요소—파라미터 구성요소 (예: 학습 가능한 가중치 또는 모멘텀을 갖는 경사하강법의 모멘텀 항) 이거나 비파라미터 구성요소 (예: 어텐션 블록) 일 수 있음—에 대해, 그것의 빈도 (frequency) 를 f_A 로 표기하고 시간 단위당 업데이트 횟수로 정의한다.

위의 업데이트 빈도가 주어지면, 우리는 연산자 $(\succ \cdot)$ 를 기반으로 기계학습 알고리즘의 구성요소들을 정렬할 수 있다. 우리는 A 가 B 보다 빠르다고 말하고 $A \succ B$ 로 표기하는데, 만약 (1) $f_A > f_B$ 이거나, 또는 (2) $f_A = f_B$ 이지만 시각 t 에서의 B 의 상태 계산이 시각 t 에서의 A 의 상태 계산을 필요로 하는 경우이다. 이 정의에서, $A \not\succ B$ 이고 $B \not\succ A$ 일 때 우리는 $A \stackrel{f}{=} B$ 로 두는데, 이는 A 와 B 가 동일한 빈도 업데이트를 갖지만 그들의 계산이 서로 독립적임을 나타낸다 (나중에 AdamW 옵티마이저에서 이러한 경우의 예를 제시한다). 위의 연산자를 기반으로, 우리는 구성요소들을 정렬된 “레벨 (level)” 들의 집합으로 정렬하는데, 여기서 (1) 동일한 레벨의 구성요소들은 동일한 빈도 업데이트를 가지며, (2) 레벨이 높을수록 그 빈도는 낮아진다. 위의 레벨과 업데이트 빈도의 정식화가 주어지면, 다음으로 우리는 중첩 학습을 정식으로 정의한다:

정의 3 (중첩 시스템, Nested System). (순서를 갖는) 중첩 시스템은 K 개의 (순서를 갖는) 레벨을 갖는 시스템으로, 각 레벨 k ($1 \leq k \leq K$) 는 최적화 문제들의 집합 $\{(\mathcal{L}_i^{(k)}, \mathcal{C}_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$ 으로 구성된다. 여기서 $\mathcal{L}_i(\cdot; \cdot)$ 는 i 번째 문제에서의 최적화 목적함수이고, $\mathcal{C}_i$ 는 그 컨텍스트 (최적화되는 데이터) 이며, Θ_i 는 그 파라미터의 가능 집합 (feasible set) 이고, 각 파라미터는 경사하강법을 사용하여 최적화된다:

$$\theta_{i,t+1}^{(k)} = \arg \min_{\Phi_i^{(k)}} \langle \Phi_i^{(k)} \mathbf{x}_{t+1}, -\nabla \mathcal{L}_i^{(k)}(\theta_{i,t}^{(k)}; \mathbf{x}_{t+1}) \rangle + \frac{1}{2\eta_{i,t+1}^{(k)}} \|\Phi_i^{(k)} - \theta_{i,t}^{(k)}\|_2^2$$

여기서 $\mathbf{x}_{t+1} \sim \mathcal{C}_i^{(k)}$ 이고, $\Phi_i^{(k)} \in \Theta_i^{(k)}$ 이다. (19)

각 최적화 과정은 자기 자신의 경사 흐름을 가지며, 따라서 우리는 때때로 그것들을 하나의 최적화 문제에 대응하는 경사 흐름의 상자 (box of gradient flow) 라고 부른다. 본 논문 전체에 걸쳐, 우리는 중첩 시스템에 대한 정의를 더 일반화하여, 일부 상자 (즉, 최적화 문제) 에 대해 비파라미터해 (non-parametric solution) 를 찾는 것도 허용한다.

위의 정의는 서로 다른 상자들 사이에 어떤 의존성 (즉, 한 상자가 다른상자의 컨텍스트나 파라미터 공간을 결정할 수 있음) 이 있는지를 명시하지않는, 중첩 시스템에 대한 일반적이고 유연한 정의를 제공한다. 다음절들에서, 우리는 서로 다른 레벨 또는 상자들 사이에서 지식/정보가 어떻게 전달될 수 있는지를 논의한다. 본 논문전체에 걸쳐, 우리는 각 최적화 과정이 연상 메모리인 중첩 시스템, 즉 연상메모리의 중첩 시스템 (Nested Systems of Associative Memories, NSAM) 에 초점을 맞춘다. 더 정식으로,

정의 4 (연상 메모리의 중첩 시스템, Nested System of Associative Memories). 연상 메모리의 중첩 시스템 (NSAM) 은 K 개의 (순서를 갖는) 레벨을 갖는 시스템으로, 각 레벨 k ($1 \leq k \leq K$) 는 최적화문제들의 집합 $\{(\mathcal{L}_i^{(k)}, \mathcal{C}_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$ 으로 구성된다. 여기서 $\mathcal{C}_i = \{(\mathbf{k}_j^{(i)}, \mathbf{v}_j^{(i)})\}_{j=1}^{L_i}$ 는 키-값 쌍 (key-value pair) 들의 집합이고, $\mathcal{L}_i(\cdot; \cdot, \cdot)$ 는 i 번째 문제에서 메모리가 학습한 매핑의 품질을 측정하며, Θ_i 는 가능한 메모리파라미터의 집합으로 각 파라미터는 경사하강법을 사용하여 최적화된다:

$$\theta_{i,t+1}^{(k)} = \arg \min_{\Phi_i^{(k)}} \langle \Phi_i^{(k)} \mathbf{k}_{t+1}^{(i)}, -\nabla \mathcal{L}_i^{(k)}(\theta_{i,t}^{(k)}; \mathbf{k}_{t+1}^{(i)}, \mathbf{v}_{t+1}^{(i)}) \rangle + \frac{1}{2\eta_{i,t+1}^{(k)}} \|\Phi_i^{(k)} - \theta_{i,t}^{(k)}\|_2^2,$$

여기서 $(\mathbf{k}_{t+1}^{(i)}, \mathbf{v}_{t+1}^{(i)}) \sim \mathcal{C}_i^{(k)}$ 이고 $\Phi_i^{(k)} \in \Theta_i^{(k)}$ 이다.

질의 (query) $\mathbf{q}$ 가 주어지면, 각 연상 메모리 $M_i^{(k)}$ 에 대해 우리는 $M_i^{(k)}(\mathbf{q})$ 를 사용하여 메모리의 순전파 과정 (즉, 검색/retrieval 과정) 을 지칭한다. NSAM 에 대한 우리의 정식화는 경사하강법을 넘어서는 임의의 최적화과정으로도 간단히 정의될 수 있지만, 그리고 처음에는 경사하강법에 의한 최적화라는 엄격한 조건이 이 정의의 모델링 능력을 제한하는 것처럼 보일 수 있지만, 본 논문 전체에 걸쳐 우리는 현대적 아키텍처들이 특정한 잘 알려진 최적화 알고리즘들과 함께 NSAM 의 사례 (instance) 로 볼 수 있음을 보인다. 그런 다음 우리는 이 직관 위에 구축하여, 여러 레벨을 쌓는 방법과 향상된 지속 학습 (continual learning) 능력을 갖는 모델을 설계하는 방법을 어떻게 더 발전시킬 수 있는지 논의한다.

여러 레벨을 쌓는 (stacking multiple levels) 새로운 차원은 중첩 학습의 중요한 특성으로, 레벨의 수를 늘림으로써 계산의 깊이 (depth of computation) 를 향상시킬 수 있다. 설계, 컨텍스트, 그리고 지식 전달의 유형에 따라, 이 계산의 깊이는 그 자체로 다음과 같은 서로 다른 개념들로 볼 수 있다: 고차 (higher-order) in-context learning 능력, 잠재계산 (latent computation, 예: Loop Transformers), 다중 메모리 시스템, 그리고 더 표현력 있는 옵티마이저 (optimizer). 나중에 우리는 이러한 모든함의 (implication) 들을 논의하지만, 다음으로 우리는 Transformer 아키텍처의 MLP 계층을 선형 또는 심층 메모리 (deep memory) 블록과 연결하는 간단한예제를 사용한다.

MLP 계층 대 선형 어텐션의 예제. 두 모델을 비교해 보자: (1) Transformer 아키텍처, 그리고 (2) 동일한 백본이지만 MLP 블록을 선형어텐션 메커니즘으로 대체한 것 (이전 계층의 키와 값을 공유하며, 그 메모리의 초기 상태는 메타 학습 (meta-learned) 됨, Behrouz et al. (2025c) 나 Sun et al. (2024) 와 유사). 우리는 두 번째 변형을 적응형 Transformer (Adaptive Transformer) 또는 AdaTransformer 라고 부른다. 두 모델 모두 다음 토큰 예측 (Next Token Prediction, NTP) 목적 함수로 경사하강법을 통해 최적화된다. 초기 예제들에서 논의했고 또한 그림 3 에 예시된 바와 같이, 두 모델 모두 두 개의 레벨을 가지며, 명료함을 위해 우리는 빨간색 (각각 파란색) 을 사용하여 첫 번째 레벨 (각각 두 번째 레벨) 의 계산/가중치를 강조한다. 더 정식으로, $X = \{\mathbf{x}_i\}_{i=1}^T$ 를 입력 토큰 시퀀스 라고 하면, 두 블록의 출력은 다음과 같이 계산된다 (단순함을 위해 $\text{MLP}(\cdot) = \cdot W_{\text{MLP}}$ 로 가정하고 정

규화 (normalization) 는 제거한다):

$$\mathbf{k}_t = \mathbf{x}_t W_k, \quad \mathbf{v}_t = \mathbf{x}_t W_v, \quad \mathbf{q}_t = \mathbf{x}_t W_q,$$

$$\mathbf{y}_{\text{attn}} = \text{Attn}(\mathbf{k}, \mathbf{v}, \mathbf{q}),$$

$$\mathbf{y}_{\text{block}} = \text{MLP}(\mathbf{y}_{\text{attn}}) = \mathbf{y}_{\text{attn}} W_{\text{MLP}}, \quad (\text{Transformer 블록})$$

$$\mathbf{k}_t = \mathbf{x}_t W_k, \quad \mathbf{v}_t = \mathbf{x}_t W_v, \quad \mathbf{q}_t = \mathbf{x}_t W_q,$$

$$\mathbf{y}_{\text{attn}} = \text{Attn}(\mathbf{k}, \mathbf{v}, \mathbf{q}),$$

$$\mathbf{y}_{\text{block}} = \mathbf{y}_{\text{attn}} W_{\text{LinAttn}}. \quad (\text{AdaTransformer 블록})$$

두 블록의 정식화는 매우 유사해 보이며 유일한 차이는 W_{MLP} 와 W_{LinAttn} 가중치의 레벨에서 온다. 즉, W_{MLP} 는 첫 번째 레벨에 있어 컨텍스트에 대해 지속적 (persistent) 인 반면, W_{LinAttn} 은 적응적 (adaptive) 이며 다음에 의해 in-context 로 업데이트된다 ($M(\cdot)$ 은 W_{LinAttn} 으로 파라미터화됨):

$$M_t = M_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top.$$

초기 변형의 선형 어텐션에서는, $M(\cdot)$ 의 초기 상태, 즉 등가적으로 W_{LinAttn} 이 영행렬 (zero matrix) $M_0 = 0$ 으로 간주된다. 그러나 더 진보된 설계 선택들 (Sun et al. 2024; Behrouz et al. 2025c) 과 유사하게, 이 초기 상태는 컨텍스트에 빠르게 적응하도록 메타 학습될 수 있다. 이 설정에서, $M_0 = W_{\text{LinAttn}}^{\text{init}}$ 의 초기상태는 NTP 목적함수로 첫 번째 레벨에서 최적화되고, 컨텍스트가 주어지면 W_{LinAttn} 은 내적 목적함수 (dot-product objective) 를 갖는 연상 메모리로서 두 번째 레벨에서 최적화된다 (Behrouz et al. 2025b).

위의 예제는 Transformer 아키텍처에서 더 진보되고 깊은 MLP 블록 (예: SwiGLU (Shazeer 2020)) 을 사용하고 그것을 순환 메모리 대응물 (Behrouz et al. 2025a) 과 비교할 때에도 유효하다. 나아가, 이 간단한 예제는 하이브리드 아키텍처를 표현력 있는 softmax 어텐션과 효율적인 순환 모델의 결합으로 보는 현재의 관점이 다소 오해의 소지가 있으며, 그것은 관습적인 Transformer 백본 설계를 따르되 MLP 블록에 대한 추가적인 in-context learning 능력을 갖는 것임을 시사한다. 우리는 이를 6 절과 7 절에서 더 논의한다.

이 소절에서 논의한 중첩 시스템과 중첩 학습 개념에 대한 요약 (takeaway) 으로서:

중첩 학습에서 레벨 쌓기 (Stacking Levels in Nested Learning): 중첩 학습은 다수의 (다층) 레벨로 구성된 계산 모델이 서로 다른 추상화 수준 (level of abstraction) 과 업데이트 빈도로 데이터로부터 학습하고 데이터를 처리할 수 있게 한다.

앞서 논의했듯이, 문헌에서는 아키텍처를 그것의 최적화 과정과 분리하여 이들을 독립적인 설계 선택으로 취급하고, 각 측면에서 최대의 표현력을 달성하는 알고리즘들을 결합하는 것을 목표로 삼는 것이 일반적이다. 그러나 실제로는, 확률적 경사하강법 (stochastic gradient descent) 으로 최적화된 Transformer 아키텍처 (Vaswani et al. 2017) 는 Adam 옵티마이저 (Kingma et al. 2014a) 를 사용한 동일한 아키텍처와 매우 다른 해 (solution) 를 학습할 수 있다. 따라서 이러한 기계학습 알고리즘과 상호작용할 때, 우리는 아키텍처축 (axes) 에서의 유사성에도 불구하고 전체적으로 학습된 모델이 서로 다른 예측을 보이거나 서로 다른 출력을 생성함을 관찰한다. 그러나 NL(Nested Learning) 의 관점에서, 기계학습 알고리즘은 최적화 문제들과 모델의 행동 (action) 들이 서로 연결된 시스템 (interconnected system) 으로 표현되며, 모델의 행동, 예측, 그리고 출력 생성은 각각의 하위 구성 요소가 아니라 이 시스템 전체에 의존한다. 이를 위해, 우리는 아키텍처와 최적화 과정이 공동으로 모델

과 그 출력을 결정하는 이러한 모델 표현을 지칭하기 위해 신경학습 모듈 (neural learning module) 이라는 용어를 정의한다. 이러한 공동표현이, 학습 (training) 단계와 시험 (test) 단계가 있는 현재의 기계학습파이프라인에서는 그다지 중요해 보이지 않을 수 있지만, 학습/시험 단계가 없는—우리가 옹호하는—지속 (continual) 설정에서는 더 중요해진다 (8 절에서 더 많은 논의 참조).

신경 학습 모듈은 상호 연결된 시스템이다 (Neural Learning Modules are Inter-connected Systems). 신경 학습 모듈의 정의에 기반하여, 하나의 중요한 질문은 아키텍처와 최적화 과정이 어떻게 상호 연결된 시스템이며 그들이 어떻게 서로에게 영향을 미칠 수 있는가이다. 신경망 학습을 위한 일반적인 정식화를 상기하자: 과제 $\mathcal{T}$, 그에 대응하는 데이터분포 $p(\mathcal{T})$, $\Phi_{\mathcal{T}}$ 로 파라미터화된 모델 $f(\cdot; \cdot)$, 그리고 목적함수 $\mathcal{L}(\cdot; \cdot)$ 가 주어졌을 때, 우리는 다음을 만족하는 파라미터 $\Phi_{\mathcal{T}}^*$ 를 학습하는 것을 목표로 한다:

$$\Phi_{\mathcal{T}}^* = \arg \min_{\Phi} \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p(\mathcal{T})} [\mathcal{L}(\Phi; \mathbf{x}, \mathbf{y})].$$

실제로, 우리는 주어진 데이터셋 $\mathcal{D}_{\text{train}}$ 과 확률적경사하강법 같은 최적화 알고리즘에 기반하여 위 문제를 최적화한다:

$$\Phi_{t+1} = \Phi_t - \eta_{t+1} \nabla_{\Phi_t} \mathcal{L}(\Phi_t; \mathbf{x}_{t+1}, \mathbf{y}_{t+1}),$$

여기서 $(\mathbf{x}_{t+1}, \mathbf{y}_{t+1}) \sim \mathcal{D}_{\text{train}}$ 이다. (23)

식 23 에서 모델 $f(\cdot; \cdot)$ 의 최적화 과정에 대한 하나의 해석은, 모델을 식 23 의 최적화 과정을 위한 데이터 생성기 (data generator) 로 보는 것이다. 즉, 3.1 절의 첫 번째 예제에서 논의했고 나중에 4 절에서 보일 것처럼, 최적화 과정은 학습 데이터와 그것의 경사 (또는 놀라움/surprise) 사이의 패턴을 압축하는 것을 목표로 하는 연상 메모리이며, 따라서 그러한 메모리를 내부적으로 학습하기 위한 데이터셋 (즉, 모델의 경사들) 은 모델에 의해 생성된다. 그러므로 모델의 유형 (type) 은 시간에 따라 서로 다른 패턴과 분포를 갖는 데이터셋 (즉, 경사) 을 생성하는 결과를 낳을 수 있다. 최적화 과정과 이러한 데이터 생성의 효과는 또한 모델 자체로 되먹임 (feedback) 되는데, 여기서 모델 파라미터의 다음 상태는 최적화 알고리즘에 의해 결정된다. 4 절에서 논의하겠지만, 옵티마이저를 모델의 경사들에 대한 연상 메모리로 바라보는 것은 각 옵티마이저가 더 나은 메모리 관리, 더 높은 압축 등과 같은 특별한 특성을 가짐을 함의한다. 따라서 그러한 알고리즘의 선택은 생성된 경사들과 파라미터 공간에서의 모델의 변화를 이해하는 것을 요구한다.

3.3 레벨 간 지식 전달 (Knowledge Transfer Between Levels)

지금까지 우리는 주로 중첩 학습의 개념과 최적화 문제들이 서로 다른 레벨에 위치하는 방식에 초점을 맞추었다. 그러나 중첩된 최적화 문제들이 (서로 다른 레벨에서) 어떻게 서로에게 영향을 미칠 수 있는지, 또는 일반적으로

그것들이 어떻게 시스템의 출력에 기여할 수 있는지를 나타내며, 따라서 상호 연결된다. 이 절에서는 서로 다른 수준 (level) 에 있는 구성 요소들 간의 몇 가지 잠재적인 지식 전달 (knowledge transfer) 방법을 논의한다. 명료함을 위해, 우리는 두 수준과 블록, 즉 대응하는 메모리 $M^{(0)}(\cdot)$ 및 $M^{(1)}(\cdot)$ 를 각각 갖는 $B^{(0)} = (L^{(0)}, C^{(0)}, \Theta^{(0)})$ 와 $B^{(1)} = (L^{(1)}, C^{(1)}, \Theta^{(1)})$ 사이의 지식 전달을 논의한다.

수준의 직접 연결 (파라미터 방식, Parametric). 첫 번째 유형의 지식 전달은 서로 다른 수준 또는 블록의 가중치를 직접 통합하는 것이다. 이를 위해, 더 낮은 빈도 (즉, 더 높은 수준) 의 메모리 시스템으로부터의 순전파 (forward pass) 또는 검색 (retrieval) 과정이 더 높은 빈도 (즉, 더 낮은 수준) 의 메모리의 파라미터에도 조건화된다:

$$M^{(0)}(\cdot) := M^{(0)}(\cdot; \Theta^{(1)}).$$

더 구체적인 정식화에서, 그리고 위 정식화의 특수한 변형으로서, 우리는 $M^{(0)}(\cdot)$ 의 출력을 더 높은 빈도 메모리의 출력 (또는 순전파) 에 기반하여 조건화할 수 있다:

$$M^{(0)}(\cdot) := M^{(0)}(\cdot; M^{(1)}(\cdot)),$$

여기서 우리는 두 번째 인수에 대한 의존성을 숨김으로써 표기를 약간 남용하였다. 이러한 유형의 지식 전달의 한 예로, 초기 메모리 상태가 0 인선형 트랜스포머 (또는 FWP) (Katharopoulos et al. 2020; Schlag et al. 2021) 에서, 더 낮은 수준 (빠른 가중치, fast weight) 의 저장된 지식은 다른 수준에서 모델의 출력에 직접 영향을 준다. 즉, 순전파 (메모리 검색) 를 다음과 같이 다시 쓸 수 있다:

$$y_t = M_t q_t = M_t \underbrace{x_t W_q}_{\text{더 높은 빈도 메모리의 순전파}}$$

(여기서 M_t 는 더 낮은 빈도 메모리의 순전파에 해당한다.)

수준의 직접 연결 (비파라미터 방식, Non-Parametric). 수준 간 직접 연결의 또 다른 형태는 위 정식화의 비파라미터적 변형으로, 블록 $B^{(1)}$ 이 비파라미터적 해 (non-parametric solution) 를 찾음으로써 최적화되는 경우이다. 따라서 저빈도 메모리의 순전파는 고빈도 메모리의 문맥 (context) 과 출력에 조건화된다:

$$M^{(0)}(\cdot) := M^{(0)}(\cdot; C^{(1)}), \quad \text{또는 유사하게,} \quad M^{(0)}(\cdot) := M^{(0)}(\cdot; M^{(1)}(\cdot; C^{(1)})).$$

이 변형의 한 예로, 트랜스포머와 소프트맥스 어텐션 모듈 (Vaswani et al. 2017) 을 들 수 있다. 위의 두 변형 모두에 대해 중요한 특성이 하나 있다: 두 개의 서로 다른 수준에 있는 블록의 어떤 상태를 통해서도 역전파 (backpropagation) 가 일어나지 않으며, 지식은 한 수준의 출력을 다른 수준의 출력/파라미터에 직접 조건화하는 것을 통해 전달된다. 따라서 이 과정에서 각 블록의 상태는 다른 블록에 대한 하이퍼파라미터 (hyperparameter) 로 취급된다.

역전파를 통한 지식 전달. 지식 전달의 또 다른 형태는 역전파를 통한 것으로, 서로 다른 수준에 있는 블록들 사이에 그래디언트 흐름 (gradient flow) 이 존재한다. 이 설계의 순전파는 위에서 논의한 순전파와 동일하다. 그러나 역전파 (backward pass) 가 주된 차이인데, 위의 두 경우에는 각 연상 메모리 (associative memory) 의 상태가 다른 것의 하이퍼파라미터로 간주된 반면, 여기서는 두 상태가 동일한 그래디언트 흐름 안에서 최적화된다. 따라서, 두 수준에 있는 두 블록의 단순한 경우에 대해 우리는 다음을 갖는다:

$$M^{(0)}(\cdot) := M^{(0)}(\cdot; M^{(1)}(\cdot)) \quad (\text{순전파, Forward Pass})$$

$$\Theta_{t+1}^{(1)} = \Theta_t^{(1)} - \eta_{t+1} \delta_t \hat{x}_{t+1}^\top,$$

$$\Theta_{t+1}^{(0)} = \Theta_t^{(0)} - \eta_{t+1} \delta_0 x_{t+1}^\top, \quad (\text{역전파, Backward Pass})$$

여기서 $\delta_{t+1}^{(0)} = J_{\phi^{(0)}}(x_{t+1}) \Theta_t^{(1)\top} \delta_{t+1}^{(1)}$, $\hat{x}_{t+1} = \phi^{(0)}(M^{(0)}(x_t))$ 이고, $\phi^{(0)}(\cdot)$ 는 비선형성 (non-linearity), $J_{\phi^{(0)}}(\cdot)$ 는 야코비안 (Jacobian) 이다. 이 설계에서 두 블록은 동일한 그래디언트 흐름 안에 있지만, 서로 다른 빈도에 기반하여 갱신된다. 이 설계의 한 예는 연속체 메모리 시스템 (continuum memory systems) 을 논의하는 7 절에서 제공한다.

초기화를 통한 지식 전달. 모델 애그노스틱 메타 러닝 (Model Agnostic Meta-Learning, MAML) (Finn et al. 2017) 은 메타 러닝 (또는 학습하는 법을 학습하기, learning to learn) 의 가장 널리 쓰이는 형태 중 하나로, 모델이 새로운 과제를 빠르게 학습할 수 있도록 전역 초기점 (global initial point) 을 학습하는 것을 목표로 한다. 중첩 학습 (nested learning) 관점에서, 두 개의 중첩된 최적화 과정이 존재하는데, 여기서 내부분제 (inner problem) 는 자신의 문맥을 순회하며 자신의 내부 목적에 기반하여 진행되고, 상위 수준 문제는 자신의 학습된 가중치를 내부 문제의 초기점으로 삼아 측정한다. 더 형식적으로, 우리는 다음을 갖는다:

$$\Theta_0^{(1)} = \arg \min_{\Phi} \mathbb{E}_{C \sim C^{(0)}} \ell \left(M^{(1)}(\cdot; \Phi), C \right),$$

여기서 상위 수준 블록은 하위 수준 문제가 가질 수 있는 모든 가능한 문맥에 걸쳐 최선의 초기값을 학습한다. 앞서 논의했듯이, 임의의 MAML 기반 학습모델은 이 경우의 한 사례이며, 더 구체적인 예로는 3.2 절과 그림 3 에서 논의된 (MLP 층 대 선형 어텐션, MLP Layer vs. Linear Attention) 의 예를 참조한다.

생성 (Generation) 과의 연결. 지식 전달의 가장 흔한 형태 중 하나는 가중치나 문맥을 생성하는 것을 통한 것이다. 즉, 하나의 저빈도 (각각 고빈도) 블록이 고빈도 (각각 저빈도) 블록의 가중치를 생성한다. 더 형식적으로,

$$\Theta^{(1)} = g \left(M^{(0)}; L^{(0)}, C^{(0)}, \Theta^{(0)} \right), \quad \text{또는} \quad \Theta^{(0)} = g \left(M^{(1)}; L^{(1)}, C^{(1)}, \Theta^{(1)} \right), \quad (\text{가중치 생성, Weight Generation})$$

$$C^{(1)} = g \left(M^{(0)}; L^{(0)}, C^{(0)}, \Theta^{(0)} \right), \quad \text{또는} \quad C^{(0)} = g \left(M^{(1)}; L^{(1)}, C^{(1)}, \Theta^{(1)} \right). \quad (\text{문맥 생성, Context Generation})$$

위 형태의 지식 전달에는 두 가지 중요한 예가 있다: (1) 하이퍼네트워크 (Hypernetworks): 대상 신경망의 가중치가 또 다른 (생성기, generator) 네트워크에 의해 생성되는 경우. (2) 최적화 과정 (Optimization process): 아키텍처가 옵티마이저의 입력을 생성하는 경우. 즉, 옵티마이저의 문맥 (또는 입력 데이터) 이 아키텍처에 의해 생성된 그래디언트인 경우이다. 이 주제에 대한 더 많은 논의는 4 절을 참조하라. 이 예는 반드시 “학습된 옵티마이저 (learned optimizers)” 에 관한 것이 아니며, 경사 하강법 (gradient descent), Adam (Kingma et al. 2014a), AdaGrad (Duchi et al. 2011) 등과 같이 일반적으로 사용되는 최적화 과정 및 알고리즘에 대해서도 유효하다는 점에 유의하라.

신경 학습 모듈 설계에 관한 참고. 위에서 우리는 가능한 지식 전달 방법의 일부 예와, 서로 다른 수준의 잠재적 연결만을 논의하였다. 그러나 NL(중첩 학습) 과 신경 학습 모듈 (neural learning module) 의 정식화는 일반적이며, 따라서 위의 특정한 방법 집합에 국한되지 않는다. 이에 따라, 중첩 학습 관점에서 신경 학습 모듈을 설계하려면 두 가지 중요한 단계와 설계 선택이 있다:

신경 학습 모듈 설계: 신경 학습 모듈을 개발하는 데에는 두 가지 상위 수준의 설계 선택이 있다: (1) 최적화 문제와 그 빈도의 설계 (즉, NSAM 내 구성 요소의 설계); (2) 수준 간 지식 전달의 설계.

지식 전달의 서로 다른 선택에 따라, 일부 학습 패러다임이 신경 학습 모듈의 일부로 간주될 수 있다는 점은 주목할 만하다: 예컨대, (1) 메타 러닝 (Meta learning), 두 수준에 있는 두 블록이 지식을 전달하며 한 수준이 다른 수준을 메타 학습하는 경우; 더 구체적으로, (2) 모델 애그노스틱 메타러닝 (MAML) (Finn et al. 2017), 지식 전달이 초기화 학습을 통해 이루어지는 경우; (3) 하이퍼네트워크 (Hypernetworks), 하나의 고빈도 블록이 다른 저빈도 블록의 가중치를 생성하는 경우; (4) 학습된 옵티마이저 (Learned optimizers), 지식 전달이 데이터 생성을 통해 이루어지는 경우 (즉, 하나의 고빈도 블록이 다른 저빈도 블록을 위한 그래디언트를 생성하는 경우).

4. 학습 모듈로서의 옵티마이저 (Optimizers as Learning Modules)

이 절에서는 역전파 과정과 신경망 최적화를 연상 메모리 및 데이터 압축 관점에서 바라보는 것으로 시작한다. 그런 다음, 모멘텀 기반 옵티마이저 (momentum-based optimizers) 와 같은 변형들이 어떻게 중첩 연상 메모리 시스템 (nested associative memory systems) 의 사례인지를 논의한다. 마지막으로, 연상 메모리 관점에서 더 높은 표현력을 갖는 심층 옵티마이저 (deep optimizers) 로 이어지는 대안적 방법들을 논의한다.

4.1 연상 메모리로서의 역전파 (Backpropagation as an Associative Memory)

역전파 (Linnainmaa 1970; Rumelhart et al. 1986) 를 통해 신경망의 가중치를 갱신하는 것은 대규모 심층 신경망 훈련의 핵심 구성 요소였다. 직관적으로, 이 최적화 과정에서는 먼저 목표 (target)

에 대한 모델 출력의 오차가 계산되고, 그런 다음 각 층은 이 오차에 대한 자신의 기여도에 기반하여 갱신된다. 이 절은 이 과정을 연상 메모리의 관점에서 설명하고, 그것이 중첩학습 패러다임 안에 어떻게 부합하는지를 논의하는 것을 목표로 한다. 명료함과 단순함을 위해, 우리는 심층 MLP 모델을 가정하지만, 이하에서 유도되는 모든 정식화는 다른 아키텍처에도 간단히 적용될 수 있다. L 개의 층을 갖고 $\{W_\ell \cdot + \mathbf{b}_\ell\}_{\ell=1}^L$ 로 파라미터화된 MLP가 주어졌을 때, 역전파에서 필요한 그라디언트는 다음과 같이 계산된다:

$$\frac{\partial \mathcal{L}}{\partial W_\ell} = \delta_\ell \hat{\mathbf{x}}_{\ell-1}^\top, \quad \text{그리고} \quad \delta_\ell = \underbrace{\mathbf{J}_{\phi_\ell}(\mathbf{z}_\ell)^\top W_{\ell+1}^\top}_{\text{층 } \ell \text{의 국소 출력 서프라이즈 (local output surprise)}}, \quad \delta_{\ell+1},$$

여기서 $\mathbf{z}_\ell = W_\ell \hat{\mathbf{x}}_{\ell-1} + \mathbf{b}_\ell$ 은 사전 활성화 (pre-activation) 이고, 따라서 $\hat{\mathbf{x}}_\ell = \phi_\ell(\mathbf{z}_\ell)$ 은 ℓ -번째 층의 출력이며, $\phi_\ell(\cdot)$ 은 그 비선형성, $\mathbf{J}_{\phi_\ell}(\cdot)$ 은 야코비안이다. 따라서 경사하강법에 의한 ℓ -번째 층의 갱신은 다음과 같이 계산된다:

$$W_{\ell,t+1} = W_{\ell,t} - \eta_{\ell,t+1} \delta_\ell \hat{\mathbf{x}}_{\ell-1}^\top.$$

여기서 $\hat{\mathbf{x}}_{\ell-1}$ 은 층의 입력이고, δ_ℓ 은 층 ℓ 의 국소 오차 신호 (local error signal) 를 측정하며, 이는 동등하게 층 ℓ 의 출력이 그 입력에 대해 갖는 서프라이즈 (surprise) 를 측정하는 지표이다. 3.1 절의 예와 유사하게, 우리는 식 30 을 다음과 같이 쓸 수 있다:

$$W_{\ell,t+1} = \arg \min_W \langle W \hat{\mathbf{x}}_{\ell-1}, \delta_\ell \rangle + \frac{1}{2\eta_{\ell,t+1}} \|W - W_{\ell,t}\|_F^2,$$

이는 각 층의 입력 $\hat{\mathbf{x}}_{\ell-1}$ 을 그것의 국소 오차신호 δ_ℓ 에 매핑하는 것을 목표로 하는 연상메모리 모듈이다 (정의 1 참조). 즉, 위 정식화는 경사 하강법과 역전파로 신경망을 훈련하는 과정이 압축 과정 (compression process) 으로 볼 수 있음을 함의하는데, 여기서 각 층은 자신의 입력과 그에 대응하는 국소 오차 신호사이의 매핑을 저장한다. 이후 § 4.5 에서, 우리는 이 관점이 역전파를 위한 더 표현력 있는 학습 규칙 (learning rules) 을 설계하는 데 어떻게 도움이 되는지를 논의한다.

서프라이즈 기반 메모리로서의 역전파를 통한 심층 신경망 훈련: 역전파로 훈련된 신경망은 예측된 출력이 얼마나 놀라운지 (surprising) 를 기억함으로써 데이터로부터 학습한다; 즉, 역전파는 각 데이터 점을 그에대응하는 예측의 오차로 매핑하는 연상 메모리이다.

역전파 $\neq$ 선형 어텐션. 식 30 에 대한 흔한 오해는 δ_ℓ 이 미리 계산된 항 (pre-computed term) 이라고 가정하여, 역전파 (적어도 선형 층에서) 가 헤비안 규칙 (Hebbian-rule) 을 복원하며, 그 결과 최적화 과정과 그라디언트에 대한 선형 어텐션 (linear attention) 수행이 동등해진다고 보는 것이다. 그러나 우리의 정식화는 역전파의 갱신 규칙이 자기 참조적 과정 (self-referential process) (Schmidhuber 1993) 임을 보여주는데, 여기서 연상 메모리의 값 (value) 이 그 자신에 의해 생성되며, 이는 그라디언트에 대한 단순한 선형어텐션보다 더 복잡한 연상 메모리로 만든다 (4.5 절 참조).

4.2 연상 메모리로서의 모멘텀 기반 옵티마이저 (Momentum-based Optimizers as Associative Memories)

모멘텀 기반 옵티마이저는 현대 머신러닝 모델 훈련의 주요 구성요소이다 (Duchi et al. 2011; Kingma et al. 2014a; Jordan et al. 2024). 모멘텀 기반 옵티마이저를 연상 메모리로 설명하기 위해, 단순한 경사 하강법 알고리즘부터 시작하자:

$$W_{t+1} = W_t - \eta_t \nabla_{W_t} \mathcal{L}(W_t; \mathbf{x}_{t+1}),$$

이는 순간 그라디언트 (서프라이즈) 에 기반하여 현재 가중치 상태를 갱신한다. 이 갱신 규칙은 이전 토큰들과 지금까지 지나온 손실 지형 (loss landscape) 을 통합하지 않으며, 그 결과 많은 시나리오에

서 더 느린 (또는 덜 강건한) 수렴을 초래한다. 이를 해결하기 위해, 모멘텀 기반 경사 하강법은 과거 그래디언트의 지수 이동 평균 (Exponential Moving Averages, EMAs) 을 통합한다:

$$W_{\ell,t+1} = W_{\ell,t} + \mathbf{m}_{\ell,t+1}$$

$$\mathbf{m}_{\ell,t+1} = \alpha_{\ell,t+1} \mathbf{m}_{\ell,t} - \eta_{\ell,t+1} \nabla_{W_{\ell,t}} \mathcal{L}(W_{\ell,t}; \mathbf{x}_{t+1}) = \alpha_{\ell,t+1} \mathbf{m}_{\ell,t} - \eta_{\ell,t+1} \boldsymbol{\delta}_{\ell} \hat{\mathbf{x}}_{\ell-1}^{\top},$$

(33)

여기서 행렬 (또는 벡터) $\mathbf{m}_t$ 는 상태 t 에서의 모멘텀 (momentum) 이며, α_t 와 η_t 는 각각 (적응적) 학습률과 모멘텀률이다. 그리고 $\boldsymbol{\delta}^{\ell}$ 와 $\hat{\mathbf{x}}^{\ell-1}$ 은 식 29 에서와 동일하게 정의된다. 식 31 및 3.1 절의 예시 중 하나와 유사하게, $\alpha_{t+1} = 1$ 이라고 가정하면, 모멘텀 항은 다음 목적함수를 경사하강법 (gradient descent) 으로 최적화한 결과로 볼 수 있다:

$$\min_{\mathbf{m}} \langle \mathbf{m} \hat{\mathbf{x}}^{\ell-1}, \boldsymbol{\delta}^{\ell} \rangle.$$

(34)

$\alpha_{t+1} \neq 1$ 인 경우는 위 최소화 문제에 대한 GD 에 모멘텀 항에 대한 ℓ_2 -정규화 (regularization) 를 더한 것과 동등하다. 따라서 모멘텀은 실제로 목적함수의 과거 경사 (gradient) 들을 어떻게 자신의 파라미터로 압축할지를 학습하는 연상 기억 (associative memory) 모듈로 볼 수 있다. 식 31 은 단순한 1-수준 (1-level) 연상 기억이며 갱신이 메모리에 직접 적용되었던 것과 대조적으로, 여기서는 모멘텀의 상태가 가중치에 대한 갱신을 결정한다. 다시 말해, 이것은 2-수준 (2-level) 최적화 절차로서, 내부 루프 (inner-loop) 는 모멘텀을 학습하고 외부 루프 (outer-loop) 는 모멘텀의 상태를 사용하여 가중치를 갱신한다.

이 관점에서, 우리는 모멘텀의 정의를 EMA (지수이동평균) 로부터 과거 경사들을 압축하거나 각 토큰의 입력을 그에 대응하는 국소 오차 (local error) 로 매핑하는 것을 목표로 하는 임의의 연상 기억 모듈로 일반화할 수 있다. 이 일반화된 모멘텀은 다음과 같이 표현될 수 있다:

$$W_{t+1}^{\ell} = W_t^{\ell} + \mathbf{m}_{t+1}^{\ell},$$

(35)

(36)

여기서 $\mathbf{m}^{\ell}$ 는 경사하강법으로 최적화된 다음 연상기억의 해 (solution) 이다:

$$\min_{\mathbf{m}} \tilde{\mathcal{L}}(\mathbf{m}; \hat{\mathbf{x}}^{\ell-1}, -\boldsymbol{\delta}^{\ell}).$$

(37)

여기서 목적함수 $\tilde{\mathcal{L}}(\cdot)$ 는 당면한 문제의 원래 목적함수와는 다르며, $\tilde{\mathcal{L}}(\cdot)$ 는 모멘텀을 정의하고 그 매핑의 품질을 측정하는 목적함수이다. 사실, 이 정식화에서 모멘텀 항은 계층의 입력에 기반하여 국소 오차율에 대해 문맥 내에서 (in-context) 적응하는 것을 목표로 한다 (모멘텀의 문맥은 경사들임을 상기하라). 대부분의 인기 있는 최적화기 (optimizer) 들은 (계산 효율성의 이유로) 원소별 (element-wise) 갱신 규칙으로 정식화되어 있다. 따라서 부록 B 에서 우리는 먼저 모멘텀의 원소별 연상 기억 정식화를 탐구하고 이를 Adam (Kingma et al. 2014a) 과 같은 인기 있는 최적화기들과 연결한다.

Adam 이 경사들의 분산을 예측하는 것을 목표로 하는 L_2 -회귀 (regression) 목적함수에 대한 최적 연상 기억으로 볼 수 있음을 보이면서, 우리는 RMSProp(Hinton et al. 2012), SignSGD 및 그 모멘텀 기반 변형들 (Bernstein et al. 2018), NAdam(Dozat 2016), AMSGrad(Reddi et al. 2016), RAdam(Liu et al. 2020), 그리고 Lion(Chen et al. 2023) 과 같은 다른 유사 알고리즘들도 경사들을 압축하는 것을 목표로 하는 연상 기억의 사례임을 논의한다. 그다음 우리는 원소별 정식화를 넘어서서 AdaGrad(Duchi et al. 2011) 역시 연상 기억 모듈임을 보인다. AdaGrad 가 Shampoo(Gupta et al. 2018) 및 Soap(Vyas et al. 2025) 과 같은 최적화기들과 연결되어 있으므로—즉, 사전조건화 (preconditioning) 항의 근사로서—우리는 이러한 모든 최적화기들이 연상 기억으로 재정식화될 수 있다고 결론짓는다. 다음으로, 우리는 사전조건화에 기반한 또 다른 부류의 최적화기들을 논의하고 이를 NL의 관점에서 더 자세히 재정식화한다:

사전조건화와 헤시안 (Hessian)의 근사. 또 다른 부류의 알고리즘은 사전조건화 알고리즘으로서, 그 아이디어는 뉴턴 알고리즘 (Newton's algorithm)의 거동을 모방하기 위해 헤시안 역행렬을 근사하는 것이다. 형식적으로, 사전조건화를 적용한 경사하강법은 다음과 같이 정의된다:

$$W_{t+1}^\ell = W_t^\ell - \eta_{t+1} P_{t+1}^{-1} g_{t+1}^\ell,$$

(38)

여기서 사전조건자 (preconditioner) P_{t+1} 는 보통 양의정부호 (positive-definite) 행렬이다. 사전조건자에 대한 중요한 해석은 변환된 좌표계에서 경사하강법을 수행하는 그것의 역할인데, 이는 경사들로부터 관심 대상 좌표계로의 매핑으로 볼 수 있다. 이에 따라, 우리는 식 38의 사전조건자를 경사들의 집합 (또는 g 로 표기된 경사들의 함수)을 우리가 선택한 계 (system), 즉 $\hat{g}$ 로 표기되는 것으로 매핑하는 연상 기억으로 재정식화하고 해석한다:

$$W_{t+1}^\ell = W_t^\ell - \eta_{t+1} P_{t+1}^{-1} g_{t+1}^\ell,$$

(39)

여기서 내부적으로 (중첩된 수준에서), P_{t+1} 은 다음 목적함수를 사용하여 이 매핑을 어떻게 수행할지를 학습한다:

$$\min_P \tilde{\mathcal{L}}(P(g); \hat{g}).$$

(40)

이 관점에서 볼 때, 주된 질문은 압축 과정을 강화할 수 있는 최선의 좌표계를 찾는 것에 관한 것이다. 가장 단순한 변형은 항등 매핑 (identity mapping)으로, 이때 우리는 계량 체계 (metric system)를 보존하고 P 를 사용하여 g (즉, 이 경우 경사들)를 그 자신으로 매핑하며, 그 결과 부록 B에서 논의하듯이 Adam(Kingma et al. 2014a)과 AdaGrad(Duchi et al. 2011)의 사전조건화 항들이 나온다. 이러한 결과들은 Adam 및 그 변형들이 연상 기억으로 표현되는 것과 더불어, 모멘텀 기반 최적화기들이 연상 기억일 뿐만 아니라 각각 경사하강법으로 최적화되는 중첩 학습 (nested learning) 문제들의 집합으로 분해될 수 있음을 보여준다. 그러나 더 일반적인 형태에서는, 더 많은 중첩 수준을 사용하고 식 40의 내부 문제들을 경사하강법으로 최적화할 수 있으며, 그 결과는 다음과 같다:

$$P_{t+1} = P_{t+1} - \zeta_{t+1} \nabla_{P_t} \tilde{\mathcal{L}}(P_t; g_{t+1}, \hat{g}_{t+1}).$$

(41)

NL 프레임워크에서, 효과적인 사전조건화를 설계하려면 $\hat{g}$ 와 $\tilde{\mathcal{L}}$ 의 올바른 선택을 찾아야 한다. 이 관점은 또한 경사/모멘텀직교화 (orthogonalization) 를 사용하는 다른 부류의 알고리즘들로 이어질 수 있다: 예를 들어 Muon 과 그 변형들 (Jordan et al. 2024; Cesista 2025; Keigwin et al. 2025) 이다. Muon 최적화기 (Jordan et al. 2024) 를 상기하면:

$$\begin{aligned} W_{t+1}^\ell &= W_t^\ell + \text{NewtonSchulz}_k \left(m_{t+1}^\ell \right) \\ m_{t+1}^\ell &= \alpha_{\ell,t+1} m_t^\ell - \eta_{\ell,t+1} \nabla_{W_t^\ell} \mathcal{L} \left(W_t^\ell; x_{t+1} \right), \end{aligned} \quad (42)$$

여기서 $\text{NewtonSchulz}_k(\cdot)$ 는 뉴턴-슐츠 (Newton-Schulz) 직교화 과정을 k 단계 수행한다. 사전조건화의 일반적 정식화에 관한 위논의로부터, $\text{NewtonSchulz}_k(\cdot)$ 연산자를 모멘텀 항의경사들로부터 적절한 계량 체계로의 매핑으로 볼 수 있다. Muon 에서 적절한좌표계의 선택은 경사들을 직교화하는 것이며, 따라서 우리는 손실함수 $\min_P \tilde{\mathcal{L}}(P; O, m)$ 를 최소화함으로써 매핑 $P(\cdot)$ 를 찾는 것을 목표로한다. 여기서 목적함수 $\tilde{\mathcal{L}}(\cdot; \cdot, \cdot)$ 는 $P(\cdot)$ 에 의한 O 로부터 m 또는 g 로의 매핑의 품질을 측정한다. 이 과정에서 중요한 난점은 파라미터 O 자체가 주어지지않는다는 것이며, 따라서 매핑은 매핑과 적절한 직교 공간 (orthogonal space) 둘 다를 학습해야 한다. 직교화를 측정하는 단순한 정식화는 목적함수를다음과 같이 정의함으로써 얻을 수 있다:

$$\tilde{\mathcal{L}}(P(g); g) = \|P(g)^\top P(g) - I\|_F^2, \quad (43)$$

여기서 $P(g)$ 는 우리가 경사들로부터 직접학습하고자 하는 직교 공간이다. 이 목적함수는 경사들 (또는 모멘텀) 과 그매핑이 상대적으로 가깝도록 보장하는 동시에 그 매핑이 직교 공간으로향하도록 한다. 위 목적함수를 최적화하여 $O = P(g)$ 를 한 단계의경사하강법으로 찾으면 다음이 나온다:

$$O_{i+1} = O_i - \zeta_{i+1} \nabla_{O_i} \tilde{\mathcal{L}}(O_i; g_i) = O_i - \zeta_{i+1} \left(O_i - g_i + 2O_i \left(O_i^\top O_i - I \right) \right), \quad (44)$$

이는 3 차 다항식 (초기값 $O_0 = g_t$) 을복원한다. 요약하면, 더 높은 빈도 (higher-frequency) 수준이 직교 매핑을학습하고, 그다음 더 낮은 빈도 (lower-frequency) 과정이 학습된 매핑을사용하여 가중치를 최적화한다. 이후 4.4 절에서, 우리는 $\text{NewtonSchulz}_k(\cdot)$ 를 메모리의 용량을 강화하기 위한 다항식매핑으로 간주하는 더 일반적인 관점을 논의한다.

4.3 최적화기에서의 긴 문맥 (Long Context in Optimizers): 직교 과제 (Orthogonal Tasks) 를 이용한 지속 학습 (Continual Learning) 의예시

훈련 시간과 테스트 시간 사이의 경계를 제거하고 오랜 기간 동안 지속적으로학습할 수 있는 모델로 나아갈 때, (온라인) 최적화기의 역할은 더욱두드러진다: 주로 더 빠르게 수렴하는 것보다 효과적인 “해 (solution)” 를찾을 필요성 때문이다. 그러나 효과적인 해를 찾으려면 “국소 최소점 (local minima)” 을 피하기 위한 목적함수에 대한 전역적 이해뿐만 아니라, 오래 전에학습한 과제들의 (치명적) 망각 (catastrophic forgetting) 을 유발할 수 있는방향을 향해 움직이는 것도 필요하다. 연상 기억 관점에서 그리고 앞서논의했듯이, 모멘텀 항은 과거 경사들의 기억일 것으로 기대되며, 이는최적화 과정이 손실 지형 (loss landscape) 에 대한 더 전역적인 관점을 갖도록돕는다. 그러나 모멘텀의 현재 설계는 경사 갱신을 매끄럽게 만드는 단순한저역 통과 필터 (low-pass filter) 로 작동하며, 따라서 최근 과거로부터의정보만을 통합하는 제한된 용량을 갖는다.

이 한계를 더 잘 예시하기 위해, $\beta > 0$ 을 모멘텀의 감쇠 (decay) 항이라 하자. 그러면 현재 모멘텀 상태에 대한 i -번째 이전 경사의 기여도는 $\beta^i(1 - \beta)$ 로 계산될 수 있다. 모멘텀 항의 현재 상태에 대한 경사들의 기여도의 누적 합 (즉, $\mathbf{S}_t = \sum_{i=0}^t \beta^i(1 - \beta)$) 과 최적화설정에서 흔히 사용되는 값 $\beta = 0.9$ 를 고려하면, 마지막 6 개의 경사 (각각 43 개의 경사) 가 누적 기여도의 최소 50%(각각 99%) 를 담당한다, 즉 $\mathbf{S}_t$ 이다. 이는 경사들, 그리고 일반적으로 과거 43 단계를 넘어서는 전역 정보가 1% 미만으로 기여함을 나타내며, 이는 목적함수 지형에 대한 이해와 효과적인 해를 찾는 능력을 제한한다. 이 단순한 예시는 현재 설계가 긴 과거 정보를 통합하는 데에서조차 제한적임을 보여주며, 하물며 모멘텀의 현재 상태에 필요한 정보를 적절히 검색 (retrieve) 하는 능력은 말할 것도 없다.

지속 학습의 설정으로 돌아와서, 직교 과제를 갖는 그 단순한 변형 중 하나를 고려하자. $\{(\mathcal{T}_i, \mathcal{D}_i, \mathcal{L}_i)\}_{i=1}^n$ 을 과제들의 집합, 그에 대응하는 데이터, 그리고 그 목적함수들이라 하자. 이때 과제 $\mathcal{T}_i$ 에 대해:

$$\mathcal{L}_i(W) = \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}_i} \left[(W^\top \mathbf{x} - \mathbf{y})^2 \right],$$

(45)

그리고 경사들은 $\{\mathbf{u}_i\}_{i=1}^n$ 의 직교 방향들에 위치한다. 과제 $\mathcal{T}_t$ (충분히 큰 $t > 1$ 에 대해) 가 주어졌을 때, 모멘텀을 갖는 경사하강법으로 문제를 최적화하고 과제 t 에서 여러 단계를 거친 후, 경사들은 이제 $\mathbf{u}_t$ 방향을 가리킨다. 이에 따라, 모멘텀 항은 점진적으로 이동하여 이제 대략 $\mathbf{u}_t$ 방향에 놓인다. 이는 과거 경사들에 대한 점진적 망각으로 이어질 수 있으며, 이는 다시 모델에서 치명적 망각을 유발할 수 있다. 즉, 최적화 과정이 가중치를 이전 과제들의 성능을 손상시키는 방향으로 이동시킬 수 있는데, 이는 주로 최적화기가 자신이 피해야 할 오래된 경사 부분공간 (gradient subspace) 에 대한 기억이 없기 때문이다. 이 실패는 모델의 용량에 관한 것이 아니라 최적화 과정의 메모리 관리에 관한 것으로, 효과적인 해를 찾는 데 실패하는 것이다.

최적화기에서의 긴 문맥 이해 (Long Context Understanding in Optimizers)

정적 모델에서 데이터/경험으로부터 지속적으로 학습할 수 있는 신경 학습 모듈로 나아가면, 최적화 과정 자체가 다양한 과제 집합에 대해 그리고 오랜 기간 동안 효과적인 해를 찾기 위해 경사 부분공간의 장기적 압축/이해로부터 이득을 얻을 수 있다.

이 관찰에 동기를 받아, 우리는 다음으로 더 나은 메모리 관리와 더 높은 메모리 용량을 갖출 수 있는 모멘텀의 더 표현력 있는 변형들을 논의한다:

4.4 연상 기억으로서의 모멘텀을 위한 더 표현력 있는 설계 (More Expressive Designs for Momentum as an Associative Memory)

지금까지 우리는 (1) 모멘텀 항이 (과거) 경사들을 자신의 파라미터로 압축하는 것을 목표로 하는 연상 기억으로 볼 수 있고, (2) 오랜 기간 동안 그리고 다양한 과제 집합에 대해 지속적으로 학습할 수 있는 모델을 개발하기 위해서는 최적화 과정이 긴 과거와 손실 지형의 전역적 속성에 대한 적절한 정보가 필요함을 논의했다. 다음으로, 우리는 중첩 학습과 연상 기억 관점이 어떻게 다양한 메모리 관리/구조를 갖는 최적화기의 설계로 이어질 수 있는지를 논의한다:

확장: 더 표현력 있는 연상 (More Expressive Association). 앞서 논의했듯이, 기본 (vanilla) 모멘텀 항은 값이 없는 (value-less) 연상 기억으로 볼 수 있다. 더 표현력 있는 연상 기억을 허용하고 연상 기억의 원래 정의 (즉, 키를 값으로 매핑) 를 따르기 위해, 우리는 값 파라미터 $\mathbf{v}_i = \mathbf{P}_i$ 로 두고, 그러면 모멘텀은 다음을 최소화하는 것을 목표로 한다:

$$\min_m \langle m \nabla \mathcal{L}(W_i; \mathbf{x}_i)^\top, \mathbf{P}_i \rangle,$$

(46)

또는 동등하게, $\langle \mathbf{m}, \mathbf{P}_i \nabla \mathcal{L}(W_i; \mathbf{x}_i) \rangle$ 를 최소화한다. 모멘텀을 갱신하기 위해 경사하강법을 사용하면 다음 갱신규칙이 나온다:

$$\begin{aligned} W_{i+1} &= W_i + \mathbf{m}_{i+1} \\ \mathbf{m}_{i+1} &= \mathbf{m}_i - \eta_t \mathbf{P}_i \nabla \mathcal{L}(W_i; \mathbf{x}_i), \end{aligned} \quad (47)$$

여기서 우리는 또한 과거를 망각하기 위한 망각 게이트 (forget gate) α_{i+1} 를 포함했다. 이 갱신 규칙은 모멘텀 GD 를 사전조건화하는것으로 볼 수 있다. 여기서, 사전조건화를 갖는 모멘텀 항은 $\mathbf{P}_i$ 와 경사 항 $\nabla \mathcal{L}(W_i; \mathbf{x}_i)$ 사이의 매핑들을 어떻게 압축할지를 학습하는 연상 기억으로 해석된다. 한편, 사전조건화의 어떤합리적인 선택 (예: 무작위 특징 (random features)) 도 모멘텀을 갖는 GD 의 초기 버전의 표현력을 향상시키는 데 사용될 수 있다. 식 39 의 사전조건화에 대한 우리의 논의는 여기와 다르다는 점에 유의하라. 식 39 는 사전조건자를 적절한 매핑으로서 학습할 수 있음을 나타내는 반면, 위정식화는 사전조건화를 사용할 때 (경사들에 대한 기억으로서의) 모멘텀 항이 경사들을 그 매핑 함수로 매핑하는 것을 목표로 함을 보여준다.

확장: 더 표현력 있는 목적함수 (More Expressive Objectives). 모멘텀의 정식화를 다시 살펴보면: 주어진 경사 $\nabla \mathcal{L}(W_i; \mathbf{x}_i)$ 에 대해, 모멘텀 매핑은 (위에서 논의했듯이) 내부 목적함수로서 내적 유사도 (dot-product similarity) 에 기반하며, 따라서 그 갱신 규칙은 헵 규칙 (Hebbian-rule) (Hebb 2005) 이다. 연상 기억 관점에서, 이 갱신 규칙은 제한된 용량 (Storkey 1997) 을 가지며 모멘텀의 갱신 규칙을 그 현재 상태와 독립적으로 만든다. 따라서 손실 지형 정보를 추적/압축하는 능력을 제한한다. 자연스러운 확장은 내부 목적함수를 L_2 -회귀 손실 (해당 키-값 매핑의 적합도를 측정하기 위한) 로 대체하고 손실함수 $\|\mathbf{m} \nabla \mathcal{L}(W_i; \mathbf{x}_i)^\top - \mathbf{P}_i\|_2^2$ 를 최소화하는 것이며, 그 결과 다음 갱신 규칙이 나온다:

$$\begin{aligned} W_{i+1} &= W_i + \mathbf{m}_{i+1}, \\ \mathbf{m}_{i+1} &= \mathbf{m}_i \left(\alpha_{i+1} - \nabla \mathcal{L}(W_i; \mathbf{x}_i) \nabla \mathcal{L}(W_i; \mathbf{x}_i)^\top \right) - \eta_t \mathbf{P}_i \nabla \mathcal{L}(W_i; \mathbf{x}_i), \end{aligned} \quad (48)$$

이 갱신은 델타 규칙 (delta-rule) (Prados et al. 1989) 에 기반하며, 따라서 메모리 (모멘텀) 가 자신의 제한된 용량 (즉, $\mathcal{O}(N)$) 을 더 잘 관리하고 과거 경사들의 계열을 더 잘 기억하도록 한다. 예를 들어, 우리는 최적화 과정 동안 과거 경사들의 일부를 잊는 것을 학습할 수 있다 (연상기억에서 선형 어텐션 (linear attention) 에서 델타 규칙으로 옮겨갈 때 일어나는 것과 유사하게). 우리는 이러한 유형의 모멘텀 항의 변형들을 델타모멘텀 (Delta Momentum) 변형이라 부른다.

확장: 더 표현력 있는 메모리 (More Expressive Memory). 모멘텀을 과거 경사들을 자신의 요소들 (파라미터들) 에 저장하는 압축기 또는 메모리로 볼 때, 그 용량은 (위와 유사하게) 갱신 규칙에만 의존하는 것이 아니라, 더 큰 용량을 허용하는 더 표현력 있는 구조도 요구한다. 현재 정식화는 과거 경사 값들을 압축하기 위해 선형 계층 (즉, 행렬값 (matrix-valued)) 에 기반하지만, 이 선형적 본성은 과거 경사들의 선형 매핑만을 학습하는 능력으로 제한할 수 있다. 이 모듈의 학습 용량을 증가시키기 위해, 우리는 모멘텀을 위한 선형 행렬값 메모리를 MLP 로 대체하는 것과 같은 더 복잡한 매핑들을 사용할 수 있다. 이 설계는 모멘텀이 더 많은 경사들을 기억하도록 하고, 따라서 최적화 과정에 더 나은 정보를 제공한다. 우리는 식 33 의 정식화를 다음과 같이 확장한다:

$$W_{i+1} = W_i + \mathbf{m}_{i+1}(\mathbf{u}_i),$$

그리고

$$\mathbf{m}_{i+1} = \alpha_{i+1}\mathbf{m}_i - \eta_t \nabla \mathcal{L}^{(2)}(\mathbf{m}_i; \mathbf{u}_i, 1),$$

(50)

여기서 $\mathbf{u}_i = \nabla \mathcal{L}(W_i; \mathbf{x}_i)$ 이고 $\nabla \mathcal{L}^{(2)}(\cdot)$ 는 모멘텀의 내부목적함수이다 (예: 내적 유사도 $\langle \mathbf{m}(\mathbf{u}_i^\top), 1 \rangle$). 우리는 이변형을 심층 모멘텀 경사하강법 (Deep Momentum Gradient Descent, DMGD) 이라 부른다. 이 예시로부터 명백하긴 하지만, 효과적인 모멘텀 모듈을 얻으려면 내부 손실함수와 모델이 신중하게 설계되어야 함을 강조할 가치가 있다.

확장: 고차 특징 맵을 갖는 메모리 (Memory with Higher-order Feature Maps). 메모리의 용량을 강화하기 위해 흔히 사용되는 기법 중 하나는 키에 고차 특징 맵 (higher-order feature maps) 을 사용하는 것이다 (Katharopoulos et al. 2020; Kacham et al. 2024). 이 기법을 모멘텀항에 사용하면, 다음을 얻을 수 있다:

$$W_{i+1} = W_i + \mathbf{m}_{i+1}$$

그리고

$$\mathbf{m}_{i+1} = \alpha_{i+1}\mathbf{m}_i - \eta_t P_i \phi(\nabla \mathcal{L}(W_i; \mathbf{x}_i)),$$

(51)

여기서 $\phi(\cdot)$ 는 (자신의 내부 목적함수를 통해 학습될 수 있는) 고차 특징 매핑이다.

확장: 비선형 출력 (Nonlinear Outputs). 모멘텀의 연상 기억 관점에 기반하여, 메모리 모듈의 표현력을 강화하는 흔한 기법 중 하나는 그 출력위에 비선형성 (non-linearity) 을 사용하는 것이다 (Sun et al. 2024; Behrouz et al. 2025c). 즉, 우리는 식 50 을 다음과 같이 재정식화한다:

$$W_{i+1} = W_i + \sigma(\mathbf{m}_{i+1}(\mathbf{u}_i)),$$

그리고

$$\mathbf{m}_{i+1} = \alpha_{i+1}\mathbf{m}_i - \eta_t \nabla \mathcal{L}^{(2)}(\mathbf{m}_i; \mathbf{u}_i, I),$$

(52)

여기서 $\sigma(\cdot)$ 는 임의의 비선형 함수 (non-linearity) 이다. 하나의 예로, 우리는 $\sigma(\cdot) = \text{NewtonSchulz}(\cdot)$ 로 두는데, 여기서 $\text{NewtonSchulz}(\cdot)$ 는 반복적 Newton-Schulz 방법 (Higham 2008) 이며, $\mathbf{m}(\cdot)$ 은 선형 계층 (linear layer) 이다. 그 결과 Muon (Jordan et al. 2024) 이 도출된다.

19

옵티마이저에서의 긴 맥락 (Long Context) 을 위한 장난감 예제 (Toy Example). 4.3 절에서 우리는, 직교하는 (orthogonal) 과제들을 포함한 연속학습 (continual learning) 과 같은 복잡한 설정에서는, 더 높은 용량 (capacity) 이나 더 나은 메모리 관리를 갖춘, 더 복잡한 모멘텀 (momentum) 항들이 필요할 수 있음을 논의했다. 다른 모멘텀 메모리 설계들의 잠재적 이득을 더 잘 예시하기 위해, 우리는 시간에 따라 변하는 곡률 (time-varying curvature) 의 장난감 예제를 사용한다. 표준 모멘텀은 저역 통과 필터 (low-pass filter) 처럼 작동하므로, 만약 지형 (landscape) 이 높은 주파수로 변

화한다면, 과거 그라디언트들의 가중 평균을 사용하려는 표준 모멘텀은 무관한 (irrelevant) 그라디언트 항들의 영향을 받아 수렴하지 연된다. 예시적 사례로 다음을 고려하자:

$$\psi(r, \theta) = r^2 + k \times (r - \theta + \alpha \sin(\omega r))^2,$$

(53) **그림 4:** 표준 모멘텀 및 우리의 델타 모멘텀 (delta momentum) 으로 함수 $\psi(r, \theta)$ 를 최적화한 결과.

그리고 이 함수를 표준 모멘텀과 우리의 델타 모멘텀을 사용해 최적화하는 것을 목표로 한다.

우리는 최적화 과정을 점 $(r_0, \theta_0) = (-3.5, 2)$ 에서 시작하여, 알고리즘 중 하나가 최적해로 수렴할 때까지 계속한다. 그 결과는 그림 4 에 시각화되어 있다. 델타 모멘텀은 더 빠르게 해를 찾는데, 이는 주로 그것의 그라디언트 의존적 가중치 감쇠 (gradient-dependent weight decay) 덕분이며, 이는 모멘텀 항이 필요할 때 감쇠하거나 멈추도록 돕는다.

4.5 단순 경사하강법과 모멘텀을 넘어서 (Going Beyond Simple Gradient Descent and Momentum)

사전 학습 (pre-training) 과정과 역전파 (backpropagation) 가 연상메모리 (associative memory) 의 한 형태라는 3.1 절의 논의로 다시 돌아와서, 이 절에서 우리는 NL(Nested Learning) 의 관점을 활용하여 경사하강법 (gradient descent) 의 더 일반적인 형태를 제시하고자 한다. 4.1 절에서 관찰한 바와 같이, 경사하강법을 사용한 역전파는 입력 데이터를, 그 예측된 출력이 야기한 놀람 (surprise) $\nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t)$ 에 매핑하는 것을 목표로 하는 연상 메모리이다:

$$W_{t+1} = W_t - \eta_{t+1} \nabla_W \mathcal{L}(W_t; \mathbf{x}_t) = W_t - \eta_{t+1} \nabla_y \mathcal{L}(W_t; \mathbf{x}_t) \otimes \mathbf{x}_t,$$

여기서 $\mathbf{x}_t \sim \mathcal{D}_{\text{train}}$ 이며,

(54)

이는 연상 메모리 관점과 근접 그라디언트 (proximal gradient) 관점에서 다음과 동등하다:

$$W_{t+1} = \arg \min_W \langle W \mathbf{x}_t, \nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t) \rangle + \frac{1}{2\eta_t} \|W - W_t\|_2^2.$$

(55)

이 단계는 그라디언트 방향의 음 (negative) 을 학습하는 것을 목표로 한다. 내부 목적함수 (inner objective) 로서 내적 유사도 (dot-product similarity) 를 사용하는 것의 주된 단점은, 그에 대응하는 갱신 규칙 (update rule) 및 학습 알고리즘이 각 데이터 샘플 (그라디언트들) 을 상태 (state) 와 독립적으로 취급한다는 점이다. 즉, 가중치의 상태와 이전 그라디언트들이 현재 상태에 대한 갱신 항에 영향을 미치지 않는다는 것을 의미한다. 이 설계는 맥락 (context) 내의 요소들이 서로 독립적인 (예: 학습을 위한 i.i.d. 샘플들) 중첩 (nested) 문제들에는 효과적일 수 있으나, 요소들이 서로 크게 의존적인 맥락 (예: 시퀀스 내의 토큰들) 에는 제한적일 수 있다. $\mathbf{u}_t = -\nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t)$ 로 정의하면, 이 과정을 L2 회귀 손실 (regression loss) 과 같은 더 표현력 있는 목적함수로 확장할 수 있다:

$$W_{t+1} = \arg \min_W \frac{1}{2} \|W \mathbf{x}_t - \mathbf{u}_t\|_2^2 + \frac{1}{2\eta_t} \|W - W_t\|_2^2.$$

(56)

$\mathbf{x}_t$ 가 정규화된 경우 (예: 정규화된 메모리 시스템이나 정규화계층을 갖는 신경망에서, $\|\mathbf{x}_t\|_2 = \lambda$), 그리고 $\eta'_t = \frac{\eta_t}{1+\eta_t}$ 로 정의하면, 우리는 Sherman-Morrison 보조정리 (lemma) 를 사용하여 다음을 얻을 수 있다 (자세한내용은 부록 C 참조):

$$\begin{aligned} W_{t+1} &= W_t \left(I - \eta'_t \mathbf{x}_t \mathbf{x}_t^\top \right) - \eta'_t \nabla_{W_t} \mathcal{L}(W_t; \mathbf{x}_t) \\ &= W_t \left(I - \eta'_t \mathbf{x}_t \mathbf{x}_t^\top \right) - \eta'_t \nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t) \otimes \mathbf{x}_t, \end{aligned}$$

여기서 $\mathbf{x}_t \sim \mathcal{D}_{\text{train}}$ 이다.

(57)

델타 규칙 (Delta rule, Prados et al. 1989) 에 기반하여 우리가 델타경사하강법 (Delta Gradient Descent, DGD) 이라 부르는 이 새로운 알고리즘은, 현재 요소들뿐만 아니라 가중치의 이전 상태도 함께 반영하여 가중치를 갱신하며, 그 결과 현재 데이터 샘플에 기반한 적응적 감쇠 항 (adaptive decay term) 을 도출한다. 다음으로, 우리는 경사하강법을 사용한 역전파과정에 대한 일반화된 관점을 논의하는데, 이는 이후 우리가 일반화된경사하강법 (Generalized Gradient Descent) 계열의 학습 규칙을 정식화하는데 도움을 줄 것이다:

20

역전파를 사용한 심층 신경망 학습은 자기 참조적과정 (Self-Referential Process) 이다: 4.1 절에서 앞서 논의했듯이, 경사하강법에 대한 흔한 오해 중 하나는 그것을 선형 순환 (linear recurrence) 의 한 형태 (예: 선형 어텐션 (linear attention)) 로 보는 것이다. 그러나 전통적인 선형 순환에서는 키 (key) 와 값 (value) 이 메모리의 상태와 독립적이며, 따라서 정식화의 병렬화 (parallelization) 를 허용한다. “연상메모리로서의 경사하강법” 관점에서 값들은 메모리 상태의 함수이며, 따라서 그것은 자기 자신의 값들을 생성함으로써 자신의 학습 과정을 제어하는 자기참조적 모델 (self-referential model, Schmidhuber 1993) 이다. 더형식적으로, 이 과정을 다음과 같이 재정식화할 수 있다:

$$\begin{aligned} W_{t+1} &= W_t + \eta_{t+1} \mathbf{v}_t \otimes \mathbf{x}_t, \\ \mathbf{v}_t &= \mathbf{f}_{W_t}(\mathbf{x}_t) = -\nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t), \end{aligned}$$

(58)

이는 각 단계에서 $\mathbf{v}_t$ 가 메모리 W_t 와 그 입력인 $\mathbf{x}_t$ 에 의해 생성됨을 의미한다.

위의 해석에 기반하여, 경사하강법을 사용한 역전파를, 학습 샘플들을 키로 압축하고 이를 자기 생성된 값들 (self-generated values) 에 매핑하여 자신의 학습 과정을 더 잘 제어하려는 임의의 자기 참조적 모델로서 일반적인 형태로 정의할 수 있다. 이 정의에 기반하면, 부록 C 에서의 위 정식화는 L2 회귀손실을 사용하는 단순한 하나의 사례일 뿐이며, 일반적으로는 다음과 같이 일반화된 경사하강법 (Generalized Gradient Descent, GGD) 을 정의할 수 있다:

정의 5 (일반화된 경사하강법 (Generalized Gradient Descent, GGD) 학습 규칙). 일반화된 경사하강법 (GGD) 학습 규칙은 데이터 샘플들을 압축하여 이를 자기 생성된 키들의 집합에 매핑하는 것을 목표로 하는 자기참조적 연상 메모리이다:

$$W_{t+1} = \arg \min_W \tilde{\mathcal{L}}(\mathbf{x}_t, \mathbf{u}_t) + \text{Ret}(W, \{\mathbf{W}_i\}_{i=t-c+1}^t),$$

(59)

여기서 $\mathbf{u}_t$ 는 자기 생성된 값이다:

$$\mathbf{u}_t = \mathbf{f}_{W_t}(\mathbf{x}_t),$$

(60)

이는 W_t 로 매개변수화된 어떤 함수 $\mathbf{f}_{W_t}(\cdot)$ 에 대한것이다. 여기서 $\tilde{\mathcal{L}}(\cdot)$ 은 매핑의 품질을 측정하고, $\text{Ret}(\cdot)$ 은 새로운 인스턴스에 대한 해가 현재 상태에서부터 너무멀리 떨어지지 않도록 보장한다.

유사하게, 이 정식화는 모멘텀 항에도 적용될 수 있으며, 그 결과 일반화된모멘텀 (Generalized Momentum, GM) 이 도출된다. 다만, 모멘텀 그 자체는 전통적인 연상 메모리이며 그것의 키와 값은 주어져 있다는 점, 더구체적으로는 더 낮은 주파수 수준 (lower-frequency level) 에 의해 생성된다는 점은 주목할 만하다. 4.2 절에서 우리는 이 정식화의 특수한 경우, 즉 $\mathcal{L}(\cdot)$ 이 $L2$ 회귀 손실인 경우를 탐구했다.

연속 학습 (Continual Learning) 설정에서의 옵티마이저에 관한주석. 위에서 논의한 바와 같이, 옵티마이저들 그 자체는 그래디언트들을 자신의 매개변수로 압축하는 것을 목표로 하는 학습 모듈 또는 연상메모리이다. 이 매개변수들은 전통적인 용어로 반드시 학습가능한 (trainable) 것은 아니지만, 실제로 모멘텀 기반 옵티마이저들은 손실지형 (loss landscape) 에 관한 지식을 저장하여, 가중치를 더 잘 갱신하도록 돕는다. 신경 학습 모듈에 대해 “사전 학습의 종료 (end of pretraining)” 가 일어날 때, 데이터/그래디언트의 분포에 관해 저장되었던 지식 (모멘텀 항 (들) 에 저장됨) 은 모델로부터 제거되며, 따라서 모멘텀 상태를 복구하지 않고 학습을 계속하면 모델이 새로운 능력을 학습하는 능력에 영향을 줄 수 있다. 모델이 연속 학습 설정에 있을 때, 데이터에 관한 지식은 전통적인 매개변수 (역전파로 최적화됨) 에 저장되지만, 모델이 자기 자신을 어떻게 최적화하는지와 목적 공간 (objective space) 에 관한 지식은 최적화의 더 낮은 주파수 수준 (예: 모멘텀 항들) 에서 최적화된다.

5 신경 학습 모듈로서의 기존 아키텍처들 (Existing Architectures as Neural Learning Modules)

트랜스포머 (Transformers, Vaswani et al. 2017) 와 순환 모델 (recurrent models; Katharopoulos et al. 2020; Schlag et al. 2021; Sun et al. 2024; Behrouz et al. 2025c) 같은 현대적 시퀀스 모델들은 최근 언어 모델 발전의 근간 (backbone) 이다. 최근, 이러한 모델들이 데이터로부터 키에 값으로의 매핑을 학습하려는 연상 메모리와 동등하다는 점이 다양한 설정과 목적 함수에서 연구되어 왔다 (Liu et al. 2024b; Sun et al. 2024; Behrouz et al. 2025b; Wang et al. 2025). 특히, 우리는 Miras (Behrouz et al. 2025b) 의 일반 프레임워크에 초점을 맞추는데, 이는 연상 메모리를 정의 1 로 정의하고, 내부 목적 함수 (“어텐션 편향 (attentional bias)” 이라고 불림) 를 임의의 함수 클래스 (즉, 메모리 아키텍처) 에 대해 어떤 최적화 알고리즘을 선택하여 최적화한다. 이 정식화만으로도 잘 알려진 아키텍처들이 연상 메모리의 중첩 시스템 (nested systems of associative memory, NSAM) 의 인스턴스임을 나타내지만, 다음으로 우리는 몇몇 학습 규칙 및 아키텍처에 대해 이동 등성을 검토한다.

21

이제부터 우리는 키 $\{\mathbf{k}_i\}_{i=1}^L$, 값 $\{\mathbf{v}_i\}_{i=1}^L$, 질의 (query) $\{\mathbf{q}_i\}_{i=1}^L$ 가 주어졌다고 가정한다. 이들은 종종 입력의 사영 (projection) 으로 정의된다. 즉,

$$\mathbf{k}_t = \mathbf{x}_t W_k, \quad \mathbf{v}_t = \mathbf{x}_t W_v, \quad \mathbf{q}_t = \mathbf{x}_t W_q.$$

(61)

이 설계에서, 사영 매개변수들 (즉, W_k, W_v, W_q) 은 더 낮은 주파수 수준에서 최적화되므로, 시퀀스 모델 구성요소 (예: 셀프 어텐션 (self-attention)) 는 더 높은 주파수를 가지며, 따라서 연상 메모리의

학습 과정은 더 낮은 수준에서 일어난다. 그에 따라, 명료함을 위해 우리는 더 높은 주파수 수준 (즉, 연상 메모리의 내부 학습 과정) 만 논의한다.

소프트맥스 어텐션 (Softmax Attention). 연상 메모리 관점에서: 키 $\{\mathbf{k}_i\}_{i=1}^L$, 값 $\{\mathbf{v}_i\}_{i=1}^L$, 질의 $\{\mathbf{q}_i\}_{i=1}^L$ 가 주어졌을 때, 소프트맥스 어텐션블록 (Bahdanau 2014; Vaswani et al. 2017) 은 Nadaraya-Watson 추정량 (estimator) 을 사용한 $\ell_2(\cdot)$ 회귀 목적함수에 대한비모수적 (non-parametric) 해로서 재정식화될 수 있다 (Fan 2018; Zhang et al. 2022):

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \sum_{i=1}^L s(\mathbf{k}_i, \mathbf{q}) \|\mathbf{v}_i - \mathcal{M}\|_2^2 = \sum_{i=1}^L \frac{s(\mathbf{k}_i, \mathbf{q})}{\sum_{j=1}^L s(\mathbf{k}_j, \mathbf{q})} \mathbf{v}_i, \quad (62)$$

여기서 L 은 시퀀스 길이이다 (Sun et al. 2024). 이 정식화는 메모리 $\mathcal{M}(\cdot)$ 을 전체 맥락에 대해 최적화한다. 그러나 하나의 설계선택으로, 최적화 과정을 과거 c 개의 토큰으로 제한할 수 있으며, 그결과 다음이 도출된다:

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \sum_{i=t-c+1}^t s(\mathbf{k}_i, \mathbf{q}_i) \|\mathbf{v}_i - \mathcal{M}\|_2^2 = \sum_{i=t-c+1}^t \frac{s(\mathbf{k}_i, \mathbf{q})}{\sum_{j=t-c+1}^t s(\mathbf{k}_j, \mathbf{q})} \mathbf{v}_i, \quad (63)$$

이는 슬라이딩 윈도우 어텐션 (sliding window attention, SWA) 과 동등하다. 따라서 어텐션과 그것의 더 표현력 있는 변형들 (Wang et al. 2025) 또한, 경사하강법이나 다른 모수적 (parametric) 방법 대신 매핑에 대한 최적의비모수적 해를 찾을 때, 정의 1 의 인스턴스들이다.

헤비안 규칙 (Hebbian Rule) 을 갖는 RNN. 현대적 순환 아키텍처의 1 세대 (예: 선형 어텐션 (Katharopoulos et al. 2020), RetNet(Sun et al. 2023), RWKV(Peng et al. 2023), lightning attention(Li et al. 2025)) 은 헤비안 유사 (Hebbian-like) 학습 규칙 (Hebb 2005) 에 기반한다. 이 부류의 모델들에서, 키와 값 사이의 매핑 품질을 측정하기 위한 내부목적함수는 내적 유사도이다. 즉, 행렬값 메모리 (matrix-valued memory) $\mathcal{M} \in \mathbb{R}^{d \times n}$, 키와 값 $\mathbf{k}, \mathbf{v} \in \mathbb{R}^d$, 목적함수 $\tilde{\mathcal{L}}(\mathcal{M}; \mathbf{k}_t, \mathbf{v}_t) := -2\langle \mathcal{M} \mathbf{k}_t, \mathbf{v}_t \rangle$, 그리고 커널 $\phi(\cdot)$ 가 주어졌을 때, 우리는 그에 상응하는 연상메모리 최적화 문제 (정의 1 참조) 를 경사하강법과 가중치 감쇠 (weight decay) 로 최적화하며, 그 결과 다음이 도출된다:

$$\mathcal{M}_t = \alpha_t \mathcal{M}_{t-1} - \underbrace{\eta \nabla_{\mathcal{M}_{t-1}} \tilde{\mathcal{L}}(\mathcal{M}_{t-1}; \phi(\mathbf{k}_t), \mathbf{v}_t)}_{-\mathbf{v}_t \phi(\mathbf{k}_t^\top)} = \alpha_t \mathcal{M}_{t-1} + \eta_t \mathbf{v}_t \phi(\mathbf{k}_t^\top), \quad (64)$$

이는 원래의 선형 어텐션 순환식 (Katharopoulos et al. 2020) 을 복원한다. α_t 에 대한 서로 다른 설정들 (즉, 1 로 고정, 학습 가능, 채널별 (channel-wise), 그리고/또는 입력의존적 (input-dependent)) 과 $\phi(\cdot)$ 에 대한 서로 다른 설정들 (즉, 항등 (identity), 다항식 커널 (polynomial kernels) 등) 이 주어지면, 위의순환식은 헤비안 규칙을 갖는 선형 어텐션의 다양한 변형들을복원한다 (Katharopoulos et al. 2020; Sun et al. 2023; Arora et al. 2024; Beck et al. 2024; Kacham et al. 2024; Peng et al. 2024). 따라서 헤비안규칙을 갖는 선형 어텐션의 변형들은, 메모리가 내적 유사도 목적함수에 기반하여 키와 값 사이의 매핑을 경사하강법으로 학습하려는 최적화 문제의과정으로 재정식화될 수 있다.

델타 규칙 (Delta Rule) 을 갖는 RNN. 위 그룹의 메모리 관리를개선하고 메모리 용량을 향상시키기 위해, 여러 연구들은 순환 신경망에서의학습 알고리즘으로 헤비안 규칙을 델타 규칙으로 대체할 것을제

안하며 (Schlag et al. 2021), 그 결과 DeltaNet(Schlag et al. 2021), Longhorn(Liu et al. 2024b), RWKV7(Peng et al. 2025b) 같은 모델들도출된다. $\mathcal{M} \in \mathbb{R}^{d \times n}$ 으로 둘 때, 델타규칙은 MSE 목적 함수 $\tilde{\mathcal{L}}_t = \|\mathcal{M}_t \mathbf{k}_t - \mathbf{v}_t\|_2^2$ 를, 지역 보존 (local retention) 으로 $\text{Ret}_t(\mathcal{M}, \mathcal{M}_{t-1}) = \|\mathcal{M}_t - \mathcal{M}_{t-1}\|_F^2$ 를, 그리고 옵티마이저로 확률적 경사하강법 (stochastic gradient descent) 을사용하여 최적화하는 것과 동등하다:

$$\mathcal{M}_t = \mathcal{M}_{t-1} - \eta_t \nabla_{\mathcal{M}_{t-1}} \tilde{\mathcal{L}}(\mathcal{M}_{t-1}; \phi(\mathbf{k}_t), \mathbf{v}_t) = \left(I - \eta_t \mathbf{k}_t \mathbf{k}_t^\top \right) \mathcal{M}_{t-1} + \eta_t \mathbf{v}_t \mathbf{k}_t^\top.$$

(65)

$$(\mathcal{M}_{t-1} \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top$$

다른 형태의 유지 게이트 (retention gate)(예: $\text{Ret}_t(M, M_{t-1}) = \|M_t - \alpha_t M_{t-1}\|_F^2$), 가중치 감쇠 (weight decay) 를 갖는 최적화 알고리즘 (예: 주어진 $q > 0$ 에대해 $\|M_t\|_q^q$ 로 정규화), 여러 단계의 경사 하강, 그리고/또는 η_t 와 α_t 같은 학습 가능매개변수의 서로 다른 정식화 (formulation) 를 사용하면, 델타 규칙 (delta rule) 의 다양한 변형들을 얻을 수 있다 (Irie et al. 2021; Liu et al. 2024b; Sun et al. 2024; Behrouz et al. 2025b; Hu et al. 2025; Peng et al. 2025b; Siems et al. 2025; Wang et al. 2025). 따라서 델타 규칙과 그변형들은 모두, 모델이 L_2 -회귀 목적함수에 기반하여 키 (key) 와값 (value) 사이의 매핑을 학습하고자 하는 최적화 문제의 인스턴스들이다.

관습적 학습 규칙을 넘어서: 오메가 (Omega), 오야 (Oja’ s), 그리고비유클리드 학습 규칙. 보다 최근에는 연상 기억 (associative memory) 관점 (정의 1 참조) 에서 아키텍처를 설계하고, 더 복잡한 내부 목적함수그리고/또는 최적화 알고리즘을 사용하려는 관심이 커지면서, 델타 및 헤비안 (Hebbian) 규칙을 넘어서는 학습 알고리즘들이등장하였다 (Irie et al. 2022a; Von Oswald et al. 2023; Behrouz et al. 2025a,b; Zhang et al. 2025). 더 구체적으로, 헤비안 규칙 (식 64 에서논의됨) 의 안정성을 향상시키기 위해, Irie et al. (2022a) 은 오야규칙 (Oja’ s rule)(Oja 1982) 에 기반한 OjaNet 을 다음의 재귀식으로도입하였다:

$$M_t = \alpha_t M_{t-1} + \eta_t \mathbf{v}_t \phi(\mathbf{k}_t)^\top - M_{t-1}^\top \mathbf{v}_t.$$

연상 기억 정식화 (정의 1 에서와 같이) 에서, 이 재귀식은 다음과 같이 경사하강의 한 단계로 간단히 재정식화될 수 있다:

$$M_t = M_{t-1} - \eta_t \nabla_{M_{t-1}} \tilde{\mathcal{L}}(M_{t-1}; \phi(\mathbf{k}_t), \mathbf{v}_t),$$

여기서 밑줄 친 항은 $M_{t-1}^\top \mathbf{v}_t - \mathbf{v}_t \phi(\mathbf{k}_t)^\top$ 이며,

$\tilde{\mathcal{L}}(M; \mathbf{k}_t, \mathbf{v}_t) = -2\langle M \mathbf{k}_t, \mathbf{v}_t \rangle + \|M^\top \mathbf{v}_t\|_2^2$ 이고 $\phi(\cdot)$ 는 커널 (kernel) 이다 (Irie et al. 2022a, 2025b). 이설계는 단일 뉴런 (single-neuron) 에 대해 단위 노름 (unit-norm) 제약을강제함으로써 헤비안 학습 규칙을 향상시키지만, 델타 학습 규칙에 기반한모델들보다 경험적으로 성능이 떨어지는 것으로 보고되었다 (Irie et al. 2022a). 더 표현력 있는 목적함수의 설계를 통해 델타 규칙을 한층 더향상시키기 위해, 최근 Behrouz et al. (2025b) 은 유클리드 공간을 넘어서내부 회귀 목적함수에 $L_p = \|\cdot\|_p^p$ 노름을 사용할 것을제안하였으며, 델타 규칙 및 그 변형들에 비해 긴 문맥 (long context) 과제에서 더 나은 경험적 성능과 강건성 (robustness) 을 보였다.

대다수의 학습 규칙이 온라인 갱신 메커니즘, 즉 각 상태에서 모델이 기억과현재의 (배치) 입력만 유지하면 되는 방식인 반면—오메가 규칙 (Omega rule)(Behrouz et al. 2025a) 은 과거의 (배치) 입력들의 집합 (또는 모든입력) 에 기반한 갱신 규칙을 제안한다. 더 구체적으로, 임의의 구조를 갖는 기억 M , 키와 값 $\mathbf{k}, \mathbf{v} \in \mathbb{R}^d$, 임의의 목적함수 $\tilde{\mathcal{L}}(M; \mathbf{k}_t, \mathbf{v}_t)$, 그리고커널 $\phi(\cdot)$ 가 주어졌을 때, 오메가 규칙은 다음과 같이 정의된다:

$$M_t = \alpha_t M_{t-1} - \sum_{i=t-c+1}^t \gamma_{t,i} \tilde{\mathcal{L}}(M_t; \phi(\mathbf{k}_i), \mathbf{v}_i),$$

여기서 $c \geq 1$ 은 캐시된 입력들의 지역 윈도우 (local window) 이다. $\gamma_{t,i} = 1$ 이고 c 가 전체 문맥 길이와 같은 특수한 경우에는, 위 설계의 최적해가 온라인 경우로 붕괴 (collapse) 하여, 갱신 규칙이 현재상태와 현재 입력에만 의존하게 됨에 유의하라 (Von Oswald et al. 2023). 아키텍처를 연상 기억으로, 따라서 최적화 문제로 표현하는 것에 관한 더 상세한 논의는 Behrouz et al. (2025b) 을 참고하기 바란다.

현대 시퀀스 모델의 게이팅에 관한 노트. 현대 언어 모델의 최근아키텍처 변화 중 하나는 선형 계층의 출력을 시퀀스 모델의 출력과 게이팅 (gating) 하는 것이다. 이 방법이 상당한 개선을 가져왔음에도 불구하고, 그것이 어떻게 성능을 향상시키는지 여전히 명확하지 않다. 그림 3 과 그에 해당하는 예제에서 논의했듯이, 순전파 신경망 (feedforward network) 과 현대의 순환 기억 모듈 (recurrent memory module)(예: 선형어텐션 (linear attention)(Katharopoulos et al. 2020) 또는 심층 기억모듈 (deep memory module)(Behrouz et al. 2025c)) 의 초기 상태가 메타학습 (meta-learn) 될 때 이들 사이의 주된 차이는, 기억 모듈에서 문맥 내학습 (in-context learning) 을 수행하고 그 상태를 문맥에 맞게 적응시키는 두번째 수준 (level) 이다. 이 관점에서, 기억의 초기값이 메타 학습되지 않을 때는 기억의 문맥 내 적응에만 의존하게 되며, 따라서 이 블록에는 사전학습 (pre-training) 의 지식을 저장하는 지속적 기억 시스템 (persistent memory system) 이 존재하지 않는다. 그러므로 초기 선형 트랜스포머변형들에서 흔한 경우인, 기억의 초기값이 메타 학습되지 않을 때는 선형어텐션의 게이팅이 지속적 기억 및 기억 모듈의 초기화 역할을 한다.

5.1 중첩 학습 (Nested Learning) 의 인간 뇌 관점재조명

1.1 절에서 우리는 인간 뇌의 구조가 어떻게 균일하고 (uniform) 재사용가능한지 (reusable), 그리고 우리가 딥러닝에서 새로운 아키텍처를 필요로 하는지, 혹은 현재 모델들의 이질성 (heterogeneity) 에 대한 우리의 믿음을 재검토할 필요가 있는지를 논의하였다. 앞선 절들에서 우리는, 신경망의 최적화 과정뿐만 아니라 신경 아키텍처도 중첩된 그리고/또는 병렬적인 최적화 문제들의 집합으로 정식화될 수 있으며, 여기서 기억 구조는 순전파 계층 (예: 심층 MLP, 선형 계층 등) 이고 목적함수는 경사 하강이나 뉴턴 방법 (Newton's method) 으로 최적화됨을 관찰하였다.

이 관점에서, 현대 아키텍처는 인공 뉴런들 (즉, 선형 또는 심층 순전파신경망) 의 집합이며, 각 뉴런 그룹은 자신만의 내부 목적함수와 그에 따른 갱신 메커니즘을 갖는다. 이를 위해, 간단한 예로서 그림 3 의 AdaTransformer 를 다시 상기해 보자: 입력 시퀀스로 $X = \{\mathbf{x}_i\}_{i=1}^T$ 가 주어졌을 때, 블록의 출력은 다음과 같이 계산된다 (단순화를 위해 $\text{MLP}(\cdot) = \cdot W_{\text{MLP}}$ 로 가정하고 정규화는 제거한다):

$$\begin{aligned} \mathbf{k}_t &= \mathbf{x}_t W_{\mathbf{k}}, \quad \mathbf{v}_t = \mathbf{x}_t W_{\mathbf{v}}, \quad \mathbf{q}_t = \mathbf{x}_t W_{\mathbf{q}}, \\ \mathbf{y}_{\text{attn},t} &= \text{Attn}(\mathbf{k}_t, \mathbf{v}_t, \mathbf{q}_t), \quad \mathbf{y}_{\text{block},t} = \mathbf{y}_{\text{attn},t} W_{\text{LinAttn},t}, \end{aligned}$$

여기서

$$W_{\text{LinAttn},t} = W_{\text{LinAttn},t-1} + \mathbf{v}_t \mathbf{k}_t^\top,$$

가장 낮은 주파수 수준 (lowest frequency level) 은 $W_{\mathbf{k}}, W_{\mathbf{v}}, W_{\mathbf{q}}$ 의 최적화를 담당하며, 이들은 모두 순전파 신경망이고 따라서 균일하다. 어텐션 자체도 회귀목적함수의 비모수적 (non-parametric) 행렬값 해 (matrix-valued solution) 이며, 이는 다시 그 구조가 인공 뉴런들 (즉, 매개변수들) 의 행렬임을 확인시켜 준다. 마지막으로, Linear Attention++ 구성 요소는 선형함수 클래스에 걸쳐 매핑들의 내적 유사도 (dot-product similarity) 를 최적화하는 것과 동등하다. 따라서 모든 매개변수는 행렬값이거나 심층순전파 계층이며, 이는 아키텍처 구성 요소들 간의 유일한 차이가 그들의 수준 (level), 목적함수, 그리고/또는 학습 갱신 규칙임을 의미한다.

현대 딥러닝 모델은 균일하고 재사용 가능한 구조를 갖는다 신경학습 모듈은 순전파 신경망들의 집합으로 구성되며, 각각은 서로 다른 수준과 시간 척도 (time scale) 에서 최적화

된다. 그러나 우리가 딥러닝 아키텍처에서 관찰하는 이질성은, 이 새로운 NL의 축 (axis)에 대한 관점의 결여에서 비롯되며, 그 결과 최적화 문제의 해 (solution) 만을 관찰하게 되어 딥러닝 아키텍처의 착시 (illusion)를 일으킨다.

6 핵심 요약 및 혼한 용어의 재조명

앞선 절들에서 우리는 중첩 학습 (nested learning)의 개념과, 인기 있는 최적화기 및 아키텍처와 같은 신경망의 잘 알려진 기존 구성 요소들이 어떻게 NL 패러다임에 속하는지를 논의하였다. 이 절에서는 핵심 요약들, 서로 다른 개념들 간의 연결, 그리고 혼한 용어들에 대한 NL 관점의 함의를 논의한다.

기억과 학습 (Memory and Learning). 오랫동안 기계 학습 모델에서 기억은 그 매개변수와 다른 구성 요소들 사이에 명확한 구분이 있는 별개의 블록으로 취급되어 왔다. 그러한 설계들은 흔히 단기 그리고/또는 장기 기억 블록을 가정하는데, 단기 기억은 지역적 문맥 (local context)을 담당하는 반면, 장기 기억은 모델 내 지속적 지식 (persistent knowledge)의 저장소이다. 그러나 인간 뇌에서 기억은, 단기 또는 장기 기억을 독립적으로 담당하는 명확히 알려진 구성 요소 없이, 분산되고 상호 연결된 시스템으로 간주된다. NL에서 우리는 신경심리학 (neuropsychology) 문헌의 기억과 학습에 대한 공통 용어에 기반을 두며, 이는 다음을 나타낸다: 기억은 입력에 의해 야기된 신경 갱신 (neural update)이며, 따라서 학습은 유용한 기억을 획득하는 과정이다 (Okano et al. 2000). 이 관점에서, 신경 학습 모듈의 어떤 수준에서든 경사 하강 (또는 다른 최적화 알고리즘)에 의한 어떠한 갱신도 기억의 한 형태로 간주된다. 흥미롭게도, 경사 하강이 (자기 참조적) 연상 기억 ((self-referential) associative memory)이라는 4.1 절의 우리 발견은 이 용어와 일치한다. 나아가, 이 용어에 기반하면, 연속체 기억 시스템 (continuum memory system)에서 신경 갱신들은 서로 다른 주파수에서 적용되고 따라서 기억들이 서로 다른 시간 척도로 저장되어, 파국적 망각 (catastrophic forgetting)에 대해 더 강건한 기억 관리로 이어진다.

중첩 학습 관점에서의 기억과 학습: 기억은 고립된 시스템이 아니며 매개변수 전반에 분산되어 있다. 특히, 입력에 의해 야기된 어떠한 갱신도 신경망에 저장된 기억이며, 그러한 기억들을 효과적으로 저장·부호화하고 일반적으로 획득하는 과정을 학습과정 (learning process)이라고 한다.

모델의 매개변수에 관한 일반적 노트 (A General Note on Parameters of a Model). 모델의 매개변수는 지식 저장, 내부 계산, 적응성의 단위를 형성하는 핵심 구성 요소 중 하나이다. 지난 수십 년 동안, 기계 학습 모델 아키텍처의 매개변수 중 (일부만이) 학습 가능한 개체 (learnable entity)로 지칭되어 왔는데, 이는 주로 그것들이 우리가 훈련 데이터에 대해 직접적이고 의도적으로 최적화해 온 유일한 구성 요소이기 때문이다. NL 관점에서, 그러한 매개변수들은 가장 낮은 주파수 수준 (즉, 가장 높은 수준)에 놓여있으며 모든 (배치) 샘플마다 갱신된다. 그러나 이들은 모델의 내부 계산, 지식 저장, 적응성에 기여하는 유일한 매개변수가 아니다. 앞서 4.2 절에서 논의했듯이, 모멘텀 (momentum)이 그러한 경우의 한 예인데, 그 매개변수들은 시간에 따라 (경사 하강으로) 갱신되며 지금까지의 모델 손실 지형 (loss landscape)에 관한 지식을 저장한다. 그러한 정보는 모델이 연속적으로 학습할 때 중요한데, 이는 효과적인 해를 찾기 위해 최적화기가 손실 지형의 전역적 속성에 대한 더 많은 정보를 가져야 하기 때문이다. 그러한 경우의 또 다른 예는 순환 신경망 (recurrent neural network)의 기억 (또는 은닉 상태 (hidden state))이다. 그러한 매개변수들은 가장 낮은 주파수 수준에서 직접 최적화되지는 않지만, 현재 문맥에 관한 중요한 지식을 저장한다. 문맥이 바뀌면, 이 매개변수들에 압축된 지식은 이 수준들 사이의 지식 전달 (knowledge transfer)의 결여로 인해 제거된다.

모델은 우리가 알던 것보다 더 많은 매개변수를 갖는다: 신경 학습 모듈의 매개변수는 사전 학습 수준에서 최적화되는 것들에 국한되지 않는다; 모델의 NL 표현에 나타나는 모든 매개변수가 그 성과 표현력 (expressivity)에 기여한다.

뉴런당 더 많은 계산 (More Computations per Neuron). NL 개념에 대한 혼한 오해 중 하나는 모델 설계와 여러 수준의 적층 (stacking)을 CMS(연속체 기억 시스템) 경우로 제한하는 것이다. 일반

적으로, 수준을적층하는 것은 모델이 계산의 깊이를 향상시키고 또한 가장 낮은 주파수수준의 각 매개변수당 더 많은 내부 계산을 수행하는 데 도움을 줄 수 있다. 그러한 설계의 한 예는 Muon 최적화기와 $\text{NewtonSchulz}_k(\cdot)$ 연산 (식 42 – 44 참조) 인데, 우리는 이것이 내부 최적화 과정과 동등함을 보였다. 이 설계에서, 모멘텀 갱신의 각 단계마다 우리는 경사 (gradient) 를직교 공간 (orthogonal space) 으로 매핑하는 방법을 학습하기 위해 k -단계의 내부 과정이 필요하다.

문맥 내 학습 (In-Context Learning). 이 논문 전반에 걸쳐 우리는 “문맥 내 학습” 의 가장 일반적인 정의를 사용하며, 그것을 주어진 문맥에적응하고 그로부터 학습하는 모델의 능력으로 지칭한다. NSAM 의 정의에 따라, 각 블록 또는 수준은 자신만의 문맥 흐름 (context flow) 을 가지며, 따라서 그문맥에 대한 어떠한 신경 갱신이나 적응도 문맥 내 학습의 한 형태로간주된다. 트랜스포머의 인기와 더불어 문맥 내 학습이 처음 연구된모델이라는 사실로 인해, 문맥 내 학습의 개념은 때때로 출력을 전체 문맥에조건화 (conditioning) 하는 것으로 지칭된다. 문맥 내 학습의 일반적 용어를고려하면, 이 정식화는 우리가 비모수적 문맥 내 학습 (non-parametric in-context learning) 이라고 지칭한, 문맥 내 학습의 인스턴스 중 하나일뿐이다. 그러나 일반적으로, 순환 모델의 기억은 문맥 내 학습을 수행하고있으며, 여기서 출력은 압축된 문맥 (compressed context) 에 조건화된다. 따라서 NL 관점에서, 모든 수준은 문맥 내 학습을 수행하지만, 자신만의 학습갱신 및 최적화 과정과 함께 자신만의 문맥 흐름에 대해서 수행한다.

이 정의에 기반하여, 문맥 내 학습은 그것의 NL 표현으로부터 투명하게드러나는 모델의 능력이며, 그 자체로 창발적 (emergent) 특성이 아니라 신경학습 모듈의 NL 표현에 여러 수준을 가지는 것의 직접적 결과이다. 이것이 ICL 이 창발적 특성이라는 이전 주장들 (Brown et al. 2020; Singh et al. 2023) 과 모순되어 보일 수 있지만, ICL 과제에서 모델의 좋은 성능이강력한 저주파수 수준 (low-frequency level) 을 요구하여 고주파수 수준 (high frequency level) 이 빠르게 적응할 수 있게 한다는 점에 유의할 만하다. 모델이 잘 훈련되지 않았을 때는, 고주파수 수준이 문맥으로부터 학습하는것을 홀로 감당해야 한다. 이러한 설정은 성능 저하로 이어질 수 있는데, 이는 주로 고주파수 매개변수들이 수렴하기에 충분한 데이터가 문맥에 없을수 있기 때문이다.

(테스트 시점) 학습/기억화 ((Test-Time) Learning/Memorization). 최근 테스트 시점 훈련 (test time training)(Sun et al. 2024; Wang et al. 2025) 또는 테스트 시점기억화 (test time memorization)(Behrouz et al. 2025b) 의 개념은 강력한시퀀스 모델을 설계하기 위한 백본 (backbone) 프레임워크로서 인기를 얻었다. 이러한 프레임워크에서는, 문맥이 주어졌을 때 새로운 구성요소/블록이 학습 규칙과 목적 함수를 사용하여 문맥을 자신의매개변수로 압축하고자 한다. 이 정식화에서 문맥이 제거되면 획득된 문맥 내지식은 그와 함께 사라진다. 앞부분에서도 논의했듯이, 이 갱신 메커니즘과학습 과정은 사실 “모수적 문맥 내 학습 (parametric in-context learning)” 의한 인스턴스이다:

테스트 시점 훈련/기억화는 문맥 내 학습의인스턴스이다 흔히 테스트 시점 훈련 및 테스트 시점 기억화라고 지칭되는개념들은 사실 모수적 문맥 내 학습의 인스턴스이며, 여기서 획득된 문맥 내지식은 현재 문맥이 제거되면 지속되지 않는다.

특히, 연속 학습 (continual learning) 설정으로 이동할 때는 테스트 시점이나훈련 시점이 존재하지 않으므로, (모수적) 문맥 내 학습을 테스트 시점학습/기억화라고 지칭하는 것은 오해를 불러일으킬 수 있다.

사전 학습과 테스트 시점 (Pre-training and Test Time). 중첩 학습관점에서, 가장 낮은 주파수 수준 (즉, 가장 높은 수준) 은 흔히 사전학습이라고 지칭되는 학습 단계에 해당한다. 따라서 사전 학습은 여러 수준중 하나이며, 자신만의 문맥 흐름 (즉, 사전 학습 데이터셋), 목적함수 (예: 다음 토큰 예측 (next token prediction)), 그리고 최적화 과정 (예: AdamW) 을갖는다. 이에 따라, 우리는 사전 학습을 문맥이 전체 사전 학습 데이터인문맥 내 학습의 가능한 인스턴스 중 하나로 해석할 수 있다.

사전 학습은 초대형 문맥 길이 (Ultra-Large Context Length) 를 갖는문맥 내 학습이다: NL 관점에서 사전 학습은 문맥 내 학습의 가능한인스턴스 중 하나일 뿐이며, 여기서 문맥은 전체 사전 학습 데이터이다. 모델에서 훈련 시점과 테스트 시점의 구분은, 가장 높

은 주파수 수준 (예: 트랜스포머의 문맥) 에서 저주파수 수준 (즉, 사전 학습) 으로의 지식 전달과정을 단절시킨 결과이다.

이 정식화와 관점은, 우리가 사전 학습 패러다임에서 데이터/세계와 연속적으로 상호작용하고 학습할 수 있는 모델들 (예: Sutton (2025)) 로 전환할 때 특히 중요하다:

연속 학습 (Continual Learning). NL 관점에서 모델에 대한 각 훈련단계는 저주파수 수준 중 하나로 정의되며, 설계상 우리는 한 수준에서의 데이터 처리를 중단하거나 (예: “사전 학습의 종료 (End of Pre-training)”), 다른 수준으로 어떠한 지식 전달도 없이 이를 계속할 수도 있다 (예: 트랜스포머에서의 관습적인 문맥 내 학습 정식화). 이에 따라, 어떤 기계학습 모델이든, 그것이 사전 학습 중이든 테스트 시점에 있든 관계없이, 연속학습을 수행하고 있는데, 이는 데이터 샘플이 주어졌을 때 출력을 제공하기 위해 내부 계산을 수행해야 하기 때문이다. 그러나 그 학습으로부터의 지식은, 주로 수준들 사이의 지식 전달의 결여로 인해, 더 지속적인 수준으로 지속되거나 전달되지 않을 수 있다.

신경 학습 모듈에는 훈련 시점이나 테스트 시점이 없다: 신경 학습 모듈에게는 훈련 시점과 테스트 시점 사이에 경계나 명확한 구분이 없다. 모델은 오직 두 가지 다른 상태만을 경험한다: 정보를 입력으로 받을 때, 혹은 고립된 학습 시스템일 때.

우리는 또한 아키텍처 설계에서의 혼란 용어 몇 가지를 다음과 같이 재조명한다:

기존 아키텍처 백본과 하이브리드 모델 (Existing Architectural Backbones and Hybrid Models). 앞서 5.1 절에서 논의했듯이, NL 관점에서 모든 현대 아키텍처는 균일하며 사실상 자신만의 문맥 흐름과 최적화 문제에 기반하여 훈련되는 순전파 계층 (선형 또는 비선형 MLP 블록) 이다. 딥러닝 관점에서 모델을 볼 때, 우리는 그러한 최적화 문제의 최종 해 (즉, 회귀손실에 대한 비모수적 해가 아니라 어텐션) 를 보게 되며, 이는 별개의 균일하지 않은 아키텍처를 갖는다는 착시로 이어진다.

순환 모델은 MLP 블록을 대체하고 있다: NL 관점에서, (심층 또는 선형 기억) 순환 모델은 그 내부 계산에 새로운 수준이 추가된 MLP 블록이다. 이에 따라, 기존 하이브리드 아키텍처는 일부 MLP 블록에 새로운 계산 수준을 추가한 관습적 트랜스포머 모델로 볼 수 있다.

기존 아키텍처를 소위 “사전 학습” 단계 동안 이해하는 또 다른 중요한 측면은, 그들이 한 수준에서 다른 수준으로 지식을 어떻게 전달하는지를 이해하는 것이다.

문맥 내 학습으로부터의 지식 전달 (Knowledge Transfer from In-Context Learning) 현대의 심층 및 선형 순환 모델이 모두 연상 기억관점을 사용하여 통합되지만, 기존 인스턴트들 사이에는 여전히 중요한 차이가 있다: Titans, Atlas, Miras, TTT 와 같은 심층 기억 모듈들은 기억의 초기 상태를 메타 학습함으로써 자신의 고주파수 수준에서 저주파수 수준으로의 지식 전달을 활용하는 반면, 대부분의 선형 기억 순환 모델은 그들의 수준들 사이에 지식 전달 과정이 없다.

상호 연결된 시스템으로서의 신경 학습 모듈 (Neural Learning Module as an Inter-Connected System). NL 의 핵심 메시지 중 하나는 신경 학습 모듈이 상호 연결된 시스템 (inter-connected system) 이라는 사실이며, 이는 각 구성 요소의 설계가 다른 부분들의 설계에 상당한 영향을 미칠 수 있음을 의미한다. 이 사실은 모든 구성 요소가 조화를 이루며 함께 작동하는 신경 학습 모듈을 어떻게 적절히 설계할 수 있는지를 더 잘 이해하기 위한 후속 및 미래 연구를 동기부여한다. 그러한 예로서, 최적화기의 문맥 (즉, 경사) 은 아키텍처 구성 요소에 의해 생성된다. 따라서 서로 다른 아키텍처는 생성된 경사 패턴에서 서로 다른 특성을 보일 수 있으며, 따라서 하나의 최적화기가 모든 아키텍처에 최선의 선택이 아닐 수 있다 (Zhang et al. 2024b).

아키텍처는 최적화기를 위한 문맥을 생성한다 신경 학습 모듈은 상호 연결된 시스템이며, 여기서 아키텍처는 최적화기를 위한 문맥 (즉, 경사) 을 생성한다. 따라서 경사의 적절한

기억 관리 (즉, 최적화 알고리즘) 는 아키텍처의 선택에 의존한다. 미래에, 모델을 신경 학습 모듈로 볼 때, 우리는 이 상호 연결된 시스템이 완벽한 조화를 이루며 작동하도록 아키텍처에 특화된 최적화기 (architecture specific optimizer) 를 설계할 필요가 있다.

최적화기 대 학습된 최적화기 (Optimizers vs. Learned Optimizers). 마지막으로, 우리는 모멘텀, 경사 하강, 그리고/또는 다른 경사기반 최적화기에 대한 우리의 정식화가, 이들이 데이터와 경사를 자신의 매개변수로 압축하고자 하는 연상 기억 모듈임을 보여준다는 점을 강조하고자한다. 그러한 갱신 및 압축 과정은 경사 하강에 기반하며 따라서 학습된 최적화기 (learned optimizer) 의 학습 과정과 매우 유사한 본질을 갖는다. NL 관점에서, 바닐라 최적화기 (vanilla optimizer) 와 학습된 최적화기는 모두같은 개념의 인스턴스이지만 서로 다른 주파수와 문맥 흐름을 갖는다: 학습된 최적화기의 매개변수는 가장 낮은 주파수 수준에 위치하지만 (즉, 다른 [것들과 함께 훈련되고 최적화되기 위해] 사전 학습 (pre-training) 에서의 파라미터), 바닐라 옵티마이저의 파라미터들은 자기 자신의 레벨에 위치하며 따라서 자신만의 그래디언트 흐름 (gradient flow) 을 갖는다.

7. 연속체 다중 시간척도 메모리 시스템 (Continuum Multi-Timescale Memory System)

기존의 아키텍처 백본 (architectural backbone) 은 (1) 시퀀스 길이에 걸쳐 정보를 능동적으로 융합하는 역할을 하는 작업 기억 (working memory) 모듈 (예: 어텐션 (attention)) 과, (2) 특징 (feature) 들에 걸쳐 정보를 융합하며 사전 학습 단계의 지속 기억 (persistent memory) 혹은 지식 저장소 (knowledge storage) 역할을 하는 피드포워드 계층 (feed-forward layer, 예: MLP) 으로 구성된다. NL(Nested Learning) 관점에서 보면, 사전학습은 학습 모듈의 가장 바깥쪽 레벨이 자신의 제한된 컨텍스트 흐름 (context flow) 에 걸쳐 갱신되는 단계이다. 따라서 연속 (continual) 설정에서 이러한 사전 학습 단계는 시간이 지나도 드물게만 갱신되며, 그에 대응하는 지식 저장소 또한 시간이 지나도 드물게만 갱신되어야 한다. 이러한 직관에 근거하여, 우리는 전통적인 장기/단기 기억 시스템의 관점을 확장하여 각 레벨 (주파수 영역 (frequency domain)) 마다 하나의 지식 저장 피드포워드를 두는 것을 제안한다.

7.1 연속체 메모리 시스템 (Continuum Memory System, CMS)

주파수 (frequency) 의 정의 (정의 2) 가 주어졌을 때, 연속체 메모리 시스템 (CMS) 은 MLP 블록들의 사슬 (chain), 즉

$$\text{MLP}^{(f_1)}(\cdot), \dots, \text{MLP}^{(f_k)}(\cdot)$$

로 형식화되며, 각각은 청크 크기 (chunk size) $C^{(\ell)} := \max_{f_i} \ell C$ 와 연관된다. 이때 입력 $x = \{x_1, \dots, x_T\}$ 가 주어지면 사슬의 출력은 다음과 같이 계산된다 (명료함을 위해 정규화 (normalization) 는 무시한다):

$$y_t = \text{MLP}^{(f_k)}(\text{MLP}^{(f_{k-1})}(\dots \text{MLP}^{(f_1)}(x_t))),$$

여기서 ℓ 번째 MLP 블록의 파라미터, 즉 $\theta^{(f_\ell)}$ 는 매 $C^{(\ell)}$ 스텝마다 갱신된다:

$$\theta_{i+1}^{(f_\ell)} = \theta_i^{(f_\ell)} - \begin{cases} \eta_t^{(\ell)} \sum_{t=i-C^{(\ell)}}^i f(\theta_t^{(f_\ell)}; x_t) & \text{if } i \equiv 0 \pmod{C^{(\ell)}}, \\ 0 & \text{otherwise.} \end{cases}$$

여기서 $\eta_t^{(\ell)}$ 는 $\theta^{(f_\ell)}$ 에 대응하는 학습률 (learning rate) 이며, $f(\cdot)$ 는 임의의 옵티마이저의 오차 성분 (error component) 이다 (예: 경사 하강법 (gradient descent) 에서의 $\nabla \mathcal{L}(\theta_t^{(f_\ell)}; x_t)$). 통상적

인 트랜스포머 (Transformer) 블록 (Vaswani et al. 2017) 은 이 형식화의 특수한 사례로서, $k = 1$ 이고 갱신 주파수가 0 인 경우이다. 여기서 $\mathcal{L}(\cdot)$ 의 목적함수는 당면한 과제에 대한 선택된 목적함수 임에 유의하라. 예를 들어 언어 모델링 (language modeling) 에서는 다음 토큰 예측 (next token prediction) 목적함수이다. 식 71 이 중요한 해석을 제공한다는 점이 주목할 만하다: 파라미터 $\theta_t^{(f_\ell)}$ 는 자신의 컨텍스트를 자신의 파라미터로 압축하는 역할을 하며, 따라서 그 컨텍스트의 추상적 지식 (abstract knowledge) 의 대표자가 된다.

3.3 절에서 논의했듯이, 서로 다른 레벨은 서로 다른 지식 전이 (knowledge transfer) 과정을 가질 수 있다. 따라서 위의 형식화가 서로 다른 레벨, 즉 서로 다른 주파수를 갖는 메모리 시스템들의 스펙트럼을 제안하기는 하지만, 그들 간의 연결은 설계에 따라 달라질 수 있다. 다음에서 우리는 몇 가지 가능한 변형 (variant) 들을 논의한다:

중첩 연속체 메모리 시스템 (Nested Continuum Memory Systems). 첫번째 변형은 완전히 중첩된 (fully nested) 연속체 메모리 시스템으로, 레벨 $s + 1$ 의 MLP 블록의 초기 상태 (initial state) 가 레벨 s 에서 메타학습 (meta-learned) 된다. 이 설계는 더 높은 차수의 인컨텍스트학습 (in-context learning) 능력을 가능하게 하며, 각 레벨은 자신만의 컨텍스트 흐름을 가지고 컨텍스트가 끝난 후 재 초기화 (re-initialized) 된다. 더 구체적으로, 임의의 $1 \leq s \leq k$ 에 대해,

$$\theta_0^{(f_{s+1})} = \arg \min_{\Phi} \mathbb{E}_{\mathcal{T} \sim \mathcal{C}^{(s)}} \ell(\Theta, \mathcal{T}; \Phi),$$

여기서 $\mathcal{C}^{(s)}$ 는 s 번째 레벨의 MLP 블록의 컨텍스트 길이이다. 이 설계에 따라, 각 블록의 최적화 과정이 끝날 때 (즉 $\lceil C^{(s)} / C^{(s+1)} \rceil$ 스텝 후) 메모리의 값은 $\theta_0^{(f_{s+1})}$ 로 재초기화된다. 각 블록이 자기 자신의 레벨에서 갖는 갱신 메커니즘은 변하지 않음에 유의하라 (즉 식 71).

순차적 연속체 메모리 시스템 (Sequential Continuum Memory Systems). 두 번째 변형에서는 MLP 블록들이 순차적으로 배치되며 (즉 레벨 s 의 MLP 블록의 출력이 레벨 $s + 1$ 의 MLP 블록의 입력이 됨), 또한 MLP 블록들의 초기 상태는 모두 가장 낮은 주파수 레벨에서 역전파 (backpropagation) 를 통해 연결된다. 임의의 $1 \leq s \leq k$ 에 대해,

$$\theta_0^{(f_s)} = \arg \min_{\Phi} \mathbb{E}_{\mathcal{T} \sim \mathcal{C}^{(1)}} \ell(\Theta, \mathcal{T}; \Phi),$$

여기서 $\mathcal{C}^{(1)}$ 은 가장 낮은 주파수 레벨의 MLP 블록의 컨텍스트 길이이다. 모든 메모리의 초기 상태가 가장 낮은 주파수에서 메타학습되므로, 모든 구성 요소의 가장 지속적인 지식은 동일한 컨텍스트 흐름의 압축 (compression) 이 된다.

독립적 (헤드별) 연속체 메모리 시스템 (Independent (Head-wise) Continuum Memory Systems). 이 변형에서는 식 73 의 지식 전이 과정은 유지하되, 식 70 의 출력 계산을 변경한다. 이전의 형식화는 메모리 시스템을 블록들의 시퀀스로 설계하여 그들의 입력/출력을 서로 종속시켰지만, 이 변형은 서로 다른 컨텍스트 길이를 갖는 독립적인 블록들을 사용한 후 이들을 집계 (aggregation) 과정을 통해 결합한다:

$$\mathbf{y}_t = \text{Agg} \left(\text{MLP}^{(f_k)}(\mathbf{x}_t), \text{MLP}^{(f_{k-1})}(\mathbf{x}_t), \dots, \text{MLP}^{(f_1)}(\mathbf{x}_t) \right).$$

위의 $\text{Agg}(\cdot)$ 는 모든 입력을 집계하여 출력을 계산하는 임의의 함수이다. 예를 들어, 하나의 직관적이고 단순한 설계 선택은 입력들의 학습 가능한 가중합 (learnable weighted sum) 을 사용하는 것이다.

CMS 설계가 연속 학습 (Continual Learning) 에 도움이 되는 이유. CMS 의 설계에 기반하여 던질 수 있는 정당한 질문은 다음과 같다: CMS 는 왜, 그리고 어떻게 더 긴 컨텍스트 길이 및 일반적으로 연속 학습에 도움이 될 수 있는가? 여기서 우리는 이 질문에 대한 간단한 답을 제공한다: CMS 의 MLP 블록들을 모델 지식의 저장소로 볼 때, 파국적 망각 (catastrophic forgetting) 은 우리가 한 블록을 갱신하고 그 결과로 그 파라미터에 저장되어있던 옛 지식이 망각될 때 발생할 수 있다. 그러나 CMS 설계에서는, 임의의 $1 \leq s \leq k$ 에 대해 어떤 블록 $\text{MLP}^{(f_s)}(\cdot)$ 를 갱신할 때, $\text{MLP}^{(f_s)}(\cdot)$ 로부터

잠재적으로 망각된 지식은 여전히 다른 구성 요소들, 예컨대 $s' < s$ 인 $\text{MLP}^{(f_{s'})}(\cdot)$ 에 저장되어 있다. 또한 이 경우 (즉지식이 $\text{MLP}^{(f_s)}(\cdot)$ 로부터는 이미 망각되었지만 $s' < s$ 인 $\text{MLP}^{(f_{s'})}(\cdot)$ 에는 여전히 남아 있는 경우), (그들의 초기 상태에 대한) 역전파를 통한 지식 전이가 그 지식을 다시 $\text{MLP}^{(f_s)}(\cdot)$ 로 되돌릴 (circle back) 수 있으며, 이는 시간 차원 (time dimension)을 통한 순환 (loop)을 형성하여 중요한 지식을 좀처럼 망각하지 않게 한다.

CMS는 충분히 효율적인가? 모델의 파라미터를 연속적인 방식으로 갱신할 때 흔히 제기되는 우려는 그 효율성이다. 따라서 CMS가 모델에 상당한 계산 오버헤드 (computational overhead)를 유발하는지 묻는 것은 정당한 질문이다. 이 질문에 답하기 위해, 5절에서 논의한 바를 상기하자. 현대의 순환 신경망 (recurrent neural network) 또한 자신의 파라미터의 일부 (즉 자신의 메모리 상태)를 연속적으로 갱신한다. 그러나 이러한 파라미터 갱신은 시퀀스 병렬화 (sequence parallelization)를 활용하며 또한 적은 수의 파라미터만을 갱신한다. 이를 위해, CMS에 대해 우리는 두 가지 점을 강조한다:

- CMS 설계에서는, 매 시점마다 갱신이 (자신의 주파수에 기반하여) 예정된 갱신 시점에 도달하는 블록들에 한정된다. 간단한 예로, 트랜스포머의 MLP 블록을 CMS로 대체한 모델을 생각해 보자 (이후 8절에서 이 변형을 Hope-Attention이라 부른다). 모델이 L_{layer} 개의 계층을 갖고, CMS에서 최고 주파수가 $\hat{f}$ 인 4개 레벨의 MLP 블록을 가지며, 은닉 차원 (hidden dimension)이 d_{in} 이라 하자. 평균적으로 갱신 비용은 $\mathcal{O}\left(\frac{1}{\hat{f}} \times \frac{L_{\text{layer}}}{5} \times d_{\text{in}}^2\right)$ 개의 파라미터에 대한 것이며, 이는 매 시점마다 오직 적은 수의 파라미터로만 구성된다.
- 식 71의 갱신 메커니즘은 모델의 지속 기억을 향상시키는 데 도움이 될뿐만 아니라, 더 높은 주파수 레벨에 대한 시퀀스 병렬화를 가능하게 한다. 더 구체적으로, 입력 x_i 에 대해 $i \not\equiv 0 \pmod{C^{(\ell)}}$ 일 때는 체크 내부에 순차적 과정이 존재하지 않으며, 따라서 $i \not\equiv 0 \pmod{C^{(\ell)}}$ 인 서로 다른 i 값에 대응하는 토큰들의 모든 계산은 병렬로 수행될 수 있다. 이러한 학습 알고리즘의 세부 사항은 Sun et al. (2024) 및 Behrouz et al. (2025c)의 학습 절차와 동일하다.

따라서 요약하면, CMS는 실제로 빠를 수 있는데, 이는 주로 매 시점마다 적은 수의 파라미터만을 갱신한다는 사실과 또한 그 설계가 시퀀스 병렬화를 가능하게 한다는 사실에 기인한다.

7.2 옵티마이저에서의 연속체 메모리 시스템

알고리즘 1: 다중 척도 모멘텀 Muon (Multi-scale Momentum Muon, M3)

입력: 초기 가중치 Θ_0 , 목적함수 $\mathcal{L}(\cdot)$, 학습률 $\eta > 0$, Newton-Schulz 스텝 수 T , 모멘텀 인자 $1 > \beta_1, \beta_2, \beta_3, \alpha \geq 0, \epsilon > 0$, 주파수 f ;

1: 모멘텀 초기화: $M_0^{(1)}, M_0^{(2)} \leftarrow 0, V_0 \leftarrow 0$; 2: **for** 저주파수 반복 $k = 0, 1, 2, \dots$ **do** 3: 느린 메모리 (Slow Memory): $M_t^{(2)} = M_{t-1}^{(2)} + \beta_3 \sum_{i=(k-1)f}^{kf} g_i$; 4: $O_t^{(2)} \leftarrow \text{Newton-Schulz}_T(M_t^{(2)})$; 5: **for** $t = kf + 1, kf + 2, \dots, (k+1)f$ **do** 6: 그래디언트 계산: $g_t = \nabla_{\Theta_t} \mathcal{L}(\Theta_t)$; 7: 1차 모멘텀 (First Momentum): $M_t^{(1)} = M_{t-1}^{(1)} + \beta_1 g_t$; 8: 2차 모멘텀 (Second Momentum): $V_t = V_{t-1} + \beta_2 g_t^2$; 9: $O_t^{(1)} \leftarrow \text{Newton-Schulz}_T(M_t^{(1)})$; 10: $\Theta_t \leftarrow \Theta_{t-1} - \eta \frac{O_t^{(1)} + \alpha O_t^{(2)}}{\sqrt{V_t + \epsilon}}$; 11: **end for** 12: **end for**

개념 증명 (proof of concept)으로서, 그리고 서로 다른 컨텍스트 흐름에서 CMS의 효과성을 뒷받침하기 위해, 이 절에서 우리는 다중 척도 모멘텀/메모리 Muon (Multi-scale Momentum/Memory Muon, M3) 옵티마이저를 제시한다. 특히 우리는 NL의 연상 기억 (associative memory) 관점을 사용하여, 최근의 그래디언트를 효과적으로 압축할 뿐만 아니라 먼 과거의 그래디언트에 관한 정보를 통합하는 능력도 갖춘 옵티마이저를 설계하고자 한다. 부록 B(식 101)에서 우리는 Adam 옵티마이저가 어떻게 연상 기억의 한 사례인지를 논의하는데, 여기서 그래디언트는 그 시점까지의 분산 (variance)으로 매핑된다. 4.3절에서 논의한 옵티마이저의 장문 컨텍스트 (long-context) 능력의 필

요성에 관한 논의에 따라, 우리는 먼저 식 102의 H 항의 단순한 연상 기억 형식화를 우리의 CMS(독립 변형, 식 74)로 대체하여, 두 레벨의 메모리 시스템을 만드는데, 이 메모리들을 $M^{(1)}$ 및 $M^{(2)}$ 라 부른다:

$$\begin{aligned} M_t^{(1)} &= M_{t-1}^{(1)} + \beta_1 g_t, \\ M_t^{(2)} &= M_t^{(2)} - \beta_2 \begin{cases} \sum_{i=t-\hat{C}}^t g_i & \text{if } t \equiv 0 \pmod{\hat{C}} \\ 0 & \text{otherwise,} \end{cases} \end{aligned}$$

여기서 $\hat{C}$ 는 저주파수 모멘텀 항을 갱신하는 체크 크기이다. 마지막으로, 모멘텀 항들을 집계하기 위해 (식 74에서의 $\text{Agg}(\cdot)$ 의 선택), 우리는 $M_t^{(2)}$ 의 계수로서 파라미터 $\alpha > 0$ 을 사용하는 단순한 가중합을 사용한다. 그래디언트를 적절한 계량 공간 (metric space)으로 매핑하는 $\text{Newton-Schulz}_T(\cdot)$ 의 중요성에 관한 우리의 논의 (4.2 절 및 식 43 참조)에 따라, Muon(Jordan et al. 2024)을 쫓아, 우리는 모멘텀항들을 가중합으로 집계하기 전에 그 출력에 $\text{Newton-Schulz}_T(\cdot)$ 를 적용한다. 이는 연상 기억이 자신의파라미터를 적절한 방향으로 갱신함으로써 자신의 용량 (capacity)을 더 잘관리하도록 돕는다. M3의 의사코드 (pseudocode)는 그림 1에 있다. 요약하면, M3는 Adam(Kingma et al. 2014a), Muon(Jordan et al. 2024), 그리고 우리의연속체 메모리 시스템 (CMS)의 결합이라고 말할 수 있다.

특히 이 옵티마이저는 CMS의 설계를 뒷받침하기 위한 개념 증명으로 설계되었다. 그러나 M3 옵티마이저 자체는 계산 오버헤드를 겪을 수 있으며, 따라서 더 큰 네트워크로 확장할 때 어려움에 직면할 수 있다 (그림 12 참조). 나아가, CMS에 기반한 M3 설계의 요점은 더 긴 컨텍스트를 얻기 위해 메모리의 갱신을 지연시키는 것임을 주목할 만하다. Devan Selvaraj (2025) 및 Pagliardini et al. (2025)의 연구와 같은 유사한 연구들은 여러 모멘텀항을 사용하면서도, 이러한 옵티마이저의 장문 컨텍스트 모멘텀에 대해 추가적인 모멘텀 항에 대한 제어된 학습률 (controlled learning rate)을 사용하여 접근한다.

7.3 애드혹 레벨 스택킹 (Ad-hoc Level Stacking): 사전 학습된 모델로 CMS 초기화하기

그림 3에 관한 논의에서, 우리는 메모리 모듈의 초기 상태가 더 낮은 주파수 레벨에서 최적화되며, 따라서 이를 바닐라 트랜스포머 아키텍처 (Vaswani et al. 2017)의 MLP 블록으로 해석할 수 있음을 관찰했다. 따라서 자연스러운 질문은, 우리가 사전 학습된 모델을 활용하여 CMS 블록을 초기화할 수 있는지의이다. NL의 중요한 장점 중 하나는

그림 5: Hope 아키텍처 백본과 트랜스포머의 비교 (명료함을 위해 정규화 및 잠재적인 데이터 종속 (data-dependent) 구성 요소는 제거되었다).

서로 다른 레벨에서 파라미터를 보고 수정할 수 있는 유연성이다. 즉, 각 레벨이 자신만의 컨텍스트 흐름과 최적화 과정을 갖기 때문에, 각 레벨의 파라미터를 독립적으로 초기화하여 모델이 해당 레벨의 컨텍스트 흐름에 더 빨리 적응하도록 도울 수 있다. 이를 위해, 이 절에서 우리는 한 레벨의 파라미터를 모델의 사전 학습된 가중치로 초기화하는 것을 제안한다. 더 형식적으로, $\{\text{MLP}^{(f_i)}(\cdot)\}_{i=1}^k$ 를 갖는 CMS와 사전 학습된 MLP 블록들의 집합 $\{\text{MLP}_{\text{pre-trained}_i}(\cdot)\}_{i=1}^k$ 가 주어졌을 때, 우리는 서로 다른 레벨에서 $\{\text{MLP}^{(f_i)}(\cdot)\}_{i=1}^k$ 를 갱신하기 위해 식 71을 사용한다. 그러나 우리는 $\{\text{MLP}_{\text{pre-trained}_i}(\cdot)\}_{i=1}^k$ 의 학습된 파라미터를 CMS 블록의 초기 상태로 사용한다: $\text{MLP}_0^{(f_i)}(\cdot) = \text{MLP}_{\text{pre-trained}_i}(\cdot)$.

이 초기화가 작동해야 하는 이유는? NL에서, 두 레벨 사이에 지식전이 과정이 있을 때, 더 높은 주파수 레벨은 더 낮은 주파수 레벨에 저장된 지식을 활용하여 자신만의 컨텍스트 흐름에 더 빨리 적응할 수 있다. 그러나 더 높은 주파수 레벨의 내부 학습률 (internal learning rate)은 적응성 (adaptability)에 대한 모델의 용량을 제어할 수 있다. 즉, 모든 블록이 사전 학습된 MLP 블록으로 초기화된 위의 경우를 생각해 보자. $\eta_t^{(\ell)} \rightarrow 0$ 으로 설정하면 갱신된 메모리 블록들이 그들의 초기 상태에 가깝게 유지되어, 적응 없이 사전 학습된 블록을 직접 사용하는 결과를 낳는다. 이후 9 절에서, 우리는 이 방법을 사용하여 사전 학습된 트랜스포머 아키텍처를 Hope의 설정으로 적응시킨다.

8. Hope: 연속체 메모리를 갖춘 자기 참조적 학습 모듈 (A Self-Referential Learning Module with Continuum Memory)

앞서 5.1 절에서 논의했듯이, 중첩 학습 (nested learning) 에서의 아키텍처는 균일 (uniform) 하다. 즉, 각각이 자신만의 컨텍스트, 갠신 주파수, 내부목적함수 (internal objective) 를 갖는 피드포워드 신경망 블록들의 집합이다. 시퀀스 모델 (sequence model)—입력 시퀀스의 토큰들에 걸쳐 정보를 융합하며 흔히 가장 높은 갠신 주파수를 갖는 블록들을 가리키는 일반적 용어—은 모델의 메모리 관리 및 인컨텍스트 학습 능력에 있어 핵심적인 구성요소이다. 앞선 논의에 이어

섹션 5에서 논의했듯이, 현대 시퀀스 모델은 연상 메모리 (associative memory) 로 볼 수 있으며 따라서 중첩 (nested) 최적화 문제이다. 이러한 관점에서, 전역 소프트맥스 어텐션 (global softmax attention) 또는 그것의 더 표현력 있는 고차 (higher-order) 변형들은, (지역) L_2 -회귀목적함수를 Nadaraya-Watson 추정량 (Fan 2018; Zhang et al. 2022) 으로 최적화하는 비모수적 (non-parametric) 해이기 때문에, 무한대의 업데이트빈도 (frequency of update) 를 갖는 완벽한 메모리 (모든 과거 토큰을 캐싱하도록 강제하는) 이다 (식 62 참조). 따라서, 유사한 목적함수에 대한 모수적 (parametric) 해 (예: 현대 RNN) 는, 파라미터 탐색 공간이 동일할 때 (즉, 행렬값 메모리 (matrix-valued memory)), 모델 크기와 데이터가 확장될 때 소프트맥스 어텐션을 능가할 것으로 기대되지 않는다. 이를 위해, 그리고 강력한 시퀀스 모델을 설계하기 위해, 우리는 Transformer 가 어디에서 한계를 갖는지, 그리고 그러한 한계를 어떻게 극복할 수 있는지를 이해할 필요가 있다.

중첩 학습 (nested learning) 관점에서, Transformer 는 2-레벨 구성요소이며, 여기서 투영 (projection) 과 MLP 블록은 첫 번째 레벨에서 최적화되고, 두 번째 레벨은 비모수적 해를 찾아 in-context learning 을 수행하고 따라서 출력을 문맥 (context) 에 조건화하는 역할을 한다. 그러나 이 설계는, 모델의 상태 추적 (state-tracking) 및 유사한 계산 능력에 관한 최근 연구들 (Merrill et al. 2024; Sanford et al. 2024; Grazi et al. 2025) 에서도 언급되었듯이, 제한된 계산 깊이 (computational depth) 를 갖는다. 더 나아가, Transformer 의 파라미터는 문맥 전체에 걸쳐 정적 (static) 이며, 이는 문맥내의 토큰들을 매핑하기 위해 찾아낸 해가 (비모수적 해이므로) 동일하게 유지된다는 것을 의미하고, 따라서 (적어도 in-context 에서) 자기 자신을 수정할 능력이 결여되어 있다. 더 구체적으로, 입력 데이터를 키 (key), 값 (value), 쿼리 (query) 로 투영하는 초기 선형 블록 W_k, W_v, W_q 는 사전 학습 (pre-training) 단계 이후에는 고정되며 (즉, 첫 번째 레벨에 있음), 따라서 Transformer 가 토큰을 문맥화하고 매핑하는 능력은 이 블록들에 저장된 지식에 의해 제한된다. 예를 들어, 1-레이어 Transformer 가 주어졌을 때, 각 토큰의 투영은 토큰 자체와 그 위치의 함수이다. 따라서, 하나의 예로, 그것은 의미가 단어 자체가 아니라 문맥에 의존하는 단어들의 다양한 가능한 인코딩을 놓칠 수 있다. 모델의 깊이를 증가시키면 이 문제가 후반 레이어에서 사라질 수 있지만, 우리는 모델의 능력을 보완하기 위해 깊이에 의존해서는 안 된다. 왜냐하면 그것은 여전히 초기 레이어에서 모델의 능력을 발휘하는 데 병목 (bottleneck) 이 되기 때문이다.

위의 도전을 극복하기 위해, 최근 짧은 컨볼루션 (short convolution) 과 canon 레이어 (Allen-Zhu 2025) 의 사용이 현대 모델에서 사실상 (de facto) 의 구성요소가 되었다. 지역 토큰들을 혼합하는 데 성공했음에도 불구하고, 여전히 모델들은 문맥에 적응하고 지역 혼합 (local mixing) 을 넘어서는 전역 정보를 포착하는 데 근본적으로 제한되어 있다. 다음 부분에서, 우리는 모든 구성요소가 in-context learning 을 수행하고, 적응하며, 자기 자신을 수정할 수 있게 하는 자기 참조적 (self-referential) Titans 를 제시함으로써 근본적인 해결책을 논의한다.

8.1 깊은 자기 참조적 Titans (Deep Self-Referential Titans)

연상 메모리 기반 블록에 대한 일반적인 정식화는, 데이터를 키, 값, 쿼리로 투영하고, 키를 값으로 매핑하는 방법과 쿼리에 기반하여 그 매핑으로부터 검색 (retrieve) 하는 방법을 학습하는 것이다. 더 형식적으로, 모수적 연상 메모리에 대해, $t = 1, \dots, L$ 에 대한 입력을 $\mathbf{x}_t \in \mathbb{R}^d$ 라 하면, 우리는 다음을 갖는다:

$$\mathbf{k}_t = \mathbf{x}_t W_{\mathbf{k}}, \quad \mathbf{v}_t = \mathbf{x}_t W_{\mathbf{v}}, \quad \mathbf{q}_t = \mathbf{x}_t W_{\mathbf{q}}, \quad \eta_t = \mathbf{x}_t W_{\eta}, \quad \alpha_t = \mathbf{x}_t W_{\alpha},$$

$$\min_{\mathcal{M}} \mathcal{L}(\mathcal{M}; \mathbf{k}_t, \mathbf{v}_t), \quad \text{최적화 알고리즘과 함께}$$

$$\mathbf{y}_t = \mathcal{M}_t \mathbf{q}_t.$$

명확성을 위해, 우리는 상위 레벨 (각각 하위 레벨) 의 계산/가중치를 강조하기 위해 빨간색 (각각 파란색) 을 사용한다. 그림 3 의 예와 유사하게, 우리는 $W_{\mathbf{k}}, W_{\mathbf{v}}, W_{\mathbf{q}}, W_{\eta}, W_{\alpha}$ 각각에 대해 새로운 레벨을 추가하고, 그것들이 in-context 로 업데이트되도록 허용할 수 있다. 효율성을 위해, 간단한 버전은 연상 메모리들의 중첩 시스템내 모든 구성요소에 대해 값을 공유하는 것이다:

$$\mathbf{k}_t = \mathcal{M}_{\mathbf{k},t-1}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v},t-1}(\mathbf{x}_t), \quad \mathbf{q}_t = \mathcal{M}_{\mathbf{q},t-1}(\mathbf{x}_t), \quad \eta_t = \mathcal{M}_{\eta,t-1}(\mathbf{x}_t), \quad \alpha_t = \mathcal{M}_{\alpha,t-1}(\mathbf{x}_t),$$

$$\min_{\mathcal{M}_{\square}} \mathcal{L}(\mathcal{M}_{\square}; \square_t, \mathbf{v}_t), \quad \text{최적화 알고리즘과 함께, } \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha\},$$

$$\min_{\mathcal{M}_{\text{mem}}} \mathcal{L}(\mathcal{M}_{\text{mem}}; \mathbf{k}_t, \mathbf{v}_t), \quad \text{최적화 알고리즘과 함께,}$$

$$\mathbf{y}_t = \mathcal{M}_{\text{mem},t}(\mathbf{q}_t),$$

여기서 모든 메모리의 초기 상태, 즉 임의의 $\square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}$ 에 대한 $\mathcal{M}_{\square,0}$ 는 모든 시퀀스/문맥에 걸쳐 메타 학습 (meta-learned) 된다. 앞서 논의했듯이, 메모리의 초기 상태에 대한 메타 학습은 빠른 적응 (fast-adaptation), 훈련 안정성, 데이터 내 노이즈에 대한 강건성 모두에 필수적이다.

이 설계는 모든 구성요소가 in-context 로 자기 자신을 적응시킬 수 있는 완전히 적응적인 (fully adaptive) 메모리를 제공한다. 그러나 그것은 (1) 여전히 자기 수정 (self-modification) 이 결여되어 있는데, 여기서 모델은 새로운 데이터에 대한 반응으로 자기 자신의 파라미터나 학습 과정을 변경한다 (Schmidhuber 2003); (2) 모든 메모리에 대해 키와 값을 공유하므로 최적이지 아닌 (suboptimal) 설계를 갖는다. 연속 학습 (continual learning) 에서는, 모델이 새로운 데이터에 대한 반응으로 일관된 가중치/지식 업데이트를 요구하며, 모델이 데이터에만 전적으로 의존하지 않고 대신 필요할 때 자기 자신을 수정하는 방법을 학습하는 것이 중요하다. 위의 점들에 동기를 부여받고, 문맥에 기반하여 자기 자신의 값을 생성하는 자기 수정 메커니즘 (Schmidhuber 1993, 2003; Irie et al. 2022b) 에서 영감을 받아, 우리는 모델이 자기 자신의 값을 생성하는 자기수정 깊은 연상 메모리 (self-modifying deep associative memory) 를 제시한다:

$$\mathbf{y}_t = \mathcal{M}_{\text{memory},t-1}(\mathbf{q}_t), \quad \mathbf{k}_t = \mathcal{M}_{\mathbf{k},t-1}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v},t-1}(\mathbf{x}_t), \quad \eta_t = \mathcal{M}_{\eta,t-1}(\mathbf{x}_t), \quad \alpha_t = \mathcal{M}_{\alpha,t-1}(\mathbf{x}_t),$$

$$\hat{\mathbf{v}}_{\square,t} = \mathcal{M}_{\square,t-1}(\mathbf{v}_t), \quad (\text{각 메모리에 대해 자기 자신의 값을 생성})$$

$$\min_{\mathcal{M}_{\square}} \mathcal{L}(\mathcal{M}_{\square}; \mathbf{k}_t, \hat{\mathbf{v}}_{\square,t}), \quad \text{최적화 알고리즘과 함께, } \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\},$$

여기서 $\mathbf{q}_t = \mathbf{x}_t W_{\mathbf{q}}$ 는 유일하게 비적응적인 투영이고, η_t 는 최적화 과정에서의 학습률 (learning rate) 이며, α_t 는 최적화 과정에서의 유지게이트 (retention gate, 망각 게이트 (forget gate) 또는 가중치 감쇠 (weight decay)) 이다. 다시 한 번 유의할 점은, 모든 메모리의 초기 상태, 즉 임의의 $\square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}$ 에 대한 $\mathcal{M}_{\square,0}$ 는 모든 시퀀스/문맥에 걸쳐 메타 학습되며, 따라서 상위 레벨 (또는 외부 루프 (outer-loop)) 에서 최적화된다는 것이다.

연상 메모리 모듈에 대한 매핑을 학습하는 것 (식 85 참조) 은 최적화알고리즘의 선택뿐만 아니라 매핑의 품질을 측정하는 목적함수 $\mathcal{L}$ 의 선택을 요구한다. 목적함수와 최적화 과정에 대한 간단하고 흔한 선택은 L_2 -회귀 손실과 경사 하강법 (gradient descent) 알고리즘이다. 목적함수로서, 우리는 L_2 -회귀 손실을 사용한다, 즉 $\mathcal{L}(\mathcal{M}; \mathbf{k}, \mathbf{v}) = \|\mathcal{M}(\mathbf{k}) - \mathbf{v}\|_2^2$. 앞서 논의했듯이 (섹션 4.5 참조), 최적화기 (optimizer)의 선택은 최적화의 문맥에 크게 의존한다. 예를 들어, 연상 메모리 관점에서 경사 하강법은 내적 유사도 (dot-product similarity)에 기반하며 따라서 각 단계에서의 업데이트는 전적으로 입력에만 기반하고 이전 데이터 샘플들을 업데이트에 통합하지 않는다. 그러나 토큰 공간 (token space)에서 최적화를 수행할 때, 우리는 토큰들이 매우 상관되어 (highly correlated) 있음을 안다. 따라서, 섹션 4.5의 논의에 따라, 우리는 가중치 감쇠를 갖는 DGD를 사용하며, 이는 다음의 일반적인 업데이트 규칙을 낳는다:

$$\mathbf{y}_t = \mathcal{M}_{\text{memory}, t-1}(\mathbf{q}_t), \quad \mathbf{k}_t = \mathcal{M}_{\mathbf{k}, t-1}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v}, t-1}(\mathbf{x}_t), \quad \boldsymbol{\eta}_t = \mathcal{M}_{\boldsymbol{\eta}, t-1}(\mathbf{x}_t), \quad \boldsymbol{\alpha}_t = \mathcal{M}_{\boldsymbol{\alpha}, t-1}(\mathbf{x}_t),$$

$$\hat{\mathbf{v}}_{\square, t} = \mathcal{M}_{\square, t-1}(\mathbf{v}_t), \quad (\text{각 메모리에 대해 자기 자신의 값을 생성})$$

$$\mathcal{M}_{\square, t} = \mathcal{M}_{\square, t-1} \left(\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top \right) - \eta_t \nabla \mathcal{L}_{\mathcal{M}_{\square, t-1}}(\mathcal{M}_{\square, t-1}; \mathbf{k}_t, \hat{\mathbf{v}}_{\square, t}), \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \boldsymbol{\eta}, \boldsymbol{\alpha}, \text{memory}\}.$$

여기서, 메모리들의 아키텍처는 임의적이며, 심지어 우리는 모든 구성요소에 대해 동일한 아키텍처를 사용하도록 강제되지 않는다. 우리는 모든 메모리의 아키텍처로 2-레이어 MLP 블록을 사용한다:

$$\mathcal{M}_{\square}(\cdot) = (\cdot) + W_{\square, 1} \sigma(W_{\square, 2}(\cdot)).$$

8.2 빠르고 병렬화 가능한 훈련 (Fast and Parallelizable Training)

위에서, 우리는 자기 자신의 잠재 값 (latent value)을 생성하는 방법을 학습하여 자기 자신을 수정할 수 있는 모델을 설계하는 방법을 논의했다. 실용적 관점에서의 주요 도전은 방법의 효율성과 그 훈련이 병렬화 가능한지 여부이다. 우리는 비선형 업데이트 규칙의 청크 단위 (chunk-wise) 훈련 알고리즘 (Sun et al. 2024; Behrouz et al. 2025c)을 따르고, 업데이트 빈도 $f_{\square} = \frac{C}{L}$ 를 사용하는데, 여기서 L 은 문맥 길이이다. 서로 다른 청크 크기 (chunk-size)를 사용하는데 제한은 없지만, 우리의 실험에서는 두 가지 다른 청크 크기 값을 사용하는데, 하나는 $\mathcal{M}_{\text{memory}}(\cdot)$ 의 업데이트를 위한 것이고 다른 하나는 자기 참조적 Titans 내 다른 모든 메모리를 위한 것이다.

더 자세히, 입력 시퀀스 $\{\mathbf{x}_t\}_{t=1}^L$ 와 청크 크기 $1 \leq C \leq L$ 이 주어졌을 때, 우리는 시퀀스를 $i = 1, \dots, \lceil \frac{L}{C} \rceil$ 에 대한 $\{\mathbf{x}_{((i-1)C+t)}\}_{t=1}^C$ 의 $\lceil \frac{L}{C} \rceil$ 개 청크로 분할하고, 그런 다음 각 청크의 끝에서 식 86의 모든 원소들을 다음 청크를 위해 생성한다. 이는 이 청크에 대한 계산을 시작하기 전에, 전체 청크에 대한 모든 원소를 병렬로 생성할 수 있게 한다. 더 나아가, 식 88에 기반하여 메모리 모듈들을 업데이트하기 위해, 우리는 이전 청크의 마지막 상태에 대한 경사를 취한다. 다시 한 번, 이는 다음 청크에 대한 모든 경사를 병렬로 계산할 수 있게 한다. 더 자세히, 이 청크 단위 업데이트 절차가 주어졌을 때, 자기 참조적 Titans에 대한 업데이트 규칙은 다음과 같이 계산된다:

$$\mathbf{y}_t = \mathcal{M}_{\text{memory}, C \times \lceil \frac{t}{C} \rceil}(\mathbf{q}_t), \quad \mathbf{k}_t = \mathcal{M}_{\mathbf{k}, C \times \lceil \frac{t}{C} \rceil}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v}, C \times \lceil \frac{t}{C} \rceil}(\mathbf{x}_t),$$

$$\hat{\mathbf{v}}_{\square, t} = \mathcal{M}_{\square, C \times \lceil \frac{t}{C} \rceil}(\mathbf{v}_t), \quad \boldsymbol{\eta}_t = \mathcal{M}_{\boldsymbol{\eta}, C \times \lceil \frac{t}{C} \rceil}(\mathbf{x}_t), \quad \boldsymbol{\alpha}_t = \mathcal{M}_{\boldsymbol{\alpha}, C \times \lceil \frac{t}{C} \rceil}(\mathbf{x}_t),$$

(각 메모리에 대해 자기 자신의 값을 생성)

$$\mathcal{M}_{\square,t} = \mathcal{M}_{\square,t-1} \left(\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top \right) - \eta_t \nabla \mathcal{L}_{\mathcal{M}_{\square,C \times \lceil \frac{t}{C} \rceil}} \left(\mathcal{M}_{\square,C \times \lceil \frac{t}{C} \rceil}; \mathbf{k}_t, \hat{\mathbf{v}}_{\square,t} \right), \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}.$$

여기서, 메모리들의 아키텍처는 임의적이며, 심지어 우리는 모든 구성요소에 대해 동일한 아키텍처를 사용하도록 강제되지 않는다. 우리는 모든 메모리의 아키텍처로 2-레이어 MLP 블록을 사용한다:

$$\mathcal{M}_{\square}(\cdot) = (\cdot) + W_{\square,1} \sigma(W_{\square,2}(\cdot)).$$

모든 경사뿐만 아니라 새로운 키, 값, 학습률, 그리고 가중치 감쇠가 현재 청크의 처리를 시작하기 전에 병렬로 계산될 수 있으므로, 위의 업데이트는 Sun et al. (2024) 과 Behrouz et al. (2025c) 에서 논의된 빠른 병렬화 가능한 이중 형식 (dual form) 을 수용한다. 자기 참조적 Titans 에 대한 위의 업데이트 규칙을 더 잘 설명하기 위해, 행렬 값 메모리의 가장 간단한 경우에 대한 순환 (recurrent) 공식을 유도해 보자. 우리는 두 가지 다른 목적 함수에 대한 순환 형식을 유도한다:

- **내적 유사도 (Dot-product similarity)** $\mathcal{L}(\mathcal{M}; \mathbf{k}, \mathbf{v}) = -\langle \mathcal{M}\mathbf{k}, \mathbf{v} \rangle$: 이 목적 함수와 선형 메모리가 주어졌을 때, 경사는 $\mathbf{v}\mathbf{k}^\top$ 로 계산되며, 이는 다음의 업데이트 규칙을 낳는다:

$$\mathcal{M}_{\square,t} = \mathcal{M}_{\square,t-1} \left(\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top \right) - \eta_t \hat{\mathbf{v}}_{\square,t} \mathbf{k}_t^\top, \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}$$

그림 6: 텍스트 분류 도메인에서의 클래스 증분 학습 (class-incremental learning), (왼쪽) CLINC 데이터셋 (Larson et al. 2019), (가운데) Banking 데이터셋 (Casanueva et al. 2020), (오른쪽) DBpedia 데이터셋 (Auer et al. 2007). Hope 로 강화된 아키텍처는 ICL 을 포함한 다른 연속 학습 방법들 중에서 가장 좋은 정확도를 달성한다.

- **L_2 -회귀 손실:** 이 목적 함수와 선형 메모리가 주어졌을 때, 경사는 $(\mathcal{M}\mathbf{k} - \mathbf{v})\mathbf{k}^\top$ 로 계산되며, 이는 다음의 업데이트 규칙을 낳는다:

$$\mathcal{M}_{\square,t} = \mathcal{M}_{\square,t-1} \left(\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top \right) - \eta_t \left(\mathcal{M}_{\square,C \times \lceil \frac{t}{C} \rceil} \mathbf{k}_t - \hat{\mathbf{v}}_{\square,t} \right) \mathbf{k}_t^\top, \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}.$$

8.3 Hope 신경 학습 모듈 (Hope Neural Learning Module)

이전 섹션들에서, 우리는 먼저 메모리의 더 지속적인 (persistent) 저장을 가능하게 하고 메모리를 서로 다른 업데이트 빈도를 갖는 블록들의 스펙트럼으로 정의하는 연속체 메모리 시스템 (Continuum Memory System, CMS) 을 논의했다. 더 큰 용량 (capacity) 과 파라미터 확장에 대한 제약으로 인해, CMS 는 종종 간단한 학습 규칙을 요구하지만 더 지속적인 지식을 저장하기 위해 더 높은 용량을 요구한다. 반면에, 이전 섹션에서, 우리는 자기 자신의 키를 생성하여 문맥에 더 잘 적응하기 위한 학습 업데이트를 생성할 수 있는 자기 수정 Titans 의 설계를 논의했다. CMS 와 대조적으로, 자기 수정 Titans 는 작은 용량을 갖지만 복잡하고 표현력 있는 학습 규칙을 사용한다. 따라서, 이 두 시스템은 상호 보완적인 것으로 보이며 그것들의 결합은 서로 다른 측면에서 모델의 표현력을 강화할 수 있다.

이를 위해, 우리는 Hope 아키텍처를 제시한다: 자기 수정 Titans 에 뒤이어 연속체 메모리 시스템을 통합하는 신경 학습 모듈. Hope 설계는 그림 5 에 예시되어 있다. 형식적으로, $t = 1, \dots, L$ 에 대한 입력을 $\mathbf{x}_t \in \mathbb{R}^d$ 라 하면, Hope 순전파 (forward pass) 는 다음과 같이 정의된다 (명확성을 위해 정규화 (normalization) 및 컨볼루션 레이어는 제거한다):

$$\mathbf{o}_t = \mathcal{M}_{\text{memory},t-1}(\mathbf{q}_t), \quad \hat{\mathbf{v}}_{\square,t} = \mathcal{M}_{\square,t-1}(\mathbf{v}_t),$$

$$\mathbf{M}_{\square,t} = \mathbf{M}_{\square,t-1} - \alpha_t \mathbf{I}$$

$$\begin{aligned}
y_t &= \text{MLP}^\wedge(f_k) (\text{MLP}^\wedge(f_{k-1}) (\\
k_t &= M_{k,t-1}(x_t), \\
\eta_t &= M_{\eta,t-1}(x_t), \\
\alpha_t &= M_{\alpha,t-1}(x_t), \quad (94) \quad (95) \\
&- \eta_t k_t k_t^\top - \eta_t \nabla L(M_{\square,t-1}) (\cdots \text{MLP}^\wedge(f_1) (\\
v_t &= M_{v,t-1}(x_t), \\
M_{\square,t-1} &; k_t, v_{\square,t}, \\
\square &\in \{k, v, q, \eta, \alpha, \text{memory}\}.
\end{aligned}$$

(96)

(97)

(ot))),

여기서 토큰 t 에 대한 블록의 출력은 y_t 이다. 우리의 실험에서는 또한 q 와 k 를 L2 정규화로 정규화하고, 윈도우 크기가 4인 국소 컨볼루션 (local convolution)도 사용한다.

Hope-어텐션 (Hope-Attention). 우리는 또한 Hope의 또 다른 변형을 사용하는데, 여기서는 자기 수정형 Titans(self-modifying Titans)를 단순히 소프트맥스 전역 어텐션 (softmax global attention) (Vaswani et al. 2017)으로 대체한다.

9. 실험 (Experiments)

이 절에서 우리는 논문 전반에 걸쳐 논의한 여러 구성 요소의 성능을 실증적으로 평가한다. 보다 구체적으로, (1) 먼저 제시한 최적화 알고리즘에 초점을 맞추고 이를 최신 기법 (state-of-the-art)들과 비교한다. (2) 다음으로 in-context 학습 및 지속 학습 (continual learning) 과제에 초점을 맞추어, 중첩 학습 (nested learning) 패러다임, 보다 구체적으로는 고차 in-context 학습 (higher order in-context learning)이 어떻게 모델의 능력을 향상시키는지 보인다. 우리는 연속체 메모리 시스템 (continuum memory system)을 단순한 MLP 계층들과 비교하고, 사전 학습된 모델이 어떻게 지속 학습자 (continual learner)로 적응될 수 있는지 논의한다. (3) 그런 다음 Hope 모델의 언어 모델링 및 장문맥 이해 (long context understanding)에 초점을 맞추고 이를 Transformer 및 현대적 순환 아키텍처 (recurrent architecture)들과 비교한다. 실험의 세부 사항과 그 설정은 해당하는 각 절에서 설명한다.

33

그림 7: (좌) RULER의 MK-NIAH (Hsieh et al. 2024), (중) LongHealth (Adams et al. 2025), (우) QASPER (Dasigi et al. 2021) 벤치마크에서 메모리 수준 (memory levels)이 모델의 in-context 학습 성능에 미치는 효과. QASPER 벤치마크 (우)의 경우 값이 낮을수록 성능이 더 우수함을 의미함에 유의하라.

9.1 Hope: 지속 학습과 장문맥 이해 (Continual Learning and Long Context Understanding)

NL의 주요 목표 중 하나는 지속 학습 능력을 향상시키는 것이며, 따라서 이 절에서 우리는 NL과 그 함의 (예: 연속체 메모리 시스템 (Continuum Memory System, CMS) 및 Hope)를 여러 지속 학습 및 장문맥 이해 과제에서 평가한다. 각 과제에 대해, 우리는 해당 벤치마크에서 보고된 최고 결과를 베이스라인으로 사용한다.

클래스 점증 학습 (Class Incremental Learning). 먼저, 우리는 세개의 데이터셋에서 클래스 점증 학습 과제에 초점을 맞춘다:

- **CLINC (Larson et al. 2019):** CLINC 는 과제 지향형대화 시스템 (task-oriented dialog system) 을 위해 설계된 다중 도메인의도 분류 (intent classification) 벤치마크로, 범위 밖 (out-of-scope, OOS) 질의 탐지에 특별히 초점을 둔다. 이 데이터셋은 10 개의 광범위한도메인 (예: 은행, 여행, 가정, 날씨, 스몰 토크 등) 에 걸친 150 개의 범위내 (in-scope) 의도 클래스를 가지며, 총 23.7K 개의 질의를 포함하는데 그중 22.5K 개는 범위 내, 1.2K 개는 범위 밖 질의이다.
- **Banking (Casanueva et al. 2020):** Banking 데이터셋은 은행 도메인의 세분화된 고객 서비스 질의에 초점을 맞춘 단일도메인 의도 분류 벤치마크이다. 이 데이터셋에서 각 예시는 짧은 고객질의 (예: “내 카드 PIN 을 어떻게 재설정하나요?”) 로, 이는 올바른 은행의도/카테고리로 분류되어야 한다. 77 개의 의도로 레이블링된 총 3083 개의 예시가 있다 (클래스가 심하게 불균형함).
- **DBpedia (Auer et al. 2007):** DBpedia 는 위키피디아에서 가져온 텍스트 분류 벤치마크로, 문서의 초록 (abstract) 이온톨로지 주제 클래스로 범주화되어야 한다. 다시 말해, 짧은 위키피디아 설명이 주어지면 그것의 상위 수준주제/카테고리 (예: 해당 문서가 책, 영화, 동물, 장소 등에 관한 것인지) 를 예측하는 것이 목표이다. 이 데이터셋은 70 개의 2 차 수준 (second-level) 클래스에 걸쳐 레이블링된 34 만 개 이상의예시를 가지고 있으나, 우리는 70-클래스 DBpedia 과제를 위해 1 만 개의학습 인스턴스와 1 천 개의 테스트 인스턴스를 샘플링한다.

Hope 모델의 백본 (backbone) 으로 우리는 Llama3-8B 와 Llama-3B (Dubey et al. 2024) 를 사용하고, 그런 다음 7.3 절에서 논의한 우리의 기법을 적용하여 MLP 블록들이 적응 (adaption) 할 수 있도록 만드는데, 이들을 서로 다른빈도의 업데이트를 갖는 서로 다른 수준에 배치하고 이어서 15B 토큰으로지속 사전 학습 (continual pre-training) 을 수행한다. Momeni et al. (2025) 를 따라, 우리는 Llama-3 모델의 단순 in-context 학습 (ICL) 능력 (MLP 블록에는 아무런 변경 없이 동일하게 15B 토큰으로 지속 사전학습을 거친 것), 탄력적 가중치 통합 (Elastic Weight Consolidation, EWC) (Kirkpatrick et al. 2017), 그리고 외부 학습자를 사용하는 In-context 지속학습 (In-context Continual Learning with an External Learner, InCA) (Momeni et al. 2025) 을 우리 평가의 베이스라인으로 사용한다. 결과는 그림 6 에 보고되어 있다. Hope 는 외부 학습자를 사용하는 모델 (즉, InCA) 을 포함한모든 지속 학습 베이스라인에 걸쳐 최고의 성능을 보인다. Hope 를 ICL 과비교했을 때, 주된 차이는 Hope 의 다중 수준 in-context 학습 (즉, 동등하게표현하면 MLP 블록에 대한 서로 다른 업데이트 빈도) 에서 비롯되며, 이는지속 학습 능력을 향상시키기 위한 CMS 설계의 효과성을 나타낸다. 나아가, InCA 및 EWC 와 비교했을 때 Hope 의 우수한 성능은 수준 간 (between levels) 지식 전이 (knowledge transfer) 가 모델의 성능에 있어 결정적인 역할을한다는 것을 나타낸다.

In-context 학습에 대한 수준의 효과 (The Effect of Levels on In-context Learning). 위 과제들에서 CMS 를 사용할 때 향상을 보였음에도불구하고, 수준 (levels) 과 그 빈도가 모델의 in-context 수준 능력에 미치는효과를 더 잘 이해하고 평가하기 위해, 우리는 in-context 질의응답과 다중키 (multi-key) 장문맥 이해를 수행한다. 보다 구체적으로, 우리는 다음을사용한다:

- **LongHealth (Adams et al. 2025):** 이것은 장문맥임상 질의응답을 위한 벤치마크로, 방대한 가상 환자 기록에 기반한다지선다형 (multiple-choice) QA 과제를 통해 LLM 이 상세한 의료 문서를추출하고 그에 대해 추론하는 능력을 시험한다. 이 데이터셋은 (다양한질병에 걸친) 20 개의 종합적인 환자 사례 문서를 포함하며,

34

표 1: Needle-In-A-Haystack 실험. 다음을 포함한다: (1) 세 가지난이도 수준을 갖는 단일 니들 (single needle)—단일 니들 과제들: S-NIAH-1(패스키 검색, passkey retrieval), S-NIAH-2(수치 니들, numerical needle), S-NIAH-3(UUID 기반 니들); (2) 다중 질의 (multi-query); (3) 다중키 (multi-key); 그리고 (4) 벤치마크의 다중 값 (multi-value) 설정.

	S-NIAH-1(패스 키 검색)			S-NIAH-2(건초 더미 속 숫자)			S-NIAH-3(건초 더미 속 UUID)		
Model	4K	8K	16K	4K	8K	16K	4K	8K	16K
Trans-former	88.6	76.4	79.8	100	98.8	94.2	78.0	69.2	40.8
Hope-Attention	100	100	100	100	98.4	94.4	76.8	68.8	42.4
RWKV-7	100	100	99.6	93.8	44.8	12.6	63.8	13.2	5.8
Comba	100	100	99.4	92.6	47.2	13.4	62.4	13.8	7.4
DLA	96.4	71.2	44.0	79.6	42.6	28.2	18.2	8.8	4.0
Titans	100	100	100	99.6	84.6	75.4	74.2	42.8	21.2
Hope	100	100	100	99.2	88.4	78.2	73.2	46.2	24.8

MK-NIAH-1(다중 키 라인 검색, multi-key line retrieval)

MQ-NIAH(다중 질의, multi-query)

그림 8: 새로운 언어의 지속 번역 (Continual Translation of a Novel Language, CTNL) 과제. 빨간색 점은 단 하나의 언어만 사용한 결과이다. 파란색 점은 지속 학습 (continual learning) 설정에서의 결과이다.

MV-NIAH(다중 값, multi-value)

Model | 4K | 8K | 16K

4K

8K

16K

4K

8K

16K

Transformer Hope-Attention

79.4 80.2

83.0 84.8

61.4 60.8

58.9 60.4

48.0 47.8

29.8 30.6

37.5 35.2

34.1 34.4

21.5 24.8

RWKV-7 Comba DLA Titans

21.4 21.4 27.4 26.4

18.8 19.4 20.0 23.6

9.6 8.2 11.8 8.2

20.4 21.8 26.4 22.8

14.8 15.2 22.0 19.8

8.6 6.4 6.4 9.4

16.2 16.5 25.6 24.6

13.4 13.5 12.8 15.1

6.8 7.2 9.6 8.2

Hope

29.4

24.8

14.8

31.7

24.8

14.2

31.4

17.2

11.4

그림 9: BABILong 벤치마크. 빨간 점은 파인튜닝된 모델의 결과이고, 파란 점은 대형 모델의 제로샷 (zero-shot) 결과이다.

각각 길이가 약 5.1K – 6.8K 단어이며, 우리는 환자 기록에서 샘플링한 200 개의 질문을 사용한다.

- **QASPER** (Dasigi et al. 2021): 이 벤치마크는 전체 길이의 NLP 연구 논문을 중심으로 한 정보 탐색 (information-seeking) QA 데이터셋이다. 구체적으로, 약 1.6K 개의 NLP 연구 논문에 근거한 약 5K 개의 QA 쌍을 포함한다. 또한, 우리는 각 논문의 전체 텍스트를 모델의 맥락 (context) 으로 사용한다.
- **MK-NIAH** (Hsieh et al. 2024): 우리는 RULER (Hsieh et al. 2024) 의 다중 키 (multiple keys) 건초 더미 속 바늘 찾기 (needle-in-haystack) 과제를 사용한다. 이 설정은 모델이 긴 텍스트 전반에 분산된 여러 정보 조각들을 단순히 위치를 찾는 것뿐 아니라 추출하는 것까지 요구한다.

베이스라인으로는, 1 레벨 메모리를 가진 Hope 와 동일한 ICL 을 사용하고, 또한 DuoAttention (Xiao et al. 2025) 도 사용한다. Cartridges (Eyuboglu et al. 2025) 와 같은 방법들이 DuoAttention 보다, 때로는 ICL 보다도 더 나은 유망한 성능을 보였다는 점은 주목할 만하다. 그러나 여기서는 주로 Hope 가 더 높은 메모리 사용량을 가지며 그들의 계산 비용에 근본적인 차이 (예: 자기학습 (self-studying) 등) 가 존재하여, 향후 연구에서 신중하고 통제된 설계로 추가 실험을 필요로 한다는 사실 때문에 그들과 Hope 의 비교는 제외한다. 우리 모델의 변형 (variants) 에 대해서는, 서로 다른 주파수 (frequency) 를 가진 서로 다른 개수의 메모리 레벨을 사용하며, 이들은 가장 낮은 주파수를 기준으로 그룹화된다. 가장 낮은 주파수의 메모리는 모델의 가장 지속적인 (persistent) 메모리에 해당하므로, 우리는 가장 낮은 주파수가 더 높은 모델일수록 더 적응적 (adaptive) 일 것으로 기대한다.

결과는 그림 7 에 보고되어 있다. 임의의 레벨 수 그리고 임의의 가장 낮은 주파수를 가진 Hope 는 ICL 베이스라인뿐만 아니라 효율적인 DuoAttention 도 능가한다. 나아가, Hope 의 변형들을 서로 비교

하면, 결과는 다음을 시사한다: (1) 메모리 레벨이 많을수록 모델의 in-context learning 능력에 도움이 되고 장기 메모리를 강화하여 긴 맥락 이해 (long-context understanding) 를 향상시킨다; 그리고 (2) 가장 낮은 주파수가 높을수록 성능이 낮아진다. 위에서 논의했듯이, 가장 낮은 주파수의 메모리는 모델의 가장 지속적인 메모리이므로

35

이를 증가시킨다는 것은 모델이 더 약하지만 더 적응적인 장기 메모리를 갖는다는 것을 의미한다. 업데이트의 주파수가 모델의 효율성에 직접적으로 영향을 미친다는 사실 때문에, “Lowest Frequency = 2K” 를 최적의 설정으로 볼 수도 있는데, 이는 훨씬 더 효율적인 순방향 전파 (forward pass) 를 제공하면서도 “Lowest Frequency = 512” 에 근접한 성능을 달성하기 때문이다.

맥락 속에서 새로운 언어 학습하기 (Learning a New Language In-Context). 이 논문 전반에 걸쳐, 우리는 사전학습 (pre-training) 이 in-context learning 과정으로 볼 수 있으며, 여기서 맥락은 전체 사전학습 데이터라고 주장하였다. 이에 따라, 우리가 신경 학습 모듈 (neural learning module) 에서 더 많은 레벨을 사용하거나 서로 다른 맥락 흐름 (context flows) 에 대해 in-context learning 을 수행할 때, 우리는 모델이 더 나은 적응성과 지속 학습 (continual learning) 능력을 보일 것으로 기대한다. 이를 위해, 우리는 기존의 두 벤치마크 MTOB (Tanzer et al. 2024) 와 Manchu (Pei et al. 2025) 를 결합하여, LLM 이 맥락 속에서 두 개의 새로운 언어를 학습하고 구문 (phrases) 을 영어로 번역할 것으로 기대되는 새로운 지속 학습 과제를 설계하였다. 그리고 우리의 새로운 언어의 지속 번역 (Continual Translation of a Novel Language, CTNL) 과제에 대해, 우리는 두 가지 설정을 고려한다: (1) 각 언어에 대해 개별적으로 모델의 성능을 학습하고 테스트하기 (빨간색). 이는 다음 설정과 비교할 때 파국적 망각 (catastrophic forgetting) 을 측정하기 위해 사용하는 베이스라인이다; 그리고 (2) 모델이 먼저 이 두 언어를 순차적으로 (파란색) 학습한 다음 구문을 영어로 번역하도록 요청받는 설정. 우리는 ICL 을 베이스라인으로 사용하고, Hope 의 다중 레벨 (multi-level) 설계의 중요성을 연구하기 위해, 각각 Hope-1, Hope-2, Hope-3 으로 지칭하는, 1 개, 2 개, 3 개의 추가 메모리 레벨을 가진 서로 다른 Hope 변형들을 사용한다.

결과는 표 8 에 보고되어 있다. 더 구체적으로, 각 점은 어느 한 설정에서 한 모델의 성능인데, 여기서 x 축 (각각 y 축) 은 Manchu → English (각각 Kalamang → English) 번역에서의 모델 ChRF 를 나타낸다. 첫 번째 설정 (지속 학습 없는 맥락 내 번역) 에서는, 모든 Hope 변형이 ICL 과 비교하여 더 낮거나 대등한 성능을 보여, CMS 설계의 중요성을 뒷받침한다. 그러나 두 번째 설정 (즉, 지속 번역) 에서는, ICL 이 극적인 성능 하락을 겪고 거의 사전학습에서 획득한 능력에만 의존한다 (즉, 맥락 속 지식에 대한 파국적 망각). 반면에, Hope 에서 메모리 레벨을 증가시키는 것은 명확한 개선을 보이며, Hope-3 은 지속 학습 없이 첫 번째 설정에서의 ICL 능력을 거의 회복한다. 이러한 결과는 지속 학습에서 CMS 설계의 중요성과 모델이 새로운 과제에 스스로 적응하는 능력을 더욱 뒷받침한다.

9.2 Hope: 긴 맥락 이해 (Long Context Understanding)

이전 절에서, 우리는 MLP 블록이 in-context learning 을 수행하도록 적응되었을 때 Hope-Attention 의 성능을 평가하였다. 이 부분에서는, Hope 가 처음부터 (from scratch) 학습을 시작할 때 긴 맥락 이해에서의 성능을 평가한다. 이를 위해, 우리는 FineWeb-Edu (Penedo et al. 2024) 와 긴 맥락 문서 (long-context documents) 의 혼합에서 약 50B 토큰을 사용하고 어휘 크기 (vocabulary size) 32K 로 모든 모델을 처음부터 학습시킨다. 모든 모델은 각 모델에 대해 튜닝된 학습률 (learning rate) 과 Behrouz et al. (2025c) 의 기본 옵티마이저 구성으로 AdamW 를 사용하여 최적화된다. 우리는 두 가지 인기 있는 벤치마크인 RULER (Hsieh et al. 2024) 와 BABILong (Kuratov et al. 2024) 에 초점을 맞춘다:

진초 더미 속 바늘 찾기 (Needle-in-a-Haystack, NIAH) 과제. 첫 번째 부분에서, 우리는 다음과 같은 서로 다른 설정의 진초 더미 속 바늘 찾기에 초점을 맞춘다: (1) 단일 바늘이지만 서로 다른 유형 (즉, pass-key, number, uid), (2) 다중 키 (multi-key), (3) 다중 질의 (multi-query), 그리고 (4) 다중 값 (multi-value) 으로, 모두 Hsieh et al. (2024) 를 따른다. 베이스라인으로는, 순수하게 Hebbian 규칙 및 Delta 규칙에 기반한 모델의 대표로서 RetNet (Sun et al. 2023) 과 DeltaNet (Schlag et al. 2021) 을 사용하고, 다양한 최신 선형 순환 (linear recurrent) 모델을 실험하여 가장 성능이 좋은

선형 모델을 사용하였다: 즉, RWKV-7 (Peng et al. 2025a) 과 Comba (Hu et al. 2025). 또 다른 베이스라인 그룹으로, 우리는 또한내적 (dot-product) 및 L2 회귀 목적함수를 가진 심층 메모리 모듈 (deep memory modules) 과도 비교한다: 즉, DLA (Behrouz et al. 2025a) 와 Titans (Behrouz et al. 2025c).

결과는 표 1 에 보고되어 있다. 다른 어텐션 없는 (attention-free) 모델들과 비교할 때, Hope 는 모든 과제와 난이도 수준에 걸쳐 최고의 성능을 달성한다. 특히, 선형 메모리와 비교할 때, 심층 메모리 모듈은 더 많은 토큰을 압축할 수 있는 더 높은 메모리 용량 덕분에 주로 더 긴 시퀀스에서 더 나은 성능을 보인다. Hope 를 Titans 와 비교하면, 특히 더 긴 맥락 길이에서 Hope 의 우수한 성능은 자기 참조 업데이트 (self-referential update) 와 CMS 설계 둘 다의 중요성을 뒷받침한다. 마지막으로, 우리는 또한 Hope-Attention 의 성능을 평가하고 이를 Transformer 와 비교하여 CMS 설계의 기여를 더 잘 이해한다. 결과는 Hope-Attention 이 Transformer 와 비교하여 더 나은 성능을 달성함을 나타내며, 이는 Hope-Attention 설계에서 CMS 를 갖는 것의 이점을 뒷받침한다.

BABILong. 다음으로, 우리는 BABILong 벤치마크 (Kuratov et al. 2024) 에서 Hope 의 성능을 평가하고 이를 다음과 비교한다: (1) GPT4 및 GPT4o-mini (Achiam et al. 2023) 와 같은 대형 모델; (2) RAG 로 증강된 버전을 가진 중간 크기의 Llama-8B 모델 (Dubey et al. 2024); 그리고 (3) 이 과제들에서의 최신 (state-of-the-art) 소형 모델들: 즉, RMT (Bulatov et al. 2022), ARMT (Rodkin et al. 2024), 그리고 Titans (Behrouz et al. 2025c). 우리는 벤치마크의 원래 설정을 따르고 파인튜닝한다

36

표 2: 언어 모델링 및 상식 추론 (common-sense reasoning) 과제에서의 모델 성능.

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑
Trans-former++	24.18	24.27	37.1	67.2	43.8
Samba*	21.07	22.85	39.2	68.9	47.8
RetNet	25.77	24.19	34.5	66.8	
DeltaNet	24.52	24.38	36.8		
RWKV-7	23.75	23.08	37.1		
Comba	22.41	22.19	37.5		

67.3 67.3 66.9

TTT Miras (Memora) DLA Titans

24.17 22.28 23.12 20.08

23.51 22.31 22.09 21.52

34.7 38.2 36.1 38.1

Hope

18.68

20.07

38.8

Wino. acc ↑

ARC-e acc ↑

ARC-c acc_n ↑

SIQA acc ↑

BoolQ acc ↑

평균 (Avg.) ↑

53.0 53.1

65.6 65.8

33.4 34.9

39.1 38.9

61.7 63.1

50.11 51.46

41.2 44.5 47.6 48.2

51.9 51.8 52.2 52.4

63.6 64.2 64.7 65.1

32.5 32.7 34.2 34.1

38.8 39.6 39.4 40.1

56.2 60.1 61.9 62.8

48.19 49.63 50.55 50.89

67.3 67.8 68.0 69.1

43.9 49.3 47.9 48.5

51.0 53.3 52.7 52.7

64.5 63.6 65.8 66.2

33.8 36.1 34.6 35.7

40.2 40.9 39.1 40.3

59.6 63.0 59.6 62.8

47.32 51.53 50.48 51.68

69.2

49.1

53.6

66.8

36.1

41.2

63.4

52.28

760M 파라미터 / 30B 토큰

1.3B 파라미터 / 100B 토큰 Transformer++ Samba*

17.92 16.15

17.73 13.21

42.6 45.2

71.4 71.5
52.3 53.8
54.1 55.8
69.9 69.1
36.5 36.7
41.8 40.6
58.4 63.0
53.38 54.46
RetNet DeltaNet RWKV-7 Comba
18.91 18.62 18.44 18.16
17.04 17.10 15.96 14.87
41.2 41.6 46.7 46.9
71.3 70.1 72.4 73.1
49.1 49.4 54.9 54.5
55.2 52.7 57.5 57.7
67.5 67.6 71.6 72.0
34.1 35.2 38.2 39.1
41.4 39.7 40.7 40.2
61.0 54.8
60.4 60.6
52.60 51.39 55.30 55.39
TTT Miras (Memora) DLA Titans
18.42 15.90 16.31 15.60
14.51 12.04 12.29 11.41
46.8 48.7 44.5 49.1
72.9 73.1 70.6 73.1
55.2 56.0 53.9 56.3
59.0 57.4 54.2 59.8
71.8 71.5 69.6 72.4
39.5 37.9 36.0 40.8
39.8 40.2 40.8 42.1
59.6 61.3 60.2 61.0
55.58 55.76 53.72 56.82
57.5
61.2
73.8

42.7

42.8

61.4

58.04

Hope 14.39 10.08 51.0 73.9

* 는 어텐션 (attention) + 선형 RNN 의 하이브리드다 (Ren et al. 2024).

소형 모델들은 Kuratov et al. (2024) 와 동일한 절차로 [평가/학습했다]. 결과는 표 9(Table 9) 에 보고되어 있다. 대형 모델들은 시퀀스 길이가 증가함에 따라 성능이 크게 하락하며, 모든 모델이 128K-256K 문맥 길이 부근에서 실패한다. RAG 로 증강된 모델도 문맥이 늘어남에 따라 성능 하락을 보이지만, 256K 문맥 길이 이후에도 상대적으로 성능을 유지할 수 있다. 미세조정 (fine-tuned) 된 모델들 중에서 Titans, ARMT, 그리고 Hope 는 1M 문맥 길이까지 경쟁력 있는 결과를 보이지만, Titans 와 ARMT 의 성능은 그 지점 이후로 빠르게 하락한다. Hope 는 주로 그 CMS 설계 덕분에 10M 문맥 길이에서도 좋은 성능을 유지한다. 주목할 점은, Hope 를 포함한 모든 소형 모델의 성능이 미세조정 없이 사용될 경우 크게 하락할 수 있다는 것이다. 그 이유는, 큰 문맥 (예: 10M) 을 압축하는 것이 고주파 (high-frequency) 수준에서의 강력한 메모리 관리에 더해, 10M 토큰 혹은 적어도 최종 답에 필요한 토큰들을 압축할 만큼 충분한 용량 (capacity) 을 요구하기 때문이다. 미세조정 단계는 모델이 자신의 저주파 (lower-frequency) 수준을 조정하여 빠르게 적응하고, 그리하여 고주파수준에서 메모리를 적절히 관리하도록 돕는다.

9.3

Hope: 언어 모델링과 상식 추론 (Language Modeling and Common-Sense Reasoning)

이 절에서 우리는 Hope 를 언어 모델의 백본 (backbone) 으로 연구하고, 다음의 설정으로 일반적인 언어 모델링 및 상식 추론 과제에서 평가하는 것을 목표로 한다:

- **데이터셋 (Datasets):** 우리는 Hope 와 베이스라인들을 Wikitext (Merity et al. 2017), LMB (Paperno et al. 2016), PIQA (Bisk et al. 2020), HellaSwag (Zellers et al. 2019), WinoGrande (Sakaguchi et al. 2021), ARC-easy (ARC-e) 및 ARC-challenge (ARC-c) (Clark et al. 2018), SIQA (Sap et al. 2019), 그리고 BoolQ (Clark et al. 2019) 벤치마크에서 평가한다.
- **베이스라인 (Baselines):** 베이스라인으로는, 9.2 절과 유사하게, 순수하게 Hebbian 규칙 혹은 Delta 규칙에 기반한 모델들의 대표로 RetNet (Sun et al. 2023) 과 DeltaNet (Schlag et al. 2021) 을 사용하고, 다른 모델들과 비교해 최고의 성능을 내는 두 개의 현대적 행렬-값 (matrix-valued) 순환 모델, 즉 RWKV-7 (Peng et al. 2025a) 과 Comba (Hu et al. 2025) 를 사용한다. 또 다른 베이스라인 그룹으로는, 다양한 [설계를 갖춘] 어텐션이 없는 (attention-free) 심층 메모리 모듈들과 비교한다.

37

표 5(Table 5): 형식 언어 인식 (formal language recognition) 과제에서 여러 모델들의 정확도.

Non-Star-Free Regular Parallel 모델 (Model)

Parity (패리티)

(aa)*

Counter (카운터) (abab)*

an bn

an bn cn

Shuffle-2

Training

Bin0

Bin1

Bin0

Bin1

Bin0

Bin1

Bin0

Bin1

Bin0

Bin1

Bin0

Bin1

LSTM Transformer

× ✓

100.0 46.4

100.0 0.0

100.0 0.0

100.0 0.0

100.0 0.0

100.0 0.0

100.0 0.0

100.0 100.0

100.0 100.0

100.0 100.0

100.0 100.0

100.0 100.0

100.0 100.0

Linear DeltaNet SRWM

✓ ✓ ×

78.1 98.2 100.0

0.0 10.1 100.0

0.0 0.0 100.0

0.0 0.0 100.0

0.0 0.0 100.0

0.0 0.0 100.0
 100.0 100.0 100.0
 100.0 100.0 100.0
 100.0 100.0 100.0
 100.0 100.0 100.0
 100.0 100.0 100.0
 100.0 100.0 100.0

Hope

✓

100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0
 100.0

내적 (dot-product), L_2 , 그리고 L_p 회귀의 내부 어텐션편향 (internal attentional bias): 즉 TTT (Sun et al. 2024), Miras (Behrouz et al. 2025b), DLA (Behrouz et al. 2025a) 및 Titans (Behrouz et al. 2025c). 마지막으로, 우리는 Transformer (Vaswani et al. 2017; Dubey et al. 2024) 뿐만 아니라 어텐션과 선형 RNN의 하이브리드인 Samba (Ren et al. 2024) 와도 비교한다.

- **학습 (Training):** 우리는 약 760M 및 1.3B 파라미터를 가진 모델을 학습하며, 각각 30B 및 100B 토큰으로 학습한다. 모든 모델은 FineWeb-Edu (Penedo et al. 2024) 와 긴 문맥 (long-context) 문서의 혼합으로, 어휘 크기 (vocabulary size) 32K 로 처음부터 (from scratch) 학습된다. 모든 모델은 각 모델에 대해 튜닝된 학습률 (learning rate) 과 Behrouz et al. (2025c) 의 기본 옵티마이저 설정으로 AdamW 를 사용하여 최적화된다.

결과는 표 2(Table 2) 에 보고되어 있다. Hope 는 언어 모델링과 상식추론 (common-sense reasoning) 벤치마크 모두에서 평균 성능 측면에서 모든 베이스라인을 능가한다. 흥미롭게도, 파라미터를 스케일링함에 따라 Hope 는 다른 어텐션-프리 (attention-free) 모델들에 비해 더 높은 성능 향상을 보인다.

9.4 Hope: 인-컨텍스트 회상 (In-context Recall) 과제와 MAD 합성벤치마크

인-컨텍스트 회상 (In-context recall) 은 흔히 어텐션-프리 모델들에게도전적인 벤치마크 중 하나로 언급된다. 이 절에서 우리는 Arora et al. (2024) 를 따라, Hope 설계의 효과를 평가하기 위해 SWDE (Lockard et al. 2019), NQ (Kwiatkowski et al. 2019), DROP (Dua et al. 2019), FDA (Arora et al. 2023), SQUAD (Rajpurkar et al. 2016), 그리고 TQA (Kembhavi et al. 2017) 에 대해 실험을 수행한다. 우리는 위의 이전 결과 동일한베이스라인 집합과 실험 설정을 사용한다. 결과는 표 3(Table 3) 에 보고되어있다. Transformer 가 최고의 성능을 달성하는 한편, Hope 는 경쟁력 있는 결과를 보이며, 모든 어텐션-프리 베이스라인을 능가하고 Transformer 와의 격차를 좁힌다.

표 3(Table 3): 짧은 인-컨텍스트 회상 과제에서의 Hope 와 베이스라인의 성능. Hope 는 모든 어텐션-프리 모델을 능가하고 Transformer 와의 격차를 좁힌다.

표 4(Table 4): MAD (Poli et al. 2024) 합성 벤치마크에서의 Hope 와 베이스라인의 성능. Hope 는 Transformer 를 포함한 모든 베이스라인을 능가한다.

	SWDE	NQ	DROP	FDA	SQUAD	TQA
Transformers	71.4	22.0	23.9	67.3	39.4	59.1
RWKV-7	52.3	17.8	21.7	32.8	28.5	56.2
Comba	53.9	19.1	21.9	35.5	30.2	56.4
Titans	60.8	20.3	22.0	37.6	31.8	57.5
Hope	65.9	21.2	22.8	41.9	33.0	57.7

	Compress.	ICR	Fuzzy ICR	Selective Copying	Memory
Transformers	49.4	100	47.9	96.2	83.7
RWKV-7	45.1	100	32.8	95.6	82.2
Comba	46.3	100	32.8	96.4	82.9
Titans	49.8	100	50.0	99.4	83.4
Hope	51.2	100	52.1	99.7	85.2

우리는 또한 회상, 암기 (memorization), 압축 (compression), 복사 (copying) 과제에서 모델의 성능을 평가하는 합성 벤치마크인 MAD 벤치마크 (Poli et al. 2024) 에서의 Hope 의 성능을 연구한다. 결과는 표 4(Table 4) 에 보고되어있다. Hope 는 베이스라인들과 비교하여 최고의 결과를 달성한다.

9.5 언어 인식 (Language Recognition) 과제

Transformer 의 중대한 한계 중 하나는 병렬화 불가능 (non-parallelizable) 과제에 있는데, 여기서는 재귀 (recurrence) 가 적절한 성능을 달성하는 데 중요한 역할을 한다. 그러한 과제의 한 예는 상태 추적 (state tracking) 문제로, 여기서 모델은 (이동 등의) 명령 시퀀스가 주어졌을 때 자신의

38

표 6(Table 6): Hope 에 대한 절제 연구 (Ablation Study). Hope 의 모든 구성 요소가 그 성능에 긍정적으로 기여한다.

모델 (Model)	언어 모델링 ppl ↓	추론 acc ↑
Hope	12.24	58.1
w/o DGD	13.41	56.5
w/o Momentum	13.58	56.9

모델 (Model)	언어 모델링 ppl ↓	추론 acc ↑
w/o weight decay	13.71	57.2
w/o CMS	13.04	57.3
w/o inner-projection k	13.77	56.9
w/o inner-projection v	13.90	55.1
w/o inner-projection q	12.19	57.4

그림 11(Figure 11): AdamW, Muon, 그리고 우리의 M3 옵티마이저로 학습한 ImageNet-21K (Ridnik et al. 2021) 에서의 ViT 테스트 및 학습손실 (loss).

그림 10(Figure 10): 문맥 사용 (context usage) 이 모델의 퍼플렉시티 (perplexity) 에 미치는 영향. 우리는 강력한 메모리 관리 능력을 가진 모델의 퍼플렉시티가 더 많은 문맥과 함께 감소하기를 기대한다.

그림 12(Figure 12): Muon, AdaMuon, 그리고 M3 옵티마이저로 140M 및 1.3B 파라미터를 가진 모델들의 학습 시간 (Training time).

상태를 추적해야 한다. 그리고 여러 연구들은 Transformer 가 이론과 실제양쪽 모두에서 비선형 재귀 (non-linear recurrence) 를 가진 모델들에 비해 상당히 저조한 성능을 보인다는 것을 보여주었다 (Merrill et al. 2024; Grazzi et al. 2025). 이 절에서 우리는 형식 언어 인식 (formal language recognition) 과제에 초점을 맞추고 Irie et al. (2023) 의 벤치마크 구성을 따른다. 결과는 표 5 (Table 5) 에 보고되어 있다. Hope 는 LSTM (Schmidhuber et al. 1997) 및 SRWM (Irie et al. 2022b) 과 같은 다른 비선형 재귀 모델들과 유사하게, 모든 과제에서 완벽한 점수 (perfect score) 를 달성한다. 그러나 Hope 의 주된 장점은, 필요할 때 병렬화 가능한 학습 (parallelizable training) 을 가지므로 언어 모델링 과제를 위해 더 큰 규모로 확장할 수 있다는 사실이다.

9.6 Hope: 절제 연구 (Ablation Studies) 와 스케일링

이 절에서 우리는 먼저 Hope 아키텍처에 대해 우리가 내린 설계 선택들의 중요성을, 절제 연구 (ablation study) 를 수행하여 연구한다. 이 연구에서는 Hope 의 구성요소들 중 하나를 한 번에 하나씩 제거하거나 변경한다. 앞선 실험들에서 우리는 일부 구성요소의 유의성을 이미 평가했음에 유의하라. 예를 들어, 표 8 과 그림 7 은 레벨의 수와 갱신의 빈도가 연속 학습 (continual learning) 에서 Hope 의 성능에 미치는 효과를 이미 보여주었다. 이 절에서는 서로 다른 변형 (variant) 들을 비교하기 위해, 우리는 언어 모델링에서 모델들의 평균 퍼플렉시티 (perplexity) 와 상식 추론 (common-sense reasoning) 과제에서의 평균 정확도를 사용한다. 결과는 표 6 에 보고되어 있다. (1) 첫번째 행은 자기 수정 (self-modifying) Titans 의 설계에서 델타 경사하강법 (Delta Gradient Descent) 을 단순한 경사 하강법 (gradient descent) 으로 대체한다. (2) 두 번째 행은 자기 수정 Titans 에서 모멘텀 (momentum) 항을 제거한다. (3) 가중치 감쇠 (weight decay) 를 제거한다. (4) Hope 의 아키텍처에서 CMS 를 제거한다. (5, 6, 7) k , v , q 에 대한 사영 (projection) 을 더 높은 빈도 레벨에서 가장 낮은 빈도 레벨로 옮긴다. Hope 의 모든 구성요소는 이 과제들에서 그 우수한 성능에 기여하며, 그것들 각각을 제거하거나 변경하면 언어 모델링에서의 모델 퍼플렉시티 및/또는 상식 추론 과제에서의 정확도를 손상시킬 수 있다.

9.7 표현력 있는 옵티마이저 (Expressive Optimizers)

이 절에서 우리는 해 (solution) 를 찾는 데 있어서의 효과성과 대규모에서의 훈련 효율성이라는 두 측면 모두에서 우리의 M3 옵티마이저의 성능을 평가한다.

ImageNet. 이 실험에서 우리는 비전 과제를 위한 ViT (Dosovitskiy et al. 2021) 아키텍처에 초점을 맞추고, 그것을 ImageNet-21K 에서 사전 훈련 (pre-train) 한다. ImageNet-21K 는 10,450 개의 클래스에 대응하는 11M 개의 이미지로 구성된다. 우리는 패치 크기 16 을 사용하며, 24M 및 86M 모

델이라는 두 스케일에 대해 각각 MLP 차원 1536 과 3072 를 사용한다. 우리는 공정한 비교를 보장하기 위해 다른 모든 구성요소를 통제하고 각 옵티마이저에 대해 하이퍼파라미터를 개별적으로 미세 조정 (finetune) 했다. 우리는 동일한 훈련 스텝 수에서 어떤 옵티마이저가 더 효과적인 해를 찾는지를 이해하기 위해 ViT 를 훈련하는 옵티마이저를 변화시킨다. 서로 다른 옵티마이저로 훈련된 모델들의 훈련 및 테스트 손실은 그림 11 에 보고되어 있다. 우리의 M3 는 AdamW 와 Muon 모두에 비해 가장 좋은 훈련/테스트 손실을 보인다.

대규모 모델에서의 효율성. ImageNet 에 필요한 모델의 크기가 작기 때문에, 우리는 언어 모델을 훈련할 때의 효율성 비교도 수행한다. 이를 위해 우리는 140M 과 1.3B 의 두 스케일을 사용하며, Muon (Jordan et al. 2024), AdaMuon (Si et al. 2025), 그리고 우리의 M3 라는 서로 다른 옵티마이저들을 사용하여 동일한 Transformer 모델을 훈련한다. 결과는 그림 12 에 보고되어 있다. 우리의 M3 옵티마이저는 다중 모멘텀 (memory, 메모리) 의 사용으로 인해 Muon 옵티마이저에 비해 상대적으로 느리며, AdaMuon 과는 대등한 (on par) 효율성을 보인다.

10 결론 (Conclusion)

이 논문에서 우리는 새로운 학습 패러다임인 중첩 학습 (Nested Learning, NL) 을 소개했다. 이 패러다임에서 현대의 머신러닝 시스템은, 각기 자신만의 맥락 흐름 (context flow) 과 갱신 빈도 (update frequency) 를 갖는, 서로 연결된 다중 레벨 최적화 문제 (multi-level optimization problem) 들로 모델링된다. 이 관점 안에서, 아키텍처와 옵티마이저는 모두, 자기 자신의 맥락 (토큰이든, 경사 (gradient) 든, 또는 더 높은 수준의 신호든) 을 내부파라미터로 압축하는, 연상 기억 (associative memory) 들의 중첩된 시스템 (nested systems) 의 사례들이다. 이 관점은 사전 훈련 (pre-training), 문맥 내 학습 (in-context learning), 그리고 연속 학습 (continual learning) 을 동일한 근본 메커니즘의 발현들로 재구성한다. 즉, 서로 다른 레벨과 시간 스케일에서 맥락을 압축하고 재사용하는 것을 학습하는 것으로서 말이다. 이 관점은 역전파 (backpropagation), 모멘텀 (momentum), 그리고 전처리 (preconditioning) 를 연상 기억 메커니즘으로 재해석하며, 기존의 폭넓은 방법군을, 더 큰 (이전에는 숨겨져 있던) 공간 안의 특정 설계지점 (design point) 들로서 설명한다.

NL 의 관점을 바탕으로, 우리는 일반화된 경사 기반 갱신 (generalized gradient-based updates) 을 도출했다. 예를 들어 델타 경사 하강법 (Delta Gradient Descent), 델타 모멘텀 (Delta Momentum), 다중 스케일 모멘텀 Muon (Multi-scale Momentum Muon, M3) 등이며, 현대의 시퀀스 아키텍처들을 중첩된 연상 기억들로 재해석했다. 메모리 처리를 향상시키기 위해, 우리는 연속체 메모리 시스템 (Continuum Memory System, CMS) 을 도입했는데, 이는 “장기/단기 메모리 블록” 이라는 전통적 관점을 일반화하는 메모리에 대한 새로운 정식화 (formulation) 이다. 자기 수정 Titans 와 CMS 에 기반한 우리의 Hope 아키텍처는 연속 학습 및 긴 맥락 추론 (long-context reasoning) 능력을 향상시키면서도, 일반적인 백본 (backbone) 으로서 경쟁력을 유지한다.

파국적 망각은 해결되었는가? (Is Catastrophic Forgetting Solved?) Hope 와 CMS 가 우리가 경험적으로 연구한 과제들에서 파국적 망각 (catastrophic forgetting) 을 줄이는 데 유망한 결과를 보여주었지만, 파국적 망각이라는 바람직하지 않은 현상이 일반적으로 “해결” 된 것은 아니다. 학습과 역전파 과정에 대한 중첩 학습의 관점에서 볼 때, 파국적 망각은 압축 (compression) 의 자연스러운 귀결이며, 여기서 네트워크의 제한된 용량 (capacity) 이 모델로 하여금 새로운 정보를 위한 용량을 보충하기 위해 잊도록 강제한다. 우리는 NL 을 목적지 (destination) 라기보다는 로드맵 (roadmap) 으로 본다. 즉, 그것은 연속 학습, 긴 맥락 추론, 현대적 옵티마이저, 그리고 자기 수정 모델에서의 진보가, 점점 더 깊어지는 정적 (static) 네트워크로부터가 아니라 레벨 (levels) 이라는 추가적인 설계축 (design axis) 을 더 잘 활용하는 데서 올 것임을 시사한다.

참고문헌 (References)

[1] Walter Pitts. “The linear theory of neuron networks: The dynamic problem” . In: The bulletin of mathematical biophysics 5 (1943), pp. 23 – 31.

- [2] Warren S McCulloch and Walter Pitts. “The statistical organization of nervous activity” . In: Biometrics 4.2 (1948), pp. 91 – 99.
- [3] Warren S McCulloch. “The brain computing machine” . In: Electrical Engineering 68.6 (1949), pp. 492 – 497.
- [4] William Beecher Scoville and Brenda Milner. “Loss of recent memory after bilateral hippocampal lesions” . In: Journal of neurology, neurosurgery, and psychiatry 20.1 (1957), p. 11.
- [5] Arthur L Samuel. “Some studies in machine learning using the game of checkers” . In: IBM Journal of research and development 3.3 (1959), pp. 210 – 229.
- [6] Seppo Linnainmaa. “The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors” . PhD thesis. Master’ s Thesis (in Finnish), Univ. Helsinki, 1970.
- [7] Kuniyiko Fukushima. “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position” . In: Biological cybernetics 36.4 (1980), pp. 193 – 202.
- [8] Erkki Oja. “Simplified neuron model as a principal component analyzer” . In: Journal of mathematical biology 15.3 (1982), pp. 267 – 273.
- [9] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors” . In: nature 323.6088 (1986), pp. 533 – 536.
- [10] Geoffrey E Hinton and David C Plaut. “Using fast weights to deblur old memories” . In: Proceedings of the ninth annual conference of the Cognitive Science Society. 1987, pp. 177 – 186.
- [11] DL Prados and SC Kak. “Neural network capacity using delta rule” . In: Electronics Letters 25.3 (1989), pp. 197 – 199.
- [12] Juergen Schmidhuber. “Learning to control fast-weight memories: An alternative to recurrent nets. Accepted for publication in” . In: Neural Computation (1992).
- [13] Tim VP Bliss and Graham L Collingridge. “A synaptic model of memory: long-term potentiation in the hippocampus” . In: Nature 361.6407 (1993), pp. 31 – 39.
- [14] Jurgen Schmidhuber. “A ‘self-referential’ weight matrix” . In: International conference on artificial neural networks. Springer. 1993, pp. 446 – 450.
- [15] Juergen Schmidhuber, Jieyu Zhao, and Marco Wiering. Simple principles of metalearning. 1996.
- [16] Uwe Frey and Richard GM Morris. “Synaptic tagging and long-term potentiation” . In: Nature 385.6616 (1997), pp. 533 – 536.
- [17] Juergen Schmidhuber and Sepp Hochreiter. “Long Short-term Memory” . In: Neural Computation MIT-Press (1997).
- [18] Amos Storkey. “Increasing the capacity of a hopfield network without sacrificing functionality” . In: International Conference on Artificial Neural Networks. Springer. 1997, pp. 451 – 456.
- [19] Richard S Sutton, Andrew G Barto, et al. Reinforcement learning: An introduction. Vol. 1. 1. 1998.
- [20] Jonathan H. Connell and Sridhar Mahadevan. “Robot Learning” . In: Robotica 17.2 (1999), pp. 229 – 235. doi: 10.1017/S0263574799271172.

- [21] Sean PA Drummond, Gregory G Brown, J Christian Gillin, John L Stricker, Eric C Wong, and Richard B Buxton. “Altered brain response to verbal learning following sleep deprivation” . In: *Nature* 403.6770 (2000), pp. 655 – 657.
- [22] Hideyuki Okano, Tomoo Hirano, and Evan Balaban. “Learning and memory” . In: *Proceedings of the National Academy of Sciences* 97.23 (2000), pp. 12403 – 12404.
- [23] Jurgen Schmidhuber. “Godel machines: self-referential universal problem solvers making provably optimal self-improvements” . In: *arXiv preprint cs/0309048* (2003).
- [24] Gyorgy Buzsaki and Andreas Draguhn. “Neuronal oscillations in cortical networks” . In: *science* 304.5679 (2004), pp. 1926 – 1929.
- [25] Donald Olding Hebb. *The organization of behavior: A neuropsychological theory*. Psychology press, 2005.
- [26] Alvaro Pascual-Leone, Amir Amedi, Felipe Fregni, and Lotfi B Merabet. “The plastic human brain cortex” . In: *Annu. Rev. Neurosci.* 28.1 (2005), pp. 377 – 401.
- [27] David J Foster and Matthew A Wilson. “Reverse replay of behavioural sequences in hippocampal place cells during the awake state” . In: *Nature* 440.7084 (2006), pp. 680 – 683.
- [28] Lisa Marshall, Halla Helgadottir, Matthias Molle, and Jan Born. “Boosting slow oscillations during sleep potentiates memory” . In: *Nature* 444.7119 (2006), pp. 610 – 613. doi: 10.1038/nature05278.
- [29] Soren Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. “Dbpedia: A nucleus for a web of open data” . In: *international semantic web conference*. Springer. 2007, pp. 722 – 735.
- [30] Timothy J. Buschman and Earl K. Miller. “Top-down versus bottom-up control of attention in the prefrontal and posterior parietal cortices” . In: *Science* 315.5820 (2007), pp. 1860 – 1862. doi: 10.1126/science.1138071.
- [31] Daoyun Ji and Matthew A Wilson. “Coordinated memory replay in the visual cortex and hippocampus during sleep” . In: *Nature neuroscience* 10.1 (2007), pp. 100 – 107.
- [32] Seung-Schik Yoo, Peter T Hu, Ninad Gujar, Ferenc A Jolesz, and Matthew P Walker. “A deficit in the ability to form new human memories without sleep” . In: *Nature neuroscience* 10.3 (2007), pp. 385 – 392.
- [33] Nicholas J Higham. *Functions of matrices: theory and computation*. SIAM, 2008.
- [34] Michael V Johnston. “Plasticity in the developing brain: implications for rehabilitation” . In: *Developmental disabilities research reviews* 15.2 (2009), pp. 94 – 101.
- [35] Adrien Peyrache, Mehdi Khamassi, Karim Benchenane, Sidney I Wiener, and Francesco P Battaglia. “Replay of rule-learning related neural patterns in the prefrontal cortex during sleep” . In: *Nature neuroscience* 12.7 (2009), pp. 919 – 926.
- [36] Susanne Diekelmann and Jan Born. “The memory function of sleep” . In: *Nature Reviews Neuroscience* 11.2 (2010), pp. 114 – 126. doi: 10.1038/nrn2762.
- [37] John Duchi, Elad Hazan, and Yoram Singer. “Adaptive subgradient methods for online learning and stochastic optimization.” In: *Journal of machine learning research* 12.7 (2011).
- [38] Juergen Fell and Nikolai Axmacher. “The role of phase synchronization in memory processes” . In: *Nature reviews neuroscience* 12.2 (2011), pp. 105 – 118.
- [39] Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. “Neural networks for machine

learning lecture 6a overview of mini-batch gradient descent” . In: Cited on 14.8 (2012), p. 2.

[40] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “Imagenet classification with deep convolutional neural networks” . In: Advances in neural information processing systems 25 (2012).

[41] Hong-Viet V. Ngo, Thomas Martinetz, Jan Born, and Matthias Molle. “Auditory closed-loop stimulation of the sleep slow oscillation enhances memory” . In: Neuron 78.3 (2013), pp. 545 – 553. doi: 10.1016/j.neuron.2013.03.006.

[42] Dzmitry Bahdanau. “Neural machine translation by jointly learning to align and translate” . In: arXiv preprint arXiv:1409.0473 (2014).

[43] James F Cavanagh and Michael J Frank. “Frontal theta as a mechanism for cognitive control” . In: Trends in cognitive sciences 18.8 (2014), pp. 414 – 421.

[44] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization” . In: arXiv preprint arXiv:1412.6980 (2014).

[45] Diederik P. Kingma and Max Welling. “Auto-Encoding Variational Bayes.” In: ICLR. Ed. by Yoshua Bengio and Yann LeCun. 2014. url: <http://dblp.uni-trier.de/db/conf/iclr/iclr2014.html#KingmaW13>.

[46] Guido Montufar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. “On the number of linear regions of deep neural networks” . In: Advances in neural information processing systems 27 (2014).

[47] Pascal Fries. “Rhythms for cognition: communication through coherence” . In: Neuron 88.1 (2015), pp. 220 – 235.

[48] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning” . In: nature 521. 7553 (2015), pp. 436 – 444.

[49] Bernhard P. Staresina, Til Ole Bergmann, Mathilde Bonnefond, Roemer van der Meij, Ole Jensen, Lorena Deuker, Christian E. Elger, Nikolai Axmacher, and Jurgen Fell. “Hierarchical nesting of slow oscillations, spindles and ripples in the human hippocampus during sleep” . In: Nature Neuroscience 18.11 (2015), pp. 1679 – 1686. doi: 10.1038/nn.4119.

[50] Jimmy Ba, Geoffrey E Hinton, Volodymyr Mnih, Joel Z Leibo, and Catalin Ionescu. “Using fast weights to attend to the recent past” . In: Advances in neural information processing systems 29 (2016).

[51] Timothy Dozat. “Incorporating nesterov momentum into adam” . In: (2016).

[52] Andrew C. Heusser, David Poeppel, Youssef Ezzyat, and Lila Davachi. “Episodic sequence memory is supported by a theta – gamma phase code” . In: Nature Neuroscience 19.10 (2016), pp. 1374 – 1380. doi: 10.1038/nn.4374.

[53] Mikael Lundqvist, Pawel Herman, Scott L. Brincat, Timothy J. Buschman, and Earl K. Miller. “Gamma and beta bursts underlie working memory” . In: Neuron 90.1 (2016), pp. 152 – 164. doi: 10.1016/j.neuron.2016.02.028.

[54] Denis Paperno, German Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernandez. “The LAMBADA dataset: Word prediction requiring a broad discourse context” . In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Ed. by Katrin Erk and Noah A. Smith. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 1525 – 1534. doi: 10.18653/v1/

P16-1144. url: <https://aclanthology.org/P16-1144/>.

[55] Ben Poole, Subhaneil Lahiri, Maithra Raghu, Jascha Sohl-Dickstein, and Surya Ganguli. “Exponential expressivity in deep neural networks through transient chaos” . In: Advances in neural information processing systems 29 (2016).

[56] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. “Squad: 100,000+ questions for machine comprehension of text” . In: arXiv preprint arXiv:1606.05250 (2016).

[57] Sashank J Reddi, Ahmed Hefny, Suvrit Sra, Barnabas Poczos, and Alex Smola. “Stochastic variance reduction for nonconvex optimization” . In: International conference on machine learning. PMLR. 2016, pp. 314 – 323.

[58] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. “Mastering the game of Go with deep neural networks and tree search” . In: nature 529.7587 (2016), pp. 484 – 489.

[59] Thomas B Christophel, P Christiaan Klink, Bernhard Spitzer, Pieter R Roelfsema, and John-Dylan Haynes. “The distributed nature of working memory” . In: Trends in cognitive sciences 21.2 (2017), pp. 111 – 124.

[60] Chelsea Finn, Pieter Abbeel, and Sergey Levine. “Model-agnostic meta-learning for fast adaptation of deep networks” . In: International conference on machine learning. PMLR. 2017, pp. 1126 – 1135.

[61] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. “Deep learning scaling is predictable, empirically” . In: arXiv preprint arXiv:1712.00409 (2017).

[62] Aniruddha Kembhavi, Minjoon Seo, Dustin Schwenk, Jonghyun Choi, Ali Farhadi, and Hannaneh Hajishirzi. “Are you smarter than a sixth grader? textbook question answering for multimodal machine comprehension” . In: Proceedings of the IEEE Conference on Computer Vision and Pattern recognition. 2017, pp. 4999 – 5007.

[63] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. “Overcoming catastrophic forgetting in neural networks” . In: Proceedings of the national academy of sciences 114.13 (2017), pp. 3521 – 3526.

[64] Takashi Kitamura, Sachie K Ogawa, Dheeraj S Roy, Teruhiro Okuyama, Mark D Morrissey, Lillian M Smith, Roger L Redondo, and Susumu Tonegawa. “Engrams and circuits crucial for systems consolidation of a memory” . In: Science 356.6333 (2017), pp. 73 – 78.

[65] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. “Pointer Sentinel Mixture Models” . In: International Conference on Learning Representations. 2017. url: <https://openreview.net/forum?id=Byj72udxe>.

[66] W Scott Terry. Learning and memory: Basic principles, processes, and procedures. Routledge, 2017.

[67] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. “Attention is All you Need” . In: Advances in Neural Information Processing Systems. Ed. by I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc., 2017. url: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

- [68] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Animashree Anandkumar. “signSGD: Compressed optimisation for non-convex problems” . In: International conference on machine learning. PMLR. 2018, pp. 560 – 569.
- [69] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. “Think you have solved question answering? try arc, the ai2 reasoning challenge” . In: arXiv preprint arXiv:1803.05457 (2018).
- [70] Jianqing Fan. Local polynomial modelling and its applications: monographs on statistics and applied probability 66. Routledge, 2018.
- [71] Vineet Gupta, Tomer Koren, and Yoram Singer. “Shampoo: Preconditioned stochastic tensor optimization” . In: International Conference on Machine Learning. PMLR. 2018, pp. 1842 – 1850.
- [72] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, et al. “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play” . In: Science 362.6419 (2018), pp. 1140 – 1144.
- [73] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. “BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions” . In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Ed. by Jill Burstein, Christy Doran, and Tamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 2924 – 2936. doi: 10.18653/v1/N19-1300. url: <https://aclanthology.org/N19-1300/>.
- [74] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. “DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs” . In: arXiv preprint arXiv:1903.00161 (2019). (앞 청크에서 이어짐: 참고문헌 [74] 의 마지막 부분 — “문단들에대한 이산적 추론을 요구하는 독해 벤치마크” .)
- [75] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Phil Bachman, Adam Trischler, and Yoshua Bengio. “Learning deep representations by mutual information estimation and maximization” . In: International Conference on Learning Representations. 2019. url: <https://openreview.net/forum?id=Bklr3j0cKX>.
- [76] Jens G Klinzing, Niels Niethard, and Jan Born. “Mechanisms of systems memory consolidation during sleep” . In: Nature neuroscience 22.10 (2019), pp. 1598 – 1610.
- [77] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. “Natural questions: a benchmark for question answering research” . In: Transactions of the Association for Computational Linguistics 7 (2019), pp. 453 – 466.
- [78] Stefan Larson, Anish Mahendran, Joseph J Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K Kummerfeld, Kevin Leach, Michael A Laurenzano, Lingjia Tang, et al. “An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction” . In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). 2019, pp. 1311 – 1316.
- [79] Colin Lockard, Prashant Shiralkar, and Xin Luna Dong. “Openceres: When open information extraction meets the semi-structured web” . In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguis-

tics: Human Language Technologies, Volume 1 (Long and Short Papers). 2019, pp. 3047 – 3056.

[80] Tsendsuren Munkhdalai, Alessandro Sordoni, Tong Wang, and Adam Trischler. “Metalearned neural memory” . In: *Advances in Neural Information Processing Systems* 32 (2019).

[81] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. “Social IQa: Commonsense Reasoning about Social Interactions” . In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4463 – 4473. doi: 10.18653/v1/D19-1454. url: <https://aclanthology.org/D19-1454/>.

[82] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. “HellaSwag: Can a Machine Really Finish Your Sentence?” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by Anna Korhonen, David Traum, and Lluís Marquez. Florence, Italy: Association for Computational Linguistics, July 2019, pp. 4791 – 4800. doi: 10.18653/v1/P19-1472. url: <https://aclanthology.org/P19-1472/>.

[83] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. “Piqa: Reasoning about physical commonsense in natural language” . In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 2020, pp. 7432 – 7439.

[84] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. “Language models are few-shot learners” . In: *Advances in neural information processing systems* 33 (2020), pp. 1877 – 1901.

[85] Inigo Casanueva, Tadas Temcinas, Daniela Gerz, Matthew Henderson, and Ivan Vulic. “Efficient Intent Detection with Dual Sentence Encoders” . In: *ACL 2020* (2020), p. 38.

[86] Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. “Orthogonal gradient descent for continual learning” . In: *International conference on artificial intelligence and statistics*. PMLR. 2020, pp. 3762 – 3773.

[87] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. “Generative adversarial networks” . In: *Communications of the ACM* 63.11 (2020), pp. 139 – 144.

[88] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. “Scaling laws for neural language models” . In: *arXiv preprint arXiv:2001.08361* (2020).

[89] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Francois Fleuret. “Transformers are rnns: Fast autoregressive transformers with linear attention” . In: *International conference on machine learning*. PMLR. 2020, pp. 5156 – 5165.

[90] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. “On the Variance of the Adaptive Learning Rate and Beyond” . In: *International Conference on Learning Representations*. 2020. url: <https://openreview.net/forum?id=rkgz2aEKDr>.

[91] Noam Shazeer. “Glu variants improve transformer” . In: *arXiv preprint arXiv:2002.05202* (2020).

[92] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. “A dataset of information-seeking questions and answers anchored in research papers” .

In: arXiv preprint arXiv:2105.03011 (2021).

[93] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale” . In: International Conference on Learning Representations. 2021. url: <https://openreview.net/forum?id=YicbFdNTTy>.

[94] Akihiro Goto, Ayaka Bota, Ken Miya, Jingbo Wang, Suzune Tsukamoto, Xinzhi Jiang, Daichi Hirai, Masanori Murayama, Tomoki Matsuda, Thomas J. McHugh, Takeharu Nagai, and Yasunori Hayashi. “Stepwise synaptic plasticity events drive the early phase of memory consolidation” . In: Science 374.6569 (2021), pp. 857 – 863. doi: 10.1126/science.abj9195. eprint: <https://www.science.org/doi/pdf/10.1126/science.abj9195>. url: <https://www.science.org/doi/abs/10.1126/science.abj9195>.

[95] Kazuki Irie, Imanol Schlag, Robert Csordas, and Juergen Schmidhuber. “Going beyond linear transformers with recurrent fast weight programmers” . In: Advances in neural information processing systems 34 (2021), pp. 7703 – 7717.

[96] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Zidek, Anna Potapenko, et al. “Highly accurate protein structure prediction with AlphaFold” . In: nature 596. 7873 (2021), pp. 583 – 589.

[97] Tal Ridnik, Emanuel Ben-Baruch, Asaf Noy, and Lihi Zelnik-Manor. “ImageNet-21K Pretraining for the Masses” . In: Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1). 2021. url: https://openreview.net/forum?id=Zkj_VcZ6ol.

[98] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. “Winogrande: An adversarial winograd schema challenge at scale” . In: Communications of the ACM 64.9 (2021), pp. 99 – 106.

[99] Imanol Schlag, Kazuki Irie, and Juergen Schmidhuber. “Linear transformers are secretly fast weight programmers” . In: International Conference on Machine Learning. PMLR. 2021, pp. 9355 – 9366.

[100] Ekin Akyurek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. “What learning algorithm is in-context learning? investigations with linear models” . In: arXiv preprint arXiv:2211.15661 (2022).

[101] Aydar Bulatov, Yury Kuratov, and Mikhail Burtsev. “Recurrent memory transformer” . In: Advances in Neural Information Processing Systems 35 (2022), pp. 11079 – 11091.

[102] Yanda Chen, Ruiqi Zhong, Sheng Zha, George Karypis, and He He. “Meta-learning via Language Model In-context Tuning” . In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2022, pp. 719 – 730.

[103] Alexandre Defossez, Leon Bottou, Francis Bach, and Nicolas Usunier. “A Simple Convergence Proof of Adam and Adagrad” . In: Transactions on Machine Learning Research (2022). issn: 2835-8856. url: <https://openreview.net/forum?id=ZPQhzTSWA7>.

[104] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. “Training compute-optimal large language models” . In: arXiv preprint arXiv:2203.15556 (2022).

- [105] Kazuki Irie, Francesco Faccio, and Jurgen Schmidhuber. “Neural differential equations for learning to program neural nets through continuous learning rules” . In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 38614 – 38628.
- [106] Kazuki Irie, Imanol Schlag, Robert Csordas, and Juergen Schmidhuber. “A modern self-referential weight matrix that learns to modify itself” . In: *International Conference on Machine Learning*. PMLR. 2022, pp. 9660 – 9677.
- [107] William Merrill, Ashish Sabharwal, and Noah A Smith. “Saturated transformers are constant-depth threshold circuits” . In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 843 – 856.
- [108] Dheeraj S Roy, Young-Gyun Park, Minyoung E Kim, Ying Zhang, Sachie K Ogawa, Nicholas DiNapoli, Xinyi Gu, Jae H Cho, Heejin Choi, Lee Kamentsky, et al. “Brain-wide mapping reveals that engrams for a single memory are distributed across multiple brain regions” . In: *Nature communications* 13.1 (2022), p. 1799.
- [109] Yufeng Zhang, Boyi Liu, Qi Cai, Lingxiao Wang, and Zhaoran Wang. “An analysis of attention via the lens of exchangeability and latent variable models” . In: *arXiv preprint arXiv:2212.14852* (2022).
- [110] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. “Gpt-4 technical report” . In: *arXiv preprint arXiv:2303.08774* (2023).
- [111] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avani Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Re. “Language models enable simple systems for generating structured views of heterogeneous data lakes” . In: *arXiv preprint arXiv:2304.09433* (2023).
- [112] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, et al. “Symbolic discovery of optimization algorithms” . In: *Advances in neural information processing systems* 36 (2023), pp. 49205 – 49233.
- [113] Ramin Hasani, Mathias Lechner, Tsun-Hsuan Wang, Makram Chahine, Alexander Amini, and Daniela Rus. “Liquid Structural State-Space Models” . In: *The Eleventh International Conference on Learning Representations*. 2023. url: <https://openreview.net/forum?id=g4OTKRKfS7R>.
- [114] Kazuki Irie, Robert Csordas, and Juergen Schmidhuber. “Practical computational power of linear transformers and their recurrent and self-referential extensions” . In: *arXiv preprint arXiv:2310.16076* (2023).
- [115] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. “CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis” . In: *The Eleventh International Conference on Learning Representations*. 2023. url: https://openreview.net/forum?id=iaYcJKpY2B_.
- [116] Bo Peng, Eric Alcaide, Quentin Gregory Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Nguyen Chung, Leon Derczynski, Xingjian Du, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartlomiej Koptyra, Hayden Lau, Jiaju Lin, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Johan S. Wind, Stanislaw Wozniak, Zhenyuan Zhang, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. “RWKV: Reinventing RNNs for the Transformer Era” . In: *The 2023 Conference on Empirical Methods in Natural Language Processing*. 2023. url: <https://openreview.net/forum?id=7SaXczaBpG>.

- [117] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Re. “Hyena hierarchy: Towards larger convolutional language models” . In: International Conference on Machine Learning. PMLR. 2023, pp. 28043 – 28078.
- [118] Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. “Are emergent abilities of large language models a mirage?” In: Advances in neural information processing systems 36 (2023), pp. 55565 – 55581.
- [119] Aaditya Singh, Stephanie Chan, Ted Moskovitz, Erin Grant, Andrew Saxe, and Felix Hill. “The transient nature of emergent in-context learning in transformers” . In: Advances in neural information processing systems 36 (2023), pp. 27801 – 27819.
- [120] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. “Retentive network: A successor to transformer for large language models” . In: arXiv preprint arXiv:2307.08621 (2023).
- [121] Johannes Von Oswald, Maximilian Schlegel, Alexander Meulemans, Seijin Kobayashi, Eyvind Niklasson, Nicolas Zucchet, Nino Scherrer, Nolan Miller, Mark Sandler, Max Vladymyrov, et al. “Uncovering mesa-optimization algorithms in transformers” . In: arXiv preprint arXiv:2309.05858 (2023).
- [122] Wenhai Wang, Zhe Chen, Xiaokang Chen, Jiannan Wu, Xizhou Zhu, Gang Zeng, Ping Luo, Tong Lu, Jie Zhou, Yu Qiao, et al. “Visionllm: Large language model is also an open-ended decoder for vision-centric tasks” . In: Advances in Neural Information Processing Systems 36 (2023), pp. 61501 – 61513.
- [123] Ekin Akyurek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. “The Surprising Effectiveness of Test-Time Training for Few-Shot Learning” . In: Forty-second International Conference on Machine Learning. 2024.
- [124] Ekin Akyurek, Bailin Wang, Yoon Kim, and Jacob Andreas. “In-context language learning: Architectures and algorithms” . In: arXiv preprint arXiv:2401.12973 (2024).
- [125] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, James Zou, Atri Rudra, and Christopher Re. “Simple linear attention language models balance the recall-throughput tradeoff” . In: Forty-first International Conference on Machine Learning. 2024. url: <https://openreview.net/forum?id=e93ffDcpH3>.
- [126] Maximilian Beck, Korbinian Poppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Gunter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. “xLSTM: Extended Long Short-Term Memory” . In: arXiv preprint arXiv:2405.04517 (2024).
- [127] Aleksandar Botev, Soham De, Samuel L Smith, Anushan Fernando, George-Cristian Muraru, Ruba Haroun, Leonard Berrada, Razvan Pascanu, Pier Giuseppe Sessa, Robert Dadashi, et al. “RecurrentGemma: Moving Past Transformers for Efficient Open Language Models” . In: arXiv preprint arXiv:2404.07839 (2024).
- [128] Jonathan Daume, Jan Kaminski, Andrea G. P. Schjetnan, Yousef Salimpour, Umais Khan, Michael Kyzar, Chrystal M. Reed, William S. Anderson, Taufik A. Valiante, Adam N. Mamelak, and Ueli Rutishauser. “Control of working memory by phase – amplitude coupling of human hippocampal neurons” . In: Nature 629 (2024), pp. 393 – 401. doi: 10.1038/s41586-024-07309-z.
- [129] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. “The

llama 3 herd of models” . In: arXiv e-prints (2024), arXiv – 2407.

[130] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. “RULER: What’s the Real Context Size of Your Long-Context Language Models?” In: First Conference on Language Modeling. 2024. url: <https://openreview.net/forum?id=kIoBbc76Sy>.

[131] K Jordan, Y Jin, V Boza, Y Jiacheng, F Cecista, L Newhouse, and J Bernstein. “Muon: An optimizer for hidden layers in neural networks, 2024b” . In: URL <https://kellerjordan.github.io/posts/muon> (2024).

[132] Praneeth Kacham, Vahab Mirrokni, and Peilin Zhong. “PolySketchFormer: Fast Transformers via Sketching Polynomial Kernels” . In: Proceedings of the 41st International Conference on Machine Learning. Ed. by Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp. Vol. 235. Proceedings of Machine Learning Research. PMLR, July 2024, pp. 22748 – 22770. url: <https://proceedings.mlr.press/v235/kacham24a.html>.

[133] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Igorevich Sorokin, Artyom Sorokin, and Mikhail Burtsev. “BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack” . In: The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track. 2024. url: <https://openreview.net/forum?id=u7m2CG84BQ>.

[134] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. “Deepseek-v3 technical report” . In: arXiv preprint arXiv:2412.19437 (2024).

[135] Bo Liu, Rui Wang, Lemeng Wu, Yihao Feng, Peter Stone, and Qiang Liu. “Longhorn: State space models are amortized online learners” . In: arXiv preprint arXiv:2407.14207 (2024).

[136] William Merrill, Jackson Petty, and Ashish Sabharwal. “The Illusion of State in State-Space Models” . In: Forty-first International Conference on Machine Learning. 2024. url: <https://openreview.net/forum?id=QZgo9JZpLq>.

[137] Guilherme Penedo, Hynek Kydlicek, Anton Lozhkov, Margaret Mitchell, Colin A Raffel, Leandro Von Werra, Thomas Wolf, et al. “The fineweb datasets: Decanting the web for the finest text data at scale” . In: Advances in Neural Information Processing Systems 37 (2024), pp. 30811 – 30849.

[138] Bo Peng, Daniel Goldstein, Quentin Anthony, Alon Albalak, Eric Alcaide, Stella Biderman, Eugene Cheah, Xingjian Du, Teddy Ferdinan, Haowen Hou, et al. “Eagle and finch: Rwkv with matrix-valued states and dynamic recurrence” . In: arXiv preprint arXiv:2404.05892 (2024).

[139] Michael Poli, Armin W Thomas, Eric Nguyen, Pragaash Ponnusamy, Bjorn Deiseroth, Kristian Kersting, Taiji Suzuki, (다음 청크로 이어짐)

Brian Hie, Stefano Ermon, Christopher Re, et al. “Mechanistic design and scaling of hybrid architectures” . In: arXiv preprint arXiv:2403.17844 (2024).

Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. “Samba: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling” . In: arXiv preprint arXiv:2406.07522 (2024).

Ivan Rodkin, Yuri Kuratov, Aydar Bulatov, and Mikhail Burtsev. “Associative recurrent memory transformer” . In: arXiv preprint arXiv:2407.04841 (2024).

Clayton Sanford, Daniel Hsu, and Matus Telgarsky. “Transformers, parallel computation,

and logarithmic depth” . In: Forty-first International Conference on Machine Learning. 2024. url: <https://openreview.net/forum?id=QCZabhKQhB>.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. “Learning to (learn at test time): Rnns with expressive hidden states” . In: arXiv preprint arXiv:2407.04620 (2024).

Garrett Tanzer, Mirac Suzgun, Eline Visser, Dan Jurafsky, and Luke Melas-Kyriazi. “A Benchmark for Learning to Translate a New Language from One Grammar Book” . In: The Twelfth International Conference on Learning Representations. 2024. url: <https://openreview.net/forum?id=tbVWug9f2h>.

Wannan Yang, Chen Sun, Roman Huszar, Thomas Hainmueller, Kirill Kiselev, and Gyorgy Buzsaki. “Selection of experience for memory by hippocampal sharp wave ripples” . In: Science 383.6690 (2024), pp. 1478 – 1483.

Ruiqi Zhang, Spencer Frei, and Peter L Bartlett. “Trained transformers learn linear models in-context” . In: Journal of Machine Learning Research 25.49 (2024), pp. 1 – 55.

Yushun Zhang, Congliang Chen, Tian Ding, Ziniu Li, Ruoyu Sun, and Zhiquan Luo. “Why transformers need adam: A hessian perspective” . In: Advances in neural information processing systems 37 (2024), pp. 131786 – 131823.

Lisa Adams, Felix Busch, Tianyu Han, Jean-Baptiste Excoffier, Matthieu Ortala, Alexander Loser, Hugo JWL Aerts, Jakob Nikolas Kather, Daniel Truhn, and Keno Bressem. “Longhealth: A question answering benchmark with long clinical documents” . In: Journal of Healthcare Informatics Research (2025), pp. 1 – 17.

Zeyuan Allen-Zhu. “Physics of Language Models: Part 4.1, Architecture Design and the Magic of Canon Layers” . In: SSRN Electronic Journal (May 2025). <https://ssrn.com/abstract=5240330>.

Shaden Alshammari, John Hershey, Axel Feldmann, William T Freeman, and Mark Hamilton. “I-Con: A Unifying Framework for Representation Learning” . In: arXiv preprint arXiv:2504.16929 (2025).

Ali Behrouz, Zeman Li, Praneeth Kacham, Majid Daliri, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. “Atlas: Learning to optimally memorize the context at test time” . In: arXiv preprint arXiv:2505.23735 (2025).

Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. “It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization” . In: arXiv preprint arXiv:2504.13173 (2025).

47

[153] [154] [155] [156] [157] [158] [159] [160] [161] [162] [163] [164] [165] [166] [167] [168] [169] [170] [171] [172]

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. “Titans: Learning to Memorize at Test Time” . In: The Thirty-ninth Annual Conference on Neural Information Processing Systems. 2025. url: <https://openreview.net/forum?id=8GjSf9Rh7Z>.

Franz Louis Cesis. Heuristic Solutions for Steepest Descent on the Stiefel manifold. July 2025. url: <https://leloykun.github.io/ponder/steepest-descent-stiefel%7D>.

Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. “Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities” . In: arXiv preprint arXiv:2507.06261 (2025).

Francesco Pappone Devan Selvaraj. Multiscale Muon. <https://publish.obsidian.md/ueaj/Machine+Learning/Research+Ideas/Multiscale+Muon>. Blogpost. Aug. 2025.

Benoit Dherin, Michael Munn, Hanna Mazzawi, Michael Wunder, and Javier Gonzalvo. “Learning without training: The implicit dynamics of in-context learning” . In: arXiv preprint arXiv:2507.16003 (2025).

Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, et al. “Cartridges: Lightweight and general-purpose long context representations via self-study” . In: arXiv preprint arXiv:2506.06266 (2025).

Samuel J Gershman, Ila Fiete, and Kazuki Irie. “Key-value memory in the brain” . In: Neuron 113.11 (2025), pp. 1694 – 1707.

Riccardo Grazi, Julien Siems, Jorg K.H. Franke, Arber Zela, Frank Hutter, and Massimiliano Pontil. “Unlocking State-Tracking in Linear RNNs Through Negative Eigenvalues” . In: The Thirteenth International Conference on Learning Representations. 2025. url: <https://openreview.net/forum?id=UvTo3tVBk2>.

Jiaxi Hu, Yongqi Pan, Jusen Du, Disen Lan, Xiaqiang Tang, Qingsong Wen, Yuxuan Liang, and Weigao Sun. “Improving Bilinear RNN with Closed-loop Control” . In: The Thirty-ninth Annual Conference on Neural Information Processing Systems. 2025.

Kazuki Irie, Robert Csordas, and Jurgen Schmidhuber. “Metalearning Continual Learning Algorithms” . In: Transactions on Machine Learning Research (2025). issn: 2835-8856. url: <https://openreview.net/forum?id=IaUh7CSD3k>.

Kazuki Irie and Samuel J Gershman. “Fast weight programming and linear transformers: from machine learning to neurobiology” . In: arXiv preprint arXiv:2508.08435 (2025).

Ben Keigwin, Dhruv Pai, and Nathan Chen. Gram-Space Manifold Muon. Oct. 2025. url: <https://blog.tilderresearch.com/vignettes/gram-space%7D>.

Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. “Minimax-01: Scaling foundation models with lightning attention” . In: arXiv preprint arXiv:2501.08313 (2025).

Saleh Momeni, Sahisnu Mazumder, Zixuan Ke, and Bing Liu. “In-context continual learning assisted by an external continual learner” . In: Proceedings of the 31st International Conference on Computational Linguistics. 2025, pp. 7292 – 7306.

Matteo Pagliardini, Pierre Ablin, and David Grangier. “The AdEMAMix Optimizer: Better, Faster, Older” . In: The Thirteenth International Conference on Learning Representations. 2025. url: <https://openreview.net/forum?id=jj7b3p5kLY>.

Renhao Pei, Yihong Liu, Peiqin Lin, Francois Yvon, and Hinrich Schutze. “Understanding In-Context Machine Translation for Low-Resource Languages: A Case Study on Manchu” . In: arXiv preprint arXiv:2502.11862 (2025).

Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Haowen Hou, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, et al. “RWKV-7” Goose” with Expressive Dynamic State Evolution” . In: arXiv preprint arXiv:2503.14456 (2025).

Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Haowen Hou, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, et al. “Rwkv-7” goose” with expressive dynamic state evolution” . In: arXiv preprint arXiv:2503.14456 (2025).

Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. “Optimizing Test-Time Compute via Meta

Reinforcement Finetuning” . In: Forty-second International Conference on Machine Learning. 2025. url: <https://openreview.net/forum?id=TqODUDsU4u>.

Chongjie Si, Debing Zhang, and Wei Shen. “Adamuon: Adaptive muon optimizer” . In: arXiv preprint arXiv:2507.11005 (2025).

48

[173] [174] [175] [176] [177] [178] [179]

[180]

Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. “DeltaProduct: Improving State-Tracking in Linear RNNs via Householder Products” . In: The Thirty-ninth Annual Conference on Neural Information Processing Systems. 2025. url: <https://openreview.net/forum?id=SoRiaijTGr>.

David Silver and Richard S Sutton. “Welcome to the era of experience” . In: Google AI 1 (2025).

Richard S. Sutton. The OaK Architecture: A Vision of SuperIntelligence from Experience. Keynote at the Reinforcement Learning Conference (RLC). Accessed: 2025-12-02. Aug. 2025. url: <https://rlj.cs.umass.edu/rlc-2025>.

Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham M. Kakade. “SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling” . In: The Thirteenth International Conference on Learning Representations. 2025. url: <https://openreview.net/forum?id=IDxZhXrpNf>.

Ke Alexander Wang, Jiaxin Shi, and Emily B Fox. “Test-time regression: a unifying framework for designing sequence models with associative memory” . In: arXiv preprint arXiv:2501.12352 (2025).

Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, junxian guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. “DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads” . In: The Thirteenth International Conference on Learning Representations. 2025. url: <https://openreview.net/forum?id=cFu7ze7xUm>.

hongzhou yu, Tianhao Cheng, Yingwen Wang, Wen He, Qing Wang, Ying Cheng, Yuejie Zhang, Rui Feng, and Xiaobo Zhang. “FineMedLM-o1: Enhancing Medical Knowledge Reasoning Ability of LLM from Supervised Fine-Tuning to Test-Time Training” . In: Second Conference on Language Modeling. 2025. url: <https://openreview.net/forum?id=7ZwuGZCpww>.

Yifan Zhang, Zhen Qin, and Quanquan Gu. “Higher-order Linear Attention” . In: arXiv preprint arXiv:2510.27258 (2025).

49

A. 중첩 학습 (Nested Learning) 과 중첩 시스템 (Nested Systems) 을 위한 일반화된 정식화

이 절에서는 3 절에서 논의한 중첩 시스템과 NSAM 의 일반화된 버전을 논의한다.

정의 6 ((일반화된) 중첩 시스템 (Nested System)). (순서가 있는) 중첩 시스템이란 K 개의 (순서가 있는) 레벨을 가진 시스템으로서, 각레벨 $1 \leq k \leq K$ 는 최적화 문제들의 집합 $\{(\mathcal{L}_i^{(k)}, \mathcal{C}_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$ 로구성된다. 여기서 $\mathcal{L}_i(\cdot; \cdot)$ 는 i 번째 문제의 최적화 목적함수이고, $\mathcal{C}_i$ 는 그것의 컨텍스트 (최적화 대상이되는 데이터) 이며, Θ_i 는 그것의 파라미터 집합이고, 각 최적화 문제는 경사 하강법 (gradient descent) 을 사용해 최적화된다. 이는등가적으로 다음과 같이 표현된다.

$$\theta_{i,t+1}^{(k)} = \arg \min_{\Phi_i^{(k)}} \mathcal{L}_i^{(k)}(\Phi_i^{(k)}; x_{t+1}) + \frac{1}{2\eta_{i,t+1}^{(k)}} \|\Phi_i^{(k)} - \theta_{i,t}^{(k)}\|_2^2 \quad \text{where } x_{t+1} \sim \mathcal{C}_i^{(k)}, \quad \text{and } \Phi_i^{(k)} \in \Theta_i^{(k)}.$$

정의 7 ((일반화된) 연상 기억 (Associative Memories) 의 중첩시스템). 연상 기억의 중첩 시스템 (NSAM) 이란 K 개의 (순서가 있는) 레벨을 가진 시스템으로서, 각 레벨 $1 \leq k \leq K$ 는 최적화 문제들의 집합 $\{(\mathcal{L}_i^{(k)}, \mathcal{C}_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$ 로구성된다. 여기서 $\mathcal{C}_i = \{(\mathbf{k}_j^{(i)}, \mathbf{v}_j^{(i)})\}_{j=1}^{L_i}$ 는정답 (ground-truth) 매핑들의 집합이고, $\mathcal{L}_i(\cdot; \cdot, \cdot)$ 는 i 번째 문제에서 기억이 학습한매핑들의 품질을 측정하는 최적화 목적 함수이며, Θ_i 는 기억 파라미터들의 집합이고, 각 최적화 문제는경사 하강법을 사용해 최적화된다.

$$\theta_{i,t+1}^{(k)} = \arg \min_{\Phi_i^{(k)}} \mathcal{L}_i^{(k)}(\Phi_i^{(k)}; \mathbf{k}_{t+1}^{(i)}, \mathbf{v}_{t+1}^{(i)}) + \frac{1}{2\eta_{i,t+1}^{(k)}} \|\Phi_i^{(k)} - \theta_{i,t}^{(k)}\|_2^2 \quad \text{where } (\mathbf{k}_{t+1}^{(i)}, \mathbf{v}_{t+1}^{(i)}) \sim \mathcal{C}_i^{(k)}, \quad \text{and } \Phi_i^{(k)} \in \Theta_i^{(k)}.$$

B. 연상 기억 모듈로서의 Adam, AdaGrad 및 기타 유사옵티마이저

역전파 (backpropagation) 에서 연쇄 법칙 (chain rule) 을 사용하지 않고 모멘텀항을 다시 살펴 보면,

$$\begin{aligned} W_{\ell,t+1} &= W_{\ell,t} + \mathbf{m}_{\ell,t+1} \\ \mathbf{m}_{\ell,t+1} &= \alpha_{\ell,t+1} \mathbf{m}_{\ell,t} - \eta_{\ell,t+1} \nabla_{W_{\ell,t}} \mathcal{L}(W_{\ell,t}; \mathbf{x}_{t+1}), \end{aligned}$$

이 되며, 모멘텀을 키 (key) 혹은 값 (value) 이 없는 연상 기억으로 해석할 수있다 (Behrouz et al. 2025b, 5 절 참조). 여기서 경사 항 $\nabla_{W_{\ell,t}} \mathcal{L}(W_{\ell,t}; \mathbf{x}_{t+1})$ 가모멘텀 안으로 압축된다. 우리는 강력한 모멘텀 항이 학습 과정에서 과거의모든 경사들을 완벽하게 기억하여 현재의 가중치 갱신을 더 잘 모델링할 수있기를 기대한다. 이를 위해, 다음과 같은 간단한 목적함수를 정의하는것에서 출발할 수 있다.

$$\tilde{\mathcal{L}}_t = \sum_{i=1}^t \|\mathbf{m}_{\ell,t} \odot \mathbf{g}_{\ell,i+1} - \mathbf{P}_{\ell,t}\|_2^2 + \lambda \|\mathbf{m}_{\ell,t}\|_F^2,$$

여기서 $\mathbf{g}_{\ell,t+1} = -\nabla_{W_{\ell,t}} \mathcal{L}(W_{\ell,t}; \mathbf{x}_{t+1})$ 이다. 이 목적함수는 경사들을 단순히 1 로 매핑하는 (이는 또한 모멘텀에서 제한적인기억 관리를 초래한다) 모멘텀 항을 찾는 것이 아니라, 경사들을 과거 데이터 샘플들의 전역적 속성 (global property) 으로 매핑하는 모멘텀 항을 찾는 것을목표로 한다. 이 전역적 속성이 더 표현력 있을수록, 모멘텀은 과거로부터압축된 정보를 더 정확하게 반영할 수 있다. 이에 따라, 식 101 의 목적함수는경사들을 $\mathbf{P}_{\ell,t+1}$ 로 매핑하는 최적의 연상 기억을다음과 같이 인정한다.

$$\mathbf{m}_{\ell,i}^{(t)*} = \left(\mathbf{H}_{\ell,i}^{(t)} + \lambda \ell \mathbf{I} \right)^{-1} \odot \mathbf{M}_{\ell,i}^{(t)} = \left(\mathbf{H}_{\ell,i}^{(t)} + \lambda \ell \mathbf{I} \right)^{-1} \odot \tilde{\mathbf{M}}_{\ell,i+1}^{(t)} \odot \mathbf{P}_{\ell,t},$$

여기서

$$\mathbf{M}_{\ell,i+1}^{(t)} = \mathbf{M}_{\ell,i}^{(t)} + \beta_1 \mathbf{g}_{\ell,i+1} \odot \mathbf{P}_{\ell,t} = \tilde{\mathbf{M}}_{\ell,i+1}^{(t)} \odot \mathbf{P}_{\ell,t},$$

(t)

$$\tilde{\mathbf{M}}_{\ell,i+1}^{(t)} = \mathbf{M}_{\ell,i}^{(t)} + \beta_1 \mathbf{g}_{\ell,i+1},$$

$$\mathbf{H}_{\ell,i+1}^{(t)} = \mathbf{H}_{\ell,i}^{(t)} + \beta_2 \mathbf{g}_{\ell,i+1} \odot \mathbf{g}_{\ell,i+1} = \mathbf{H}_{\ell,i}^{(t)} + \beta_2 \mathbf{g}_{\ell,i+1}^2.$$

이 해 (solution) 가 주어지면, 갱신 단계 (update step) 를 다음과 같이 쓸 수있다:

$$W_{\ell_{i+1}} = W_{\ell_i} - \eta_t \mathbf{m}_{\ell_i}^{(t)*} = W_{\ell_i} - \eta_i \left(\mathbf{H}_{\ell_i}^{(t)} + \lambda_\ell \mathbf{I} \right)^{-1} \odot \tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)} \odot \mathbf{P}_{\ell_t}.$$

우리는 간단한 경우, 즉 $\mathbf{P}_{\ell_t}$ 가 과거 그래디언트들의제곱의 합인 경우에서 출발한다. 다시 말해 $\mathbf{P}_{\ell_t} = \sum_{i=1}^t \mathbf{g}_{\ell_{i+1}}^2$ 이다. $\lambda \rightarrow 0$ 의 선택으로, 식 103 은 모멘텀 (momentum) 을 갖는 단순경사하강법 (gradient descent) 을 복원한다. 즉, $\lambda = 0$ 이라 두면다음을 얻는다:

$$W_{\ell_{i+1}} = W_{\ell_i} - \eta_t \mathbf{m}_{\ell_i}^{(t)*} = W_{\ell_i} - \eta_i \underbrace{\left(\mathbf{H}_{\ell_i}^{(t)} \right)^{-1}}_{\mathbf{H}_{\ell_i}^{(t)} / \beta_2} \odot \tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)} \odot \mathbf{P}_{\ell_t} = W_{\ell_i} - \eta_t \beta_2 \tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)},$$

이는 $\tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)}$ 의 정의에 기반하면, 이갱신 규칙 (update rule) 은 모멘텀을 갖는 경사하강법과 동등하다. 다음으로, 우리는 좀 더 정교한 설계 선택을 탐구하는데, 여기서는 $\mathbf{P}_{\ell_t}$ 를 토큰 $t + 1$ 이전 데이터 표본들의분산 (variance) 으로 사용한다. 이 경우 형식적으로 $\mathbf{P}_{\ell_t} = \sqrt{\sum_{i=1}^t \mathbf{g}_{\ell_{i+1}}^2}$ 이며, 따라서 갱신 규칙은 다음과 같다:

$$W_{\ell_{i+1}} = W_{\ell_i} - \eta_t \mathbf{m}_{\ell_i}^{(t)*} = W_{\ell_i} - \eta_i \left(\mathbf{H}_{\ell_i}^{(t)} + \lambda_\ell \mathbf{I} \right)^{-1} \odot \tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)} \odot \underbrace{\mathbf{P}_{\ell_t}}_{\frac{1}{\sqrt{\mathbf{H}_{\ell_i}^{(t)}}} / \beta_2} \approx W_{\ell_i} - \frac{\eta_t}{\sqrt{\beta_2}} \frac{\tilde{\mathbf{M}}_{\ell_i}^{(t)}}{(\mathbf{H}_{\ell_i}^{(t)})^{1/2} + \varepsilon}$$

이는 널리 쓰이는 Adam 옵티마이저 (Kingma et al. 2014a) 와 동등하다. 따라서, Adam 은 식 101 에서 정의된 L_2 회귀 목적함수 (regression objective) 가 주어졌을 때의 최적 연상 메모리 (associative memory) 이다. 실제로, 각 상태 (state) 에서의 Adam 의 갱신 요소는 그래디언트와 (과거데이터 표본의 전역적 성질로서의) 그 분산 사이의 사상 (mapping) 을 학습하는것을 목표로 한다. 이 정식화 (formulation) 에 관한 한 가지 흥미로운 점은요소들의 빈도 (frequency) 에 관한 것이다. Adam 옵티마이저의 첫 번째와 두번째 모멘텀 (momentum) 을 살펴보면, 이 두 메모리의 갱신 빈도는 서로 동일한데, 둘 다 각 표본 이후에 갱신되기 때문이다. 더 나아가, 이들 각각의계산은 병렬로 수행될 수 있으며 따라서 서로 독립적이다. 그에 따라, 이는어떠한 내부 그래디언트 흐름 (internal gradient flow) 도 없는 두 요소가동일한 빈도, 따라서 동일한 레벨 (level) 에 놓이는 몇 안 되는 예 중하나이다 (3.2 절 참조).

요소별 (element-wise) 갱신을 넘어서, 우리는 식 101 을 외적 (outer-product) 연산으로 다음과 같이 재정식화한다:

$$\tilde{\mathcal{L}}_t = \sum_{i=1}^t \left\| \mathbf{m}_{\ell_i} \mathbf{g}_{\ell_{i+1}} - \mathbf{P}_{\ell_t} \right\|_2^2 + \lambda_\ell \left\| \mathbf{m}_{\ell_t} \right\|_F^2,$$

여기서 $\mathbf{g}_{\ell_{t+1}} = -\nabla_{W_{\ell_t}} \mathcal{L}(W_{\ell_t}; \mathbf{x}_{t+1})$ 이다. 식 102 와 유사하게, 위 목적함수에 대한 최적해 (optimal solution) 를 찾는것은 다음과 같이 정의된다:

$$\mathbf{m}_{\ell_i}^{(t)*} = \left(\mathbf{H}_{\ell_i}^{(t)} + \lambda_\ell \mathbf{I} \right)^{-1} \mathbf{M}_{\ell_i}^{(t)} = \left(\mathbf{H}_{\ell_i}^{(t)} + \lambda_\ell \mathbf{I} \right)^{-1} \tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)} \mathbf{P}_{\ell_t}, \quad \text{where}$$

$$\mathbf{M}_{\ell_{i+1}}^{(t)} = \mathbf{M}_{\ell_i}^{(t)} + \beta_1 \mathbf{P}_{\ell_t} \mathbf{g}_{\ell_{i+1}}^\top = \mathbf{P}_{\ell_t} \tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)},$$

$$\tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)} = \mathbf{M}_{\ell_i}^{(t)} + \beta_1 \mathbf{g}_{\ell_{i+1}}^\top,$$

$$\mathbf{H}_{\ell_{i+1}}^{(t)} = \mathbf{H}_{\ell_i}^{(t)} + \beta_2 \mathbf{g}_{\ell_{i+1}} \mathbf{g}_{\ell_{i+1}}^\top.$$

Adam 과 유사한 선택, 즉 $\mathbf{P}_{\ell_t}$ 를 지금까지의 그라디언트들의 분산으로 두어 $\mathbf{P}_{\ell_t} = \sqrt{\sum_{i=1}^t \mathbf{g}_{\ell_{i+1}} \mathbf{g}_{\ell_{i+1}}^\top}$ 라하면, 위 해는 다음과 같이 단순화될 수 있다:

$$W_{\ell_{i+1}} = W_{\ell_i} - \eta_t \mathbf{m}_{\ell_i}^{(t)*} = W_{\ell_i} - \eta_t \left(\mathbf{H}_{\ell_i}^{(t)} + \lambda_t \mathbf{I} \right)^{-1} \mathbf{P}_{\ell_t}^{-1/2} \tilde{\mathbf{M}}_{\ell_{i+1}}^{(t)} \approx W_{\ell_i} - \frac{\eta_t}{\sqrt{\beta_2}} \underbrace{\mathbf{H}_{\ell_i}^{(t)}}_{\frac{1}{\sqrt{\mathbf{H}_{\ell_i}^{(t)}}^2} / \beta_2} \mathbf{M}_{\ell_i}^{(t)}$$

위 정식화는 모멘텀을 갖는 AdaGrad(Defossez et al. 2022) 이며 따라서 AdaGrad(Duchi et al. 2011) 를 일반화한다. 즉, $\beta_1 = 1$ 일 때가 이에 해당한다. 마찬가지로, Adam 옵티마이저 (Kingma et al. 2014a) 가 RMSProp(Hinton et al. 2012), SignSGD 및 그 모멘텀 기반 변형들 (Bernstein et al. 2018), NAdam(Dozat 2016), AMSGrad(Reddi et al. 2016), RAdam(Liu et al. 2020), Lion(Chen et al. 2023) 등과 같은 다른 알고리즘들과 갖는 연관성에 기반하고, 또한 AdaGrad 가 Shampoo(Gupta et al. 2018), Soap(Vyas et al. 2025) 와 같은 옵티마이저들과 갖는 연관성—즉, 사전조건화 (preconditioning) 항의 근사로서의 연관성—을 고려하면, 우리는 이 모든 옵티마이저들이 그라디언트를 압축 (compress) 하는 것을 목표로 하는 연상 메모리로 재정식화될 수 있다고 결론지을 수 있다.

C. 정규화를 포함한 델타 경사하강법 (Delta Gradient Descent with Normalization)

4.5 절에서 우리는 경사하강법이 연상 메모리로 볼 수 있으며, 다음과 같이 재정식화될 수 있음을 논의하였다:

$$W_{t+1} = \arg \min_W \langle W \mathbf{x}_t, \nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t) \rangle + \frac{1}{2\eta_t} \|W - W_t\|_2^2,$$

여기서 각 단계는 그라디언트 방향의 음수 (negative) 를 학습하는 것을 목표로 한다. 이 학습 규칙이 현재 그라디언트에만 의존하는 갱신 항들을 사용한다는 사실에 근거하여, 우리는 $\mathbf{u}_t = -\nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t)$ 로 정의하였고, 위 과정을 L_2 회귀 손실이라는 더 표현력 있는 목적함수를 갖는 델타 경사하강법 (Delta Gradient Descent) 으로 확장하였다:

$$W_{t+1} = \arg \min_W \frac{1}{2} \|\mathbf{W} \mathbf{x}_t - \mathbf{u}_t\|_2^2 + \frac{1}{2\eta_t} \|W - W_t\|_2^2.$$

우리는 $\mathbf{x}_t$ 가 정규화 (normalized) 되어 있다고 가정한다 (예를 들어, 정규화된 메모리 시스템에서 혹은 정규화 계층을 갖는 신경망에서, $\|\mathbf{x}_t\|_2 = \lambda$). 위 목적함수를 최적화하기 위해 그라디언트를 취하면,

$$2(W_{t+1} \mathbf{x}_t - \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t)) \mathbf{x}_t^\top + 2\eta_t (W_{t+1} - W_t) = 0,$$

$$W_{t+1} (\mathbf{x}_t \mathbf{x}_t^\top + \eta_t \mathbf{I}) = \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top + \eta_t W_t,$$

\$\$

\$\$

$$\Rightarrow W_{t+1} = \left(\nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top + \eta_t W_t \right) (\mathbf{x}_t \mathbf{x}_t^\top + \eta_t \mathbf{I})^{-1}.$$

이는 다음과 같은 결과를 낳는다:

$(\mathbf{x}_t \mathbf{x}_t^\top + \eta_t \mathbf{I})^{-1}$ 항을 셔먼-모리슨 보조정리 (Sherman-Morrison lemma) 로 계산하기 위해, 우리는 다음을 얻는다:

$$\left(x_t x_t^\top + \eta_t I\right)^{-1} = \frac{1}{\eta_t} \left(I - \frac{1}{\lambda^2 + \eta_t} x_t x_t^\top\right),$$

그리고 따라서:

$$\begin{aligned} W_{t+1} &= \left(\nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top + \eta_t W_t\right) \frac{1}{\eta_t} \left(I - \frac{1}{\lambda^2 + \eta_t} x_t x_t^\top\right) \\ \Rightarrow W_{t+1} &= W_t \left(I - \frac{1}{\lambda^2 + \eta_t} x_t x_t^\top\right) + \frac{1}{\eta_t} \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top - \underbrace{\frac{1}{\lambda^2 \eta_t + \eta_t^2} \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top x_t x_t^\top}_{\frac{\lambda}{\lambda^2 \eta_t + \eta_t^2} \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top} \\ &= \left(\frac{\lambda}{\lambda^2 + \eta_t}\right) \left(-\frac{1}{\lambda^2 \eta_t + \eta_t^2}\right) \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top - \frac{1}{\eta_t} x_t x_t^\top \\ \Rightarrow W_{t+1} &= W_t \left(I - \alpha_t x_t x_t^\top\right) - \beta \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top \\ \Rightarrow W_{t+1} &= W_t (I - \dots) \end{aligned}$$

번역·검증 고지

이 문서는 분석이나 평가를 추가하지 않는 것을 원칙으로 하며, 원문의 주장 강도와 수치를 유지하도록 작성되었다. 한국어 초안 작성과 구조 검수에는 Codex GPT-5.6 을 사용했고, 고정 source hash·원문 보존 부록·역대조 표를 통해 검증했다. 불일치가 있을 때에는 원문이 우선한다.

원문 구조·그림 보존 부록

아래 부록은 고정 원문의 equation, table, figure, caption 과 section 배치를 대조하기 위한 검증판이다. arXiv/proceedings rights; translated derivative and source plate are internal-only

Nested Learning: The Illusion of Deep Learning Architecture

Ali Behrouz , Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni



Abstract

Over the last decades, developing more powerful neural architectures and simultaneously designing optimization algorithms to effectively train them have been the core of research efforts to enhance the capability of machine learning models. Despite the recent progresses, particularly in developing Language Models (LMs), there are fundamental challenges and unanswered questions about how such models can *continually learn/memorize, self-improve, and find effective solutions*. In this paper, we present a new learning paradigm, called Nested Learning (NL), that coherently represents a machine learning model with a set of nested, multi-level, and/or parallel optimization problems, each of which with its own “*context flow*”. Through the lenses of NL, existing deep learning methods learn from data through *compressing* their own context flow, and *in-context learning* naturally emerges in large models. NL suggests a philosophy to design more expressive learning algorithms with more “*levels*”, resulting in higher-order in-context learning and potentially unlocking effective continual learning capabilities. In addition to its neuro-scientific motivation, we advocate for NL by presenting three core contributions: (1) Expressive Optimizers: We show that known gradient-based optimizers, such as Adam, SGD with Momentum, etc., are in fact associative memory modules that aim to compress the gradients’ information (by gradient descent). Building on this insight, we present other “more expressive” optimizers with deep memory and/or more powerful learning rules; (2) Self-Modifying Learning Module: Taking advantage of NL’s insights on learning algorithms, we present a sequence model that learns how to modify itself by learning its own update algorithm; and (3) Continuum Memory System: We present a new formulation for memory system that generalizes the traditional viewpoint of “long-term/short-term memory”. Combining our self-modifying sequence model with the continuum memory system, we present a continual learning module, called HOPE, showing promising results in language modeling, knowledge incorporation, and few-shot generalization tasks, continual learning, and long-context reasoning tasks.

“We cannot solve our problems with the same thinking we used when we created them!”

— Attributed to Albert Einstein

1 Introduction

For decades, AI research has focused on designing machine learning algorithms that learn from data (Pitts 1943; McCulloch et al. 1948; McCulloch 1949; Samuel 1959) or experience (Sutton et al. 1998; Connell et al. 1999; Silver et al. 2025); often by optimizing an objective $\mathcal{L}(\theta)$ over parameters $\theta \in \Theta$ with gradient-based methods. While traditional machine learning techniques required careful engineering and domain expertise to design feature extractors, limiting their ability to directly process and learn from natural data (LeCun et al. 2015), deep representation learning offered a fully automated alternative to discover the representations needed for the task. Thereafter, deep learning has been an inseparable part of the large-scale computational models with seminal success in chemistry and biology (Jumper et al. 2021), games (Silver et al. 2016, 2018), computer vision (Krizhevsky et al. 2012; Dosovitskiy et al. 2021), and multimodal and natural language understanding (Achiam et al. 2023; Liu et al. 2024a; Comanici et al. 2025).

Stacking of multiple layers, as it is done in deep learning models, provides the models with better expressive power in representing complex features, and more internal computations (e.g., #FLOPS) (Montúfar et al. 2014; Poole et al. 2016; Hestness et al. 2017), all of which are critical and desirable characteristics for static tasks that require in-distribution predictions over an a-priori fixed set. This deep design, however, is not a universal solution to all the challenges and cannot help the expressive power of the models in multiple aspects, for example: (i) The computational depth of deep models might not change with more layers (Merrill et al. 2022; Sanford et al. 2024), leaving their ability to implement complex algorithms untouched compared to traditional shallow approaches (Merrill et al. 2024); (ii) The capacity of some class of parameters might show marginal improvement with increasing the depth/width of the model (Kaplan et al. 2020); (iii) The training process might converge to a suboptimal solution, mainly due to the suboptimal choice of the optimizer

[§]Correspondence to: {alibehrouz, razaviyayn, mirrokni}@google.com and peilin.zhong@columbia.edu.

[§]A version of this work is published at Neural Information Processing Systems (NeurIPS) 2025.

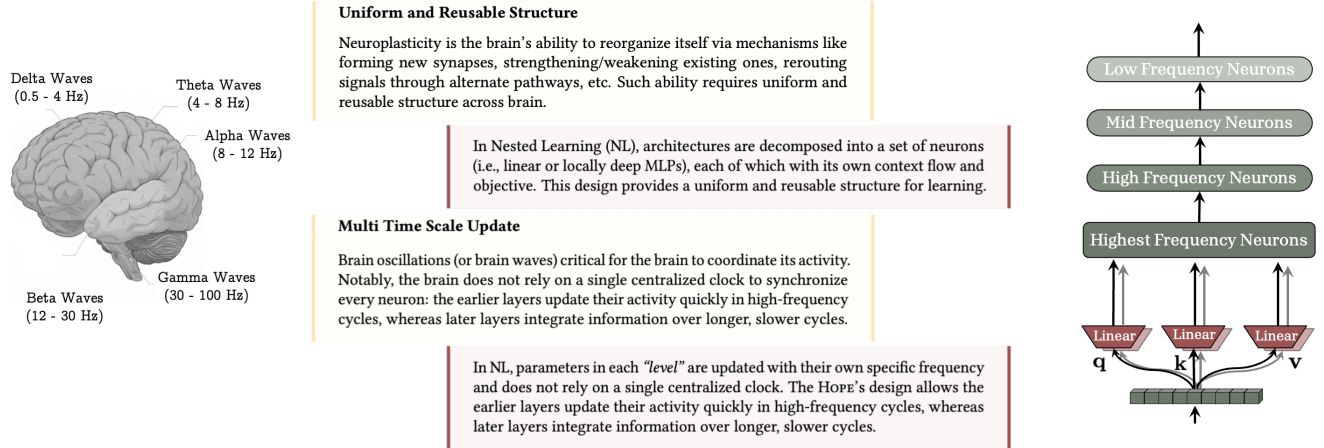


Figure 1: The uniform and reusable structure as well as multi time scale update in the brain are the key components to unlock the continual learning in humans. Nested Learning (NL) allows for multi time-scale update for each component of the brain, while showing that well-known architectures such as Transformers are in fact linear layers with different frequency updates.

or its hyperparameters; and (iv) The model's ability to fast adapt to a new task, continually learn, and/or generalize to out-of-distribution data might not change with stacking more layers and requires more careful designs.

The core part of the efforts to overcome the above challenges and to enhance the capability of deep learning models concentrate on: (1) developing more expressive class of parameters (i.e., neural architectures) (Fukushima 1980; Schmidhuber et al. 1997; Krizhevsky et al. 2012; Vaswani et al. 2017; Behrouz et al. 2025c); (2) introducing objectives that can better model the tasks (Rumelhart et al. 1986; Kingma et al. 2014b; Hjelm et al. 2019; Goodfellow et al. 2020; Alshammari et al. 2025); (3) designing more efficient/effective optimization algorithms to find better solutions or with more resilience to forgetting (Kingma et al. 2014a; Gupta et al. 2018; Farajtabar et al. 2020; Jordan et al. 2024); and (4) scaling the model size to enhance its expressivity, when the "right" choice of architecture, objective, and optimization algorithms are made (Brown et al. 2020; Kaplan et al. 2020; Hoffmann et al. 2022). Collectively, these advancements and new findings on scaling patterns of deep models have established the foundations upon which Large Language Models (LLMs) have been built.

The development of LLMs marks a pivotal milestone in deep learning research: a paradigm shift from task-specific models to more general-purpose systems with various emergent capabilities as a result of scaling the "right" architectures (Brown et al. 2020; Schaeffer et al. 2023). Despite all their success and remarkable capabilities in diverse sets of tasks (Nijkamp et al. 2023; Wang et al. 2023; Comanici et al. 2025), LLMs are largely static after their initial deployment phase, meaning that they successfully perform tasks learned during pre- or post-training, but are unable to continually acquire new capabilities beyond their immediate context. The only adaptable component of LLMs is their *in-context learning* ability—a (known to be emergent) characteristic of LLMs that enables fast adaption to the context and so perform zero- or few-shot tasks (Brown et al. 2020). Beyond in-context learning, recent efforts to overcome the static nature of LLMs either are computationally expensive, require external components, lack generalization, and/or might suffer from catastrophic forgetting (Akyürek et al. 2024a; Eyuboglu et al. 2025; yu et al. 2025), which has led researchers to question if there is a need to revisit how to design machine learning models and if a new learning paradigm beyond stacking of layers is required to unleash the capabilities of LLMs in continual setups.

Current Models only Experience the Immediate Present. As an analogy and to better illustrate the static nature of LLMs, we use the example of anterograde amnesia—a neurological condition where a person cannot form new long-term memories after the onset of the disorder, while existing memories remain intact (Scoville et al. 1957). This condition limits the person's knowledge and experiences to a short window of present and long past—before the onset of the disorder—which results in continuously experiencing the immediate present as if it were always new. The memory processing system of current LLMs suffer from a similar pattern. Their knowledge is limited to either, the immediate context that fits into their context window, or the knowledge in MLPs that stores long-past, before the onset of "*end of pre-training*." This analogy, has motivated us to take inspiration from neurophysiology literature and how brain consolidate its short-term memories.

1.1 Human Brain Perspective and Neurophysiological Motivation

Human brain is highly efficient and effective when it comes to continual learning, which is often attributed to neuroplasticity—the brain’s remarkable capacity to change itself in response to new experiences, memories, learning, and even damage (Pascual-Leone et al. 2005; Johnston 2009). Recent studies support that the formation of Long-term memory involves at least two distinct but complementary consolidation processes (Frey et al. 1997; Goto et al. 2021; Yang et al. 2024): (1) A rapid “online” consolidation (also known as synaptic consolidation) phase occurs immediately or soon after learning, even during wakefulness. This is when new and initially fragile memory traces are stabilized and begin transferring from short-term to long-term storage; (2) An “offline” consolidation (also known as systems consolidation) process repeats the replay of the recently encoded patterns—during sharp-wave ripples (SWRs) in the hippocampus, coordinated with cortical sleep spindles and slow oscillations—strengthens and reorganizes the memory and supports transfer to cortical sites (Foster et al. 2006; Ji et al. 2007; Peyrache et al. 2009).

Coming back to the analogy of anterograde amnesia, evidence indicates that the condition can impact both stages, but especially the online consolidation phase, mainly due to the fact that hippocampus is the gateway for encoding new declarative memories, and so its damage means new information never will be stored in long-term memory. As mentioned above, the design of LLMs, and more specifically Transformer-based backbones, suffers from a similar condition after the pre-training phase. That is, the information provided in the context, never impacts the long-term memory parameters (e.g., feedforward layers), and so the model is not capable of acquiring new knowledge or skill, unless the information is still stored in the short-term memory (e.g., in-context or attention). To this end, although the second stage is equally, or even more, crucial for the consolidation of memories, and its absence can damage the process and might cause loss of memory (Drummond et al. 2000; Yoo et al. 2007), in this work, we focus on the first stage: memory consolidation as an online process. As discussed earlier, the memory processing, its online consolidation, and so continual learning ability of humans are known to be highly relied on the neuroplasticity as well as neural oscillations (Bliss et al. 1993; Buzsaki et al. 2004; Klinzing et al. 2019).

Multi Time scale Processing System. Brain oscillations (also known as brainwaves)—a rhythmic fluctuations in brain activity—is not mere byproducts of brain function but is increasingly understood to play a crucial role in various cognitive functions such as attention, memory, and decision-making, and to be a core mechanism for organizing neural computation, coordinating communication between brain regions, and gating the synaptic plasticity that underlies learning and memory (Fell et al. 2011; Cavanagh et al. 2014; Fries 2015). These brainwaves are the results of the brain coordinating its computations in different timescales and frequency updates, where each frequency determines how often groups of brain neurons become active and share updated information. More specifically, such neural oscillations are typically categorized into distinct frequencies, each of which has been associated with different cognitive functions and, critically, different timescales of information processing: ranging from (1) fast Gamma waves (frequency of 30-150 Hz) that are mainly associated with sensory information to (2) Beta waves (frequency of 13 - 30 Hz) that are mainly associated with active thinking (Buzsaki et al. 2004; Buschman et al. 2007; Lundqvist et al. 2016), and (3) slow Delta and Theta waves (frequency of 0.5 - 8 Hz), mainly responsible for memory consolidation and learning (Marshall et al. 2006; Diekelmann et al. 2010; Ngo et al. 2013; Staresina et al. 2015; Heusser et al. 2016; Daume et al. 2024).

In deep learning models, however, the weights of the architectures are fixed at test time and also it is common in pre-training to use the same update rate for all the blocks/layers in the model. Later, in Section 6, however, we show that in-context learning provides an extreme case of this design and in fact, Transformer architectures are based on two extreme frequencies of update: i.e., ∞ and 0 for attention and MLP blocks, respectively.

Brain’s Uniform and Reusable Structure. As discussed earlier, neuroplasticity is the brain’s remarkable capability to change itself in response to new memories, knowledge, and even damage (Pascual-Leone et al. 2005; Johnston 2009). This characteristic suggests a uniform architecture where neural elements are not rigidly dedicated to one function but are instead reusable, capable of being flexibly redeployed to support different cognitive needs. One real-world example of neural reusability is hemispherectomy—the surgical removal or disabling of one cerebral hemisphere, usually to alleviate severe epilepsy. Amazingly, if this surgery is done in childhood, patients can lead largely normal lives into adulthood with high functioning cognition and intact neural network organization that contains all the same core brain networks present in a typical two-hemisphere brain (networks for language, vision, etc.). This extraordinary outcome provides real-life proof of the brain’s uniform architecture. That is, even half a brain can reallocate resources and reorganize so that the person can function extremely well. Such cases, along with documented instances of individuals living relatively normally with missing pieces of cortex, highlight the brain’s uniform and reusable structure.

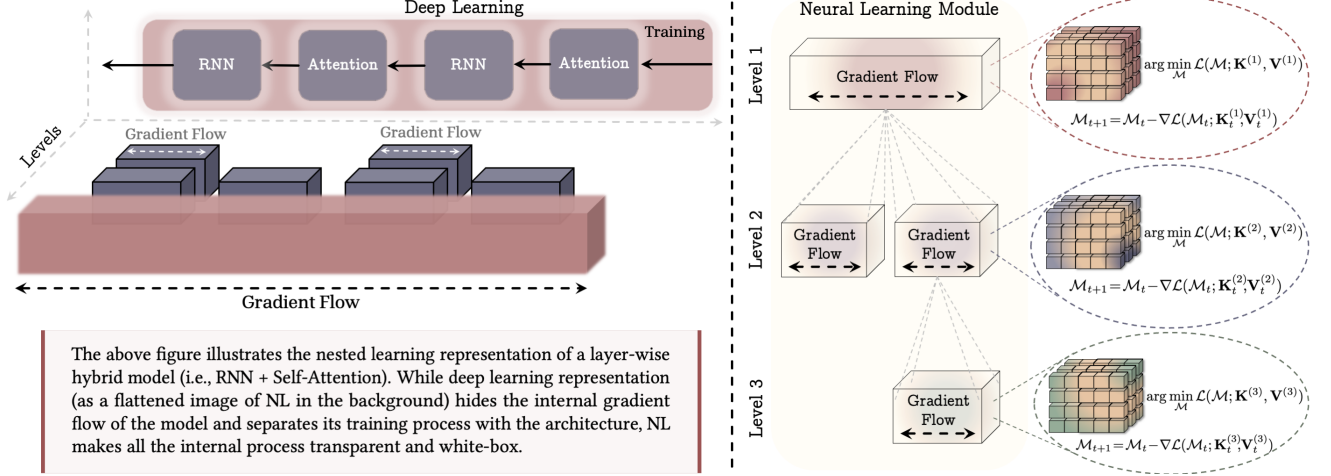


Figure 2: Nested Learning Paradigm that represent a machine learning model and its training procedure as a set of nested optimization problems. **(Left)** An example of Hybrid architecture. While deep learning perspective, as the flattened image of NL, does not provide insight about the depth of computation in the blocks, NL transparently represent all the inner gradient flows. **(Right)** A Neural Learning Module: A computational model that learns how to compress its own context flow. For example, the first level corresponds to the model’s most outer-loop training, often refer to as “pre-training” step.

Furthermore, this interpretation of brain’s uniform and reusable structure demonstrate that memory in human brain is not an isolated system in some specific areas, and mainly is distributed across brain. That is, contrary to the traditional models of memory that often implied that different types of memory reside in distinct brain structures (e.g. short-term memory in frontal cortex vs. long-term memory in the hippocampus and cortex), modern research advocate for distributed neural circuits memory processing across multiple regions (Christophel et al. 2017; Kitamura et al. 2017; Roy et al. 2022).

The modern deep learning architectures in recent years, however, at least on the surface, seem to be heterogeneous and are based on a combination of a subset of self-attention variants (Vaswani et al. 2017), modern recurrent neural networks (Katharopoulos et al. 2020; Schlag et al. 2021; Behrouz et al. 2025c; Peng et al. 2025b), canon layers (Allen-Zhu 2025), global convolutions (Hasani et al. 2023; Poli et al. 2023), and MLP blocks (Shazeer 2020). This raises the question of whether we need a new uniform architecture, or if our beliefs about the heterogeneity of current models need to be revisited.

1.2 Contributions and Roadmap

In this paper, we aim to present a unifying learning paradigm that not only provides new insights about existing algorithms, methods, and architectures, but it also reveals a new dimension to stacking layers in deep learning with enhancement of the computational depth, and continual learning ability of models. After discussing preliminary concepts and backgrounds in §2, we present:

Nested Learning Paradigm (§3). To answer the questions raised above and to provide new insights on overcoming the design challenges in continual learning, architecture design, and computational depth of modern deep learning models, we present Nested Learning (NL)—a learning paradigm that allows each component of the machine learning model to have its own internal gradient flow on its own context in multiple levels, representing a model and its learning process (i.e., optimization) as an inter-connected system of nested, multi-level, and/or parallel optimization problems. We argue that the optimization process and the learning algorithms/architectures are fundamentally the same concepts but are in different levels of a system with different context (i.e., gradient vs. tokens). Furthermore, they are two inter-connected components where the learning algorithm/architecture generates the context for optimizers (i.e., the gradients), advocating for designing architecture-specific optimizers. We discuss different ways of knowledge transfer between levels, resulting in unifying and generalizing concepts like meta-learning, in-context learning, recurrent neural networks, hypernetworks, etc.

Optimizers and Architectures as Learning Module (§4, §5). Building on the NL’s viewpoint, we argue that training a *deep neural network* with backpropagation process and gradient descent is a compression and an optimization problem that aims to train an associative memory to map layers’ inputs to their corresponding local error in the prediction. Accordingly, we argue that pre-training is a form of in-context learning, where the context is the entire pre-training data and the layers are compressing the context into their parameters. We demonstrate that such arguments are also valid for other popular gradient-based optimizers and they are associative memories that aim to compress the gradients into their parameters. From NL’s terminology, gradient-based optimizers such as gradient descent with momentum, Adam (Kingma et al. 2014a), and AdaGrad (Duchi et al. 2011) can be decomposed into a two-level nested optimization problems, each of which is optimized with a simple gradient descent. In particular, this viewpoint makes it apparent that for compressing the gradients, in theory, Adam is the optimal associative memory with respect to the element-wise L_2 regression objective.

We revisit previous findings on representing architectures as associative memories (Behrouz et al. 2025b) and decompose their optimization process into a set of nested optimization problems, all of which optimized with gradient descent. Building on the above findings—i.e., popular gradient-based optimizers and modern architectures are both a set of nested and/or parallel optimization problems—we argue that the combination of these two—i.e., training an architecture with a specific optimizer—can also be represented as a set of nested and/or parallel optimization problem. Therefore, a neural learning module (a joint system of architecture and its training/optimization process) is a uniform model, in which all elements are linear or deep MLPs, while they are optimizing their own internal objective in different levels with different frequencies.

Building upon the associative memory perspective of optimizers, we design a set of new learning updates (optimization steps) with more expressive memory structure or memory management in compressing the gradients. In particular, we argue that the choice of optimizer depends on the context of the optimization. A powerful optimizer for compressing the gradients might not be the best choice for compressing the tokens. To this end, we present a new variant of gradient descent, called Delta Gradient Descent (DGD), that its update not only depends on the current input, but also the state of the weight of the neural network, resulting in capturing the dependencies of data samples without i.i.d. assumption.

Main Takeaways and Revisiting Common Terms: Continual and In-context Learning, Pre-Training, and Learning (§6). We discuss the main takeaways of NL about principal concepts and revisit some common terms: (1) We argue that continual learning can be viewed as a learning problem on a sequences of incoming contexts or episodes where different levels are responsible for compressing their own in-context knowledge and transfer it to higher levels. Based on this, we advocate for designing models and pipelines that do not rely on test/training phase, and rather continually manage their knowledge and memory; (2) In-context learning is the characteristic of “having multiple nested levels”. Accordingly, Transformers in-context learning comes from being a non-parametric solution to a certain regression objective on tokens, while modern recurrent models uses parametric learning processes in their lower levels; (3) We further revisit other terminologies such as learning/memorization, hybrid architectures, looped architectures, and learned optimizers.

Continuum Memory System, Self-Referential Titans, and Hope (§7, §8). We generalize the traditional viewpoint of “long-term/short-term memory” (LSM) by presenting the Continuum Memory Systems (CMSs) and see memory as a distributed inter-connected system with a spectrum of frequency updates. In this design, higher-frequency neurons are responsible for fast adaption but store memories/knowledge for a short period of time, while lower frequency neurons are responsible for more persistent knowledge. Comparing to LSM, we show that this multi-frequency design results in a loop process for memory of the model, meaning that knowledge can partially be recovered when it is forgotten. While we mainly design this memory system as a replacement of MLP blocks in Transformers, we take advantage of this intuition to design Multi-scale Momentum Muon (M3) optimizer—an optimization algorithm with multiple momentum terms—further supporting the importance of CMSs design in different contexts.

Evaluations (§9). To support the effectiveness of our proofs-of-concept as well as the importance of nested learning design, we perform experimental evaluation on (1) Continual learning and in-context learning tasks including (i) learning new language, (ii) class incremental learning, and (iii) question/answering on a new corpus; (2) Long context understanding tasks, including needle-in-a-haystack (Hsieh et al. 2024) and BABILong (Kuratov et al. 2024) benchmarks; (3) Language modeling and common-sense reasoning tasks; (4) In-context recall and memorization tasks; (5) Language recognition tasks; and (6) comparing different optimizers, including our M3 optimizer. Our results indicate the effectiveness of NL viewpoint in designing models with continual learning ability, having multiple levels of computations, and self-referential process.

2 Preliminaries

In this section we discuss the notations and review the background concepts.

Notations. We let $x \in \mathbb{R}^{N \times d_{\text{in}}}$ be the input, $\mathcal{M}_t$ represent the state of memory/model $\mathcal{M}$ at time t , $\mathbf{K}$ be the keys, $\mathbf{V}$ be the values, and $\mathbf{Q}$ be the query matrices. We use bold lowercase letters with subscript t to refer to the vector corresponds to the input t (i.e., $\mathbf{k}_t$, $\mathbf{v}_t$, and $\mathbf{q}_t$). We further refer to the distribution of any random variable $\mathcal{T}$ as $p(\mathcal{T})$. Throughout the paper, we use simple MLPs with $\mathcal{L}_{\mathcal{M}} \geq 1$ layers and residual connection as the architecture of the memory module $\mathcal{M}(\cdot)$. When it is needed, we parameterized the memory module with $\theta_{\mathcal{M}} \supseteq \{W_1, W_2, \dots, W_{\mathcal{L}_{\mathcal{M}}}\}$, which at least includes the parameters of linear layers in the MLP. We use superscript with parenthesis to refer to parameters in different *levels* of nested learning (different update frequency): i.e., either by using level index $W^{(\ell)}$ or its corresponding frequency $W^{(f_{\ell})}$.

Gradient Descent. Gradient descent is one of the most widely used optimization algorithms for high-dimensional problems, and its variants are standard tools for training large models. Given an objective $\mathcal{L}(\cdot; \cdot)$, the stochastic gradient descent (SGD) with step size $\eta_t > 0$ (a.k.a. learning rate) updates parameters by:

$$W_{t+1} = W_t - \eta_t \nabla_{W_t} \mathcal{L}(W_t; \mathbf{x}_t), \quad (1)$$

for data sample $\mathbf{x}_t$ from training set. The gradient descent formulation admits several equivalent characterizations that are useful in analysis. One of those equivalent formulations is steepest-descent in the Euclidean metric, where one step of gradient descent is equivalent to:

$$W_{t+1} = \arg \min_W \left\{ \langle \nabla_W \mathcal{L}(W_t; \mathbf{x}_t), W \rangle + \frac{1}{2\eta_t} \|W - W_t\|_2^2 \right\}, \quad (2)$$

which is minimizing a first-order Taylor approximation regularized by a quadratic proximal term. The GD step above is precisely a proximal update on the linearization of $\mathcal{L}(\cdot; \cdot)$ at W_t , revealing an implicit bias toward small moves in L_2 -distance. Accumulating steps (with a constant learning rate η) yields the *follow-the-regularized-leader* (FTRL) form

$$W_{t+1} = \arg \min_W \left\{ \left\langle \sum_{s=1}^t \nabla \mathcal{L}(W_s; \mathbf{x}_s), W \right\rangle + \frac{1}{2\eta} \|W - W_1\|_2^2 \right\}, \quad (3)$$

whose solution is $W_{t+1} = W_1 - \eta \sum_{s=1}^t \nabla \mathcal{L}(W_s; \mathbf{x}_s)$. These two formulations are used in this paper interchangeably. However, our discussions and formulations are generally valid for many other optimization algorithms as well.

Meta Learning. Designing an effective machine learning model often requires making decisions about its architecture parameterized by $\theta \in \Theta$, objective $\mathcal{L}(\theta)$, and an optimizer, aiming to iteratively optimize the objective. Meta learning paradigm (or learning to learn) (Schmidhuber et al. 1996; Finn et al. 2017; Akyürek et al. 2022; Irie et al. 2025a) aim to automate a part of such decisions by modeling it as a *two-level* optimization procedure, in which the outer model aims to learn to set parameters for the inner procedure to maximize the performance across a set of tasks. That is, given an objective parameterized by a parameter Φ : i.e., $\ell(\theta, \mathcal{D}; \Phi)$, one can formalize the outer loop process as optimizing parameter Φ over a set of tasks:

$$\Phi^* = \arg \min_{\Phi} \mathbb{E}_{\mathcal{T}_i \sim p(\mathcal{T})} \left[\ell(\theta, \mathcal{T}_i; \Phi) \right], \quad (4)$$

where $p(\mathcal{T})$ is the distribution of tasks. While initial studies on meta-learning used supervised settings for the outer loop (Schmidhuber et al. 1996), recently, a more flexible family of methods that use an unsupervised process for the outer loop has gained popularity (Finn et al. 2017; Brown et al. 2020; Akyürek et al. 2022; Chen et al. 2022; Qu et al. 2025). In addition to the growing interest to use meta learning methods for a diverse set of downstream tasks (Finn et al. 2017; Munkhdalai et al. 2019; Chen et al. 2022; Irie et al. 2025a; Qu et al. 2025), in recent years, it also has shown popularity as a paradigm to design powerful sequence models (Sun et al. 2024; Behrouz et al. 2025b,c).

Fast Weight Programmers (FWPs). Fast weight programmers (more recently refer to as linear Transformers) (Hinton et al. 1987; Schmidhuber 1992; Ba et al. 2016; Schlag et al. 2021), which also have been motivated from neuroscience perspective (Gershman et al. 2025), are recurrent neural networks whose memory (or hidden state) is *matrix-valued*: a time-varying

fast-weight matrix $\mathcal{M}_t \in \mathbb{R}^{d_{\text{out}} \times d_{\text{key}}}$ that serves as a short-term memory. A separate “programmer” (slow net) maps each input $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ to query, key, and value vectors and updates the fast weights online. A basic (Hebbian/outer-product) FWP—often called *vanilla* FWP—update its parameters with:

$$\mathcal{M}_t = \alpha_t \mathcal{M}_{t-1} + \mathbf{v}_t \phi(\mathbf{k}_t)^\top, \quad (5)$$

and retrieve from memory with $y_t = \mathcal{M}_t \phi(q_t)$, where $\phi(\cdot)$ is an element-wise feature map (often applied to both keys and queries). Unlike traditional RNNs or early variants of modern RNNs (Schmidhuber et al. 1997; Sun et al. 2023; Botev et al. 2024) with vector states, the *matrix* $\mathcal{M}_t$ is the recurrent state; it is *written* by rank-one update and *read* by a matrix–vector multiplication, providing a compact, learnable key–value memory with constant state size across time.

In-context Learning. The concept of “in-context learning” initially defined by Brown et al. (2020) as the ability of a language model to leverage knowledge acquired during pre-training in order to infer and perform a new task solely based on its context (e.g., few examples, or natural language instructions). This broad and general definition, which simply is applicable for any language model with any architectural backbones and/or objective, later was formalized in a way that only described in-context learning for Transformer architectures trained with next token prediction objectives. Accordingly, despite extensive research on the algorithms/problems that a transformer-based model can learn in-context (Akyürek et al. 2022, 2024b; Zhang et al. 2024a; Dherin et al. 2025), the in-context learning as its general form is relatively underexplored. Throughout this paper, we use the most general definition of “in-context learning” and refer to it as the ability of a model to adapt itself to and learn from a given context. Our NL formulation connects ICL with the concept of associative memory, offering a unified explanation for the ICL capabilities of models regardless of their architectural backbone and/or objectives.

3 Nested Learning

This section discusses the motivations, formal definitions, and general high-level implications of Nested Learning (NL). We start with a formulation of associative memory and then by using step-by-step examples, we build the intuition behind architecture decomposition and its connection to modeling a neural network as an integrated system of optimization problems. We aim to first show how existing methods and concepts in deep learning fall under the NL paradigm and then we present new formulations that go beyond traditional methods and/or provide insights on how to improve existing algorithms and designs.

3.1 Associative Memory

Associative memory—the ability to form and retrieve connections between events—is a fundamental mental process and is an inseparable component of human learning (Terry 2017). Often in the literature, the concept of memorization and learning are used interchangeably; in neuropsychology literature, however, these two are clearly distinguished. More specifically, following neuropsychology literature (Okano et al. 2000), we build our terminology based on the following definition of memory and learning:

Learning vs. Memorization:

Memory is a neural update caused by an input, and learning is the process for acquiring effective and useful memory.

In this work, our goal is to first show that all the elements of a computational sequence model, including optimizers and neural networks, are *associative memory systems* that compress their own *context flow*. Broadly speaking, associative memory is an operator that maps a set of keys to a set of values. We follow the general definition of associative memory by Behrouz et al. (2025b):

Definition 1 (Associative Memory). *Given a set of keys $\mathcal{K} \subseteq \mathbb{R}^{d_k}$ and values $\mathcal{V} \subseteq \mathbb{R}^{d_v}$, associative memory is an operator $\mathcal{M}(\cdot)$ that maps the set of keys $\mathcal{K}$ to values $\mathcal{V}$. To learn such mapping from the data, an objective $\tilde{\mathcal{L}}(\cdot; \cdot)$ measures the quality of the mapping and $\mathcal{M}$ can be computed by:*

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \tilde{\mathcal{L}}(\mathcal{M}(\mathcal{K}); \mathcal{V}). \quad (6)$$

While the operator itself is a memory and the mapping acts as a memorization process (i.e., memorizing the connections of events in the context), acquiring such effective operator based on the data, is a learning process. Notice that, here, keys and values can be any arbitrary event that memory aims to map them and are not limited to tokens. Later, we will discuss that given a context flow, keys and values might be tokens, gradients, sub-sequences, etc. Furthermore, while the term of associative memory is more common in neuroscience and neuropsychology literature, the above formulation is also closely related to data compression and low-dimensional representation. That is, one can interpret the optimization process in Equation 6 as the training process of a network $\mathcal{M}(\cdot)$ that aims to compress the mappings into its parameters, representing them in a lower dimensional space.

In sequence modeling, where keys and values are input tokens (e.g., tokenized text), the choice of objective and the optimization process for solving Equation 6 can result in distinct sequence modeling architectures (see Liu et al. 2024b and Behrouz et al. 2025b) such as global/local softmax attention (Vaswani et al. 2017), or other modern recurrent models (Katharopoulos et al. 2020; Sun et al. 2023; Behrouz et al. 2025c). This simple formulation of sequence models provides us with better understanding of their internal process and also a tool to simply compare their modeling power based on their objective and optimization process. In the following, using step-by-step examples, we discuss how this formulation can be applied to all components of a neural architecture (including its optimization process in pre-training) and in fact, how a model is an integrated system of multi-level, nested, potentially parallel memories, each of which with its own context flow.

A Simple Example of MLP Training. We start with a simple example, in which we aim to train a 1-layer MLP (parameterized with W) for task $\mathcal{T}$ and on dataset $\mathcal{D}_{\text{train}} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{D}_{\text{train}}|}\}$ by optimizing the objective $\mathcal{L}(\cdot; \cdot)$ with gradient descent. In this case, the training process objective is to solve the following optimization problem:

$$W^* = \arg \min_W \mathcal{L}(W; \mathcal{D}_{\text{train}}), \quad (7)$$

whose optimization by (stochastic/online) gradient descent results in a weight update rule:

$$W_{t+1} = W_t - \underbrace{\eta_{t+1} \nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1})}_{\text{Surprise}} = W_t - \underbrace{\eta_{t+1} \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \otimes \mathbf{x}_{t+1}}_{\text{Surprise in the Output}}, \quad \text{where } \mathbf{x}_{t+1} \sim \mathcal{D}_{\text{train}}, \quad (8)$$

where $y_{t+1} = W\mathbf{x}_{t+1}$ is the output of the model for input $\mathbf{x}_{t+1}$ and we used the simplifying notation: $\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) := \frac{\partial \mathcal{L}}{\partial y} |_{y=W\mathbf{x}_{t+1}}$. In this case, $\nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1})$ is the surprise metric that shows how much the current input is different from previously observed data. Similarly, $\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$ is the surprised metric for the output (or more accurately *local surprise signal in representation space* that quantifies the mismatch between the current output and the structure the objective $\mathcal{L}(\cdot; \cdot)$ enforces)—measuring how much surprising the model’s prediction is for this input. Given this formulation, one can let the surprise value of output be $u_{t+1} = \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$ and reformulate the backpropagation process as the solution to an optimization problem on finding an associative memory that maps input data points $\mathcal{D}_{\text{train}} = \{\mathbf{x}_t\}_{t=1}^{|\mathcal{D}_{\text{train}}|}$ to their corresponding $u_{t+1} = \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$. That is, we let $\mathcal{M}(\cdot) = W_t \cdot$ parametrizes the memory, and use dot-product similarity to measure the quality of W_t ’s mapping between $\mathbf{x}_{t+1}$ and $\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$:

$$W_{t+1} = \arg \min_W \langle W\mathbf{x}_{t+1}, u_{t+1} \rangle + \frac{1}{2\eta_{t+1}} \|W - W_t\|_2^2 = \arg \min_W \langle W\mathbf{x}_t, \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \rangle + \frac{1}{2\eta_{t+1}} \|W - W_t\|_2^2. \quad (9)$$

Therefore, this formulation translates the training phase of the model as a process of acquiring effective memory that maps data samples to their Local Surprise Signal (LSS) in representation space—measuring how surprising its corresponding output is. This gradient can be viewed as an error in the prediction (with gradient being zero when the loss is minimized). Later in Section 4, we discuss the backpropagation process as an associative memory in more details, but as a preliminary takeaway from this simple example:

Training a Linear Layer with Backpropagation as a Surprise-based Memory:

A linear layer trained with backpropagation learns from data by memorizing how surprising their predicted outputs are; i.e., backpropagation can be viewed as an associative memory that maps each data sample to the error of its corresponding prediction.

Accordingly, in this example, our model has a *single gradient flow* over the data samples, which is only active over dataset $\mathcal{D}_{\text{train}} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{D}_{\text{train}}|}\}$ and *will be frozen for any other data samples afterwards (i.e., inference or test time)*.

In the above example, we can replace the gradient descent algorithm with its momentum-based variant, resulting in the update rule of:

$$W_{t+1} = W_t - \mathbf{m}_{t+1}, \quad (10)$$

$$\mathbf{m}_{t+1} = \mathbf{m}_t + \eta_{t+1} \nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1}) = \mathbf{m}_t + \eta_{t+1} \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \otimes \mathbf{x}_{t+1}. \quad (11)$$

In Equation 11, given the previous state of Equation 10 (at time t), the value of $\nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1})$ or similarly $\nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1})$ does not depend on the output of recurrence in Equation 11 and so can be pre-computed beforehand: Letting $u_{t+1} = \nabla_W \mathcal{L}(W_t; \mathbf{x}_{t+1})$, Equation 11 can be reformulated as:

$$W_{t+1} = W_t - \mathbf{m}_{t+1}, \quad (12)$$

$$\mathbf{m}_{t+1} = \arg \min_{\mathbf{m}} -\langle \mathbf{m}, \nabla_{W_t} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \rangle + \frac{1}{2\eta_{t+1}} \|\mathbf{m} - \mathbf{m}_t\|_2^2 = \arg \min_{\mathbf{m}} -\langle \mathbf{m} \mathbf{x}_{t+1}, \nabla_{y_{t+1}} \mathcal{L}(W_t; \mathbf{x}_{t+1}) \rangle + \frac{1}{2\eta_{t+1}} \|\mathbf{m} - \mathbf{m}_t\|_2^2. \quad (13)$$

Given this formulation, one can interpret the momentum term as either: (1) a value-less associative memory that compress the gradients into its parameters, or (2) an associative memory that learns how to map data points to their corresponding LSS-value. Interestingly, this formulation reveals that gradient descent with momentum can be viewed as a two-level optimization procedure, where the memory is optimized by simple gradient descent algorithm¹.

Concluding the above examples, we observed that the training process of a 1-layer MLP with: (1) Gradient descent is a *1-level* associative memory that learns how to map data points to their corresponding LSS-value; and (2) Gradient descent with momentum is a *2-level* associative memory (or optimization process) that the inner-level learns to store gradient values into its parameters, and then the outer-level updates the slow weight (i.e., W_t) with the value of the inner-level memory. While these are the most simple examples with respect to both architecture and optimizer algorithms, one might ask if similar conclusion can be made in more complex setups.

An Example of Architectural Decomposition. In the next example, we replace the MLP module in our previous example with a linear attention (Katharopoulos et al. 2020). That is, we aim to train a 1-layer linear attention for task $\mathcal{T}$ and on a sequence of $\mathcal{D}_{\text{train}} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{D}_{\text{train}}|}\}$ by optimizing the objective $\mathcal{L}$ with gradient descent. Recalling the unnormalized linear attention formulation:

$$\mathbf{k}_t = W_k \mathbf{x}_t, \quad \mathbf{v}_t = W_v \mathbf{x}_t, \quad \mathbf{q}_t = W_q \mathbf{x}_t, \quad (14)$$

$$\mathcal{M}_t = \mathcal{M}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad (15)$$

$$y_t = \mathcal{M}_t \mathbf{q}_t. \quad (16)$$

As discussed in earlier studies (Liu et al. 2024b; Behrouz et al. 2025b), the recurrence in Equation 15 can be reformulated as the optimization process of a matrix-valued associative memory $\mathcal{M}_t(\cdot)$ to compress the mappings of keys and values into its parameters. Specifically, in Definition 1, if we let $\tilde{\mathcal{L}}(\mathcal{M}_{t-1}; \mathbf{k}_t, \mathbf{v}_t) := -\langle \mathcal{M}_{t-1} \mathbf{k}_t, \mathbf{v}_t \rangle$ and aim to optimize the memory with gradient descent, the memory update rule is: (Note that $\nabla \tilde{\mathcal{L}}(\mathcal{M}_{t-1}; \mathbf{k}_t, \mathbf{v}_t) = -\mathbf{v}_t \mathbf{k}_t^\top$ and we let learning rate $\eta_t = 1$)

$$\mathcal{M}_{t+1} = \arg \min_{\mathcal{M}} -\langle \mathcal{M} \mathbf{k}_{t+1}, \mathbf{v}_{t+1} \rangle + \frac{1}{2} \|\mathcal{M} - \mathcal{M}_t\|_2^2 \quad (17)$$

$$\Rightarrow \mathcal{M}_{t+1} = \mathcal{M}_t - \nabla \tilde{\mathcal{L}}(\mathcal{M}_t; \mathbf{k}_{t+1}, \mathbf{v}_{t+1}) = \mathcal{M}_t + \mathbf{v}_{t+1} \mathbf{k}_{t+1}^\top, \quad (18)$$

which is equivalent to the update rule of an unnormalized linear attention in Equation 15. Also, as we observed in the first example, training a linear layer with gradient descent can be viewed as a 1-level optimization of an associative memory (Equation 8) and so the general training/updating process of projection layers (i.e., W_k , W_v , and W_q) is itself an optimization process of associative memory. Therefore, training a linear attention with gradient descent can be seen as a two-level optimization process, where the outer-loop (also known as training process) optimizes the projection layers with gradient descent, while the inner-loop optimizes the inner memory of $\mathcal{M}_t$ with gradient descent.

In the examples we discussed so far, we have two associative memories, each of which has their own optimization process and gradient flow. That is, in the optimization of the outer-level parameters of W_k , W_v , and W_q there is no gradient

¹We use the term two- or multi-level to describe the optimization *procedure*. This differs from classical multi-level optimization, where the optimization *problems* are arranged hierarchically.

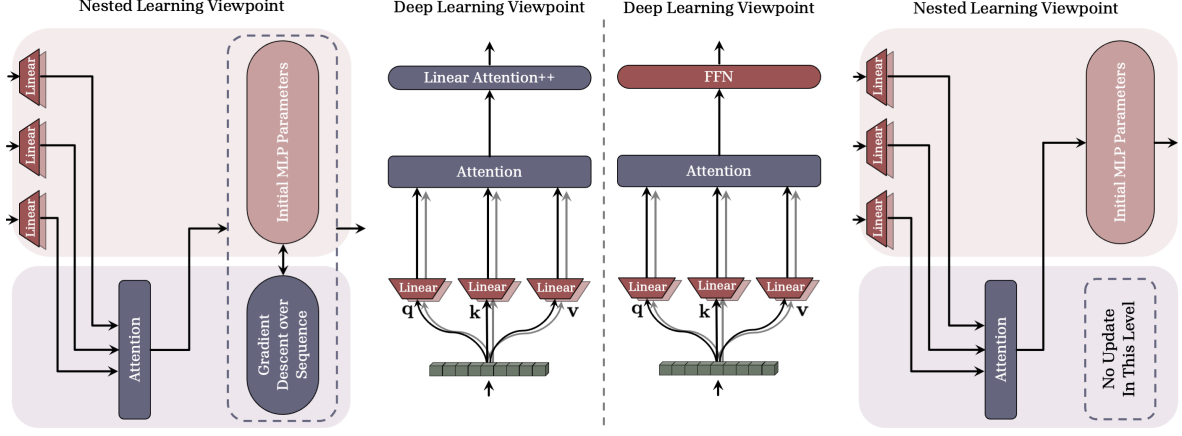


Figure 3: An example of comparing a FFN (e.g., MLP) with linear attention in a Transformer-based backbone, optimizing with gradient descent. The **red components** are blocks in the first level (with frequency 1), while **blue components** are blocks in the second level (frequency L). Linear attention with learnable initial memory state (referred to as Linear Attention++) is the same as an MLP layer but with in-context learning ability and adaptation to the input sequence.

with respect to parameter $\mathcal{M}(\cdot)$ and so there is no backpropagation through it. Similarly, in the inner-level, there is no backpropagation through projection layers and they are considered frozen. Furthermore, it is notable that in this example, the above formulation is also closely connected to FWPs perspective of linear attentions (Schlag et al. 2021), where projections are considered slow weights, and memory update in Equation 15 is the fast weight update rule.

Architectural Decomposition with More Levels. In both examples above, we discussed how they can be viewed as a 2-level optimization process (coinciding with their FWPs interpretations). In practice, however, we may need to use more powerful optimization process, and/or more powerful recurrent update rules for memory. As a simple example, assume we use gradient descent with momentum to train a linear attention model. As we saw above, the linear attention component can be decomposed into two nested optimization processes. Similarly, the model here can be represented as a 2-level optimization problem, where (1) the inner level optimizes the memory to compress the context using gradient descent (Equation 17), and (2) the outer level optimizes the projection layers with gradient descent with momentum. Interestingly, we saw that “gradient descent with momentum” algorithm itself can be viewed as a 2-level optimization process where the momentum term itself is an associative memory that compress the past gradients into its parameters.

3.2 Nested Optimization Processes

In the previous section, we provided examples to demonstrate how one can decompose a machine learning model into a set of nested or multi-level optimization procedures. Next, we first present a formalization of nested learning problems and then define Neural Learning Module—an integrated computational system that learns from data.

In previous sections, we decomposed the model into a set of optimization process. However, it is still unclear if we can define a hierarchy (or order) over these processes, and uniquely represent the model in this format. Inspired by the hierarchy of brain waves that indicates the information processing frequency rate of each part (discussed in Section 1), we use the update rate of each optimization process to order the components in multiple levels. To this end, we let the one update step over one data point to be the unit of time, and define the update frequency rate of each component as:

Definition 2 (Update Frequency). *For any component of A , which can be a parametric component (e.g., learnable weights or momentum term in gradient descent with momentum) or a non-parametric component (e.g., attention block), we define its frequency, denoted by f_A , as its number of updates per unit of time.*

Given the above update frequency, we can order the components of a machine learning algorithm based on operator $(\cdot \succ \cdot)$. We say A is faster than B and denote $A \succ B$ if: (1) $f_A > f_B$, or (2) $f_A = f_B$ but the computation of the B ’s state at time t requires the computation of A ’s state at time t . In this definition, when $A \not\succ B$ and $B \not\succ A$, we let $A \stackrel{f}{=} B$, which indicates that A and B has the same frequency update, but their computation is independent of each other (Later, we provide an

example of this cases in AdamW optimizer). Based on the above operator, we sort the components into an ordered set of “levels”, where (1) components in the same level have the same frequency update, and (2) the higher the level is, the lower its frequency. Given the above formulations of levels and update frequency, we next formally define nested learning:

Definition 3 (Nested System). *A (ordered) nested system is a system with K (ordered) levels such that each level k , $1 \leq k \leq K$, consists of a set of optimization problems $\{(\mathcal{L}_i^{(k)}, C_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$, where $\mathcal{L}_i(\cdot; \cdot)$ is the optimization objective in the i -th problem, C_i is its context (the data that is optimized on), Θ_i is the feasible set of its parameters, and each parameter is optimized using gradient descent:*

$$\theta_{i_{t+1}}^{(k)} = \arg \min_{\Phi_i^{(k)}} \langle \Phi_i^{(k)} x_{t+1}, -\nabla \mathcal{L}_i^{(k)}(\theta_{i_t}^{(k)}; x_{t+1}) \rangle + \frac{1}{2\eta_{i_{t+1}}^{(k)}} \|\Phi_i^{(k)} - \theta_{i_t}^{(k)}\|_2^2 \quad \text{where } x_{t+1} \sim C_i^{(k)}, \text{ and } \Phi_i^{(k)} \in \Theta_i^{(k)}. \quad (19)$$

Notice that each optimization process has its own gradient flow, and therefore sometimes we refer to them as a box of gradient flow corresponding to an optimization problem. Through this paper, we further generalize our definition of nested systems, and allow finding non-parametric solutions for some boxes (i.e., optimization problems).

The above definition provides a general flexible definition for a nested system that does not specify if there is any dependence among different boxes (i.e., a box can determines the context or the parameter space of another box). In the next sections, we discuss how knowledge/information can be transferred between different levels or boxes. Throughout this paper, we focus on the Nested Systems of Associative Memories (NSAM) that is a nested system, in which each optimization process is an associative-memory. More formally,

Definition 4 (Nested System of Associative Memories). *A nested system of associative memory (NSAM) is a system with K (ordered) levels such that each level k , $1 \leq k \leq K$, consists of a set of optimization problems $\{(\mathcal{L}_i^{(k)}, C_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$, where $C_i = \{(\mathbf{k}_j^{(i)}, \mathbf{v}_j^{(i)})\}_{j=1}^{L_i}$ is a set of key-value pairs, $\mathcal{L}_i(\cdot; \cdot, \cdot)$ measures the quality of memory learned mappings in the i -th problem, Θ_i is the set of feasible memory parameters with each parameter optimized using gradient descent:*

$$\theta_{i_{t+1}}^{(k)} = \arg \min_{\Phi_i^{(k)}} \langle \Phi_i^{(k)} \mathbf{k}_{t+1}^{(i)}, -\nabla \mathcal{L}_i^{(k)}(\theta_{i_t}^{(k)}; \mathbf{k}_{t+1}^{(i)}, \mathbf{v}_{t+1}^{(i)}) \rangle + \frac{1}{2\eta_{i_{t+1}}^{(k)}} \|\Phi_i^{(k)} - \theta_{i_t}^{(k)}\|_2^2, \quad (20)$$

where $(\mathbf{k}_{t+1}^{(i)}, \mathbf{v}_{t+1}^{(i)}) \sim C_i^{(k)}$ and $\Phi_i^{(k)} \in \Theta_i^{(k)}$.

Given a query $\mathbf{q}$, for each associative memory $\mathcal{M}_i^{(k)}$, we use $\mathcal{M}_i^{(k)}(\mathbf{q})$ to refer to the forward pass process (i.e., retrieval process) of the memory. While our formulation of NSAM can simply be defined by any optimization process beyond gradient descent, and initially, it may seem that the strict condition of *optimization by gradient descent* can limit the modeling power of the definition, through this paper, we show that modern architectures alongside certain well-known optimization algorithms can be viewed as instances of NSAM. We then build on top of this intuition and discuss how to go further with stacking multiple levels and design models with enhanced continual learning capabilities.

The new dimension of stacking multiple levels is an important characteristic of nested learning, where the depth of computation can be enhanced with increasing the number of levels. Depending on the design, context, and the type of knowledge transfer, this depth of computation can itself be viewed as different concepts such as: higher-order in-context learning ability, latent computation (e.g., Loop Transformers), multiple memory systems, and more expressive optimizers. Later, we discuss all such implications, but next, we use a simple example that connects MLP layers in Transformer architectures with linear or deep memory blocks.

Example of an MLP Layer vs. Linear Attention. Let us compare two models: (1) a Transformer architecture, and (2) the same backbone but with replacing the MLP block with a linear attention mechanism (sharing the keys and values from previous layer), in which the initial state of its memory is meta-learned (similar to Behrouz et al. (2025c) or Sun et al. (2024)). We refer to the second variant as Adaptive Transformer or AdaTransformer. Both models are optimized on Next Token Prediction (NTP) objective with gradient descent. As discussed in the initial examples, and also illustrated in Figure 3, both models have two levels, and for the sake of clarity, we use red (resp. blue) to highlight computations/weight in the first level (resp. second level). More formally, let $X = \{x_i\}_{i=1}^T$ be an input sequence of tokens, the output of both blocks are computed as (For the sake of simplicity, we assume $\text{MLP}(\cdot) = \cdot W_{\text{MLP}}$, and remove normalizations):

$ \begin{aligned} \mathbf{k}_t &= \mathbf{x}_t \mathbf{W}_k, & \mathbf{v}_t &= \mathbf{x}_t \mathbf{W}_v, & \mathbf{q}_t &= \mathbf{x}_t \mathbf{W}_q, \\ \mathbf{y}_{\text{attn}} &= \text{Attn}(\mathbf{k}, \mathbf{v}, \mathbf{q}), \\ \mathbf{y}_{\text{block}} &= \text{MLP}(\mathbf{y}_{\text{attn}}) = \mathbf{y}_{\text{attn}} \mathbf{W}_{\text{MLP}}, \quad (\text{Transformer Block}) \end{aligned} $	$ \begin{aligned} \mathbf{k}_t &= \mathbf{x}_t \mathbf{W}_k, & \mathbf{v}_t &= \mathbf{x}_t \mathbf{W}_v, & \mathbf{q}_t &= \mathbf{x}_t \mathbf{W}_q, \\ \mathbf{y}_{\text{attn}} &= \text{Attn}(\mathbf{k}, \mathbf{v}, \mathbf{q}), \\ \mathbf{y}_{\text{block}} &= \mathbf{y}_{\text{attn}} \mathbf{W}_{\text{LinAttn}}. \quad (\text{AdaTransformer Block}) \end{aligned} $
--	--

The formulation of both blocks seem to be very similar and the only difference comes from the level of $\mathbf{W}_{\text{MLP}}$ and $\mathbf{W}_{\text{LinAttn}}$ weights. That is, while $\mathbf{W}_{\text{MLP}}$ is in the first level and so is persistent with respect to the context, $\mathbf{W}_{\text{LinAttn}}$ is adaptive and is updated in-context by $(\mathcal{M}(\cdot))$ is parametrized by $\mathbf{W}_{\text{LinAttn}}$:

$$\mathcal{M}_t = \mathcal{M}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top. \quad (21)$$

In earlier variants of linear attention, the initial state of $\mathcal{M}(\cdot)$ or equivalently $\mathbf{W}_{\text{LinAttn}}$ is considered as zero matrix, $\mathcal{M}_0 = \mathbf{0}$. Similar to more advanced design choices (Sun et al. 2024; Behrouz et al. 2025c), however, this initial state can be meta-learned to adapt fast to a context. In this setting, the initial state of $\mathcal{M}_0 = \mathbf{W}_{\text{LinAttn}_{\text{init}}}$ is optimized in the first level with NTP objective, while given the context, $\mathbf{W}_{\text{LinAttn}}$ is optimized in the second level as an associative memory with dot-product objective (Behrouz et al. 2025b).

The above example is also valid when using more advanced and deep MLP blocks in Transformer architecture (such as SwiGLU (Shazeer 2020)) and compared it with its recurrent memory counterpart (Behrouz et al. 2025a). Furthermore, this simple example implies that the current perspective on hybrid architectures as the combination of expressive softmax attention and efficient recurrent models is somewhat misleading and it follows the conventional Transformer backbone design but with additional in-context learning capabilities for MLP blocks. We discuss this further in Section 6 and Section 7. As a takeaway of the discussion in this subsection about the concept of nested systems and nested learning:

Stacking Levels in Nested Learning:

Nested learning allows computational models that are composed of multiple (multi-layer) levels to learn from and process data with different levels of abstraction and frequencies of update.

As discussed earlier, it is common in the literature to separate architectures from their optimization processes and to treat them as independent design choices, with the aim of combining algorithms that achieve the greatest expressive power in each aspect. In practice, however, a Transformer architecture (Vaswani et al. 2017) that is optimized with stochastic gradient descent can learn a very different solution than the same architecture when Adam optimizer is used (Kingma et al. 2014a). Accordingly, when interacting with such machine learning algorithms, we observe that despite the similarity in the architecture axes, the overall trained models show different predictions or generates different outputs. From the NL’s viewpoint, however, a machine learning algorithm is represented as an interconnected system of optimization problems and model’s actions, predictions, and generation of output depends on this system as a whole, not necessarily each of its sub-components. To this end, we define the term of neural learning module to refer to this representation of a model, where architecture and optimization process jointly determine the model and its outputs. While such joint representation might not seem significant in the current machine learning pipelines, where there is a training and then test phases, it becomes more important in the continual setup we advocate for, where there is no training/test phases (see more discussions in Section 8).

Neural Learning Modules are Inter-connected Systems. Based on the definition of neural learning module, one important question is how the architecture and optimization process are interconnected systems and how they can affect each other. Recall the general formulation for training a neural network: Given a task $\mathcal{T}$, its corresponding data distribution $p(\mathcal{T})$, a model $f(\cdot; \cdot)$ parameterized by $\Phi_{\mathcal{T}}$, and an objective $\mathcal{L}(\cdot; \cdot)$ we aim to learn parameters $\Phi_{\mathcal{T}}^*$ such that:

$$\Phi_{\mathcal{T}}^* = \arg \min_{\Phi} \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p(\mathcal{T})} [\mathcal{L}(\Phi; \mathbf{x}, \mathbf{y})]. \quad (22)$$

In practice, we optimize the above problem based on a given dataset $\mathcal{D}_{\text{train}}$ and an optimization algorithm such as stochastic gradient descent:

$$\Phi_{t+1} = \Phi_t - \eta_{t+1} \nabla_{\Phi_t} \mathcal{L}(\Phi_t; \mathbf{x}_{t+1}, \mathbf{y}_{t+1}), \quad \text{where } (\mathbf{x}_{t+1}, \mathbf{y}_{t+1}) \sim \mathcal{D}_{\text{train}}. \quad (23)$$

One interpretation for the optimization process of model $f(\cdot; \cdot)$ in Equation 23 is to see the model as the data generator for the optimization process in Equation 23. That is, as discussed in the first example in Section 3.1, and later we will show in Section 4, the optimization process is an associative memory that aims to compress the patterns between the training data and its gradients (or surprise) and so the dataset for internally training such memory (i.e., the gradients of the model) is generated by the model. Therefore, the type of the model can result in generating dataset (i.e., gradients) with different patterns and distributions over time. The effect of the optimization process and this data generation also fed back at the model itself, where the next state of the parameters in the model are determined by the optimization algorithms. As we will discuss in Section 4, looking at optimizers as associative memories on the gradients of the model implies that each optimizer has some special traits such as better memory management, higher compression, etc. Therefore, the choice of such algorithms requires understanding the generated gradients and also changes of the model in the parameter space.

3.3 Knowledge Transfer Between Levels

So far, we mainly focused on the concept of nested learning and how optimization problems are located in different levels. It is, however, still unclear that how nested optimization problems (in different levels) can affect each other, or in general how they can contribute to the output of the system, and so be interconnected. In this section, we discuss several potential knowledge transfer methods between components in different levels. For the sake of clarity, we discuss the knowledge transfer between two levels and blocks, i.e., $\mathcal{B}^{(0)} = (\mathcal{L}^{(0)}, \mathcal{C}^{(0)}, \Theta^{(0)})$ and $\mathcal{B}^{(1)} = (\mathcal{L}^{(1)}, \mathcal{C}^{(1)}, \Theta^{(1)})$ with corresponding memories $\mathcal{M}^{(0)}(\cdot)$ and $\mathcal{M}^{(1)}(\cdot)$, respectively:

Direct Connection of Levels (Parametric). The first type of knowledge transfer is to directly incorporate the weights in different levels or blocks. To this end, the forward pass or the retrieval process from the lower-frequency (i.e., higher-level) memory system is also conditioned on the parameters of the higher-frequency (i.e., lower-level) memory:

$$\mathcal{M}^{(0)}(\cdot) := \mathcal{M}^{(0)}(\cdot; \Theta^{(1)}). \quad (24)$$

In a more specific formulation, and as a special variant of the above formulation, we can condition the output of $\mathcal{M}^{(0)}(\cdot)$ based on the output (or forward pass) of the higher-frequency memory:

$$\mathcal{M}^{(0)}(\cdot) := \mathcal{M}^{(0)}(\cdot; \mathcal{M}^{(1)}(\cdot)), \quad (25)$$

where we slightly abused notation by hiding the dependence on the second argument. As an example of this type of knowledge transfer, in linear Transformer (or FWP) (Katharopoulos et al. 2020; Schlag et al. 2021), where the initial memory state is zero, the stored knowledge of a lower level (fast weight) directly affect the output of the model in another level. That is, one can re-write the forward pass (memory retrieval) as:

$$\mathbf{y}_t = \mathcal{M}_t \mathbf{q}_t = \mathcal{M}_t \underbrace{\left[\mathbf{x}_t \mathbf{W}_q \right]}_{\substack{\text{Forward pass of the} \\ \text{lower-frequency memory}}} \quad (26)$$

Forward pass of the
higher-frequency memory

Direct Connection of Levels (Non-Parametric). Another form of direct connection between levels is a non-parametric variant of the above formulation, where the block $\mathcal{B}^{(1)}$ is optimized by finding non-parametric solution. Therefore, the forward pass of the low-frequency memory is conditioned on the context and output of higher-frequency memory:

$$\mathcal{M}^{(0)}(\cdot) := \mathcal{M}^{(0)}(\cdot; \mathcal{C}^{(1)}), \quad \text{or similarly,} \quad \mathcal{M}^{(0)}(\cdot) := \mathcal{M}^{(0)}(\cdot; \mathcal{M}^{(1)}(\cdot; \mathcal{C}^{(1)})). \quad (27)$$

As an example of this variant, one can refer to Transformers and softmax attention module (Vaswani et al. 2017). There is an important characteristic for both above variants: there is no backpropagation through any state of the blocks in two different levels and knowledge transfers through direct conditioning the output of one level on the other's output/parameters. Therefore, in this process the state of each block is treated as hyperparameter for the other.

Knowledge Transfer via Backpropagation. Another form of knowledge transfer is through backpropagation, where there is a gradient flow between blocks in different levels. The forward pass of this design is the same as the forward

pass discussed above. The backward pass, however, is the main difference, where in the above two cases the state of each associative memory is considered as hyperparameters of the other but here both states are optimized in the same gradient flow. Therefore, for a simple case of two blocks in two levels, we have:

$$\mathcal{M}^{(0)}(\cdot) := \mathcal{M}^{(0)}(\cdot; \mathcal{M}^{(1)}(\cdot)) \quad (\text{Forward Pass})$$

$$\begin{cases} \Theta_{t+1}^{(1)} = \Theta_t^{(1)} - \eta_{t+1}^{(1)} \delta_1 \hat{\mathbf{x}}_{t+1}^\top, \\ \Theta_{t+1}^{(0)} = \Theta_t^{(0)} - \eta_{t+1}^{(0)} \delta_0 \mathbf{x}_{t+1}^\top, \end{cases} \quad (\text{Backward Pass})$$

where $\delta_{t+1}^{(0)} = J_{\phi^{(0)}}(\mathbf{x}_{t+1})^\top (\Theta_{t+1}^{(1)} \delta_{t+1}^{(1)})$, $\hat{\mathbf{x}}_{t+1} = \phi^{(0)}(\mathcal{M}^{(0)}(\mathbf{x}_t))$, $\phi^{(0)}(\cdot)$ is non-linearity, and $J_{\phi^{(0)}}(\cdot)$ is the Jacobian. In this design, two blocks are in the same flow of gradient, but are updated based on different frequencies. We provide an example this design in Section 7, when we discuss continuum memory systems.

Knowledge Transfer via Initialization. Model Agnostic Meta-Learning (MAML) (Finn et al. 2017), is one of the most popular form of meta-learning (or learning to learn) that aims to learn a global initial point for a model so it could learn fast a new task. From the nested learning perspective, there are two nested optimization processes, in which the inner problem iterates over its own context and based on its internal objective, the higher-level problem measures its own learned weight as the initial point of the inner problem. More formally, we have

$$\Theta_0^{(1)} = \arg \min_{\Phi} \mathbb{E}_{C \sim C^{(0)}} \left[\ell(\mathcal{M}^{(1)}(\cdot; \Phi), C) \right], \quad (28)$$

where the higher level block learns the best initial value over all possible contexts the lower level problem might have. As discussed earlier, any MAML-based learning model is an instance of this case, but as a more specific example, we refer to the example of (MLP Layer vs. Linear Attention), which is discussed in Section 3.2 and Figure 3.

Connections with Generation. One of the most common form of knowledge transfer is through generating weights or context. That is, one lower-frequency (resp. higher-frequency) block generates the weight of a higher-frequency (resp. lower-frequency) block. More formally,

$$\begin{cases} \Theta^{(1)} = g(\mathcal{M}^{(0)}; \mathcal{L}^{(0)}, C^{(0)}, \Theta^{(0)}), \text{ or} \\ \Theta^{(0)} = g(\mathcal{M}^{(1)}; \mathcal{L}^{(1)}, C^{(1)}, \Theta^{(1)}), \end{cases} \quad (\text{Weight Generation}) \quad \begin{cases} C^{(1)} = g(\mathcal{M}^{(0)}; \mathcal{L}^{(0)}, C^{(0)}, \Theta^{(0)}), \text{ or} \\ C^{(0)} = g(\mathcal{M}^{(1)}; \mathcal{L}^{(1)}, C^{(1)}, \Theta^{(1)}). \end{cases} \quad (\text{Context Generation})$$

There are two important examples of the above form for knowledge transfer: (1) Hypernetworks: where the weights of a targeted neural network is generated by another (generator) network. (2) Optimization process: where the architecture generates the input for the optimizer. That is, the context (or input data) of an optimizer is the gradients that are generated by the architecture. For more discussion on this topic, see Section 4. Note that this example is not necessarily about “learned optimizers” and it is valid for the commonly used optimization process and algorithms such as gradient descent, Adam (Kingma et al. 2014a), AdaGrad (Duchi et al. 2011), etc.

A Note on Designing Neural Learning Modules. In the above, we discussed only some examples of possible knowledge transfer methods and also the potential connections of different levels. The formulation of NL and neural learning module, however, is general and so is not limited to the above specific set of methods. Accordingly, to design a neural learning module from a nested learning perspective, there are two important steps and design choices:

Designing Neural Learning Modules:

There are two high-level design choices in developing a neural learning module: (1) The design of optimization problems and their frequency (i.e., designing components in NSAM); (2) The design of knowledge transfer between levels.

It is notable that with different choices of knowledge transfer, some learning paradigms can be seen as a part of a neural learning model: E.g., (1) Meta learning, when two blocks in two levels transfer their knowledge with one level meta-learns the other; more specifically, (2) Model Agnostic Meta Learning (MAML) (Finn et al. 2017), when knowledge transfer is through learning the initialization; (3) Hypernetworks, when one higher-frequency block generates the weights for the

other lower-frequency block; (4) Learned optimizers, when the knowledge transfer is through data generation (i.e., one high-frequency block generates the gradient for the other lower-frequency block).

4 Optimizers as Learning Modules

In this section, we start with viewing backpropagation process and optimizing a neural network from the associative memory and data compression perspective. Then, we discuss how variants such as momentum-based optimizers are instances of nested associative memory systems. Finally, we discuss alternative methods leading to deep optimizers with higher expressive power from the associative memory perspective.

4.1 Backpropagation as an Associative Memory

Updating the weights of a neural network through backpropagation (Linnainmaa 1970; Rumelhart et al. 1986) has been the critical component of training large-scale deep neural networks. Intuitively, in this optimization process, first, the error of the model’s output with respect to target is calculated, and then each layer is updated based on its contribution to this error. This section aims to explain this process through the lens of associative memory and discuss how it fits within the nested learning paradigm. For the sake of clarity and simplicity, we assume a deep MLP model, but all the derived formulations in the following can simply be adapted to other architectures as well. Given an MLP with L layers parameterized with $\{W_\ell \cdot + \mathbf{b}_\ell\}_{\ell=1}^L$, the required gradients in backpropagation are computed as:

$$\frac{\partial \mathcal{L}}{\partial W_\ell} = \delta_\ell \hat{\mathbf{x}}_{\ell-1}^\top, \quad \text{and} \quad \delta_\ell = \underbrace{J_{\phi_\ell}(\mathbf{z}_\ell)^\top (W_{\ell+1}^\top \delta_{\ell+1})}_{\text{local output surprise for layer } \ell}, \quad (29)$$

where $\mathbf{z}_\ell = W_\ell \hat{\mathbf{x}}_{\ell-1} + \mathbf{b}_\ell$ is pre-activation, and so $\hat{\mathbf{x}}_\ell = \phi_\ell(\mathbf{z}_\ell)$ is the output of ℓ -th layer, $\phi_\ell(\cdot)$ is its non-linearity, and $J_{\phi_\ell}(\cdot)$ is the Jacobian. Therefore, the update of the ℓ -th layer with gradient descent is computed as:

$$W_{\ell,t+1} = W_{\ell,t} - \eta_{\ell,t+1} \delta_\ell \hat{\mathbf{x}}_{\ell-1}^\top. \quad (30)$$

Here, $\hat{\mathbf{x}}_{\ell-1}$ is the input of the layer and δ_ℓ measures the local error signal for layer ℓ or equivalently is a metric that measures the surprise of layer ℓ ’s output given its input. Similar to our example in Section 3.1, we can write Equation 30 as:

$$W_{\ell,t+1} = \arg \min_W \langle W \hat{\mathbf{x}}_{\ell-1}, \delta_\ell \rangle + \frac{1}{2\eta_{\ell,t+1}} \|W - W_{\ell,t}\|_F^2, \quad (31)$$

which is an associative memory module that aim to map the input of each layer $\hat{\mathbf{x}}_{\ell-1}$ to its local error signal, δ_ℓ (see Definition 1). That is, the above formulation implies that *the training process of a neural network with gradient descent and backpropagation can be viewed as a compression process*, in which each layer stores the mappings between its input and the corresponding local error signal. Later in §4.5, we discuss how this viewpoint helps with designing more expressive learning rules for backpropagation.

Training a Deep Neural Network with Backpropagation as a Surprise-based Memory:

A neural network trained with backpropagation learns from data by memorizing how surprising their predicted outputs are; i.e., backpropagation is an associative memory that maps each data point to the error in its corresponding prediction.

Backpropagation $\neq$ Linear Attention. A common misinterpretation for Equation 30 is to assume δ_ℓ is a pre-computed term and so backpropagation (at least on a linear layer) recovers Hebbian-rule, resulting in the equivalency of the optimization process and performing linear attention on gradients. Our formulation, however, shows that the update rule in backpropagation is a self-referential process (Schmidhuber 1993), where the values of the associative memory is generated by itself, making it a more complex associative memory than a simple linear attention on gradients (see Section 4.5).

4.2 Momentum-based Optimizers as Associative Memories

Momentum-based optimizers are the major components of modern machine learning models’ training (Duchi et al. 2011; Kingma et al. 2014a; Jordan et al. 2024). To explain momentum-based optimizers as associative memories, let us start from

a simple gradient descent algorithm:

$$W_{t+1} = W_t - \eta_t \nabla_{W_t} \mathcal{L}(W_t; \mathbf{x}_{t+1}), \quad (32)$$

which updates the current state of the weights based on the momentary gradient (surprise). This update rule does not incorporate the previous tokens and also the loss landscape that have been traversed so far, resulting in slower (or less robust) convergence in many scenarios. To fix this, momentum-based gradient descent methods incorporate an Exponential Moving Averages (EMAs) of past gradients:

$$\begin{aligned} W_{\ell_{t+1}} &= W_{\ell_t} + \mathbf{m}_{\ell_{t+1}} \\ \mathbf{m}_{\ell_{t+1}} &= \alpha_{\ell,t+1} \mathbf{m}_{\ell_t} - \eta_{\ell,t+1} \nabla_{W_{\ell_t}} \mathcal{L}(W_{\ell_t}; \mathbf{x}_{t+1}) = \alpha_{\ell,t+1} \mathbf{m}_{\ell_t} - \eta_{\ell,t+1} \boldsymbol{\delta}_\ell \hat{\mathbf{x}}_{\ell-1}^\top, \end{aligned} \quad (33)$$

where matrix (or vector) $\mathbf{m}_t$ is the momentum at state t and α_t and η_t are (adaptive) learning and momentum rates, respectively, and $\boldsymbol{\delta}_\ell$ and $\hat{\mathbf{x}}_{\ell-1}$ are defined the same as in Equation 29. Similar to Equation 31 and one of the examples in Section 3.1, assuming $\alpha_{t+1} = 1$, the momentum term can be viewed as the result of optimizing the following objective with gradient descent:

$$\min_m \langle \mathbf{m} \hat{\mathbf{x}}_{\ell-1}, \boldsymbol{\delta}_\ell \rangle. \quad (34)$$

The case of $\alpha_{t+1} \neq 1$ is equivalent to GD on the above minimization plus an ℓ_2 -regularization on the momentum term. Thus, momentum can indeed be viewed as an associative memory module that learns how to compress the past gradients of the objective into its parameters. Contrary to Equation 31, which was a simple 1-level associative memory and the update was directly applied to the memory, here the state of the momentum determines the update for the weights. In other words, it is a 2-level optimization procedure, in which the inner-loop learns the momentum and the outer-loop uses the state of the momentum to update the weights.

From this perspective, we can generalize the definition of momentum from EMAs to any arbitrary associative memory module that aims to compress the past gradients or maps the input of each token to its corresponding local error. This generalized momentum can be expressed as:

$$W_{\ell_{t+1}} = W_{\ell_t} + \mathbf{m}_{\ell_{t+1}}, \quad (35)$$

$$(36)$$

where $\mathbf{m}_\ell$ is the solution of the following associative memory, optimized by gradient descent:

$$\min_m \tilde{\mathcal{L}}(\mathbf{m}; \hat{\mathbf{x}}_{\ell-1}, -\boldsymbol{\delta}_\ell). \quad (37)$$

Here, the objective $\tilde{\mathcal{L}}(\cdot)$ is different from the original objective of the problem at hand, and $\tilde{\mathcal{L}}(\cdot)$ is the objective that defines the momentum and measures the quality of its mappings. In fact, the momentum term in this formulation aims to adapt in-context (recall that the context of the momentum is the gradients) to the local error rates based on the input of the layer. Most popular optimizers are formulated as element-wise update rule (for computational efficiency reasons) and so in Appendix B, we first explore the element-wise associative memory formulation of momentum and connect it to popular optimizers such as Adam (Kingma et al. 2014a). Showing that Adam can be viewed as the optimal associative memory to the L_2 -regression objective that aims to predict the variance of gradients, we discuss other similar algorithms such as RMSProp (Hinton et al. 2012), SignSGD and its momentum-based variants (Bernstein et al. 2018), NAdam (Dozat 2016), AMSGrad (Reddi et al. 2016), RAdam (Liu et al. 2020), and Lion (Chen et al. 2023) are also instances of an associative memory that aims to compress the gradients. We then go beyond element-wise formulation and show that AdaGrad (Duchi et al. 2011) is also an associative memory module. Due to the connection of AdaGrad with optimizers such as Shampoo (Gupta et al. 2018) and Soap (Vyas et al. 2025)—i.e., as the approximation of the preconditioning term—we then conclude that all these optimizers can be re-formulated as associative memory. Next, we discuss another class of optimizers based on preconditioning and reformulate them from NL’s perspective in more details:

Preconditioning and Approximation of Hessian. Another class of algorithms is preconditioning algorithms where the idea is to approximate Hessian inverse to mimic the behavior of Newton’s algorithm. Formally, gradient descent with preconditioning is defined as:

$$W_{\ell_{t+1}} = W_{\ell_t} - \eta_{t+1} \mathbf{P}_{t+1}^{-1} \mathbf{g}_{\ell_{t+1}}, \quad (38)$$

where *preconditioner* P_{t+1} is often a positive-definite matrix. A critical interpretation of preconditioner is their role in performing gradient descent in a transformed coordinate system, which can be viewed as a mapping from gradients to that system of interest. Accordingly, we reformulate and interpret the preconditioner in Equation 38 as an associative memory that maps the set of gradients (or a function of gradients denoted as $\mathbf{g}$) to the system of our choice, denoted as $\hat{\mathbf{g}}$:

$$W_{\ell_{t+1}} = W_{\ell_t} - \eta_{t+1} P_{t+1}^{-1} (\mathbf{g}_{\ell_{t+1}}), \quad (39)$$

where internally (in a nested level), P_{t+1} learns how to perform this mapping using an objective:

$$\min_P \tilde{\mathcal{L}}(P(\hat{\mathbf{g}}); \mathbf{g}). \quad (40)$$

Given this viewpoint, the main question is about finding the best coordinate system that can empower the compression process. The most simple variant is an identity mapping, where we preserve the metric system and use P to map $\mathbf{g}$ (i.e., gradients in this case) to itself, resulting in preconditioning terms in Adam (Kingma et al. 2014a) and AdaGrad (Duchi et al. 2011), as discussed in Appendix B. These results, along with the representation of Adam and its variants as associative memories, show that not only momentum-based optimizers are associative memories, but they also can be decomposed into a set of nested learning problems, each of which optimized with gradient descent. In a more general form, however, one can use more nested levels and optimize the inner problems in Equation 40 with gradient descent, resulting in:

$$P_{t+1} = P_{t+1} - \zeta_{t+1} \nabla_{P_t} \tilde{\mathcal{L}}(P_t; \mathbf{g}_{t+1}, \hat{\mathbf{g}}_{t+1}). \quad (41)$$

In the NL framework, to design an effective preconditioning, one needs to find the right choice of $\hat{\mathbf{g}}$ and $\tilde{\mathcal{L}}$. This viewpoint can also lead to other classes of algorithms with gradient/momentum orthogonalization: e.g., Muon and its variants (Jordan et al. 2024; Cesista 2025; Keigwin et al. 2025). Recalling Muon optimizer (Jordan et al. 2024):

$$\begin{aligned} W_{\ell_{t+1}} &= W_{\ell_t} + \text{NewtonSchulz}_k(\mathbf{m}_{\ell_{t+1}}) \\ \mathbf{m}_{\ell_{t+1}} &= \alpha_{\ell, t+1} \mathbf{m}_{\ell_t} - \eta_{\ell, t+1} \nabla_{W_{\ell_t}} \mathcal{L}(W_{\ell_t}; \mathbf{x}_{t+1}), \end{aligned} \quad (42)$$

where $\text{NewtonSchulz}_k(\cdot)$ performs k steps of Newton-Schulz orthogonalization process. From the above discussion about the general formulation of preconditioning, one can see $\text{NewtonSchulz}_k(\cdot)$ operator as a mapping from gradients of momentum term to a proper metric system. The choice of proper coordinate system in Muon is to orthogonalize the gradients and so we aim to find a mapping $P(\cdot)$ by minimizing a loss function $\min_P \tilde{\mathcal{L}}(P; \mathbf{O}, \mathbf{m})$ where objective $\tilde{\mathcal{L}}(\cdot; \cdot, \cdot)$ measures the quality of mapping from $\mathbf{O}$ to either $\mathbf{m}$ or $\mathbf{g}$ by $P(\cdot)$. A critical challenge in this process is that the parameter $\mathbf{O}$ itself is not given and so the mapping requires learning both the mapping and the proper orthogonal space. A simple formulation measuring orthogonalization, can be achieved by defining the objective as:

$$\tilde{\mathcal{L}}(P(\mathbf{g}); \mathbf{g}) = \|P(\mathbf{g})^\top P(\mathbf{g}) - \mathbf{I}\|_F^2, \quad (43)$$

where $P(\mathbf{g})$ is the orthogonal space that we aim to directly learn from gradients. This objective ensures that the gradients (or momentum) and their mapping are relatively close while the mapping is to an orthogonal space. Optimizing the above objective to find $\mathbf{O} = P(\mathbf{g})$ with one step of gradient descent results in:

$$\mathbf{O}_{i+1} = \mathbf{O}_i - \zeta_{i+1} \nabla_{\mathbf{O}_i} \tilde{\mathcal{L}}(\mathbf{O}_i; \mathbf{g}_t) = \mathbf{O}_i - \zeta_{i+1} (\mathbf{O}_i - \mathbf{g}_t + 2\mathbf{O}_i (\mathbf{O}_i^\top \mathbf{O}_i - \mathbf{I})), \quad (44)$$

which recovers the 3-degree polynomial (initial value $\mathbf{O}_0 = \mathbf{g}_t$). In a summary, the higher-frequency level learns the orthogonal mapping and then the lower-frequency process use the learned mapping to optimize the weights. Later, in Section 4.4, we discuss a more general viewpoint that considers $\text{NewtonSchulz}_k(\cdot)$ as a polynomial mapping to enhance the capacity of the memory.

4.3 Long Context in Optimizers: An Example of Continual Learning with Orthogonal Tasks

When removing the boundary between train and test time, and moving towards models that can continually learn for a long period of time, the role of (online) optimizers becomes more prominent: mainly due to the need for finding effective "solutions" rather than converging faster. Finding an effective solution, however, requires global understanding of the objective to avoid "local minima" as well as moving toward directions that might cause (catastrophic) forgetting of long past learned tasks. From associative memory perspective and as discussed earlier, the momentum term is expected to be a

memory of past gradients, helping the optimization process to have a more global view of the loss landscape. The current design of momentum, however, acts as a simple low-pass filter that smoothifies the gradient updates and thus has limited capacity with only incorporating information from recent past.

To better illustrate this limitation, let $\beta > 0$ be the momentum's decay term, and so the contribution of i -th gradient before to the current state of the momentum can be calculated as $\beta^i (1 - \beta)$. Considering the cumulative sum of gradients' contributions to the current state of the momentum term (i.e., $S_t = \sum_{i=0}^t \beta^i (1 - \beta)$) and the commonly used value of $\beta = 0.9$ in optimization setups, the last 6 gradients (resp. 43 gradients) are responsible for at least 50% (resp. 99%) of the cumulative contribution, i.e., S_t . This indicates that gradients and generally the global information beyond only past 43 steps contribute less than 1%, limiting the understanding of the objective landscape and the ability to find effective solutions. This simple example shows that the current design is limited even in incorporating the long past information, let alone its ability to properly retrieve information needed for the current state of the momentum.

Coming back to the setup of continual learning and considering one of its simple variants with orthogonal tasks. We let $\{(\mathcal{T}_i, \mathcal{D}_i, \mathcal{L}_i)\}_{i=1}^n$ be the set of tasks, their corresponding data, and their objectives such that for task $\mathcal{T}_i$:

$$\mathcal{L}_i(W) = \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}_i} [(W^\top \mathbf{x} - \mathbf{y})^2], \quad (45)$$

and gradients live in orthogonal directions of $\{\mathbf{u}_i\}_{i=1}^n$. Given the task $\mathcal{T}_t$ (for large enough $t > 1$), when optimizing the problem with gradient descent with momentum, and after many steps on task t , the gradients now point along $\mathbf{u}_t$. Accordingly, the momentum term is gradually shifted and now is approximately in $\mathbf{u}_t$ direction. This can lead to gradual forgetting about past gradients, which in turn may cause catastrophic forgetting in the model. That is, the optimization process can move the weights in a direction that damages the performance on previous tasks, mainly because the optimizer has no memory of the old gradient subspace that it should avoid. This failure is not about the capacity of the model but the memory management of the optimization process, failing at finding an effective solution.

Long Context Understanding in Optimizers

Moving from static models to neural learning modules that can continually learn from data/experience, the optimization process itself can benefit from long-term compression/understanding of gradient subspace to find effective solutions over diverse sets of tasks and for a long period of time..

Motivated by this observation, we next discuss more expressive variants of momentum that are capable of better memory management and higher memory capacity:

4.4 More Expressive Designs for Momentum as an Associative Memory

So far we discussed that (1) momentum term can be viewed as an associative memory that aims to compress the (past) gradients into its parameters; and (2) for developing models that can continually learn for a long period of time and on diverse sets of tasks, the optimization process need proper information about long past and the global properties of loss landscape. Next, we discuss how Nested Learning and associative memory viewpoint could result in designing optimizers with diverse memory management/structure:

Extension: More Expressive Association. As discussed earlier, vanilla momentum term can be viewed as a value-less associative memory. To allow more expressive associative memory and following the original definition of associative memory (i.e., mapping keys to values), we let value parameter $\mathbf{v}_i = \mathbf{P}_i$ and so the momentum aims to minimize:

$$\min_{\mathbf{m}} \langle \mathbf{m} \nabla \mathcal{L}(W_i; \mathbf{x}_i)^\top, \mathbf{P}_i \rangle, \quad (46)$$

or equivalently, minimizes $\langle \mathbf{m}, \mathbf{P}_i \nabla \mathcal{L}(W_i; \mathbf{x}_i) \rangle$. Using gradient descent for updating the momentum results in the update rule of:

$$\begin{aligned} W_{i+1} &= W_i + \mathbf{m}_{i+1} \\ \mathbf{m}_{i+1} &= \mathbf{m}_i - \eta_i \mathbf{P}_i \nabla \mathcal{L}(W_i; \mathbf{x}_i), \end{aligned} \quad (47)$$

where we also included a forget gate α_{i+1} for forgetting the past. This update rule can be viewed as preconditioning the momentum GD. Here, the momentum term with preconditioning is interpreted as an associative memory that learns

how to compress the mappings between P_i and the gradient term $\nabla \mathcal{L}(W_i; \mathbf{x}_i)$. While any reasonable choice (e.g., random features) of preconditioning may also be used to improve the expressivity of the initial version of GD with momentum. Note that our discussion on preconditioning in Equation 39 is different from here and indicates that one can learn the preconditioner as a proper mapping, while the above formulation shows when using preconditioning, momentum term (as a memory for gradients) aims to map them to their mapping function.

Extension: More Expressive Objectives. Revisiting the formulation of momentum: for a given gradients $\nabla \mathcal{L}(W_i; \mathbf{x}_i)$, the momentum mapping is based on dot-product similarity as the internal objective (as discussed above) and so its update rule is a Hebbian-rule (Hebb 2005). From an associative memory perspective, this update rule has a limited capacity (Storkey 1997) and makes the update rule of momentum independent of its current state. Thus it limits its ability to track/compress the loss landscape information. A natural extension is to replace the internal objective with L_2 -regression loss (for measuring the corresponding key-value mapping fitness) and minimize the loss function $\|\mathbf{m} \nabla \mathcal{L}(W_i; \mathbf{x}_i)^\top - P_i\|_2^2$, resulting in the update rule of:

$$W_{i+1} = W_i + \mathbf{m}_{i+1}, \quad (48)$$

$$\mathbf{m}_{i+1} = \mathbf{m}_i (\alpha_{i+1} - \nabla \mathcal{L}(W_i; \mathbf{x}_i)^\top \nabla \mathcal{L}(W_i; \mathbf{x}_i)) - \eta_t P_i \nabla \mathcal{L}(W_i; \mathbf{x}_i), \quad (49)$$

This update is based on delta-rule (Prados et al. 1989) and so it allows the memory (momentum) to better manage its limited capacity (i.e., $O(N)$) and better memorize the series of past gradients. For example, we may learn to forget some of the past gradients during the optimization process (similar to what happens when we move from linear attention to delta rule in associate memory). We refer to the variants of this type of momentum term as Delta Momentum variants.

Extension: More Expressive Memory. Viewing momentum as a compressor or a memory that store past gradients into its elements (parameters), its capacity not only depends on its update rule (similar to the above), but it also requires more expressive structure that allows for larger capacity. The current formulation is based on a linear layer (i.e., matrix-valued) to compress the past gradient values, but this linear nature may limit the capability to only learn linear mappings of past gradients. To increase the learning capacity of this module, one can use more complex mappings such as replacing a linear matrix-valued memory for momentum with an MLP. This design allows momentum to memorize more gradients and so provides better information for the optimization process. We extend the formulation in Equation 33 as:

$$W_{i+1} = W_i + \mathbf{m}_{i+1}(\mathbf{u}_i), \quad \text{and} \quad \mathbf{m}_{i+1} = \alpha_{i+1} \mathbf{m}_i - \eta_t \nabla \mathcal{L}^{(2)}(\mathbf{m}_i; \mathbf{u}_i, \mathbb{1}), \quad (50)$$

where $\mathbf{u}_i = \nabla \mathcal{L}(W_i; \mathbf{x}_i)$ and $\nabla \mathcal{L}^{(2)}(\cdot)$ is the internal objective of momentum (e.g., dot product similarity $\langle \mathbf{m}(\mathbf{u}_i^\top), \mathbb{1} \rangle$). We refer to this variant as Deep Momentum Gradient Descent (DMGD). While it is clear from this example, it is worth emphasizing that the internal loss function and the model need to be designed carefully to obtain an effective momentum module.

Extension: Memory with Higher-order Feature Maps. One of the commonly used techniques to enhance the capacity of a memory is to use higher-order feature maps on the keys (Katharopoulos et al. 2020; Kacham et al. 2024). Using this technique on the momentum term, one can obtain:

$$W_{i+1} = W_i + \mathbf{m}_{i+1} \quad \text{and} \quad \mathbf{m}_{i+1} = \alpha_{i+1} \mathbf{m}_i - \eta_t P_i \phi(\nabla \mathcal{L}(W_i; \mathbf{x}_i)), \quad (51)$$

where $\phi(\cdot)$ is a higher-order feature mapping (that may be learned through its internal objective).

Extension: Nonlinear Outputs. Building upon the associative memory perspective of the momentum, one common technique to enhance the representation power of memory module is to use non-linearity on top of its output (Sun et al. 2024; Behrouz et al. 2025c). That is, we re-formulate Equation 50 as:

$$W_{i+1} = W_i + \sigma(\mathbf{m}_{i+1}(\mathbf{u}_i)), \quad \text{and} \quad \mathbf{m}_{i+1} = \alpha_{i+1} \mathbf{m}_i - \eta_t \nabla \mathcal{L}^{(2)}(\mathbf{m}_i; \mathbf{u}_i, \mathbf{I}), \quad (52)$$

where $\sigma(\cdot)$ is an arbitrary non-linearity. As an example, we let $\sigma(\cdot) = \text{NewtonSchulz}(\cdot)$, where $\text{Newton-Schulz}(\cdot)$ is the iterative Newton-Schulz method (Higham 2008), and $\mathbf{m}(\cdot)$ be a linear layer; resulting in Muon (Jordan et al. 2024).

A Toy Example for Long Context in Optimizer. In Section 4.3, we discussed that in complex setups, including continual learning with orthogonal tasks, we may need more complex momentum terms, either with higher capacity or better memory management. To better illustrate the potential gains of other momentum memory designs, we use a toy example of a time-varying curvature. Since the standard momentum acts as a low-pass filter, if the landscape changes at a high frequency, then standard momentum which aims to use the weighted average of past gradients, will be under the influence of irrelevant gradient terms, delaying the convergence. As an illustrative example, consider:

$$\psi(r, \theta) = r^2 + k \times (r - \theta + \alpha \sin(\omega r))^2, \quad (53)$$

and aims to optimize it using a standard momentum and our delta momentum. We start the optimization process from point $(r_0, \theta_0) = (-3.5, 2)$ and continue until one of the algorithms converge to the optimal solution. The result is visualized in Figure 4. The delta momentum finds the solution faster, mainly due to its gradient-dependent weight decay that helps the momentum term to decay or stop when it is needed.

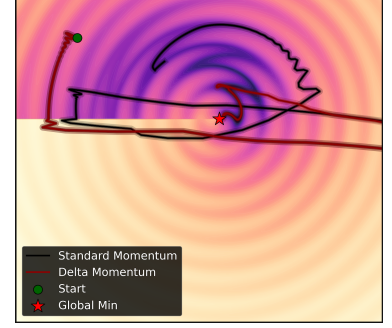


Figure 4: Optimization of function $\psi(r, \theta)$ with standard momentum and our delta momentum.

4.5 Going Beyond Simple Gradient Descent and Momentum

Coming back to the discussion in Section 3.1 about the pre-training process and backpropagation being a form of associative memory, in this section, we aim to take advantage of NL’s viewpoint and present a more general form for gradient descent. As observed in Section 4.1, backpropagation with gradient descent is an associative memory that aims to map the input data to the surprised caused by its predicted output $\nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t)$:

$$W_{t+1} = W_t - \eta_{t+1} \nabla_W \mathcal{L}(W_t; \mathbf{x}_t) = W_t - \eta_{t+1} \nabla_y \mathcal{L}(W_t; \mathbf{x}_t) \otimes \mathbf{x}_t, \quad \text{where } \mathbf{x}_t \sim \mathcal{D}_{\text{train}}, \quad (54)$$

which from the associative memory perspective and proximal gradient viewpoint is equivalent to:

$$W_{t+1} = \arg \min_W \langle W \mathbf{x}_t, \nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t) \rangle + \frac{1}{2\eta_t} \|W - W_t\|_2^2. \quad (55)$$

This step aims at learning the negative of the gradient direction. The main drawback of the dot-product similarity as the inner objective is that its corresponding update rule and so learning algorithm treats each data sample (gradients) independent of the state, meaning that the state of the weights and so previous gradients do not affect the update term to the current state. While this design can be effective for nested problems with independent elements in their context (e.g., i.i.d. samples for training), it can be restrictive for context with highly dependent elements (e.g., tokens in a sequence). Defining $\mathbf{u}_t = -\nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t)$, one can extend this process to more expressive objectives such as $\mathcal{L}_2$ regression loss:

$$W_{t+1} = \arg \min_W \frac{1}{2} \|W \mathbf{x}_t - \mathbf{u}_t\|_2^2 + \frac{1}{2\eta_t} \|W - W_t\|_2^2. \quad (56)$$

For the cases that $\mathbf{x}_t$ is normalized (e.g. in normalized memory systems or in neural networks with normalization layers, $\|\mathbf{x}_t\|_2 = \lambda$), and by defining $\eta'_t = \frac{\eta_t}{1+\eta_t}$, we can use Sherman-Morrison lemma to get (see Appendix C for the details):

$$\begin{aligned} W_{t+1} &= W_t (\mathbf{I} - \eta'_t \mathbf{x}_t \mathbf{x}_t^\top) - \eta'_t \nabla_{W_t} \mathcal{L}(W_t; \mathbf{x}_t) \\ &= W_t (\mathbf{I} - \eta'_t \mathbf{x}_t \mathbf{x}_t^\top) - \eta'_t \nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t) \otimes \mathbf{x}_t, \quad \text{where } \mathbf{x}_t \sim \mathcal{D}_{\text{train}}. \end{aligned} \quad (57)$$

This new algorithm, which based on Delta rule (Prados et al. 1989) we refer to as Delta Gradient Descent (DGD), updates the weights not only with respect to the current elements, but it also incorporates the previous state of weights, resulting in an adaptive decay term based on the current data sample. Next, we discuss a generalized viewpoint about the process of backpropagation with gradient descent, which later will help us to formulate Generalized Gradient Descent family of learning rules:

Training a Deep Neural Network with Backpropagation is a Self-Referential Process:

As discussed earlier in Section 4.1, one common misinterpretation for gradient descent is to view it as a form of linear recurrence (e.g., linear attention). In a conventional linear recurrence, however, keys and values are independent of the state of the memory, and so allows for parallelization of the formulation. The values in “gradient descent as associative memory” viewpoint are a function of the state of the memory, and so it is a self-referential model (Schmidhuber 1993) that controls its own learning process by generating its own values. More formally, one can reformulate the process as:

$$\begin{aligned} W_{t+1} &= W_t + \eta_{t+1} v_t \otimes x_t, \\ v_t &= f_{W_t}(x_t) = -\nabla_{y_t} \mathcal{L}(W_t; x_t), \end{aligned} \quad (58)$$

which means that at each step v_t is generated by memory W_t and x_t as its input.

Based on the above interpretation, one can define backpropagation with gradient descent in a general form as *any* self-referential model that aims to compress training samples as keys and map them to *self-generated* values to better control its own learning process. Based on this definition, our above formulation of Appendix C, is only a simple instance that uses $\mathcal{L}_2$ regression loss; in general, however, one can define Generalized Gradient Descent (GGD) as:

Definition 5 (Generalized Gradient Descent (GGD) Learning Rule). *Generalized Gradient Descent (GGD) learning rule is a self-referential associative memory that aims to compress data samples and map them to a set of self-generated keys:*

$$W_{t+1} = \arg \min_W \tilde{\mathcal{L}}(x_t, u_t) + \text{Ret}(W, \{W_i\}_{i=t-c+1}^t), \quad (59)$$

where u_t is a self-generated value:

$$u_t = f_{W_t}(x_t), \quad (60)$$

for some function $f_{W_t}(\cdot)$ parameterized by W_t . Here, $\tilde{\mathcal{L}}(\cdot)$ measures the quality of the mapping, and $\text{Ret}(\cdot)$ ensures that the solution for the new instance is not far away from the current state.

Similarly, this formulation can be adapted for the momentum term, resulting in Generalized Momentum (GM). However, it is notable that the momentum itself, is a conventional associative memory and its keys and values are given, or more specifically are generated by a lower-frequency level. In Section 4.2, we explored a special case of this formulation, where $\mathcal{L}(\cdot)$ is L_2 regression loss.

A Note on Optimizers in Continual Learning Setup. As discussed above, optimizers themselves are learning modules or associative memories that aim to compress the gradients into their parameters. These parameters are not necessarily trainable in the conventional terminology but indeed momentum-based optimizers store the knowledge about the loss landscape, helping them to better update the weights. When the “end of pretraining” happens for a neural learning module, the knowledge stored about the distribution of gradients/data, which are stored in the momentum term(s) is removed from the model and so continuing the training without recovering the momentum states can affect the model’s ability to learn new capabilities. When the model is in continual learning setup, the knowledge about data is stored in the conventional parameters (optimized with backpropagation), but the knowledge about how the model optimizes itself and about the objective space are optimized in the lower-frequency levels of optimization (e.g., momentum terms).

5 Existing Architectures as Neural Learning Modules

Modern sequence models such as Transformers (Vaswani et al. 2017) and recurrent models (Katharopoulos et al. 2020; Schlag et al. 2021; Sun et al. 2024; Behrouz et al. 2025c) are the backbones of recent advances in language models. Recently, the equivalency of such models with associative memories that aim to learn a mapping from keys to values from data have been studied in different settings and objectives (Liu et al. 2024b; Sun et al. 2024; Behrouz et al. 2025b; Wang et al. 2025). Particularly, we focus on the general framework of Miras (Behrouz et al. 2025b), which defines associative memory as Definition 1 and optimizes the internal objective (called “attentional bias”) with a choice of optimization algorithm on an arbitrary class of functions (i.e., memory architecture). While this formulation alone indicates that the well-known architectures are instances of nested systems of associative memory (NSAM), next, we review this equivalency for some learning rules and architectures.

From now on, we assume that keys $\{\mathbf{k}_i\}_{i=1}^L$, values $\{\mathbf{v}_i\}_{i=1}^L$, and queries $\{\mathbf{q}_i\}_{i=1}^L$ are given: they often are defined as the projections of the input, i.e.,

$$\mathbf{k}_t = \mathbf{x}_t \mathbf{W}_k, \quad \mathbf{v}_t = \mathbf{x}_t \mathbf{W}_v, \quad \mathbf{q}_t = \mathbf{x}_t \mathbf{W}_q. \quad (61)$$

In this design, since projection parameters (i.e., $\mathbf{W}_k$, $\mathbf{W}_v$, and $\mathbf{W}_q$) are optimized in a lower frequency level, the sequence model component (e.g., self-attention) has a higher frequency and so the learning process of the associative memory happens in a lower level. Accordingly, for the sake of clarity, we only discuss the higher frequency level (i.e., the internal learning process of the associative memory).

Softmax Attention. From the associative memory viewpoint: given keys $\{\mathbf{k}_i\}_{i=1}^L$, values $\{\mathbf{v}_i\}_{i=1}^L$, and queries $\{\mathbf{q}_i\}_{i=1}^L$, Softmax attention block (Bahdanau 2014; Vaswani et al. 2017) can be reformulated as a non-parametric solution to the $\ell_2(\cdot)$ regression objective with Nadaraya-Watson estimators (Fan 2018; Zhang et al. 2022):

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \sum_{i=1}^L \|\mathbf{s}(\mathbf{k}_i, \mathbf{q}) \mathbf{v}_i - \mathcal{M}\|_2^2 = \sum_{i=1}^L \frac{\mathbf{s}(\mathbf{k}_i, \mathbf{q})}{\sum_{j=1}^L \mathbf{s}(\mathbf{k}_j, \mathbf{q})} \mathbf{v}_i, \quad (62)$$

where L is the sequence length (Sun et al. 2024). This formulation optimizes the memory $\mathcal{M}(\cdot)$ with respect to the entire context; however, one design choice can be to limit the optimization process to the past c tokens, resulting in:

$$\mathcal{M}^* = \arg \min_{\mathcal{M}} \sum_{i=t-c+1}^t \|\mathbf{s}(\mathbf{k}_i, \mathbf{q}_i) \mathbf{v}_i - \mathcal{M}\|_2^2 = \sum_{i=t-c+1}^t \frac{\mathbf{s}(\mathbf{k}_i, \mathbf{q})}{\sum_{j=t-c+1}^t \mathbf{s}(\mathbf{k}_j, \mathbf{q})} \mathbf{v}_i, \quad (63)$$

which is equivalent to the sliding window attention (SWA). Therefore, attention and its more expressive variants (Wang et al. 2025) also are instances of Definition 1, when instead of gradient descent or other parametric methods, we find the optimal non-parametric solution to the mapping.

RNNs with Hebbian Rule. The first generation of modern recurrent architectures (e.g., Linear attention (Katharopoulos et al. 2020), RetNet (Sun et al. 2023), RWKV (Peng et al. 2023), lightening attention (Li et al. 2025)) are based on Hebbian-like learning rules (Hebb 2005). For this class of models, the inner objective to measure the quality of mapping between keys and values is the dot-product similarity. That is, given a matrix-valued memory $\mathcal{M} \in \mathbb{R}^{d \times n}$, keys and values $\mathbf{k}, \mathbf{v} \in \mathbb{R}^d$, objective $\tilde{\mathcal{L}}(\mathcal{M}; \mathbf{k}_t, \mathbf{v}_t) := -2\langle \mathcal{M} \mathbf{k}_t, \mathbf{v}_t \rangle$, and a kernel $\phi(\cdot)$, we optimize the equivalent associative memory optimization problem (see Definition 1) with gradient descent and weight decay, resulting in:

$$\mathcal{M}_t = \alpha_t \mathcal{M}_{t-1} - \underbrace{\eta \nabla_{\mathcal{M}_{t-1}} \tilde{\mathcal{L}}(\mathcal{M}_{t-1}; \phi(\mathbf{k}_t), \mathbf{v}_t)}_{-\mathbf{v}_t \phi(\mathbf{k}_t)^\top} = \alpha_t \mathcal{M}_{t-1} + \eta_t \mathbf{v}_t \phi(\mathbf{k}_t)^\top, \quad (64)$$

which recovers the original linear attention recurrence (Katharopoulos et al. 2020). Given different settings for α_t (i.e., either is 1, learnable, channel-wise, and/or input-dependent) and also $\phi(\cdot)$ (i.e., identity, polynomial kernels, etc.), the above recurrence recovers different variants of linear attention with Hebbian rule (Katharopoulos et al. 2020; Sun et al. 2023; Arora et al. 2024; Beck et al. 2024; Kacham et al. 2024; Peng et al. 2024). Therefore, the variants of linear attention with Hebbian rule can be reformulated as the process of an optimization problem, in which the memory aims to learn the mapping between keys and values based on dot-product similarity objective, with gradient descent.

RNNs with Delta Rule. To improve the memory management and to enhance the memory capacity of the above group, several studies suggest replacing Hebbian rule with Delta rule as the learning algorithm in recurrent neural networks (Schlag et al. 2021), resulting in models such as DeltaNet (Schlag et al. 2021), Longhorn (Liu et al. 2024b), and RWKV7 (Peng et al. 2025b). When letting $\mathcal{M} \in \mathbb{R}^{d \times n}$, delta rule is equivalent to optimizing MSE objective $\tilde{\mathcal{L}}_t = \|\mathcal{M}_t \mathbf{k}_t - \mathbf{v}_t\|_2^2$ with $\text{Ret}_t(\mathcal{M}, \mathcal{M}_{t-1}) = \|\mathcal{M}_t - \mathcal{M}_{t-1}\|_F^2$ as local retention, and stochastic gradient descent as the optimizer:

$$\mathcal{M}_t = \mathcal{M}_{t-1} - \underbrace{\eta_t \nabla_{\mathcal{M}_{t-1}} \tilde{\mathcal{L}}(\mathcal{M}_{t-1}; \phi(\mathbf{k}_t), \mathbf{v}_t)}_{(\mathcal{M}_{t-1} \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top} = (\mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathcal{M}_{t-1} + \eta_t \mathbf{v}_t \mathbf{k}_t^\top. \quad (65)$$

Using other forms of retention gates (e.g., $\text{Ret}_t(\mathcal{M}, \mathcal{M}_{t-1}) = \|\mathcal{M}_t - \alpha_t \mathcal{M}_{t-1}\|_F^2$), optimization algorithms with weight decay (e.g., regularizing with $\|\mathcal{M}_t\|_q^q$ for a given $q > 0$), multiple steps of gradient descent, and/or different formulations of

learnable parameters such as η_t and α_t can result in diverse variants of delta rule (Irie et al. 2021; Liu et al. 2024b; Sun et al. 2024; Behrouz et al. 2025b; Hu et al. 2025; Peng et al. 2025b; Siems et al. 2025; Wang et al. 2025). Therefore, Delta rule and its variants are all instances of an optimization problem, in which the model aims to learn a mapping between keys and values based on the L_2 -regression objective.

Beyond Conventional Learning Rules: Omega, Oja’s, and Non-Euclidean Learning Rules. More recently, there have been growing interests in designing architectures from the associative memory perspective (see Definition 1) and use more complex internal objectives, and/or optimization algorithms, resulting in learning algorithms beyond Delta and Hebbian rules (Irie et al. 2022a; Von Oswald et al. 2023; Behrouz et al. 2025a,b; Zhang et al. 2025). More specifically, to enhance the stability of Hebbian rule (discussed in Equation 64), Irie et al. (2022a) introduced OjaNet based on Oja’s rule (Oja 1982) with the following recurrence:

$$\mathcal{M}_t = \alpha_t \mathcal{M}_{t-1} + \eta_t \mathbf{v}_t (\phi(\mathbf{k}_t)^\top - \mathcal{M}_{t-1}^\top \mathbf{v}_t). \quad (66)$$

In the associative memory formulation (as in Definition 1), this recurrence can simply be reformulated as one step of gradient descent as:

$$\mathcal{M}_t = \mathcal{M}_{t-1} - \eta_t \underbrace{\nabla_{\mathcal{M}_{t-1}} \tilde{\mathcal{L}}(\mathcal{M}_{t-1}; \phi(\mathbf{k}_t), \mathbf{v}_t)}_{\mathcal{M}_{t-1}^\top \mathbf{v}_t - \mathbf{v}_t \phi(\mathbf{k}_t)^\top}, \quad (67)$$

where $\tilde{\mathcal{L}}(\mathcal{M}; \mathbf{k}_t, \mathbf{v}_t) = -2\langle \mathcal{M} \mathbf{k}_t, \mathbf{v}_t \rangle + \|\mathcal{M}^\top \mathbf{v}_t\|_2^2$ and $\phi(\cdot)$ is a kernel (Irie et al. 2022a, 2025b). Although this design enhances the Hebbian learning rule by enforcing a unit-norm constraint for the single-neuron, it has been reported to empirically underperform models based on Delta learning rule (Irie et al. 2022a). To further enhance the Delta rule through the design of more expressive objectives, recently, Behrouz et al. (2025b) suggested going beyond Euclidean spaces and use $L_p = \|\cdot\|_p^p$ norm for the internal regression objective, showing better empirical performance and robustness in long context tasks compared to Delta rule and its variants.

While the majority of learning rules are online update mechanisms—meaning that at each state, the models only need to keep the memory and the *current* (batch of) input—Omega rule (Behrouz et al. 2025a) suggest an update rule based on a set of past (batches of) inputs (or all inputs). More specifically, given a memory $\mathcal{M}$ with an arbitrary structure, keys and values $\mathbf{k}, \mathbf{v} \in \mathbb{R}^d$, an arbitrary objective $\tilde{\mathcal{L}}(\mathcal{M}; \mathbf{k}_t, \mathbf{v}_t)$, and a kernel $\phi(\cdot)$, Omega rule is defined as:

$$\mathcal{M}_t = \alpha_t \mathcal{M}_{t-1} - \sum_{i=t-c+1}^t \gamma_{t,i} \tilde{\mathcal{L}}(\mathcal{M}_i; \phi(\mathbf{k}_i), \mathbf{v}_i), \quad (68)$$

where $c \geq 1$ is the local window of cached inputs. Note that in the special case of $\gamma_{t,i} = 1$ and c equal to the entire context length, the optimal solution of the above design collapses into an online case, where the update rule only depends on the current state and the current input (Von Oswald et al. 2023). For further discussion with more details about representing architectures as associative memories and so an optimization problem, we refer the reader to Behrouz et al. (2025b).

A Note on Gating in Modern Sequence Models. One of the recent architectural changes in modern language models is the gating of a linear layer’s output with the output of the sequence model. Despite significant improvement resulted by this method, it is still unclear that how it enhances the performance. As we discussed in Figure 3 and its corresponding example, the main difference between feedforward network and modern recurrent memory modules (e.g., linear attention (Katharopoulos et al. 2020) or deep memory modules (Behrouz et al. 2025c)) when their initial state of the memory is meta-learned, is the second level in memory modules that perform in-context learning and adapt its state with the context. From this viewpoint, when the initial value of the memory is not meta-learned, it only relies on the in-context adaption of the memory and so there is no persistent memory system that stores the knowledge of pre-training in this block. Therefore, when the initial value of memory is not meta-learned, which is common in earlier variants of linear transformers, the gating of linear attention acts as a persistent memory and the initialization of the memory module.

5.1 Revisiting the Human Brain Perspective of Nested Learning

In Section 1.1, we discussed how structure in human brain is uniform and reusable, and if we need a new architecture in deep learning, or if our beliefs about the heterogeneity of current models need to be revisited. In the previous sections,

we observed that both optimization process of neural networks as well as neural architectures can be formulated as a set of nested and/or parallel optimization problems, in which the memory structure is a feedforward layer (e.g., either Deep MLPs, linear layers, etc.) and the objective is optimized with gradient descent or Newton’s methods.

From this perspective, modern architectures, are a set of artificial neurons (i.e., linear or deep feedforward networks), and each group of neurons has their own internal objective and so update mechanism. To this end, as a simple example, let us recall the AdaTransformer in Figure 3: Given $X = \{\mathbf{x}_i\}_{i=1}^T$ as the input sequence, the output of the block is computed as (For the sake of simplicity, we assume $\text{MLP}(\cdot) = \cdot W_{\text{MLP}}$, and remove normalizations):

$$\begin{aligned} \mathbf{k}_t &= \mathbf{x}_t W_k, & \mathbf{v}_t &= \mathbf{x}_t W_v, & \mathbf{q}_t &= \mathbf{x}_t W_q, \\ \mathbf{y}_{\text{attn}_t} &= \text{Attn}(\mathbf{k}_t, \mathbf{v}_t, \mathbf{q}_t), \\ \mathbf{y}_{\text{block}_t} &= \mathbf{y}_{\text{attn}_t} W_{\text{LinAttn}_t}, & \text{where} \\ W_{\text{LinAttn}_t} &= W_{\text{LinAttn}_{t-1}} + \mathbf{v}_t \mathbf{k}_t^\top, \end{aligned} \tag{69}$$

The lowest frequency level is responsible for the optimization of W_k , W_v , and W_q , all of which are feedforward networks and so are uniform. The attention itself is also the non-parametric matrix-valued solution of a regression objective, again verifying that the structure is a matrix of artificial neurons (i.e., parameters). Finally, the Linear Attention++ component is equivalent to the optimizing the dot-product similarity of the mappings over the linear class of functions. Therefore, all the parameters are matrix-valued or deep feedforward layers, meaning that the only difference in the components of an architecture is their level, objective, and/or learning update rule.

Modern Deep Learning Models Have Uniform and Reusable Structure

Neural learning modules consist of a set of feedforward networks, each of which are optimized in different levels and time scales. The heterogeneity we observe in deep learning architectures, however, is due to the lack of view to this new NL’s axis, resulting in observing only the solution of optimization problems and so causing the illusion of deep learning architectures.

6 Takeaways and Revisiting Common Terms

In the previous sections, we discussed the concept of nested learning and how existing well-known components of neural networks such as popular optimizers and architectures fall under the NL paradigm. In this section, we discuss the takeaways, connection of different concepts, and the implications of NL perspective on common terms.

Memory and Learning. For a long period of time, in machine learning models, memory have been treated as a separate block with a *clear* distinction between its parameters and other components. Such designs often assume a short and/or long-term memory blocks, where short-term memory is responsible for the local context, while long-term memory is the storage for the persistent knowledge in models. In human brain, however, memory is considered as a distributed interconnected system without a clear known components that are independently responsible for short or long-term memory. In NL, we build upon a common terminology for memory and learning in neuropsychology literature, indicating that: Memory is a neural update caused by an input and so learning is the process of acquiring useful memory (Okano et al. 2000). From this viewpoint, any update by gradient descent (or any other optimization algorithms) in any levels of neural learning module is considered as a form of memory. Interestingly, our findings in Section 4.1 on gradient descent being (self-referential) associative memory is aligned with this terminology. Furthermore, based on this terminology, in continuum memory system, the neural updates are applied at different frequencies and so memories are stored with different time scales, resulting in more robust memory management with respect to catastrophic forgetting.

Memory and Learning from Nested Learning Perspective:

Memory is not an isolated system and is distributed throughout the parameters. Particularly, any update that is caused by the input is a stored memory in the neural network, and the process of effectively store, encode, and in general acquire such memories is referred to as learning process.

A General Note on Parameters of a Model. The parameters of a model are one of its critical components that shape the units for knowledge storage, internal computations, and adaptability. Over the past decades, only (a subset of) the parameters in the architecture of machine learning models have been referred to as the *learnable* entities, mainly due to the fact that they are the only components that we have been directly and intentionally optimizing over the training data. From the NL viewpoint, such parameters are placed in the lowest frequency level (the highest level) and are updated for every (batch of) samples. They are, however, are not the only parameters that contributes to the model internal computation, knowledge storage, and adaptability. As discussed earlier in Section 4.2, momentum is an example of such cases, where its parameters are updated over time (by gradient descent) and stores the knowledge about the loss landscape of the model so far. Such information are critical when the model is continually learn, mainly due to the fact that to find an effective solution, the optimizer needs to have more information about the global properties of the loss landscape. Another example of such cases is the memory (or hidden state) of recurrent neural networks. Although those parameters are not directly optimized in the lowest frequency level, they store important knowledge about the current context. With the change of the context, the compressed knowledge in these parameters are removed due to the lack of knowledge transfer between these levels.

Models Have More Parameters Than We Knew:

The parameters of a neural learning module are not limited to those optimized in the pre-training level; all parameters that appear in the NL representation of the model contribute to its performance and expressivity.

More Computations per Neuron. One common misinterpretation about the concept of NL is to restrict the model design and stacking multiple levels to CMS case. In general, stacking levels can help the model to enhance the depth of computations and also perform more internal computations per each parameter in the lowest frequency level. One example of such designs is Muon optimizer and $\text{NewtonSchulz}_k(\cdot)$ operation (see Equation 42 - 44), which we showed that is equivalent to an internal optimization process. In this design, per each step of momentum update, we need k -steps of internal process to learn how to map gradients to an orthogonal space.

In-Context Learning. Throughout this paper, we use the most general definition of “in-context learning” and refer to it as the ability of a model to adapt to and learn from a given context. Following the definition of NSAM, each block or level has its own context flow and so any neural update or adaptation to that context is considered as a form of in-context learning. Due to the popularity of Transformers as well as being the first model that in-context learning is studied for, the concept of in-context learning sometimes is referred to as conditioning the output on the entire context. Considering the general term of in-context learning, this formulation is only one of the instances of in-context learning, which we referred to as *non-parametric in-context learning*. In general, however, the memory in recurrent models is performing in-context learning, in which the output is conditioned on the *compressed context*. Therefore, from NL perspective, all the levels are performing in-context learning but on their own context flow with their own learning update and optimization process.

Building on this definition, in-context learning is a model’s capability that is transparent from its NL representation, and per se it is not an emergent characteristic but a direct consequence of having multiple levels in the NL representation of the neural learning module. Although this might seem contradictory with previous claims about ICL being an emergent characteristic (Brown et al. 2020; Singh et al. 2023), it is notable that the good performance of the model in ICL tasks also requires a powerful low-frequency level, enabling the high frequency level to adapt fast. When the model is not well-trained, the higher-frequency level is on its own to learn from the context. This setup might result in a poor performance, mainly due to the fact that there might be not enough data in context to allow the high-frequency parameters to converge.

(Test-Time) Learning/Memorization. Recently, the concept of test time training (Sun et al. 2024; Wang et al. 2025) or test time memorization (Behrouz et al. 2025b) has gained popularity as a backbone framework to design powerful sequence models. In these frameworks, given the context, a new component/block aims to compress the context into its parameters using a learning rule and objective function. In this formulation when the context is removed the acquired in-context knowledge diminishes along with it. As also discussed in the previous part, this update mechanism and learning process is indeed an instance of “*parametric in-context learning*”:

Test Time Training/Memorization are Instances of In-Context Learning

The concepts commonly referred to as test-time training and test-time memorization are in fact instances of parametric in-context learning, where the acquired in-context knowledge does not persist once the current context is removed.

Notably, when moving toward continual learning setup, there is no test-time or training-time and so it can be misleading to refer to (parametric) in-context learning as test-time learning/memorization.

Pre-training and Test Time. From the nested learning perspective, the lowest frequency level (i.e., the highest level) corresponds to a learning phase that is often referred to as pre-training. Accordingly, pre-training is one of the levels and so has its own context flow (i.e., pre-training dataset), objective (e.g., next token prediction), and optimization process (e.g., AdamW). Accordingly, one can interpret the pre-training as one of the possible instances of in-context learning, where the context is the entire pre-training data.

Pre-training is In-Context Learning with Ultra-Large Context Length:

From NL's viewpoint pre-training is only one of the possible instances of in-context learning, where the context is the entire pre-training data. The distinction of training and test time in models is the results of disconnecting the knowledge transfer process from the highest frequency level (e.g., the context of Transformers) to the low frequency levels (i.e., pre-training).

This formulation and viewpoint is specifically important when we shift from pre-training paradigm to models that are capable of continually interact and learn from data/world (e.g., Sutton (2025)):

Continual Learning. From NL's perspective each phase of training for a model is defined as one of the low-frequency levels, which by design, we might want to stop the data processing in one level (e.g., "End of Pre-training"), or continue it without any knowledge transfer to other levels (e.g., conventional formulation of in-context learning in Transformers). Accordingly, any machine learning model, no matter if it is during its pre-training or at test time, is performing continual learning as given a data sample, it requires performing internal computations to provide the output. However, the knowledge from that learning might not last or transfer to more persistent levels, mainly due to the lack of knowledge transfer between levels.

No Training or Test Time in Neural Learning Modules:

For a neural learning module, there is no boarder and clear distinction between training and test time. The model only experiences two different states: when it receives information as input, or when it is an isolated learning system.

We also revisit some of the common terms in the architectural design in the following:

Existing Architectural Backbones and Hybrid Models. As discussed earlier in Section 5.1, from the NL's perspective all modern architectures are uniform and in fact are feedforward layers (linear or non-linear MLP blocks) that are trained based on their own context flow and optimization problem. When viewing models from deep learning perspective, we see the final solution of such optimization problem (i.e., we see attention rather than a non-parametric solution to a regression loss), which results in the illusion of having distinct and non-uniform architectures.

Recurrent Models are Replacing MLP Blocks:

From NL's viewpoint, (deep or linear memory) recurrent models are MLP blocks that a new level is added to their internal computation. Accordingly, existing hybrid architectures can be seen as conventional Transformer models, when we added a new level of computation to some of the MLP blocks.

Another important aspect of understanding the existing architectures during their so-called "pre-training" phase is to understand how they transfer their knowledge from one level to another.

Knowledge Transfer from In-Context Learning

Although both modern deep and linear recurrent models are unified using associative memory perspective, there is still an important difference between the existing instances: While deep memory modules such as Titans, Atlas, Miras, and TTT take advantage of knowledge transfer from their high-frequency level to their lower-frequency level through meta-learning the initial state of memory, most linear memory recurrent models have no knowledge transfer process between their levels.

Neural Learning Module as an Inter-Connected System. One of the critical messages in NL is the fact that neural learning modules are inter-connected systems, meaning that the design of each component can significantly affect the design of other parts. This fact motivates follow up and future studies to better understand how one can properly design a neural learning module with all components work together in a harmony. As an example of such, the context of optimizers (i.e., gradients) is generated by the architecture component. Therefore, different architectures might show different characteristics in the generated gradient patterns and so one optimizer might not be the best option for all the architectures (Zhang et al. 2024b).

Architectures Generates the Context for Optimizers

Neural learning modules are inter-connected systems, where architecture generates the context for optimizers (i.e., gradients). Therefore, the proper memory management of gradients (i.e., optimization algorithm) relies on the choice of architectures. In future, when viewing models as a neural learning modules, we need to design architecture specific optimizers so this inter-connected system works perfectly in harmony.

Optimizers vs. Learned Optimizers. Finally, we want to emphasize that our formulation of momentum, gradient descent, and/or other gradient-based optimizers show that they are associative memory modules aiming to compress the data and gradients into their parameters. Such update and compression process is based on gradient descent and so has a very similar nature to the learning process of learned optimizers. From the NL’s viewpoint, both vanilla optimizers as well as learned optimizers are instances of the same concept but with different frequency and context flow: Although the parameters of learned optimizers are located in the lowest frequency level (i.e., to train and be optimized along with other parameters in the pre-training), the parameters of vanilla optimizers are located in their own level and so has their own gradient flow.

7 Continuum Multi-Timescale Memory System

Existing architectural backbones consist of (1) a *working memory* module (e.g., attention), which is responsible to actively fuse the information across sequence length, and (2) a feed-forward layer (e.g., MLP) that fuse information across features and acts as the persistent memory or knowledge storage of pre-training phase. From the NL perspective, pre-training is the phase that the most outer level of the learning module is updated over its *limited* context flow. Accordingly, in the continual setup, such pre-training phase is also rarely updated over time, and so its corresponding knowledge storage needs to rarely be updated over time. Given this intuition, we extend the traditional view-point of long-term/short-term memory system and suggest a knowledge storage feed-forward for each level (frequency domain).

7.1 Continuum Memory System (CMS)

Given the definition of frequency (Definition 2), Continuum Memory System (CMS) is formalized as a chain of MLP blocks $\text{MLP}^{(f_1)}(\cdot), \dots, \text{MLP}^{(f_k)}(\cdot)$, each of which associated with a chunk size of $C^{(\ell)} := \frac{\max_i C^{(i)}}{f_i}$ such that given input $x = \{x_1, \dots, x_T\}$ the output of the chain is calculated as (we disregard normalizations for the sake of clarity):

$$y_t = \text{MLP}^{(f_k)}(\text{MLP}^{(f_{k-1})}(\dots \text{MLP}^{(f_1)}(x_t))), \quad (70)$$

where the parameters of ℓ -th MLP block, i.e., $\theta^{(f_\ell)}$, are updated every $C^{(\ell)}$ steps:

$$\theta_{i+1}^{(f_\ell)} = \theta_i^{(f_\ell)} - \begin{cases} \sum_{t=i-C^{(\ell)}}^i \eta_t^{(\ell)} f(\theta_t^{(f_\ell)}; x_t) & \text{if } i \equiv 0 \pmod{C^{(\ell)}}, \\ 0 & \text{otherwise.} \end{cases} \quad (71)$$

Here $\eta_t^{(\ell)}$ are learning rates corresponds to $\theta^{(f_\ell)}$, and $f(\cdot)$ is the error component of an arbitrary optimizer (e.g., $\nabla \mathcal{L}(\theta_t^{(f_\ell)}; x_t)$ in gradient descent). The conventional Transformer block (Vaswani et al. 2017) is a special instance of this formulation, where $k = 1$ and frequency of update is zero. Note that the objective of $\mathcal{L}(\cdot)$ is the objective of choice for the task at hand, e.g., for language modeling it is next token prediction objective. It is notable that Equation 71 provides an important interpretation: parameters $\theta_t^{(f_\ell)}$ are responsible for compressing their own context into the their parameters and so they are a representative of abstract knowledge of their context.

As discussed in Section 3.3, different levels might have different process of knowledge transfer. Accordingly, while the above formulation suggests a spectrum of memory systems in different levels and so with different frequencies, their connections can vary based on the design. In the following, we discuss some potential variants:

Nested Continuum Memory Systems. The first variant is a fully nested continuum memory system, in which the initial state of the MLP block in level $s + 1$ is meta-learned in level s . This design allows for higher-order in-context learning ability, where each of the levels has its own context flow and re-initialized after the end of the context. More specifically, given an arbitrary $1 \leq s \leq k$,

$$\theta_0^{(f_{s+1})} = \arg \min_{\Phi} \mathbb{E}_{\mathcal{T} \sim C^{(s)}} \left[\ell(\Theta, \mathcal{T}; \Phi) \right], \quad (72)$$

where $C^{(s)}$ is the context length of the MLP block in s -th level. Following this design, at the end of the optimization process of each block (i.e., after $\lceil C^{(s)} / C^{(s+1)} \rceil$ steps.) the value of the memory will be re-initialized to $\theta_0^{(f_{s+1})}$. Note that the update mechanism of each block in its own level remain unchanged (i.e., Equation 71).

Sequential Continuum Memory Systems. In the second variant, the MLP blocks are located sequentially (i.e., the output of the MLP block in level s is the input for the MLP block in level $s + 1$) and also the initial state of MLP blocks are all connected through backpropagation in the lowest frequency level. Given an arbitrary $1 \leq s \leq k$,

$$\theta_0^{(f_s)} = \arg \min_{\Phi} \mathbb{E}_{\mathcal{T} \sim C^{(1)}} \left[\ell(\Theta, \mathcal{T}; \Phi) \right], \quad (73)$$

where $C^{(1)}$ is the context length of the MLP block in the lowest frequency level. Since the initial state of all memories are meta-learned in the lowest frequency, the most persistent knowledge of all components is the compression of the same context flow.

Independent (Head-wise) Continuum Memory Systems. In this variant, we keep the knowledge transfer process in Equation 73, but change the output computation in Equation 70. While the previous formulation designs the memory system as a sequence of blocks, and so making their input/out dependent to each other, this variant uses independent blocks with different context length and then combine them using an aggregation process:

$$\mathbf{y}_t = \text{Agg} \left(\text{MLP}^{(f_k)}(\mathbf{x}_t), \text{MLP}^{(f_{k-1})}(\mathbf{x}_t), \dots, \text{MLP}^{(f_1)}(\mathbf{x}_t) \right). \quad (74)$$

The above $\text{Agg}(\cdot)$ is an arbitrary function that aggregates all the inputs to compute the output. For example, one straightforward and simple design choice is to use a learnable weighted sum of the input.

CMS Design Helps with Continual Learning. Based on the design of CMS, a fair question is to ask: Why and how CMS can help with longer context length and generally continual learning. Here, we provide a simple answer to this question: Viewing MLP blocks in CMS as the storage of model’s knowledge catastrophic forgetting can happen when we update a block and as its result, the old knowledge stored in its parameters are forgotten. In CMS design, however, when updating an arbitrary block of $\text{MLP}^{(f_s)}(\cdot)$ for some $1 \leq s \leq k$, the potentially forgotten knowledge from $\text{MLP}^{(f_s)}(\cdot)$ is still stored in other components such as $\text{MLP}^{(f_{s'})}(\cdot)$, where $s' < s$. Also, in this case (i.e., the knowledge is already forgotten from $\text{MLP}^{(f_s)}(\cdot)$ but it is still in $\text{MLP}^{(f_{s'})}(\cdot)$ for $s' < s$) the knowledge transfer through backpropagation (for their initial state) can circle back the knowledge to $\text{MLP}^{(f_s)}(\cdot)$, resulting in a loop through time dimension, and so hardly forgetting important knowledge.

Is CMS Efficient Enough? A common concern when updating the parameters of a model in a continual manner is its efficiency. Therefore, a fair question is to ask if CMS causes significant computational overhead for the model. To answer this question, let us recall from Section 5 that modern recurrent neural networks are also continually updating a subset of their parameters (i.e., their memory state). These parameter updates, however, take advantage of sequence parallelization as well as updating only a small number of parameters. To this end, for CMS, we highlight two points:

- In the CMS design, at each time, updates are restricted to blocks approaching their scheduled update time (based on their frequency). As a simple example, consider a Transformers but with replacing its MLP blocks with CMS (later in

Section 8, refer to this variant as HOPE-Attention). Let the model have L_{layer} layers, 4 levels of MLP blocks in CMS with highest frequency of $\hat{f}$, and hidden dimension of d_{in} . On average, the update cost is for $\mathcal{O}\left(\frac{1}{\hat{f}} \times \frac{L_{\text{layer}}}{5} \times d_{\text{in}}^2\right)$ of parameters, which consists of only a small number of parameters at each time.

- The update mechanism of Equation 71, not only helps with the enhancing the persistent memory of the model, but it also unlocks the sequence parallelization for higher frequency levels. More specifically, for input x_i when $i \not\equiv 0 \pmod{C^{(\ell)}}$ there is no sequential process inside the chunk and so all the computations for tokens correspond to different values of $i \not\equiv 0 \pmod{C^{(\ell)}}$ can be done in parallel. The details of such training algorithm is the same as the training procedure in Sun et al. (2024) and Behrouz et al. (2025c).

Therefore, in summary, CMS can be fast in practice, mainly due to the fact that it updates only small number of parameters at each time, and also its design unlocks sequence parallelization.

7.2 Continuum Memory System In Optimizers

As a proof of concept and to support the effectiveness of CMS in different context flows, in this section we present Multi-scale Momentum/Memory Muon (M3) optimizer. Particularly, we aim to use NL’s associative memory viewpoint to design an optimizer that not only compress the recent gradients effectively, but it also has a capability of incorporating the information about long past gradients. In Appendix B (Equation 101) we discuss that how Adam optimizer is an instance of associative memory, in which the gradients are mapped to their variance until that point. Following the discussion about the need of long-context capability of optimizers in Section 4.3, we first replace the simple associative memory formulation of H term in Equation 102 with our CMS (independent variant, Equation 74) with a two-level memory system, which we refer to the memories as $M^{(1)}$ and $M^{(2)}$:

$$\begin{aligned} M_t^{(1)} &= M_{t-1}^{(1)} + \beta_1 g_t, \\ M_t^{(2)} &= M_t^{(2)} - \beta_2 \begin{cases} \sum_{i=t-\hat{C}}^t g_i & \text{if } t \equiv 0 \pmod{\hat{C}} \\ 0 & \text{otherwise,} \end{cases} \end{aligned} \quad (75)$$

where $\hat{C}$ is the chunk size that we update the lower-frequency momentum term. Finally, to aggregate the momentum terms (the choice of $\text{Agg}(\cdot)$ in Equation 74), we use a simple weighted summation with the use of parameter $\alpha > 0$ as the coefficient of $M_t^{(2)}$. Following our discussion on the importance of $\text{Newton-Schulz}_T(\cdot)$ to map the gradients to a proper metric space (see Section 4.2 and Equation 43), following Muon (Jordan et al. 2024), we use $\text{Newton-Schulz}_T(\cdot)$ on the output of the momentum terms, before aggregating them with weighted sum. This helps the associative memory to better manage its capacity by updating its parameters in a proper direction. The pseudocode for M3 is in Figure 1. In summary, one can say that M3 is the combination of Adam (Kingma et al. 2014a), Muon (Jordan et al. 2024), and our Continuum Memory System.

Notably, this optimizer is designed as a proof-of-concept to support the design of CMS. The M3 optimizer per se, however, might suffer from computational overhead and so face challenges when scaling to larger networks (see Figure 12). Furthermore, it is notable that the main point of M3 design based on CMS is to delay the update of the memory to gain longer-context. Similar studies such as the work of Devan Selvaraj (2025) and Pagliardini et al. (2025), while using multiple momentum terms, approach this long-context momentum for optimizers using a controlled learning rate for the additional momentum terms.

7.3 Ad-hoc Level Stacking: Initializing CMS with Pre-Trained Models

In our discussion on Figure 3, we observed that the initial state of the memory modules are optimized in lower-frequency levels and so one can interpret them as MLP blocks in the vanilla Transformer architectures (Vaswani et al. 2017). Therefore, a natural question is if we can leverage pre-trained models to initialize CMS blocks. One of the important advantages

Algorithm 1 Multi-scale Momentum Muon (M3)

Input: Initial weights Θ_0 , objective $\mathcal{L}(\cdot)$, learning rate $\eta > 0$, Newton-Schulz steps T , momentum factor $1 > \beta_1, \beta_2, \beta_3, \alpha \geq 0, \epsilon > 0$, frequency f ;

- 1: Initialize momentums: $M_0^{(1)}, M_0^{(2)} \leftarrow \mathbf{0}, V_0 \leftarrow \mathbf{0}$;
- 2: **for** lower-frequency iteration $k = 0, 1, 2, \dots$ **do**
- 3: Slow Memory: $M_t^{(2)} = M_{t-1}^{(2)} + \beta_3 \sum_{i=(k-1)f}^{kf} g_i$;
- 4: $O_t^{(2)} \leftarrow \text{Newton-Schulz}_T(M_t^{(2)})$;
- 5: **for** $t = kf + 1, kf + 2, \dots, (k+1)f$ **do**
- 6: Compute Gradient: $g_t = \nabla_{\Theta_t} \mathcal{L}(\Theta_t)$;
- 7: First Momentum: $M_t^{(1)} = M_{t-1}^{(1)} + \beta_1 g_t$;
- 8: Second Momentum: $V_t = V_{t-1} + \beta_2 g_t^2$;
- 9: $O_t^{(1)} \leftarrow \text{Newton-Schulz}_T(M_t^{(1)})$;
- 10: $\Theta_t \leftarrow \Theta_{t-1} - \eta \frac{O_t^{(1)} + \alpha O_t^{(2)}}{\sqrt{V_t + \epsilon}}$;
- 11: **end for**
- 12: **end for**

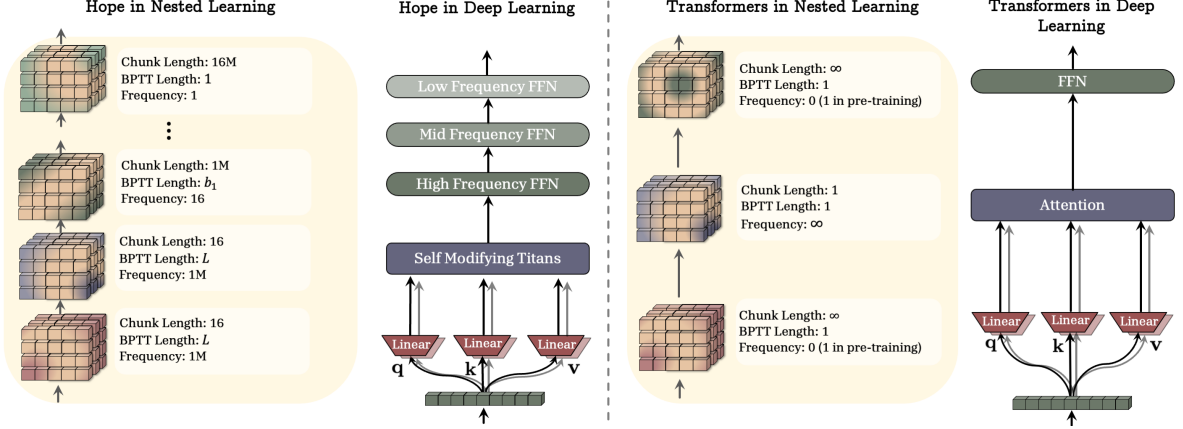


Figure 5: A comparison of Hope architectural backbone with Transformers (Normalization and potential data-dependent components are removed for the sake of clarity).

of NL is its flexibility to view and modify parameters in different levels. That is, since each level has its own context flow and optimization process, one can simply initialize the parameters in each level independently so that it helps the model to adapt faster to the levels' context flow. To this end, in this section, we suggest initializing the parameters in a level with a model's pre-trained weights. More formally, given a CMS with $\{\text{MLP}^{(f_i)}(\cdot)\}_{i=1}^k$, and a set of pre-trained MLP blocks $\{\text{MLP}_{\text{pre-trained}_i}(\cdot)\}_{i=1}^k$ we use Equation 71 to update $\{\text{MLP}^{(f_i)}(\cdot)\}_{i=1}^k$ in different levels; we, however, use the trained parameters of $\{\text{MLP}_{\text{pre-trained}_i}(\cdot)\}_{i=1}^k$ as the initial state of CMS blocks: $\text{MLP}_0^{(f_i)}(\cdot) = \text{MLP}_{\text{pre-trained}_i}(\cdot)$.

Why This Initialization Should Work? In NL, when there is a knowledge transfer process between two levels, the higher frequency level can take advantage of the knowledge stored in the lower frequency level and so adapt faster to its own context flow. The internal learning rate in the higher-frequency level, however, can control the capacity of the model for adaptability. That is, consider the above case, where all the blocks are initialized with pre-trained MLP blocks, setting $\eta_t^{(\ell)} \rightarrow 0$ keeps the updated memory blocks close to their initial states, resulting in directly using of pre-trained blocks, without adaption. Later in Section 9, we use this method to adapt pre-trained Transformer architectures to the HOPE's setup.

8 HOPE: A Self-Referential Learning Module with Continuum Memory

As we discussed earlier in Section 5.1, architectures in nested learning are uniform, i.e., a set of feedforward neural network blocks, each of which with its own context, update frequency, and internal objective. Sequence models—a common term to refer to blocks often with the highest update frequency that fuse information across tokens in the input sequence—are critical components for memory management and in-context learning ability of models. Following our earlier discussion in Section 5, modern sequence models can be seen as associative memories and so are nested optimization problems. From this perspective, global softmax attention or its more expressive higher-order variants are perfect memories (forcing to cache all past tokens) with frequency update of infinity as they are non-parametric solutions for optimizing (local) L_2 -regression objective with Nadaraya-Watson estimators (Fan 2018; Zhang et al. 2022) (see Equation 62). Therefore, parametric solutions (e.g., modern RNNs) for the similar objectives and when the parameter search space are the same (i.e., matrix-valued memory) are not expected to outperform softmax attention when the model size and data scales. To this end, and to design powerful sequence models, we need to understand where Transformers are limited and how one can overcome such limitations.

From the nested learning perspective, Transformers are two-level components, where projections and MLP blocks are optimized in the first level and the second level is responsible for in-context learning with finding the non-parametric solution and so conditioning the output on the context. This design, however, has limited computational depth as also stated in recent studies on state-tracking and similar computational capabilities of models (Merrill et al. 2024; Sanford et al. 2024; Grazi et al. 2025). Furthermore, Transformers' parameters are static throughout their context, meaning that their found solution to map tokens in the context (since it is non-parametric solution) remains the same and so they lack

the ability to modify themselves (at least in-context). More specifically, the initial linear blocks, W_k , W_v , and W_q , that projects input data to keys, values, and queries, are fixed after the pre-training stage (i.e., are in the first level) and so the Transformer’s ability to contextualize and map tokens is bounded by the knowledge stored in these blocks. For example, given a 1-layer Transformer, the projection of each token is a function of the token itself and its position; therefore, as an example, it can miss the diverse possible encodings of words whose meaning depend on the context, rather than the word itself. Although with increasing the depth of the model this issue might fade in later layers, we should not rely on the depth to compensate the models ability as it still is a bottleneck to unleash the capability of the model in earlier layers.

To overcome the above challenge, recently, the use of short convolutions and canon layers (Allen-Zhu 2025) have become a de facto component in modern models. Despite their success in mixing local tokens, still the models are fundamentally limited to adapt to the context and capture the global information beyond the local mixing. In the next part, we discuss a fundamental solution by presenting self-referential Titans that allows all the components to perform in-context learning, and adapt and modify themselves:

8.1 Deep Self-Referential Titans

A general formulation for the associative memory-based blocks is to project the data into keys, values, and queries and learns how to map keys to values and how to retrieve from the mapping based on queries. More formally, for a parametric associative memory, let $\mathbf{x}_t \in \mathbb{R}^d$ for $t = 1, \dots, L$ be the input, we have:

$$\mathbf{k}_t = \mathbf{x}_t W_k, \quad \mathbf{v}_t = \mathbf{x}_t W_v, \quad \mathbf{q}_t = \mathbf{x}_t W_q, \quad \eta_t = \mathbf{x}_t W_\eta, \quad \alpha_t = \mathbf{x}_t W_\alpha, \quad (76)$$

$$\min_{\mathcal{M}} \mathcal{L}(\mathcal{M}; \mathbf{k}_t, \mathbf{v}_t), \quad \text{with an optimization algorithm} \quad (77)$$

$$\mathbf{y}_t = \mathcal{M}_t \mathbf{q}_t. \quad (78)$$

For the sake of clarity, we use red (resp. blue) to highlight computations/weight in the upper level (resp. lower level). Similar to example in Figure 3, we can add a new level for each of W_k , W_v , W_q , W_η , and W_α and allow them to be updated in-context. For the sake of efficiency, a simple version is to share the values for all the components in the nested system of associative memories:

$$\mathbf{k}_t = \mathcal{M}_{\mathbf{k},t-1}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v},t-1}(\mathbf{x}_t), \quad \mathbf{q}_t = \mathcal{M}_{\mathbf{q},t-1}(\mathbf{x}_t), \quad \eta_t = \mathcal{M}_{\eta,t-1}(\mathbf{x}_t), \quad \alpha_t = \mathcal{M}_{\alpha,t-1}(\mathbf{x}_t), \quad (79)$$

$$\min_{\mathcal{M}_\square} \mathcal{L}(\mathcal{M}_\square; \square_t, \mathbf{v}_t), \quad \text{with an optimization algorithm, } \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha\}, \quad (80)$$

$$\min_{\mathcal{M}_{\text{mem}}} \mathcal{L}(\mathcal{M}_{\text{mem}}; \mathbf{k}_t, \mathbf{v}_t), \quad \text{with an optimization algorithm,} \quad (81)$$

$$\mathbf{y}_t = \mathcal{M}_{\text{mem},t}(\mathbf{q}_t), \quad (82)$$

where the initial states of all memories, i.e., $\mathcal{M}_{\square,0}$ for any $\square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}$ are meta-learned across all sequences/contexts. As discussed earlier, the meta-learning of the initial states of memories is essential for both fast-adaption, training stability, robustness to noise in the data.

This design provides a fully adaptive memory, where all the components can adapt themselves in-context. It, however, (1) still lacks self-modification, where the model in response to new data changes its own parameters or learning process (Schmidhuber 2003); (2) has suboptimal design as it shares of keys and values for all the memories. In continual learning, where the model requires consistent weight/knowledge update in response to new data, it is critical for the model to not solely rely on data, and instead learns how to modify itself when it is needed. Motivated by the above points, and inspired by the self-modifying mechanisms that generate their own values based on the context (Schmidhuber 1993, 2003; Irie et al. 2022b), we present self-modifying deep associative memory where the models generate their own values:

$$\mathbf{y}_t = \mathcal{M}_{\text{memory},t-1}(\mathbf{q}_t), \quad \mathbf{k}_t = \mathcal{M}_{\mathbf{k},t-1}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v},t-1}(\mathbf{x}_t), \quad \eta_t = \mathcal{M}_{\eta,t-1}(\mathbf{x}_t), \quad \alpha_t = \mathcal{M}_{\alpha,t-1}(\mathbf{x}_t), \quad (83)$$

$$\hat{\mathbf{v}}_{\square,t} = \mathcal{M}_{\square,t-1}(\mathbf{v}_t), \quad (\text{Generating its own values for each memory}) \quad (84)$$

$$\min_{\mathcal{M}_\square} \mathcal{L}(\mathcal{M}_\square; \mathbf{k}_t, \hat{\mathbf{v}}_{\square,t}), \quad \text{with an optimization algorithm, } \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}, \quad (85)$$

where $\mathbf{q}_t = \mathbf{x}_t W_q$ is the only non-adaptive projection, η_t is the learning rate in optimization process, and α_t is the retention gate (forget gate or weight decay) in the optimization process. Note that, again, the initial states of all memories, i.e., $\mathcal{M}_{\square,0}$

for any $\square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}$ are meta-learned across all sequences/contexts, and so are optimized in the higher levels (or outer-loop).

Learning the mappings for associative memory modules (see Equation 85) requires a choice of optimization algorithm as well as an objective $\mathcal{L}$ that measures the quality of mappings. A simple and common choice for objective and optimization process are L_2 -regression loss, and gradient descent algorithm. As for the objective, we use L_2 -regression loss, i.e., $\mathcal{L}(\mathcal{M}; \mathbf{k}, \mathbf{v}) = \|\mathcal{M}(\mathbf{k}) - \mathbf{v}\|_2^2$. As discussed earlier (see Section 4.5), the choice of optimizer highly depends on the context of optimization. For example, gradient descent from associative memory perspective is based on dot-product similarity and so the update at each step, is solely based on the input and does not incorporate the previous data samples to the update. When performing optimization in the token space, however, we know tokens are highly correlated. Therefore, following our discussion in Section 4.5, we use our DGD with weight decay, resulting in general update rule of:

$$\mathbf{y}_t = \mathcal{M}_{\text{memory}, t-1}(\mathbf{q}_t), \quad \mathbf{k}_t = \mathcal{M}_{\mathbf{k}, t-1}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v}, t-1}(\mathbf{x}_t), \quad \eta_t = \mathcal{M}_{\eta, t-1}(\mathbf{x}_t), \quad \alpha_t = \mathcal{M}_{\alpha, t-1}(\mathbf{x}_t), \quad (86)$$

$$\hat{\mathbf{v}}_{\square, t} = \mathcal{M}_{\square, t-1}(\mathbf{v}_t), \quad (\text{Generating its own values for each memory}) \quad (87)$$

$$\mathcal{M}_{\square, t} = \mathcal{M}_{\square, t-1}(\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top) - \eta_t \nabla \mathcal{L}_{\mathcal{M}_{\square, t-1}}(\mathcal{M}_{\square, t-1}; \mathbf{k}_t, \hat{\mathbf{v}}_{\square, t}), \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}. \quad (88)$$

Here, the architecture of the memories are arbitrary and even we are not forced to use the same architecture for all components. We use a 2-layer MLP block as the architecture of all the memories:

$$\mathcal{M}_{\square}(\cdot) = (\cdot) + W_{\square, 1} \sigma(W_{\square, 2}(\cdot)). \quad (89)$$

8.2 Fast and Parallelizable Training

In the above, we discussed how to design a model that can learn to generate its own latent values and so modify itself. The main challenge from the practical point of view is the efficiency of the method and if its training is parallelizable. We follow the chunk-wise training algorithm of non-linear update rules (Sun et al. 2024; Behrouz et al. 2025c) and use update frequency of $\bar{f}_{\square} = \frac{L}{C_{\square}}$, where L is the context length. While there is no limitation to use different chunk-sizes, in our experiments, we use two different value of chunk sizes, one for the update of $\mathcal{M}_{\text{memory}}(\cdot)$ and the other for all the other memories in the self-referential Titans.

In more details, given an input sequence $\{\mathbf{x}_t\}_{t=1}^L$ and chunk size $1 \leq C \leq L$, we split the sequence into $\lceil \frac{L}{C} \rceil$ chunks of $\{\mathbf{x}_{((i-1)C+t)}\}_{t=1}^C$ for $i = 1, \dots, \lceil \frac{L}{C} \rceil$, and then generate all elements in Equation 86 at the end of each chunk for the next chunk. This allows for generating all the elements for the entire chunk in parallel, before starting the computation for this chunk. Furthermore, to update the memory modules based on Equation 88, we take the gradient with respect to the last state of the previous chunk. Again, this allows for computing all the gradients for the next chunk in parallel. In more details, given this chunk-wise updating procedure, the update rule for the self-referential Titans is computed as:

$$\mathbf{y}_t = \mathcal{M}_{\text{memory}, C \times \lceil \frac{L}{C} \rceil}(\mathbf{q}_t), \quad \mathbf{k}_t = \mathcal{M}_{\mathbf{k}, C \times \lceil \frac{L}{C} \rceil}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v}, C \times \lceil \frac{L}{C} \rceil}(\mathbf{x}_t), \quad \eta_t = \mathcal{M}_{\eta, C \times \lceil \frac{L}{C} \rceil}(\mathbf{x}_t), \quad \alpha_t = \mathcal{M}_{\alpha, C \times \lceil \frac{L}{C} \rceil}(\mathbf{x}_t),$$

$$\hat{\mathbf{v}}_{\square, t} = \mathcal{M}_{\square, C \times \lceil \frac{L}{C} \rceil}(\mathbf{v}_t), \quad (\text{Generating its own values for each memory})$$

$$\mathcal{M}_{\square, t} = \mathcal{M}_{\square, t-1}(\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top) - \eta_t \nabla \mathcal{L}_{\mathcal{M}_{\square, C \times \lceil \frac{L}{C} \rceil}}(\mathcal{M}_{\square, C \times \lceil \frac{L}{C} \rceil}; \mathbf{k}_t, \hat{\mathbf{v}}_{\square, t}), \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}. \quad (90)$$

Here, the architecture of the memories are arbitrary and even we are not forced to use the same architecture for all components. We use a 2-layer MLP block as the architecture of all the memories:

$$\mathcal{M}_{\square}(\cdot) = (\cdot) + W_{\square, 1} \sigma(W_{\square, 2}(\cdot)). \quad (91)$$

Since all the gradients as well as new keys, values, learning-rates, and weight decays can be computed in parallel before starting the processing of the current chunk, the above updates accepts the fast parallelizable dual form that is discussed by Sun et al. (2024) and Behrouz et al. (2025c). To better illustrate the above update rule for self-referential Titans, let us derive the recurrent formula for the simplest case of matrix-valued memory. We derive the recurrent form for two different objectives:

- Dot-product similarity $\mathcal{L}(\mathcal{M}; \mathbf{k}, \mathbf{v}) = -\langle \mathcal{M} \mathbf{k}, \mathbf{v} \rangle$: Given this objective and linear memory, the gradient is calculated as $\mathbf{v} \mathbf{k}^\top$, which results in update rule of:

$$\mathcal{M}_{\square, t} = \mathcal{M}_{\square, t-1}(\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top) - \eta_t \hat{\mathbf{v}}_{\square, t} \mathbf{k}_t^\top, \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\} \quad (92)$$

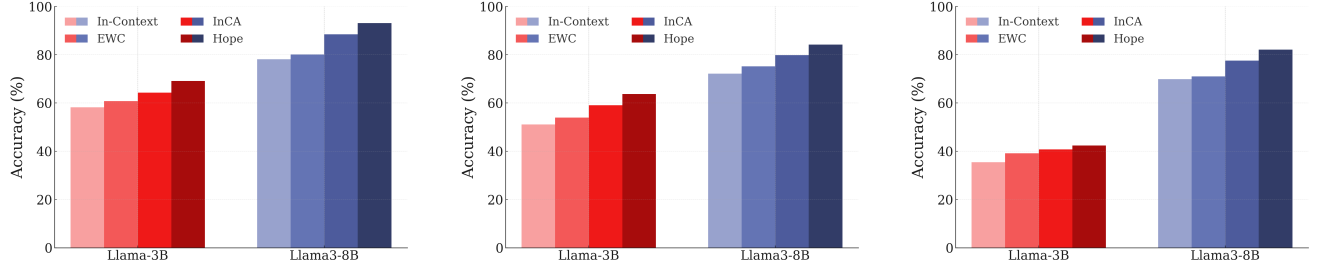


Figure 6: Class-incremental learning in the text classification domain on (Left) CLINC dataset (Larson et al. 2019), (Middle) Banking dataset (Casanueva et al. 2020), and (Right) DBpedia dataset (Auer et al. 2007). HOPE-enhanced architecture achieves the best accuracy among other continual learning methods, including ICL.

- L_2 -regression loss: Given this objective and linear memory, the gradient is calculated as $(\mathcal{M}\mathbf{k} - \mathbf{v})\mathbf{k}^\top$, which results in update rule of:

$$\mathcal{M}_{\square,t} = \mathcal{M}_{\square,t-1} (\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top) - \eta_t \left(\mathcal{M}_{\square, C \times \lceil \frac{1}{\epsilon} \rceil} \mathbf{k}_t - \hat{\mathbf{v}}_{\square,t} \right) \mathbf{k}_t^\top, \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}. \quad (93)$$

8.3 Hope Neural Learning Module

In the previous sections, we first discussed Continuum Memory System (CMS) that allows for more persistent storage of memories and defines memory as a spectrum of blocks with different frequencies of update. Due to the larger capacity and constraints for scaling the parameters, often CMS requires simple learning rule but higher capacity to store more persistent knowledge. On the other hand, in the previous section, we discussed the design of a self-modifying Titans, where it can generate its own keys and so learning update to better adapt to the context. Contrary to CMS, the self-modifying Titans has a small capacity but is using a complex and expressive learning rule. Accordingly, these two systems seem to be complementary and their combination can enhance the model expressiveness from different aspects.

To this end, we present HOPE architecture: A neural learning module that incorporates self-modifying Titans followed by Continuum Memory System. The HOPE design is illustrated in Figure 5. Formally, let $\mathbf{x}_t \in \mathbb{R}^d$ for $t = 1, \dots, L$ be the input, the HOPE forward pass is defined as (we remove the normalization and convolution layers for the sake of clarity):

$$\mathbf{o}_t = \mathcal{M}_{\text{memory},t-1}(\mathbf{q}_t), \quad \mathbf{k}_t = \mathcal{M}_{\mathbf{k},t-1}(\mathbf{x}_t), \quad \mathbf{v}_t = \mathcal{M}_{\mathbf{v},t-1}(\mathbf{x}_t), \quad \eta_t = \mathcal{M}_{\eta,t-1}(\mathbf{x}_t), \quad \alpha_t = \mathcal{M}_{\alpha,t-1}(\mathbf{x}_t), \quad (94)$$

$$\hat{\mathbf{v}}_{\square,t} = \mathcal{M}_{\square,t-1}(\mathbf{v}_t), \quad (95)$$

$$\mathcal{M}_{\square,t} = \mathcal{M}_{\square,t-1} (\alpha_t \mathbf{I} - \eta_t \mathbf{k}_t \mathbf{k}_t^\top) - \eta_t \nabla \mathcal{L}_{\mathcal{M}_{\square,t-1}}(\mathcal{M}_{\square,t-1}; \mathbf{k}_t, \hat{\mathbf{v}}_{\square,t}), \quad \square \in \{\mathbf{k}, \mathbf{v}, \mathbf{q}, \eta, \alpha, \text{memory}\}. \quad (96)$$

$$\mathbf{y}_t = \text{MLP}^{(f_k)}(\text{MLP}^{(f_{k-1})}(\dots \text{MLP}^{(f_1)}(\mathbf{o}_t))), \quad (97)$$

where the block’s output for token t is $\mathbf{y}_t$. In our experiments, we also normalize $\mathbf{q}$ and $\mathbf{k}$ with L_2 normalization and also use local convolutions with window size of 4.

Hope-Attention. We also use another variant of HOPE, in which we simply replace the self-modifying Titans with softmax global attention (Vaswani et al. 2017).

9 Experiments

In this section, we empirically evaluate the performance of different components we discussed throughout the paper. More specifically, (1) we first focus on the presented optimization algorithms and compare them with state-of-the-art methods; (2) We then focus on in-context and continual learning tasks and show how the nested learning paradigm and more specifically higher order in-context learning enhances the capabilities of models. We compare the continuum memory system with simple MLP layers and discuss how a pre-trained model can be adapted to be a continual learner; (3) We then focus on the language modeling and long context understanding of HOPE model and compare it with Transformers and modern recurrent architectures. The details of the experiments and their setups are explained in the corresponding sections.

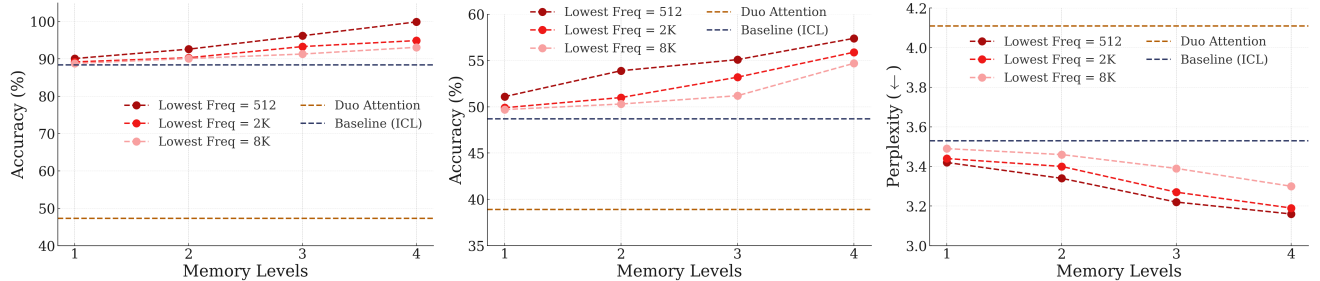


Figure 7: The effect of memory levels on the in-context learning performance of the model in (Left) MK-NIAH of RULER (Hsieh et al. 2024), (Middle) LongHealth (Adams et al. 2025), (Right) QASPER (Dasigi et al. 2021) benchmarks. Note that for QASPER benchmark (Right), the lower values indicate better performance.

9.1 HOPE: Continual Learning and Long Context Understanding

One of the main goals of NL is to enhance the continual learning capabilities, and so in this section, we evaluate NL and its implications such as Continuum Memory System (CMS) and HOPE on multiple continual learning and long context understanding tasks. For each of the tasks, we use the best reported results on the benchmark as the baselines.

Class Incremental Learning. First, we focus on class-incremental learning tasks on three datasets:

- **CLINC** (Larson et al. 2019): CLINC is a multi-domain intent classification benchmark designed for task-oriented dialog systems, with a special focus on detecting out-of-scope (OOS) queries. It has 150 in-scope intent classes spanning 10 broad domains (e.g. Banking, Travel, Home, Weather, Small Talk, etc.) with 23.7K total queries, of which 22.5K are in-scope and 1.2K are out-of-scope.
- **Banking** (Casanueva et al. 2020): Banking dataset is a single-domain intent classification benchmark focused on fine-grained customer service queries in the banking domain. In this dataset, each example is a short customer query (e.g. “How can I reset my card PIN?”) that must be classified into the correct banking intent/category. There are 3083 total examples labeled with the 77 intents (heavily imbalanced classes).
- **DBpedia** (Auer et al. 2007): DBpedia is a text classification benchmark from Wikipedia where article abstracts are expected to be categorized into ontology topic classes. In other words, given a short Wikipedia description, the goal is to predict its high-level topic/category (such as whether the article is about a book, a film, an animal, a place, etc.). The dataset has over 340K examples labeled across those 70 second-level classes, but we sample 10K training and 1K test instances for the 70-class DBpedia task.

As the backbone of our HOPE models we use Llama3-8B and Llama-3B (Dubey et al. 2024), and then employ our technique discussed in Section 7.3 to make the MLP blocks capable of adaption, placing them in different levels with different frequency updates followed by continual pre-training with 15B tokens. Following Momeni et al. (2025), we use simple in-context learning (ICL) capability of Llama-3 models (with the same process of continual pre-training with 15B tokens but without any change in the MLP blocks), Elastic Weight Consolidation (EWC) (Kirkpatrick et al. 2017), and In-context Continual Learning with an External Learner (InCA) (Momeni et al. 2025) as the baselines of our evaluation. The results are reported in Figure 6. HOPE shows the best performance across all continual learning baselines, including models with external learner (i.e., InCA). Comparing HOPE with ICL, the main difference comes from HOPE’s multiple levels of in-context learning (or equivalently, different frequency of updates for MLP blocks), indicating the effectiveness of CMS’s design for enhancing continual learning capabilities. Furthermore, the superior performance of HOPE, compared to InCA and EWC, indicates that the knowledge transfer between levels plays a critical role in the performance of the model.

The Effect of Levels on In-context Learning. Despite showing improvement when using CMS in the above tasks, to better understand and evaluate the effect of levels and their frequency on the in-context level ability of the model, we perform in-context question/answering and multi-key long context understanding. More specifically, we use:

- **LongHealth** (Adams et al. 2025): This is a benchmark for long-context clinical question answering with multiple-choice QA tasks based on extensive fictional patient records, testing an LLM’s ability to extract and reason over detailed medical documents. The dataset includes 20 comprehensive patient case documents (across various diseases),

Table 1: Needle-In-A-Haystack experiments with: (1) Single needle with three levels of difficulty: single-needle tasks—S-NIAH-1 (passkey retrieval), S-NIAH-2 (numerical needle), and S-NIAH-3 (UUID-based needle); (2) multi-query; (3) multi-key; and (4) multi-value settings of the benchmark.

Model	S-NIAH-1 (pass-key retrieval)			S-NIAH-2 (number in haystack)			S-NIAH-3 (uuid in haystack)		
	4K	8K	16K	4K	8K	16K	4K	8K	16K
Transformer	88.6	76.4	79.8	100	98.8	94.2	78.0	69.2	40.8
HOPE-Attention	100	100	100	100	98.4	94.4	76.8	68.8	42.4
RWKV-7	100	100	99.6	93.8	44.8	12.6	63.8	13.2	5.8
Comba	100	100	99.4	92.6	47.2	13.4	62.4	13.8	7.4
DLA	96.4	71.2	44.0	79.6	42.6	28.2	18.2	8.8	4.0
Titans	100	100	100	99.6	84.6	75.4	74.2	42.8	21.2
HOPE	100	100	100	99.2	88.4	78.2	73.2	46.2	24.8

Model	MK-NIAH-1 (multi-key line retrieval)			MQ-NIAH (multi-query)			MV-NIAH (multi-value)		
	4K	8K	16K	4K	8K	16K	4K	8K	16K
Transformer	79.4	83.0	61.4	58.9	48.0	29.8	37.5	34.1	21.5
HOPE-Attention	80.2	84.8	60.8	60.4	47.8	30.6	35.2	34.4	24.8
RWKV-7	21.4	18.8	9.6	20.4	14.8	8.6	16.2	13.4	6.8
Comba	21.4	19.4	8.2	21.8	15.2	6.4	16.5	13.5	7.2
DLA	27.4	20.0	11.8	26.4	22.0	6.4	25.6	12.8	9.6
Titans	26.4	23.6	8.2	22.8	19.8	9.4	24.6	15.1	8.2
HOPE	29.4	24.8	14.8	31.7	24.8	14.2	31.4	17.2	11.4

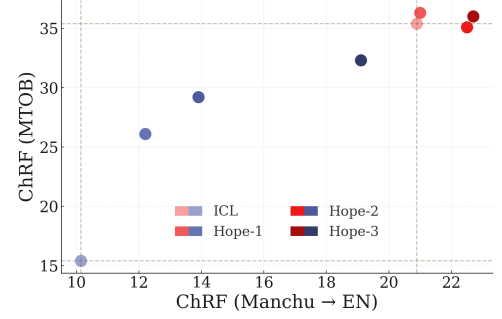


Figure 8: Continual Translation of a Novel Language (CTNL) task. Red points are the results with only one language. Blue points are the results in the continual learning setup.

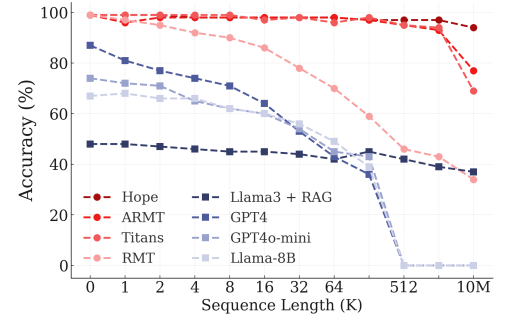


Figure 9: BABILong Benchmark. Red points are the results of fine-tuned models, and Blue points are the large models' zero-shot results.

each about 5.1K–6.8K words in length, and we use 200 questions sampled from patient records.

- QASPER (Dasigi et al. 2021): This benchmark is an information-seeking QA dataset centered on full-length NLP research papers. In particular, it contains around 5K QA pairs grounded in around 1.6L NLP research papers. Also, we use the full text of each paper as the context for the model.
- MK-NIAH (Hsieh et al. 2024): We use the task of multiple keys in needle-in-haystack from RULER (Hsieh et al. 2024). This setup requires models to not only locate but also extract multiple pieces of information distributed throughout a long text.

As for the baseline, we use ICL, which is the same as HOPE with 1-level of memory, and also DuoAttention (Xiao et al. 2025). It is notable that methods such as Cartridges (Eyuboglu et al. 2025) have shown promising performance, even better than DuoAttention and sometimes ICL. Here, however, we exclude their comparison with HOPE mainly due to the fact that HOPE has higher memory usage and there are fundamental differences in their computational costs (e.g., self-studying, etc.), requiring further experiments with careful and controlled design in the future studies. For the variants of our model, we use different number of memory levels with different frequencies, which are grouped based on the lowest frequency. Note that the lowest-frequency memory corresponds to the most persistent memory of the model and so we expect models with higher lowest frequency to be more adaptive.

The results are reported in Figure 7. HOPE with any number of levels and also with any lowest frequency outperforms both ICL baseline as well as the efficient DuoAttention. Furthermore, comparing the HOPE’s variants with themselves, the results suggest that: (1) The more levels of memory can help with in-context learning capability of the model and enhances its long-term memory and so long-context understanding; and (2) The higher the lowest-frequency the lower the performance. As discussed above, the lowest-frequency memory is the most persistent memory of the model and so

increasing it means that the model has a weaker but more adaptive long-term memory. Due to the fact that the frequency of update directly affects the efficiency of the model, one might find “Lowest Frequency = 2K” an optimal setup as it offers significantly more efficient forward pass, while achieves close performance with “Lowest Frequency = 512”.

Learning a New Language In-Context. Throughout the paper, we argued that pre-training can be seen as an in-context learning process, where the context is the entire pre-training data. Accordingly, when we use more levels in the neural learning module, or performing in-context learning on different context flows, we expect the model to show better adaptability and continual learning capabilities. To this end, with combining two existing benchmarks of MTOB (Tanzer et al. 2024) and Manchu (Pei et al. 2025), we design a new continual learning task that the LLM learns two new languages in context and is expected to translate phrases to English. Then for our Continual Translation of a Novel Language (CTNL) task, we consider two setups: (1) Learning and testing the performance of the model on each language separately (in red color). This is a baseline we use to measure the catastrophic forgetting when comparing with the next setting; and (2) The model, first, learns these two languages in a sequential manner (in blue color) and then is asked to translate the phrases to English. We use ICL as the baseline, and to study the importance of multi-level design of HOPE, we use different variants of HOPE with 1, 2, and 3 additional levels of memory, which we refer to as HOPE-1, HOPE-2, and HOPE-3, respectively.

The results are reported in Table 8. More specifically, each point is the performance of one model in one of the setups such that the x -axis (resp. y -axis) indicates the model ChRF in translation of Manchu $\rightarrow$ English (resp. Kalamang $\rightarrow$ English). In the first setup (in-context translation without continual learning), all HOPE’s variants performs better or on-par compared to ICL, supporting the importance of CMS design. In the second setup (i.e., continual translation), however, ICL faces dramatic performance drop and almost rely on its capabilities achieved in its pre-training (i.e., catastrophic forgetting about the knowledge in the context). On the other hand, increasing the memory levels in the HOPE shows clear improvement and HOPE-3 almost recovers the ICL capability in the first setup, without continual learning. These results further support the importance of CMS design in continual learning and the ability of the model to adapt itself to the new tasks.

9.2 HOPE: Long Context Understanding

In the previous section, we evaluated the performance of HOPE-Attention, when the MLP blocks are adapted to perform in-context learning. In this part, we evaluate the performance of HOPE in long context understanding, when it starts learning from scratch. To this end, we use about 50B tokens from a mixture of FineWeb-Edu (Penedo et al. 2024) and long-context documents with a vocabulary size of 32K to train all the models from scratch. All models are optimized using AdamW with tuned learning rate for each model and with the default optimizer configuration in Behrouz et al. (2025c). We focus on two popular benchmarks of RULER (Hsieh et al. 2024) and BABILong (Kuratov et al. 2024):

Needle-in-a-Haystack (NIAH) Tasks. In the first part, we focus on the needle-in-a-haystack with different setups of: (1) single needle but different types (i.e., pass-key, number, and uuid), (2) multi-key, (3) multi-query, and (4) multi-value, all follows Hsieh et al. (2024). As for the baselines, we use RetNet (Sun et al. 2023) and DeltaNet (Schlag et al. 2021) as the representative of the models *purely* based on Hebbian- and Delta-rule, and experimented with diverse set of modern *linear* recurrent models and so used the linear models with the best performance: i.e., RWKV-7 (Peng et al. 2025a) and Comba (Hu et al. 2025). As another group of baselines, we also compare with *deep* memory modules with dot-product and L_2 regression objectives: i.e., DLA (Behrouz et al. 2025a), and Titans (Behrouz et al. 2025c).

The results are reported in Table 1. Comparing with other attention-free models, HOPE achieves the best performance across all tasks and levels of difficulties. Particularly, comparing to linear memories, deep memory modules show better performance in longer sequences, mainly due to their higher memory capacity to compress more tokens. Comparing HOPE with Titans, the superior performance of HOPE, specifically in longer context length supports the importance of both self-referential update as well as the CMS design. Finally, we also evaluate the performance of HOPE-Attention and compare it with Transformers to better understand the contribution of CMS design. The results indicate that HOPE-Attention achieves a better performance compared to Transformers, supporting the advantage of having CMS in HOPE-Attention design.

BABILong. Next, we evaluate the HOPE’s performance on BABILong benchmark (Kuratov et al. 2024) and compare it with (1) large models such as GPT4 and GPT4o-mini (Achiam et al. 2023); (2) middle-size Llama-8B model (Dubey et al. 2024) with its RAG augmented version; and (3) the state-of-the-art small models in this tasks: i.e., RMT (Bulatov et al. 2022), ARMT (Rodkin et al. 2024), and Titans (Behrouz et al. 2025c). We follow the original setup of the benchmark and fine-tune

Table 2: Performance of models on language modeling and common-sense reasoning tasks.

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	SIQA acc ↑	BoolQ acc ↑	Avg. ↑
760M params / 30B tokens											
Transformer++	24.18	24.27	37.1	67.2	43.8	53.0	65.6	33.4	39.1	61.7	50.11
Samba*	21.07	22.85	39.2	68.9	47.8	53.1	65.8	34.9	38.9	63.1	51.46
RetNet	25.77	24.19	34.5	66.8	41.2	51.9	63.6	32.5	38.8	56.2	48.19
DeltaNet	24.52	24.38	36.8	67.3	44.5	51.8	64.2	32.7	39.6	60.1	49.63
RWKV-7	23.75	23.08	37.1	67.3	47.6	52.2	64.7	34.2	39.4	61.9	50.55
Comba	22.41	22.19	37.5	66.9	48.2	52.4	65.1	34.1	40.1	62.8	50.89
TTT	24.17	23.51	34.7	67.3	43.9	51.0	64.5	33.8	40.2	59.6	47.32
Miras (Memora)	22.28	22.31	38.2	67.8	49.3	53.3	63.6	36.1	40.9	63.0	51.53
DLA	23.12	22.09	36.1	68.0	47.9	52.7	65.8	34.6	39.1	59.6	50.48
Titans	20.08	21.52	38.1	69.1	48.5	52.7	66.2	35.7	40.3	62.8	51.68
HOPE	18.68	20.07	38.8	69.2	49.1	53.6	66.8	36.1	41.2	63.4	52.28
1.3B params / 100B tokens											
Transformer++	17.92	17.73	42.6	71.4	52.3	54.1	69.9	36.5	41.8	58.4	53.38
Samba*	16.15	13.21	45.2	71.5	53.8	55.8	69.1	36.7	40.6	63.0	54.46
RetNet	18.91	17.04	41.2	71.3	49.1	55.2	67.5	34.1	41.4	61.0	52.60
DeltaNet	18.62	17.10	41.6	70.1	49.4	52.7	67.6	35.2	39.7	54.8	51.39
RWKV-7	18.44	15.96	46.7	72.4	54.9	57.5	71.6	38.2	40.7	60.4	55.30
Comba	18.16	14.87	46.9	73.1	54.5	57.7	72.0	39.1	40.2	60.6	55.39
TTT	18.42	14.51	46.8	72.9	55.2	59.0	71.8	39.5	39.8	59.6	55.58
Miras (Memora)	15.90	12.04	48.7	73.1	56.0	57.4	71.5	37.9	40.2	61.3	55.76
DLA	16.31	12.29	44.5	70.6	53.9	54.2	69.6	36.0	40.8	60.2	53.72
Titans	15.60	11.41	49.1	73.1	56.3	59.8	72.4	40.8	42.1	61.0	56.82
HOPE	14.39	10.08	51.0	73.9	57.5	61.2	73.8	42.7	42.8	61.4	58.04

* is a hybrid of attention + linear RNN (Ren et al. 2024).

the small models with the same process as Kuratov et al. (2024).

The results are reported in Table 9. Large models show significant performance drop with increasing the sequence length, where all fail around 128K-256K context length. The RAG augmented model also show a drop with increasing the context, but it is able to relatively maintain its performance after 256K context length. Among fine-tuned models, Titans, ARMT, and HOPE show competitive results until 1M context length, but the performance of both Titans and ARMT drop fast after that point. HOPE maintains its good performance even for 10M context length, mainly due to its CMS design. It is notable that the performance of all small models, including HOPE, can drop significantly when used without fine-tuning. The reason is, compressing large context (e.g., 10M), in addition to a powerful memory management in the high-frequency level, requires enough capacity to compress 10M tokens or at least tokens that are needed for the final answer. The fine-tuning step helps the models to adjust their lower-frequency levels to adapt fast and so properly manage their memory in the higher-frequency levels.

9.3 HOPE: Language Modeling and Common-Sense Reasoning

In this section, we aim to study HOPE as a backbone of a language model and evaluate it on common language modeling and common-sense reasoning tasks with the setup of:

- **Datasets:** We evaluate HOPE and baselines on Wikitext (Merity et al. 2017), LMB (Paperno et al. 2016), PIQA (Bisk et al. 2020), HellaSwag (Zellers et al. 2019), WinoGrande (Sakaguchi et al. 2021), ARC-easy (ARC-e) and ARC-challenge (ARC-c) (Clark et al. 2018), SIQA (Sap et al. 2019), and BoolQ (Clark et al. 2019) benchmarks.
- **Baselines:** As for the baselines, similar to Section 9.2, we use RetNet (Sun et al. 2023) and DeltaNet (Schlag et al. 2021) as the representatives of the models that are *purely* based on Hebbian- or Delta-rule, and two modern *matrix-valued* recurrent models with the best performance compared to others: i.e., RWKV-7 (Peng et al. 2025a) and Comba (Hu et al. 2025). As another group of baselines, we compare with attention-free *deep* memory modules with diverse

Table 5: Accuracies of various models on the formal language recognition tasks.

Model	Parallel Training	Non-Star-Free Regular						Counter					
		Parity		$(aa)^*$		$(abab)^*$		$a^n b^n$		$a^n b^n c^n$		Shuffle-2	
		Bin0	Bin1	Bin0	Bin1	Bin0	Bin1	Bin0	Bin1	Bin0	Bin1	Bin0	Bin1
LSTM	✗	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Transformer	✓	46.4	0.0	0.0	0.0	0.0	0.0	100.0	100.0	100.0	100.0	100.0	100.0
Linear	✓	78.1	0.0	0.0	0.0	0.0	0.0	100.0	100.0	100.0	100.0	100.0	100.0
DeltaNet	✓	98.2	10.1	0.0	0.0	0.0	0.0	100.0	100.0	100.0	100.0	100.0	100.0
SRWM	✗	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
HOPE	✓	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

internal attentional bias of dot-product, L_2 , and L_p regression: i.e., TTT (Sun et al. 2024), Miras (Behrouz et al. 2025b), DLA (Behrouz et al. 2025a) and Titans (Behrouz et al. 2025c). Finally, we also compare with Transformers (Vaswani et al. 2017; Dubey et al. 2024) as well as the hybrid of attention and linear RNN, Samba (Ren et al. 2024).

- Training: We train models with about 760M and 1.3B parameters, trained with 30B and 100B tokens, respectively, from a mixture of FineWeb-Edu (Penedo et al. 2024) and long-context documents with a vocabulary size of 32K to train all the models from scratch. All models are optimized using AdamW with tuned learning rate for each model and with the default optimizer configuration in Behrouz et al. (2025c).

The results are reported in Table 2. HOPE outperforms all the baselines on the average performance in both language modeling and common-sense reasoning benchmarks. Interestingly, with scaling the parameters, HOPE show higher performance gain compare to other attention-free models.

9.4 HOPE: In-context Recall Tasks and MAD Synthetic Benchmark

In-context recall is often referred to as one of the challenging benchmarks for attention-free models. In this section, we follow Arora et al. (2024) and perform experiments on SWDE (Lockard et al. 2019), NQ (Kwiatkowski et al. 2019), DROP (Dua et al. 2019), FDA (Arora et al. 2023), SQUAD (Rajpurkar et al. 2016), and TQA (Kembhavi et al. 2017) to evaluate the effectiveness of HOPE’s design. We use the same set of baselines and experimental setup as the above previous section. The results are reported in Table 3. Transformers achieve the best performance, while HOPE show competitive results, outperforming all the attention-free baselines and closing the gap with Transformers.

Table 3: The performance of HOPE and baselines in short in-context recall tasks. HOPE outperforms all attention-free models and close the gap with Transformers.

	SWDE	NQ	DROP	FDA	SQUAD	TQA
Transformers	71.4	22.0	23.9	67.3	39.4	59.1
RWKV-7	52.3	17.8	21.7	32.8	28.5	56.2
Comba	53.9	19.1	21.9	35.5	30.2	56.4
Titans	60.8	20.3	22.0	37.6	31.8	57.5
HOPE	65.9	21.2	22.8	41.9	33.0	57.7

Table 4: Performance of HOPE and baselines on the synthetic benchmark of MAD (Poli et al. 2024). HOPE outperforms all the baselines, including Transformers.

	Compress.	ICR	Fuzzy ICR	Selective Copying	Memory
Transformers	49.4	100	47.9	96.2	83.7
RWKV-7	45.1	100	32.8	95.6	82.2
Comba	46.3	100	32.8	96.4	82.9
Titans	49.8	100	50.0	99.4	83.4
HOPE	51.2	100	52.1	99.7	85.2

We also study the performance of HOPE on MAD benchmark (Poli et al. 2024), which is a synthetic benchmark, evaluating the performance of models in recall, memorization, compression, and copying tasks. The results are reported in Table 4. HOPE achieves the best results compared to baselines.

9.5 Language Recognition Tasks

One of the critical limitations of Transformers is on non-parallelizable tasks, where the recurrence plays a significant rule in achieving proper performance. An example of such tasks is state tracking problem, where the model needs to track its

Table 6: Ablation Study on HOPE. All components of HOPE are positively contributing to its performance.

Model	Language Modeling ppl ↓	Reasoning acc ↑
HOPE	12.24	58.1
w/o DGD	13.41	56.5
w/o Momentum	13.58	56.9
w/o weight decay	13.71	57.2
w/o CMS	13.04	57.3
w/o inner-projection k	13.77	56.9
w/o inner-projection v	13.90	55.1
w/o inner-projection q	12.19	57.4

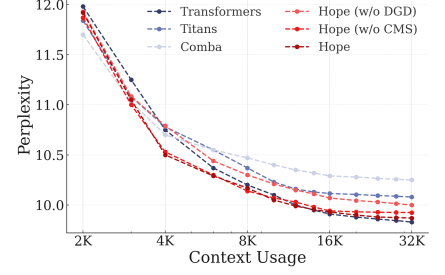


Figure 10: The effect of context usage on the perplexity of the model. We expect the perplexity of a model with powerful memory management to decrease with more context.

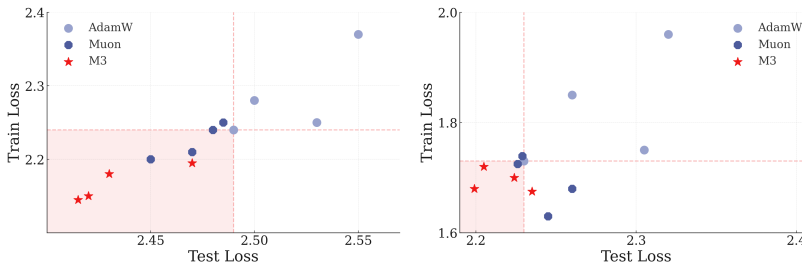


Figure 11: ViT test and train loss on ImageNet-21K (Ridnik et al. 2021), trained with AdamW, Muon, and our M3 optimizers.

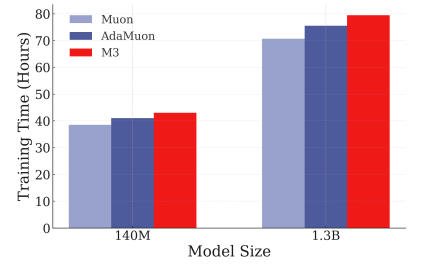


Figure 12: Training time of models with 140M and 1.3B parameters with Muon, AdaMuon, and M3 optimizers.

state given a sequence of instructions (movement, etc.), and several studies have shown that Transformers both in theory and practice significantly underperforms models with non-linear recurrence (Merrill et al. 2024; Grazzi et al. 2025). In this section, we focus on formal language recognition tasks and follow the construction of the benchmark by Irie et al. (2023). The results are reported in Table 5. HOPE achieves the perfect score on all the tasks, similar to other non-linear recurrent models; e.g., LSTM Schmidhuber et al. 1997 and SRWM (Irie et al. 2022b). The main advantage of HOPE, however, is the fact that when it is needed, it has parallelizable training and so can scale to larger scales for language modeling tasks.

9.6 HOPE: Ablation Studies and Scaling

In this section, we first study the importance of design choices we have made for the HOPE architecture by performing ablation studies, where we remove or change one of the HOPE’s components at a time. Note that, in the previous experiments, we have evaluated the significance of some components: E.g., Table 8 and Figure 7 have already been shown the effect of number of levels as well as the frequency of update on the performance of HOPE in continual learning. In this section, to compare different variants, we use the average perplexity of models in language modeling as well as the average accuracy in common-sense reasoning tasks. The results are reported in Table 6. (1) The first row, replaces Delta Gradient Descent with a simple gradient descent in the design of self-modifying Titans; (2) The second row removes the momentum term in the self-modifying Titans; (3) removes the weight decay; (4) removes the CMS from HOPE’s architecture; (5, 6, 7) transfer the projections for k, v, q from higher-frequency level to the lowest-frequency level. All the components of HOPE contribute to its superior performance in these tasks and removing or changing each of them can damage the model’s perplexity in language modeling and/or accuracy in common-sense reasoning tasks.

9.7 Expressive Optimizers

In this section, we evaluate the performance of our M3 optimizer from both aspects of effectiveness in finding the solution and also efficiency of training in large scales:

ImageNet. In this experiment, we focus on ViT (Dosovitskiy et al. 2021) architecture for vision tasks and pre-train it on ImageNet-21K, which consists of 11M images corresponding to 10,450 classes. We use a patch size of 16, MLP dimension of 1536 and 3072 for the two scales of 24M and 86M models, respectively. We control all other components and finetuned hyperparameters for each optimizer separately to make sure a fair comparison. We vary the optimizer for training the ViT to understand which of the optimizers find more effective solutions in the same number of training steps. The training and test loss of trained models with different optimizers are reported in Figure 11. Our M3 shows the best training/test loss compared to both AdamW and Muon.

Efficiency in Large Models. Due to the small size of the model that is needed for ImageNet, we also perform an efficiency comparison when training a language model. To this end, we use two scales of 140M and 1.3B and train the same Transformer model using different optimizers of Muon (Jordan et al. 2024), AdaMuon (Si et al. 2025), and our M3. The results are reported in Figure 12. Our M3 optimizer, due to the use of multiple momentums (memories) is relatively slower compare to the Muon optimizer, and show on par efficiency with AdaMuon.

10 Conclusion

In this paper, we introduced *Nested Learning (NL)* a new learning paradigm, in which modern machine learning systems are modeled as inter-connected, multi-level optimization problems, each with its own context flow and update frequency. Within this view, both architectures and optimizers are instances of nested systems of associative memories that compress their own context (either is tokens, gradients, or higher-level signals) into internal parameters. This perspective reframes pre-training, in-context learning, and continual learning as manifestations of the same underlying mechanism: learning to compress and reuse context at different levels and time scales. This perspective recasts backpropagation, momentum, and preconditioning as associative memory mechanisms, and explains a wide family of existing methods as specific design points in a larger (previously hidden) space.

Building on NL’s viewpoint, we derived generalized gradient-based updates: e.g., Delta Gradient Descent, Delta Momentum, Multi-scale Momentum Muon (M3), and reinterpreted modern sequence architectures nested associative memories. To enhance the memory processing, we introduced Continuum Memory System (CMS), a new formulation for memory that generalizes the traditional viewpoint of “long-term/short-term memory blocks”. Our HOPE architecture based on a self-modifying Titans and CMS improves continual learning and long-context reasoning capabilities, while remaining competitive as a general backbone.

Is Catastrophic Forgetting Solved? While HOPE and CMS have shown promising results in reducing catastrophic forgetting in the tasks we empirically studied, the undesirable phenomenon of catastrophic forgetting is not “solved” in general. From nested learning viewpoint on learning and backpropagation process, catastrophic forgetting is a natural consequence of compression, where the limited capacity of the network forces the model to forget so that it retains capacity for new information. We view NL as a roadmap rather than a destination: it suggests that progress on continual learning, long-context reasoning, modern optimizers, and self-modifying models will come from better exploiting the extra design axis of *levels* rather than from ever-deeper static networks.

References

- [1] Walter Pitts. “The linear theory of neuron networks: The dynamic problem”. In: *The bulletin of mathematical biophysics* 5 (1943), pp. 23–31.
- [2] Warren S McCulloch and Walter Pitts. “The statistical organization of nervous activity”. In: *Biometrics* 4.2 (1948), pp. 91–99.
- [3] Warren S McCulloch. “The brain computing machine”. In: *Electrical Engineering* 68.6 (1949), pp. 492–497.
- [4] William Beecher Scoville and Brenda Milner. “Loss of recent memory after bilateral hippocampal lesions”. In: *Journal of neurology, neurosurgery, and psychiatry* 20.1 (1957), p. 11.
- [5] Arthur L Samuel. “Some studies in machine learning using the game of checkers”. In: *IBM Journal of research and development* 3.3 (1959), pp. 210–229.
- [6] Seppo Linnainmaa. “The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors”. PhD thesis. Master’s Thesis (in Finnish), Univ. Helsinki, 1970.
- [7] Kuniyiko Fukushima. “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position”. In: *Biological cybernetics* 36.4 (1980), pp. 193–202.
- [8] Erkki Oja. “Simplified neuron model as a principal component analyzer”. In: *Journal of mathematical biology* 15.3 (1982), pp. 267–273.
- [9] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536.
- [10] Geoffrey E Hinton and David C Plaut. “Using fast weights to deblur old memories”. In: *Proceedings of the ninth annual conference of the Cognitive Science Society*. 1987, pp. 177–186.
- [11] DL Prados and SC Kak. “Neural network capacity using delta rule”. In: *Electronics Letters* 25.3 (1989), pp. 197–199.
- [12] Juergen Schmidhuber. “Learning to control fast-weight memories: An alternative to recurrent nets. Accepted for publication in”. In: *Neural Computation* (1992).
- [13] Tim VP Bliss and Graham L Collingridge. “A synaptic model of memory: long-term potentiation in the hippocampus”. In: *Nature* 361.6407 (1993), pp. 31–39.
- [14] Jürgen Schmidhuber. “A ‘self-referential’ weight matrix”. In: *International conference on artificial neural networks*. Springer. 1993, pp. 446–450.
- [15] Juergen Schmidhuber, Jieyu Zhao, and Marco Wiering. *Simple principles of metalearning*. 1996.
- [16] Uwe Frey and Richard GM Morris. “Synaptic tagging and long-term potentiation”. In: *Nature* 385.6616 (1997), pp. 533–536.
- [17] Juergen Schmidhuber and Sepp Hochreiter. “Long Short-term Memory”. In: *Neural Computation MIT-Press* (1997).
- [18] Amos Storkey. “Increasing the capacity of a hopfield network without sacrificing functionality”. In: *International Conference on Artificial Neural Networks*. Springer. 1997, pp. 451–456.
- [19] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*. Vol. 1. 1. 1998.
- [20] Jonathan H. Connell and Sridhar Mahadevan. “Robot Learning”. In: *Robotica* 17.2 (1999), pp. 229–235. DOI: 10.1017/S0263574799271172.
- [21] Sean PA Drummond, Gregory G Brown, J Christian Gillin, John L Stricker, Eric C Wong, and Richard B Buxton. “Altered brain response to verbal learning following sleep deprivation”. In: *Nature* 403.6770 (2000), pp. 655–657.
- [22] Hideyuki Okano, Tomoo Hirano, and Evan Balaban. “Learning and memory”. In: *Proceedings of the National Academy of Sciences* 97.23 (2000), pp. 12403–12404.
- [23] Jürgen Schmidhuber. “Gödel machines: self-referential universal problem solvers making provably optimal self-improvements”. In: *arXiv preprint cs/0309048* (2003).
- [24] Gyorgy Buzsaki and Andreas Draguhn. “Neuronal oscillations in cortical networks”. In: *science* 304.5679 (2004), pp. 1926–1929.
- [25] Donald Olding Hebb. *The organization of behavior: A neuropsychological theory*. Psychology press, 2005.
- [26] Alvaro Pascual-Leone, Amir Amedi, Felipe Fregni, and Lotfi B Merabet. “The plastic human brain cortex”. In: *Annu. Rev. Neurosci.* 28.1 (2005), pp. 377–401.
- [27] David J Foster and Matthew A Wilson. “Reverse replay of behavioural sequences in hippocampal place cells during the awake state”. In: *Nature* 440.7084 (2006), pp. 680–683.
- [28] Lisa Marshall, Halla Helgadóttir, Matthias Mölle, and Jan Born. “Boosting slow oscillations during sleep potentiates memory”. In: *Nature* 444.7119 (2006), pp. 610–613. DOI: 10.1038/nature05278.
- [29] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. “Dbpedia: A nucleus for a web of open data”. In: *international semantic web conference*. Springer. 2007, pp. 722–735.

- [30] Timothy J. Buschman and Earl K. Miller. “Top-down versus bottom-up control of attention in the prefrontal and posterior parietal cortices”. In: *Science* 315.5820 (2007), pp. 1860–1862. DOI: 10.1126/science.1138071.
- [31] Daoyun Ji and Matthew A Wilson. “Coordinated memory replay in the visual cortex and hippocampus during sleep”. In: *Nature neuroscience* 10.1 (2007), pp. 100–107.
- [32] Seung-Schik Yoo, Peter T Hu, Ninad Gujar, Ferenc A Jolesz, and Matthew P Walker. “A deficit in the ability to form new human memories without sleep”. In: *Nature neuroscience* 10.3 (2007), pp. 385–392.
- [33] Nicholas J Higham. *Functions of matrices: theory and computation*. SIAM, 2008.
- [34] Michael V Johnston. “Plasticity in the developing brain: implications for rehabilitation”. In: *Developmental disabilities research reviews* 15.2 (2009), pp. 94–101.
- [35] Adrien Peyrache, Mehdi Khamassi, Karim Benchenane, Sidney I Wiener, and Francesco P Battaglia. “Replay of rule-learning related neural patterns in the prefrontal cortex during sleep”. In: *Nature neuroscience* 12.7 (2009), pp. 919–926.
- [36] Susanne Diekelmann and Jan Born. “The memory function of sleep”. In: *Nature Reviews Neuroscience* 11.2 (2010), pp. 114–126. DOI: 10.1038/nrn2762.
- [37] John Duchi, Elad Hazan, and Yoram Singer. “Adaptive subgradient methods for online learning and stochastic optimization.” In: *Journal of machine learning research* 12.7 (2011).
- [38] Juergen Fell and Nikolai Axmacher. “The role of phase synchronization in memory processes”. In: *Nature reviews neuroscience* 12.2 (2011), pp. 105–118.
- [39] Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. “Neural networks for machine learning lecture 6a overview of mini-batch gradient descent”. In: *Cited on* 14.8 (2012), p. 2.
- [40] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “Imagenet classification with deep convolutional neural networks”. In: *Advances in neural information processing systems* 25 (2012).
- [41] Hong-Viet V. Ngo, Thomas Martinetz, Jan Born, and Matthias Mölle. “Auditory closed-loop stimulation of the sleep slow oscillation enhances memory”. In: *Neuron* 78.3 (2013), pp. 545–553. DOI: 10.1016/j.neuron.2013.03.006.
- [42] Dzmitry Bahdanau. “Neural machine translation by jointly learning to align and translate”. In: *arXiv preprint arXiv:1409.0473* (2014).
- [43] James F Cavanagh and Michael J Frank. “Frontal theta as a mechanism for cognitive control”. In: *Trends in cognitive sciences* 18.8 (2014), pp. 414–421.
- [44] Diederik P Kingma and Jimmy Ba. “Adam: A method for stochastic optimization”. In: *arXiv preprint arXiv:1412.6980* (2014).
- [45] Diederik P. Kingma and Max Welling. “Auto-Encoding Variational Bayes.” In: *ICLR*. Ed. by Yoshua Bengio and Yann LeCun. 2014. URL: <http://dblp.uni-trier.de/db/conf/iclr/iclr2014.html#KingmaW13>.
- [46] Guido Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. “On the number of linear regions of deep neural networks”. In: *Advances in neural information processing systems* 27 (2014).
- [47] Pascal Fries. “Rhythms for cognition: communication through coherence”. In: *Neuron* 88.1 (2015), pp. 220–235.
- [48] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. In: *nature* 521.7553 (2015), pp. 436–444.
- [49] Bernhard P. Staresina, Til Ole Bergmann, Mathilde Bonnefond, Roemer van der Meij, Ole Jensen, Lorena Deuker, Christian E. Elger, Nikolai Axmacher, and Jürgen Fell. “Hierarchical nesting of slow oscillations, spindles and ripples in the human hippocampus during sleep”. In: *Nature Neuroscience* 18.11 (2015), pp. 1679–1686. DOI: 10.1038/nn.4119.
- [50] Jimmy Ba, Geoffrey E Hinton, Volodymyr Mnih, Joel Z Leibo, and Catalin Ionescu. “Using fast weights to attend to the recent past”. In: *Advances in neural information processing systems* 29 (2016).
- [51] Timothy Dozat. “Incorporating nesterov momentum into adam”. In: (2016).
- [52] Andrew C. Heusser, David Poeppel, Youssef Ezzyat, and Lila Davachi. “Episodic sequence memory is supported by a theta-gamma phase code”. In: *Nature Neuroscience* 19.10 (2016), pp. 1374–1380. DOI: 10.1038/nn.4374.
- [53] Mikael Lundqvist, Pawel Herman, Scott L. Brincat, Timothy J. Buschman, and Earl K. Miller. “Gamma and beta bursts underlie working memory”. In: *Neuron* 90.1 (2016), pp. 152–164. DOI: 10.1016/j.neuron.2016.02.028.
- [54] Denis Paperno, German Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernandez. “The LAMBADA dataset: Word prediction requiring a broad discourse context”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Katrin Erk and Noah A. Smith. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 1525–1534. DOI: 10.18653/v1/P16-1144. URL: <https://aclanthology.org/P16-1144/>.
- [55] Ben Poole, Subhaneil Lahiri, Maithra Raghu, Jascha Sohl-Dickstein, and Surya Ganguli. “Exponential expressivity in deep neural networks through transient chaos”. In: *Advances in neural information processing systems* 29 (2016).

- [56] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. “Squad: 100,000+ questions for machine comprehension of text”. In: *arXiv preprint arXiv:1606.05250* (2016).
- [57] Sashank J Reddi, Ahmed Hefny, Suvrit Sra, Barnabas Poczos, and Alex Smola. “Stochastic variance reduction for nonconvex optimization”. In: *International conference on machine learning*. PMLR. 2016, pp. 314–323.
- [58] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. “Mastering the game of Go with deep neural networks and tree search”. In: *nature* 529.7587 (2016), pp. 484–489.
- [59] Thomas B Christophel, P Christiaan Klink, Bernhard Spitzer, Pieter R Roelfsema, and John-Dylan Haynes. “The distributed nature of working memory”. In: *Trends in cognitive sciences* 21.2 (2017), pp. 111–124.
- [60] Chelsea Finn, Pieter Abbeel, and Sergey Levine. “Model-agnostic meta-learning for fast adaptation of deep networks”. In: *International conference on machine learning*. PMLR. 2017, pp. 1126–1135.
- [61] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. “Deep learning scaling is predictable, empirically”. In: *arXiv preprint arXiv:1712.00409* (2017).
- [62] Aniruddha Kembhavi, Minjoon Seo, Dustin Schwenk, Jonghyun Choi, Ali Farhadi, and Hannaneh Hajishirzi. “Are you smarter than a sixth grader? textbook question answering for multimodal machine comprehension”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern recognition*. 2017, pp. 4999–5007.
- [63] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. “Overcoming catastrophic forgetting in neural networks”. In: *Proceedings of the national academy of sciences* 114.13 (2017), pp. 3521–3526.
- [64] Takashi Kitamura, Sachie K Ogawa, Dheeraj S Roy, Teruhiro Okuyama, Mark D Morrissey, Lillian M Smith, Roger L Redondo, and Susumu Tonegawa. “Engrams and circuits crucial for systems consolidation of a memory”. In: *Science* 356.6333 (2017), pp. 73–78.
- [65] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. “Pointer Sentinel Mixture Models”. In: *International Conference on Learning Representations*. 2017. URL: <https://openreview.net/forum?id=Byj72udxe>.
- [66] W Scott Terry. *Learning and memory: Basic principles, processes, and procedures*. Routledge, 2017.
- [67] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [68] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Animashree Anandkumar. “signSGD: Compressed optimisation for non-convex problems”. In: *International conference on machine learning*. PMLR. 2018, pp. 560–569.
- [69] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. “Think you have solved question answering? try arc, the ai2 reasoning challenge”. In: *arXiv preprint arXiv:1803.05457* (2018).
- [70] Jianqing Fan. *Local polynomial modelling and its applications: monographs on statistics and applied probability* 66. Routledge, 2018.
- [71] Vineet Gupta, Tomer Koren, and Yoram Singer. “Shampoo: Preconditioned stochastic tensor optimization”. In: *International Conference on Machine Learning*. PMLR. 2018, pp. 1842–1850.
- [72] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, et al. “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play”. In: *Science* 362.6419 (2018), pp. 1140–1144.
- [73] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. “BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 2924–2936. DOI: 10.18653/v1/N19-1300. URL: <https://aclanthology.org/N19-1300/>.
- [74] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. “DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs”. In: *arXiv preprint arXiv:1903.00161* (2019).
- [75] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Phil Bachman, Adam Trischler, and Yoshua Bengio. “Learning deep representations by mutual information estimation and maximization”. In: *International Conference on Learning Representations*. 2019. URL: <https://openreview.net/forum?id=Bklr3j0cKX>.

- [76] Jens G Klinzing, Niels Niethard, and Jan Born. “Mechanisms of systems memory consolidation during sleep”. In: *Nature neuroscience* 22.10 (2019), pp. 1598–1610.
- [77] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. “Natural questions: a benchmark for question answering research”. In: *Transactions of the Association for Computational Linguistics* 7 (2019), pp. 453–466.
- [78] Stefan Larson, Anish Mahendran, Joseph J Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K Kummerfeld, Kevin Leach, Michael A Laurenzano, Lingjia Tang, et al. “An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 1311–1316.
- [79] Colin Lockard, Prashant Shiralkar, and Xin Luna Dong. “Openceres: When open information extraction meets the semi-structured web”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 2019, pp. 3047–3056.
- [80] Tsendsuren Munkhdalai, Alessandro Sordani, Tong Wang, and Adam Trischler. “Metalearned neural memory”. In: *Advances in Neural Information Processing Systems* 32 (2019).
- [81] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. “Social IQa: Commonsense Reasoning about Social Interactions”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4463–4473. DOI: 10.18653/v1/D19-1454. URL: <https://aclanthology.org/D19-1454/>.
- [82] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. “HellaSwag: Can a Machine Really Finish Your Sentence?” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by Anna Korhonen, David Traum, and Lluís Marquez. Florence, Italy: Association for Computational Linguistics, July 2019, pp. 4791–4800. DOI: 10.18653/v1/P19-1472. URL: <https://aclanthology.org/P19-1472/>.
- [83] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. “Piqa: Reasoning about physical commonsense in natural language”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 2020, pp. 7432–7439.
- [84] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.
- [85] Inigo Casanueva, Tadas Temcinas, Daniela Gerz, Matthew Henderson, and Ivan Vulic. “Efficient Intent Detection with Dual Sentence Encoders”. In: *ACL 2020* (2020), p. 38.
- [86] Mehrdad Farajtabar, Navid Azizan, Alex Mott, and Ang Li. “Orthogonal gradient descent for continual learning”. In: *International conference on artificial intelligence and statistics*. PMLR. 2020, pp. 3762–3773.
- [87] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. “Generative adversarial networks”. In: *Communications of the ACM* 63.11 (2020), pp. 139–144.
- [88] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. “Scaling laws for neural language models”. In: *arXiv preprint arXiv:2001.08361* (2020).
- [89] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. “Transformers are rnns: Fast autoregressive transformers with linear attention”. In: *International conference on machine learning*. PMLR. 2020, pp. 5156–5165.
- [90] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. “On the Variance of the Adaptive Learning Rate and Beyond”. In: *International Conference on Learning Representations*. 2020. URL: <https://openreview.net/forum?id=rkgz2aEKDr>.
- [91] Noam Shazeer. “Glu variants improve transformer”. In: *arXiv preprint arXiv:2002.05202* (2020).
- [92] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. “A dataset of information-seeking questions and answers anchored in research papers”. In: *arXiv preprint arXiv:2105.03011* (2021).
- [93] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations*. 2021. URL: <https://openreview.net/forum?id=YicbFdNTTy>.
- [94] Akihiro Goto, Ayaka Bota, Ken Miya, Jingbo Wang, Suzune Tsukamoto, Xinzhi Jiang, Daichi Hirai, Masanori Murayama, Tomoki Matsuda, Thomas J. McHugh, Takeharu Nagai, and Yasunori Hayashi. “Stepwise synaptic plasticity events drive the early phase of memory consolidation”. In: *Science* 374.6569 (2021), pp. 857–863. DOI:

- 10.1126/science.abj9195. eprint: <https://www.science.org/doi/pdf/10.1126/science.abj9195>. URL: <https://www.science.org/doi/abs/10.1126/science.abj9195>.
- [95] Kazuki Irie, Imanol Schlag, Robert Csordas, and Juergen Schmidhuber. “Going beyond linear transformers with recurrent fast weight programmers”. In: *Advances in neural information processing systems* 34 (2021), pp. 7703–7717.
 - [96] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, et al. “Highly accurate protein structure prediction with AlphaFold”. In: *nature* 596.7873 (2021), pp. 583–589.
 - [97] Tal Ridnik, Emanuel Ben-Baruch, Asaf Noy, and Lihi Zelnik-Manor. “ImageNet-21K Pretraining for the Masses”. In: *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*. 2021. URL: https://openreview.net/forum?id=Zkj_VcZ6o1.
 - [98] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. “Winogrande: An adversarial winograd schema challenge at scale”. In: *Communications of the ACM* 64.9 (2021), pp. 99–106.
 - [99] Imanol Schlag, Kazuki Irie, and Juergen Schmidhuber. “Linear transformers are secretly fast weight programmers”. In: *International Conference on Machine Learning*. PMLR. 2021, pp. 9355–9366.
 - [100] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. “What learning algorithm is in-context learning? investigations with linear models”. In: *arXiv preprint arXiv:2211.15661* (2022).
 - [101] Aydar Bulatov, Yury Kuratov, and Mikhail Burtsev. “Recurrent memory transformer”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 11079–11091.
 - [102] Yanda Chen, Ruiqi Zhong, Sheng Zha, George Karypis, and He He. “Meta-learning via Language Model In-context Tuning”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2022, pp. 719–730.
 - [103] Alexandre Défossez, Leon Bottou, Francis Bach, and Nicolas Usunier. “A Simple Convergence Proof of Adam and Adagrad”. In: *Transactions on Machine Learning Research* (2022). ISSN: 2835-8856. URL: <https://openreview.net/forum?id=ZPQhzTSA7>.
 - [104] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. “Training compute-optimal large language models”. In: *arXiv preprint arXiv:2203.15556* (2022).
 - [105] Kazuki Irie, Francesco Faccio, and Jürgen Schmidhuber. “Neural differential equations for learning to program neural nets through continuous learning rules”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 38614–38628.
 - [106] Kazuki Irie, Imanol Schlag, Róbert Csordás, and Juergen Schmidhuber. “A modern self-referential weight matrix that learns to modify itself”. In: *International Conference on Machine Learning*. PMLR. 2022, pp. 9660–9677.
 - [107] William Merrill, Ashish Sabharwal, and Noah A Smith. “Saturated transformers are constant-depth threshold circuits”. In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 843–856.
 - [108] Dheeraj S Roy, Young-Gyun Park, Minyoung E Kim, Ying Zhang, Sachie K Ogawa, Nicholas DiNapoli, Xinyi Gu, Jae H Cho, Heejin Choi, Lee Kamensky, et al. “Brain-wide mapping reveals that engrams for a single memory are distributed across multiple brain regions”. In: *Nature communications* 13.1 (2022), p. 1799.
 - [109] Yufeng Zhang, Boyi Liu, Qi Cai, Lingxiao Wang, and Zhaoran Wang. “An analysis of attention via the lens of exchangeability and latent variable models”. In: *arXiv preprint arXiv:2212.14852* (2022).
 - [110] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. “Gpt-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).
 - [111] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. “Language models enable simple systems for generating structured views of heterogeneous data lakes”. In: *arXiv preprint arXiv:2304.09433* (2023).
 - [112] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, et al. “Symbolic discovery of optimization algorithms”. In: *Advances in neural information processing systems* 36 (2023), pp. 49205–49233.
 - [113] Ramin Hasani, Mathias Lechner, Tsun-Hsuan Wang, Makram Chahine, Alexander Amini, and Daniela Rus. “Liquid Structural State-Space Models”. In: *The Eleventh International Conference on Learning Representations*. 2023. URL: <https://openreview.net/forum?id=g40TKRKfS7R>.
 - [114] Kazuki Irie, Robert Csordas, and Juergen Schmidhuber. “Practical computational power of linear transformers and their recurrent and self-referential extensions”. In: *arXiv preprint arXiv:2310.16076* (2023).

- [115] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. “CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis”. In: *The Eleventh International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=iaYcJKpY2B_.
- [116] Bo Peng, Eric Alcaide, Quentin Gregory Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Nguyen Chung, Leon Derczynski, Xingjian Du, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Jiaju Lin, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Johan S. Wind, Stanisław Wozniak, Zhenyuan Zhang, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. “RWKV: Reinventing RNNs for the Transformer Era”. In: *The 2023 Conference on Empirical Methods in Natural Language Processing*. 2023. URL: <https://openreview.net/forum?id=7SaXczaBpG>.
- [117] Michael Poli, Stefano Massaro, Eric Nguyen, Daniel Y Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. “Hyena hierarchy: Towards larger convolutional language models”. In: *International Conference on Machine Learning*. PMLR. 2023, pp. 28043–28078.
- [118] Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. “Are emergent abilities of large language models a mirage?” In: *Advances in neural information processing systems* 36 (2023), pp. 55565–55581.
- [119] Aaditya Singh, Stephanie Chan, Ted Moskovitz, Erin Grant, Andrew Saxe, and Felix Hill. “The transient nature of emergent in-context learning in transformers”. In: *Advances in neural information processing systems* 36 (2023), pp. 27801–27819.
- [120] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. “Retentive network: A successor to transformer for large language models”. In: *arXiv preprint arXiv:2307.08621* (2023).
- [121] Johannes Von Oswald, Maximilian Schlegel, Alexander Meulemans, Seijin Kobayashi, Eyvind Niklasson, Nicolas Zucchet, Nino Scherrer, Nolan Miller, Mark Sandler, Max Vladymyrov, et al. “Uncovering mesa-optimization algorithms in transformers”. In: *arXiv preprint arXiv:2309.05858* (2023).
- [122] Wenhai Wang, Zhe Chen, Xiaokang Chen, Jiannan Wu, Xizhou Zhu, Gang Zeng, Ping Luo, Tong Lu, Jie Zhou, Yu Qiao, et al. “Visionllm: Large language model is also an open-ended decoder for vision-centric tasks”. In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 61501–61513.
- [123] Ekin Akyürek, Mehul Damani, Adam Zweiger, Linlu Qiu, Han Guo, Jyothish Pari, Yoon Kim, and Jacob Andreas. “The Surprising Effectiveness of Test-Time Training for Few-Shot Learning”. In: *Forty-second International Conference on Machine Learning*. 2024.
- [124] Ekin Akyürek, Bailin Wang, Yoon Kim, and Jacob Andreas. “In-context language learning: Architectures and algorithms”. In: *arXiv preprint arXiv:2401.12973* (2024).
- [125] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalina, Silas Alberti, James Zou, Atri Rudra, and Christopher Re. “Simple linear attention language models balance the recall-throughput tradeoff”. In: *Forty-first International Conference on Machine Learning*. 2024. URL: <https://openreview.net/forum?id=e93ffDcpH3>.
- [126] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. “xLSTM: Extended Long Short-Term Memory”. In: *arXiv preprint arXiv:2405.04517* (2024).
- [127] Aleksandar Botev, Soham De, Samuel L Smith, Anushan Fernando, George-Cristian Muraru, Ruba Haroun, Leonard Berrada, Razvan Pascanu, Pier Giuseppe Sessa, Robert Dadashi, et al. “RecurrentGemma: Moving Past Transformers for Efficient Open Language Models”. In: *arXiv preprint arXiv:2404.07839* (2024).
- [128] Jonathan Daume, Jan Kamiński, Andrea G. P. Schjetnan, Yousef Salimpour, Umaish Khan, Michael Kyzar, Chrystal M. Reed, William S. Anderson, Taufik A. Valiante, Adam N. Mamelak, and Ueli Rutishauser. “Control of working memory by phase–amplitude coupling of human hippocampal neurons”. In: *Nature* 629 (2024), pp. 393–401. DOI: 10.1038/s41586-024-07309-z.
- [129] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. “The llama 3 herd of models”. In: *arXiv e-prints* (2024), arXiv–2407.
- [130] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekeshe, Fei Jia, and Boris Ginsburg. “RULER: What’s the Real Context Size of Your Long-Context Language Models?” In: *First Conference on Language Modeling*. 2024. URL: <https://openreview.net/forum?id=kIoBbc76Sy>.
- [131] K Jordan, Y Jin, V Boza, Y Jiacheng, F Cecista, L Newhouse, and J Bernstein. “Muon: An optimizer for hidden layers in neural networks, 2024b”. In: *URL https://kellerjordan.github.io/posts/muon* (2024).
- [132] Praneeth Kacham, Vahab Mirrokni, and Peilin Zhong. “PolySketchFormer: Fast Transformers via Sketching Polynomial Kernels”. In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by Ruslan Salakhutdinov,

- Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp. Vol. 235. *Proceedings of Machine Learning Research*. PMLR, July 2024, pp. 22748–22770. URL: <https://proceedings.mlr.press/v235/kacham24a.html>.
- [133] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Igorevich Sorokin, Artyom Sorokin, and Mikhail Burtsev. “BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack”. In: *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 2024. URL: <https://openreview.net/forum?id=u7m2CG84BQ>.
 - [134] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. “Deepseek-v3 technical report”. In: *arXiv preprint arXiv:2412.19437* (2024).
 - [135] Bo Liu, Rui Wang, Lemeng Wu, Yihao Feng, Peter Stone, and Qiang Liu. “Longhorn: State space models are amortized online learners”. In: *arXiv preprint arXiv:2407.14207* (2024).
 - [136] William Merrill, Jackson Petty, and Ashish Sabharwal. “The Illusion of State in State-Space Models”. In: *Forty-first International Conference on Machine Learning*. 2024. URL: <https://openreview.net/forum?id=QZgo9JZpLq>.
 - [137] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin A Raffel, Leandro Von Werra, Thomas Wolf, et al. “The fineweb datasets: Decanting the web for the finest text data at scale”. In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 30811–30849.
 - [138] Bo Peng, Daniel Goldstein, Quentin Anthony, Alon Albalak, Eric Alcaide, Stella Biderman, Eugene Cheah, Xingjian Du, Teddy Ferdinan, Haowen Hou, et al. “Eagle and finch: RwkV with matrix-valued states and dynamic recurrence”. In: *arXiv preprint arXiv:2404.05892* (2024).
 - [139] Michael Poli, Armin W Thomas, Eric Nguyen, Pragaash Ponnusamy, Björn Deiseroth, Kristian Kersting, Taiji Suzuki, Brian Hie, Stefano Ermon, Christopher Ré, et al. “Mechanistic design and scaling of hybrid architectures”. In: *arXiv preprint arXiv:2403.17844* (2024).
 - [140] Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. “Samba: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling”. In: *arXiv preprint arXiv:2406.07522* (2024).
 - [141] Ivan Rodkin, Yuri Kuratov, Aydar Bulatov, and Mikhail Burtsev. “Associative recurrent memory transformer”. In: *arXiv preprint arXiv:2407.04841* (2024).
 - [142] Clayton Sanford, Daniel Hsu, and Matus Telgarsky. “Transformers, parallel computation, and logarithmic depth”. In: *Forty-first International Conference on Machine Learning*. 2024. URL: <https://openreview.net/forum?id=QCZabHkQhB>.
 - [143] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. “Learning to (learn at test time): Rnns with expressive hidden states”. In: *arXiv preprint arXiv:2407.04620* (2024).
 - [144] Garrett Tanzer, Mirac Suzgun, Eline Visser, Dan Jurafsky, and Luke Melas-Kyriazi. “A Benchmark for Learning to Translate a New Language from One Grammar Book”. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=tbVWug9f2h>.
 - [145] Wannan Yang, Chen Sun, Roman Huszár, Thomas Hainmueller, Kirill Kiselev, and György Buzsáki. “Selection of experience for memory by hippocampal sharp wave ripples”. In: *Science* 383.6690 (2024), pp. 1478–1483.
 - [146] Ruiqi Zhang, Spencer Frei, and Peter L Bartlett. “Trained transformers learn linear models in-context”. In: *Journal of Machine Learning Research* 25.49 (2024), pp. 1–55.
 - [147] Yushun Zhang, Congliang Chen, Tian Ding, Ziniu Li, Ruoyu Sun, and Zhiquan Luo. “Why transformers need adam: A hessian perspective”. In: *Advances in neural information processing systems* 37 (2024), pp. 131786–131823.
 - [148] Lisa Adams, Felix Busch, Tianyu Han, Jean-Baptiste Excoffier, Matthieu Ortala, Alexander Löser, Hugo JWL Aerts, Jakob Nikolas Kather, Daniel Truhn, and Keno Bressem. “Longhealth: A question answering benchmark with long clinical documents”. In: *Journal of Healthcare Informatics Research* (2025), pp. 1–17.
 - [149] Zeyuan Allen-Zhu. “Physics of Language Models: Part 4.1, Architecture Design and the Magic of Canon Layers”. In: *SSRN Electronic Journal* (May 2025). <https://ssrn.com/abstract=5240330>.
 - [150] Shaden Alshammari, John Hershey, Axel Feldmann, William T Freeman, and Mark Hamilton. “I-Con: A Unifying Framework for Representation Learning”. In: *arXiv preprint arXiv:2504.16929* (2025).
 - [151] Ali Behrouz, Zeman Li, Praneeth Kacham, Majid Daliri, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. “Atlas: Learning to optimally memorize the context at test time”. In: *arXiv preprint arXiv:2505.23735* (2025).
 - [152] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. “It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization”. In: *arXiv preprint arXiv:2504.13173* (2025).

- [153] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. “Titans: Learning to Memorize at Test Time”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025. URL: <https://openreview.net/forum?id=8GjSf9Rh7Z>.
- [154] Franz Louis Cesista. *Heuristic Solutions for Steepest Descent on the Stiefel manifold*. July 2025. URL: <https://leloykun.github.io/ponder/steepest-descent-stiefel%7D>.
- [155] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. “Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities”. In: *arXiv preprint arXiv:2507.06261* (2025).
- [156] Francesco Pappone Devan Selvaraj. *Multiscale Muon*. <https://publish.obsidian.md/ueaj/Machine+Learning/Research+Ideas/Multiscale+Muon>. Blogpost. Aug. 2025.
- [157] Benoit Dherin, Michael Munn, Hanna Mazzawi, Michael Wunder, and Javier Gonzalvo. “Learning without training: The implicit dynamics of in-context learning”. In: *arXiv preprint arXiv:2507.16003* (2025).
- [158] Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, et al. “Cartridges: Lightweight and general-purpose long context representations via self-study”. In: *arXiv preprint arXiv:2506.06266* (2025).
- [159] Samuel J Gershman, Ila Fiete, and Kazuki Irie. “Key-value memory in the brain”. In: *Neuron* 113.11 (2025), pp. 1694–1707.
- [160] Riccardo Grazi, Julien Siems, Jörg K.H. Franke, Arber Zela, Frank Hutter, and Massimiliano Pontil. “Unlocking State-Tracking in Linear RNNs Through Negative Eigenvalues”. In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=UvTo3tVBk2>.
- [161] Jiaxi Hu, Yongqi Pan, Jusen Du, Disen Lan, Xiaqiang Tang, Qingsong Wen, Yuxuan Liang, and Weigao Sun. “Improving Bilinear RNN with Closed-loop Control”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025.
- [162] Kazuki Irie, Róbert Csordás, and Jürgen Schmidhuber. “Metalearning Continual Learning Algorithms”. In: *Transactions on Machine Learning Research* (2025). ISSN: 2835-8856. URL: <https://openreview.net/forum?id=IaUh7CSD3k>.
- [163] Kazuki Irie and Samuel J Gershman. “Fast weight programming and linear transformers: from machine learning to neurobiology”. In: *arXiv preprint arXiv:2508.08435* (2025).
- [164] Ben Keigwin, Dhruv Pai, and Nathan Chen. *Gram-Space Manifold Muon*. Oct. 2025. URL: <https://blog.tilderresearch.com/vignettes/gram-space%7D>.
- [165] Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. “Minimax-01: Scaling foundation models with lightning attention”. In: *arXiv preprint arXiv:2501.08313* (2025).
- [166] Saleh Momeni, Sahisnu Mazumder, Zixuan Ke, and Bing Liu. “In-context continual learning assisted by an external continual learner”. In: *Proceedings of the 31st International Conference on Computational Linguistics*. 2025, pp. 7292–7306.
- [167] Matteo Pagliardini, Pierre Ablin, and David Grangier. “The AdEMAMix Optimizer: Better, Faster, Older”. In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=jj7b3p5kLY>.
- [168] Renhao Pei, Yihong Liu, Peiqin Lin, François Yvon, and Hinrich Schütze. “Understanding In-Context Machine Translation for Low-Resource Languages: A Case Study on Manchu”. In: *arXiv preprint arXiv:2502.11862* (2025).
- [169] Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Haowen Hou, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, et al. “RWKV-7” Goose” with Expressive Dynamic State Evolution”. In: *arXiv preprint arXiv:2503.14456* (2025).
- [170] Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Haowen Hou, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, et al. “Rwkv-7” goose” with expressive dynamic state evolution”. In: *arXiv preprint arXiv:2503.14456* (2025).
- [171] Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. “Optimizing Test-Time Compute via Meta Reinforcement Finetuning”. In: *Forty-second International Conference on Machine Learning*. 2025. URL: <https://openreview.net/forum?id=Tq0DUDsU4u>.
- [172] Chongjie Si, Debing Zhang, and Wei Shen. “Adamuon: Adaptive muon optimizer”. In: *arXiv preprint arXiv:2507.11005* (2025).

- [173] Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. “DeltaProduct: Improving State-Tracking in Linear RNNs via Householder Products”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025. URL: <https://openreview.net/forum?id=SoRiaijTGr>.
- [174] David Silver and Richard S Sutton. “Welcome to the era of experience”. In: *Google AI 1* (2025).
- [175] Richard S. Sutton. *The OaK Architecture: A Vision of SuperIntelligence from Experience*. Keynote at the Reinforcement Learning Conference (RLC). Accessed: 2025-12-02. Aug. 2025. URL: <https://rlj.cs.umass.edu/rlc-2025>.
- [176] Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham M. Kakade. “SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling”. In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=IDxZhXrpNf>.
- [177] Ke Alexander Wang, Jiaxin Shi, and Emily B Fox. “Test-time regression: a unifying framework for designing sequence models with associative memory”. In: *arXiv preprint arXiv:2501.12352* (2025).
- [178] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, junxian guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. “DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads”. In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=cFu7ze7xUm>.
- [179] hongzhou yu, Tianhao Cheng, Yingwen Wang, Wen He, Qing Wang, Ying Cheng, Yuejie Zhang, Rui Feng, and Xiaobo Zhang. “FineMedLM-o1: Enhancing Medical Knowledge Reasoning Ability of LLM from Supervised Fine-Tuning to Test-Time Training”. In: *Second Conference on Language Modeling*. 2025. URL: <https://openreview.net/forum?id=7ZwuGZCopw>.
- [180] Yifan Zhang, Zhen Qin, and Quanquan Gu. “Higher-order Linear Attention”. In: *arXiv preprint arXiv:2510.27258* (2025).

A Generalized Formulation for Nested Learning and Nested Systems

In this section, we discuss the generalized version of nested systems and NSAM, we discussed in Section 3:

Definition 6 ((Generalized) Nested System). *A (ordered) nested system is a system with K (ordered) levels such that each level $1 \leq k \leq K$ consists of a set of optimization problems $\{(\mathcal{L}_i^{(k)}, C_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$, where $\mathcal{L}_i(\cdot; \cdot)$ is the optimization objective in the i -th problem, C_i is its context (the data that is optimized on), Θ_i is the set of its parameters, and each optimization problem is optimized using gradient descent, or equivalently:*

$$\theta_{i,t+1}^{(k)} = \arg \min_{\Phi_i^{(k)}} \mathcal{L}_i^{(k)}(\Phi_i^{(k)}; x_{t+1}) + \frac{1}{2\eta_{i,t+1}^{(k)}} \|\Phi_i^{(k)} - \theta_{i,t}^{(k)}\|_2^2 \quad \text{where } x_{t+1} \sim C_i^{(k)}, \quad \text{and } \Phi_i^{(k)} \in \Theta_i^{(k)}. \quad (98)$$

Definition 7 ((Generalized) Nested System of Associative Memories). *A nested system of associative memory (NSAM) is a system with K (ordered) levels such that each level $1 \leq k \leq K$ consists of a set of optimization problems $\{(\mathcal{L}_i^{(k)}, C_i^{(k)}, \Theta_i^{(k)})\}_{i=1}^{N_k}$, where $C_i = \{(\mathbf{k}_j^{(i)}, \mathbf{v}_j^{(i)})\}_{j=1}^{L_i}$ is a set of ground-truth mappings, $\mathcal{L}_i(\cdot; \cdot, \cdot)$ is the optimization objective that measures the quality of memory learned mappings in the i -th problem, Θ_i is the set of memory parameters, and each optimization problem is optimized using gradient descent:*

$$\theta_{i,t+1}^{(k)} = \arg \min_{\Phi_i^{(k)}} \mathcal{L}_i^{(k)}(\Phi_i^{(k)}; \mathbf{k}_{t+1}^{(i)}, \mathbf{v}_{t+1}^{(i)}) + \frac{1}{2\eta_{i,t+1}^{(k)}} \|\Phi_i^{(k)} - \theta_{i,t}^{(k)}\|_2^2 \quad \text{where } (\mathbf{k}_{t+1}^{(i)}, \mathbf{v}_{t+1}^{(i)}) \sim C_i^{(k)}, \quad \text{and } \Phi_i^{(k)} \in \Theta_i^{(k)}. \quad (99)$$

B Adam, AdaGrad, and Other Similar Optimizers as Associative Memory Modules

Revisiting the momentum term without using the chain rule in backpropagation,

$$\begin{aligned} W_{\ell,t+1} &= W_{\ell,t} + \mathbf{m}_{\ell,t+1} \\ \mathbf{m}_{\ell,t+1} &= \alpha_{\ell,t+1} \mathbf{m}_{\ell,t} - \eta_{\ell,t+1} \nabla_{W_{\ell,t}} \mathcal{L}(W_{\ell,t}; \mathbf{x}_{t+1}), \end{aligned} \quad (100)$$

one can interpret the momentum as a key or value-less associative memory (Behrouz et al. 2025b, see Section 5), where the gradient terms $\nabla_{W_{\ell,t}} \mathcal{L}(W_{\ell,t}; \mathbf{x}_{t+1})$ are compressed into the momentum. We expect from a powerful momentum term to perfectly memorize all the past gradients in the training process so it can better model the current update to the weights. To this end, one can start from defining a simple objective as:

$$\tilde{\mathcal{L}}_t = \sum_{i=1}^t \|\mathbf{m}_{\ell,t} \odot \mathbf{g}_{\ell,i+1} - \mathbf{P}_{\ell,t}\|_2^2 + \lambda_{\ell} \|\mathbf{m}_{\ell,t}\|_F^2, \quad (101)$$

where $\mathbf{g}_{\ell,t+1} = -\nabla_{W_{\ell,t}} \mathcal{L}(W_{\ell,t}; \mathbf{x}_{t+1})$. This objective aims to find a momentum term that does not simply map the gradients to 1 (which also results in limited memory management in momentum), but it maps gradients to a global property of past data samples. The more expressive this global property is, the more accurately the momentum can incorporate the compressed information from past. Accordingly, the objective in Equation 101 admits an optimal associative memory that maps gradients to $\mathbf{P}_{\ell,t+1}$ as:

$$\begin{aligned} \mathbf{m}_{\ell,i}^{(t)*} &= \left[\left(\mathbf{H}_{\ell,i}^{(t)} + \lambda_{\ell} \mathbf{I} \right)^{-1} \right] \odot \left(\mathbf{M}_{\ell,i}^{(t)} \right) = \left[\left(\mathbf{H}_{\ell,i}^{(t)} + \lambda_{\ell} \mathbf{I} \right)^{-1} \right] \odot \tilde{\mathbf{M}}_{\ell,i+1}^{(t)} \odot \mathbf{P}_{\ell,t}, \quad \text{where} \\ \mathbf{M}_{\ell,i+1}^{(t)} &= \mathbf{M}_{\ell,i}^{(t)} + \beta_1 \mathbf{g}_{\ell,i+1} \odot \mathbf{P}_{\ell,t} = \tilde{\mathbf{M}}_{\ell,i+1}^{(t)} \odot \mathbf{P}_{\ell,t}, \\ \tilde{\mathbf{M}}_{\ell,i+1}^{(t)} &= \mathbf{M}_{\ell,i}^{(t)} + \beta_1 \mathbf{g}_{\ell,i+1}, \\ \mathbf{H}_{\ell,i+1}^{(t)} &= \mathbf{H}_{\ell,i}^{(t)} + \beta_2 \mathbf{g}_{\ell,i+1} \odot \mathbf{g}_{\ell,i+1} = \mathbf{H}_{\ell,i}^{(t)} + \beta_2 \mathbf{g}_{\ell,i+1}^2. \end{aligned} \quad (102)$$

Given this solution, one can write the update step as:

$$W_{\ell,i+1} = W_{\ell,i} - \eta_t \mathbf{m}_{\ell,i}^{(t)*} = W_{\ell,i} - \eta_i \left[\left(\mathbf{H}_{\ell,i}^{(t)} + \lambda_{\ell} \mathbf{I} \right)^{-1} \right] \odot \tilde{\mathbf{M}}_{\ell,i+1}^{(t)} \odot \mathbf{P}_{\ell,t}. \quad (103)$$

We start with a simple case, where P_{ℓ_t} is the summation of the square of past gradients: i.e., $P_{\ell_t} = \sum_{i=1}^t g_{\ell_{i+1}}^2$. With the choice of $\lambda \rightarrow 0$, Equation 103 recovers simple gradient descent with momentum. That is, let $\lambda = 0$, we have:

$$W_{\ell_{i+1}} = W_{\ell_i} - \eta_t m_{\ell_i}^{(t)*} = W_{\ell_i} - \eta_i \left(H_{\ell_i}^{(t)} \right)^{-1} \odot \tilde{M}_{\ell_{i+1}}^{(t)} \odot \underbrace{P_{\ell_t}}_{H_{\ell_i}^{(t)} / \beta_2} = W_{\ell_i} - \eta_t \beta_2 \tilde{M}_{\ell_{i+1}}^{(t)}, \quad (104)$$

which based on the definition of $\tilde{M}_{\ell_{i+1}}^{(t)}$, this update rule is equivalent to gradient descent with momentum. Next, we explore a more sophisticated design choice, where we use P_{ℓ_t} as the variance of data samples before token $t + 1$. In this case, formally, $P_{\ell_t} = \sqrt{\sum_{i=1}^t g_{\ell_{i+1}}^2}$ and so the update rule is as follows:

$$W_{\ell_{i+1}} = W_{\ell_i} - \eta_t m_{\ell_i}^{(t)*} = W_{\ell_i} - \eta_i \left[\left(H_{\ell_i}^{(t)} + \lambda_\ell I \right)^{-1} \right] \odot \tilde{M}_{\ell_{i+1}}^{(t)} \odot \underbrace{P_{\ell_t}}_{H_{\ell_i}^{(t)\frac{1}{2}} / \sqrt{\beta_2}} \approx W_{\ell_i} - \frac{\eta_t}{\sqrt{\beta_2}} \frac{\tilde{M}_{\ell_i}^{(t)}}{H_{\ell_i}^{(t)1/2} + \varepsilon} \quad (105)$$

which is equivalent to the popular Adam optimizer (Kingma et al. 2014a). Therefore, Adam is an optimal associative memory given the L_2 regression objective that is defined in Equation 101. In fact, the update component of Adam at each state aims to learn a mapping between the gradients and their variance (as a global property of past data samples). One interesting point about this formulation is about the frequency of elements. Looking at the first and second momentum in Adam optimizer, the frequency of update for both of these memories are the same as both are updated after each sample. Furthermore, the computation of each of them can be done in parallel and so are independent. Accordingly, it is one of the few examples that two components without any internal gradient flow, are placed in the same frequency and so level (see Section 3.2).

Going beyond element-wise update, we reformulate Equation 101 with outer-product operation as:

$$\tilde{\mathcal{L}}_t = \sum_{i=1}^t \|m_{\ell_t} g_{\ell_{i+1}} - P_{\ell_t}\|_2^2 + \lambda_\ell \|m_{\ell_t}\|_F^2, \quad (106)$$

where $g_{\ell_{i+1}} = -\nabla_{W_{\ell_t}} \mathcal{L}(W_{\ell_t}; x_{t+1})$. Similar to Equation 102, finding the optimal solution to the above objective is defined as:

$$m_{\ell_i}^{(t)*} = \left[\left(H_{\ell_i}^{(t)} + \lambda_\ell I \right)^{-1} \right] \left(M_{\ell_i}^{(t)} \right) = \left[\left(H_{\ell_i}^{(t)} + \lambda_\ell I \right)^{-1} \right] \tilde{M}_{\ell_{i+1}}^{(t)} P_{\ell_t}, \quad \text{where} \quad (107)$$

$$M_{\ell_{i+1}}^{(t)} = M_{\ell_i}^{(t)} + \beta_1 P_{\ell_t} g_{\ell_{i+1}}^\top = P_{\ell_t} \tilde{M}_{\ell_{i+1}}^{(t)}, \quad (108)$$

$$\tilde{M}_{\ell_{i+1}}^{(t)} = M_{\ell_i}^{(t)} + \beta_1 g_{\ell_{i+1}}^\top, \quad (109)$$

$$H_{\ell_{i+1}}^{(t)} = H_{\ell_i}^{(t)} + \beta_2 g_{\ell_{i+1}} g_{\ell_{i+1}}^\top. \quad (110)$$

With a similar choice as Adam, i.e., letting P_{ℓ_t} be the variance of the gradients so far, $P_{\ell_t} = \sqrt{\sum_{i=1}^t g_{\ell_{i+1}} g_{\ell_{i+1}}^\top}$, then the above solution can be simplified as:

$$W_{\ell_{i+1}} = W_{\ell_i} - \eta_t m_{\ell_i}^{(t)*} = W_{\ell_i} - \eta_i \left[\left(H_{\ell_i}^{(t)} + \lambda_\ell I \right)^{-1} \right] \underbrace{P_{\ell_t}}_{H_{\ell_i}^{(t)\frac{1}{2}} / \sqrt{\beta_2}} \tilde{M}_{\ell_{i+1}}^{(t)} \approx W_{\ell_i} - \frac{\eta_t}{\sqrt{\beta_2}} H_{\ell_i}^{(t)-1/2} \tilde{M}_{\ell_i}^{(t)} \quad (111)$$

The above formulation is the AdaGrad with momentum (Défossez et al. 2022) and so generalizes the AdaGrad (Duchi et al. 2011), i.e., when $\beta_1 = 1$. Similarly, based on the connection of Adam optimizer (Kingma et al. 2014a) with other algorithms such as RMSProp (Hinton et al. 2012), SignSGD and its momentum-based variants (Bernstein et al. 2018), NAdam (Dozat 2016), AMSGrad (Reddi et al. 2016), RAdam (Liu et al. 2020), and Lion (Chen et al. 2023), as well as considering AdaGrad's connection with optimizers such as Shampoo (Gupta et al. 2018) and Soap (Vyas et al. 2025)—i.e., as the approximation of the preconditioning term—we can conclude that all these optimizers can be re-formulated as associative memory that aims to compress the gradients.

C Delta Gradient Descent with Normalization

In Section 4.5 we discussed that gradient descent can be seen as an associative memory, and be reformulated as:

$$W_{t+1} = \arg \min_W \langle W \mathbf{x}_t, \nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t) \rangle + \frac{1}{2\eta_t} \|W - W_t\|_2^2, \quad (112)$$

where each step aims at learning the negative of the gradient direction. Due to the fact that this learning rule only uses update terms that depend on the current gradient, we defined $\mathbf{u}_t = -\nabla_{y_t} \mathcal{L}(W_t; \mathbf{x}_t)$, and extended the above process to Delta Gradient Descent with more expressive objective of $\mathcal{L}_2$ regression loss:

$$W_{t+1} = \arg \min_W \frac{1}{2} \|W \mathbf{x}_t - \mathbf{u}_t\|_2^2 + \frac{1}{2\eta_t} \|W - W_t\|_2^2. \quad (113)$$

We assume that $\mathbf{x}_t$ is normalized (e.g. in normalized memory systems or in neural networks with normalization layers, $\|\mathbf{x}_t\|_2 = \lambda$). Taking gradient to optimize the above objective,

$$2(W_{t+1} \mathbf{x}_t - \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t)) \mathbf{x}_t^\top + 2\eta_t (W_{t+1} - W_t) = 0, \quad (114)$$

which results in:

$$W_{t+1} (\mathbf{x}_t \mathbf{x}_t^\top + \eta_t I) = \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top + \eta_t W_t, \quad (115)$$

$$\Rightarrow W_{t+1} = (\nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top + \eta_t W_t) (\mathbf{x}_t \mathbf{x}_t^\top + \eta_t I)^{-1}. \quad (116)$$

To compute $(\mathbf{x}_t \mathbf{x}_t^\top + \eta_t I)^{-1}$ term with Sherman-Morrison lemma, we have:

$$(\mathbf{x}_t \mathbf{x}_t^\top + \eta_t I)^{-1} = \frac{1}{\eta_t} \left(I - \frac{1}{\lambda^2 + \eta_t} \mathbf{x}_t \mathbf{x}_t^\top \right), \quad (117)$$

and so:

$$W_{t+1} = (\nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top + \eta_t W_t) \frac{1}{\eta_t} \left(I - \frac{1}{\lambda^2 + \eta_t} \mathbf{x}_t \mathbf{x}_t^\top \right) \quad (118)$$

$$\Rightarrow W_{t+1} = W_t \left(I - \frac{1}{\lambda^2 + \eta_t} \mathbf{x}_t \mathbf{x}_t^\top \right) + \frac{1}{\eta_t} \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top - \underbrace{\frac{1}{\lambda^2 \eta_t + \eta_t^2} \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top \mathbf{x}_t \mathbf{x}_t^\top}_{\frac{\lambda}{\lambda^2 \eta_t + \eta_t^2} \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top} \quad (119)$$

$$\Rightarrow W_{t+1} = W_t \left(I - \frac{1}{\lambda^2 + \eta_t} \mathbf{x}_t \mathbf{x}_t^\top \right) - \left(\frac{\lambda}{\lambda^2 \eta_t + \eta_t^2} - \frac{1}{\eta_t} \right) \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top \quad (120)$$

$$\Rightarrow W_{t+1} = W_t (I - \alpha_t \mathbf{x}_t \mathbf{x}_t^\top) - \beta \nabla_{y_t} \mathcal{L}(W_t, \mathbf{x}_t) \mathbf{x}_t^\top \quad (121)$$